

基于稳定通量与物质坐标的 药材干燥建模及停机评估

摘要

在热风干燥条件下，我们结合环境观测、非线性物性及收缩数据，建立圆柱药材的径向热质传递与全域停机评估模型。两种情形均保留表面水分交换：先不计蒸发潜热，再按干密度换算质量通量、从对流供热中扣除耗热（潜热 2.45 MJ/kg）。每问均用 BDF、Radau 计算两种情形，主答案保留四位小数。

针对问题一，为求七时刻五位置状态，插值环境数据，首先由环形控制体能量守恒建立**恒热物性导热与非线性水分扩散模型**；然后设置轴心对称与表面对流边界，采用表层加密有限体积离散，以变阶后向差分公式（BDF）求解、Radau IIA 同网格复核。1800 s 时，不计潜热，两算法均得出中心、表面温度为 **33.5753、36.7856 °C**，含水率 2.5500、1.5102 kg/kg（表 1、2）；最后计入潜热，两算法均得温度 **10.5466、11.9026 °C**，含水率不变（表 S5、S6）。另检查网格、能量收支与端面影响。

针对问题二，首先建立**温度与含水率双向反馈的非线性热质耦合模型**；然后随状态更新物性，以积分通量处理扩散退化，BDF 联立推进、Radau 同网格核对前 3 h 截面。3 h 时，不计潜热，两算法均得中心、表面温度 **49.8495、49.9664 °C** 及含水率 1.7662、1.0081 kg/kg（表 3、4）；最后计入潜热，两算法均得出温度为 **30.5356、31.1755 °C** 及含水率 2.2830、0.8830 kg/kg（表 S11、S12）。另作冻结物性与网格加密检验。

针对问题三，先延续第二问动力系统，建立**最大含水率约束的干燥终点模型**。BDF 与 Radau 分别定位 0.15 kg/kg 临界事件，再延时、向上舍入并回代全节点确定严格停机时间。不计潜热，两算法的临界、严格时间均为 **57.4740、57.4754 h**；计入潜热，均为 **60.5653、60.5668 h**（双算法时刻见表 S15，两种含水率截面依次见表 5、S18）。网格加密检查事件误差；无潜热扰动与八角点测试仅评估有限情景风险。

针对问题四，以保形分段三次 Hermite 插值（PCHIP）重建半径，由干基连续性建立**均匀收缩下的物质坐标热质传递模型**，将移动域映射到固定域，然后用附录 4 物性、有限体积法及 BDF/Radau 双算法核对；含潜热时同步更新收缩干密度。不计潜热，两算法的临界、严格时间均为 **51.0885、51.0899 h**；计入潜热，均为 **56.5979、56.5994 h**（表 S21），严格终点半径均为 1.2000 cm（表 S20）。两种含水率截面依次见表 6、S24；零输运、干基收支与无潜热四格对照另作检验。

数值改进：以近状态保护的积分通量替代端点平均，同为 160 区间时第三问事件偏差由 299.27 s 降至 2.56 s（表 S13）。该处理保留题给扩散定律，仅修正界面离散，减少扩散退化对判停时刻的影响。八组 BDF/Radau 核对的最大温差约 2.91×10^{-6} °C、最大临界时差约 0.00176 s（表 S4–S21），仅支持时间求解一致性。

针对本题的创新：将稳定扩散通量、干基守恒的收缩坐标、表面潜热修正与全节点事件回代结合，兼顾扩散退化、收缩质量测度与严格达标判据。潜热项与收缩干密度同步更新，使热边界修正与水分质量换算采用一致的干基口径。全节点回代将停机判据落实到未舍入状态，避免仅凭平均含水率判停。创新在于本题的模型适配，而非新积分算法。相变位置与气固交换依然需要实验校准。

关键词：药材干燥；非线性扩散；有限体积法；物质坐标；蒸发潜热

一、问题重述

1.1 背景与要求

热风与药材交换热量，同时水分由内部向表面迁移并与环境交换。外部观测容易反映表层状态，而产品达标要求约束整个内部，因此只研究平均含水率或表面浓度不足以确定停机时刻。题目给出长 25 cm、初始半径 2 cm 的圆柱药材，初温 28℃、初始干基含水率 2.55 kg/kg，以及烘房环境、收缩半径和三组经验物性。

问题一要求附录 2 条件下 0 至 1800 s 的温湿变化，在七个指定时刻、五个径向位置列出数值，并输出逐秒、每 0.1 cm 的温度与含水率。问题二要求统一采用附录 3 研究整个过程，正文给出前 3 h 逐 0.5 h 的截面，工作簿给出全过程逐秒数据。问题三延续这一动力系统，以各处含水率低于 0.15 kg/kg 确定时间，并给出每 6 h 截面和每 60 s 数据。问题四改用附录 4 及附件 2 的半径变化，重新计算干燥时间和变域截面；固定位置超出药材后不能再赋予内部含水率。

六张核心结果表按题目顺序编号：表 1、2 对应问题一温度与含水率，表 3、4 对应问题二温度与含水率，表 5、6 对应问题三、四含水率。它们报告无潜热基线的结果；潜热扩展与验证等辅助表采用 S 前缀，避免与题目指定表混淆。

二、问题分析与总体思路

2.1 传递机制与问题关系

多孔介质干燥模型需要区分液态水迁移、气相输运及相变能量交换。Datta 将食品过程中的热质传递模型按机制与简化程度联系起来^[1]。据此，我们以题给有效扩散模型为基线，另将侧表面蒸发列为条件扩展。模型未求解气相压力和内部相变，有效热容量方程不能代表完整多相能量模型。

图 1 将圆柱截面简化为径向位置 r ：轴心满足对称条件，侧表面承担热质交换，内部扩散决定温湿场的非均匀程度。问题四再以 $R(t)$ 描述收缩，变域与物质点映射见图 15。

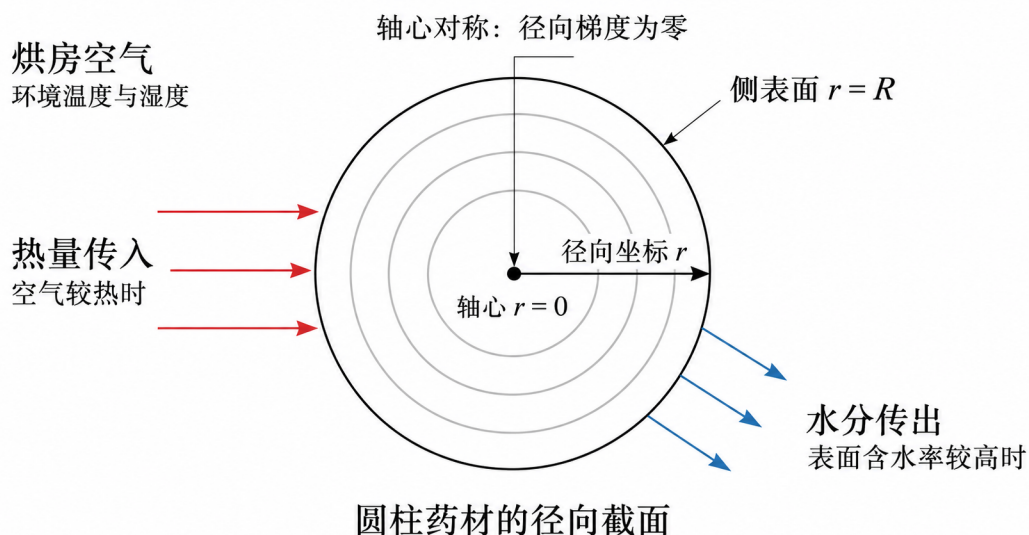


图 1: 圆柱药材径向传热传质的几何与边界示意

同心圆仅表示等半径位置，不是孔隙或实际计算网格。轴心为实体内点，零梯度不代表内部没有扩散；红、蓝箭头分别对应 $T_a > T_s$ 时的热输入和 $C_s > C_a$ 时的水分外流。含潜热时，表面净热输入还须扣除蒸发耗热。

2.2 分层求解与验证路线

四问共用几何与环境输入，但只有第二、三问连续使用同一变物性动力系统，分别计算场分布和达标时间。第一问独立求解短时过程，第四问另用物性和半径序列。图2按这一关系连接，第二问仍从原始初态开始。各流程图围绕无潜热基线展开，表面蒸发扩展只需修改热边界并重新求解。端面诊断、物性与几何四格对照及八角点压力检验均针对基线。

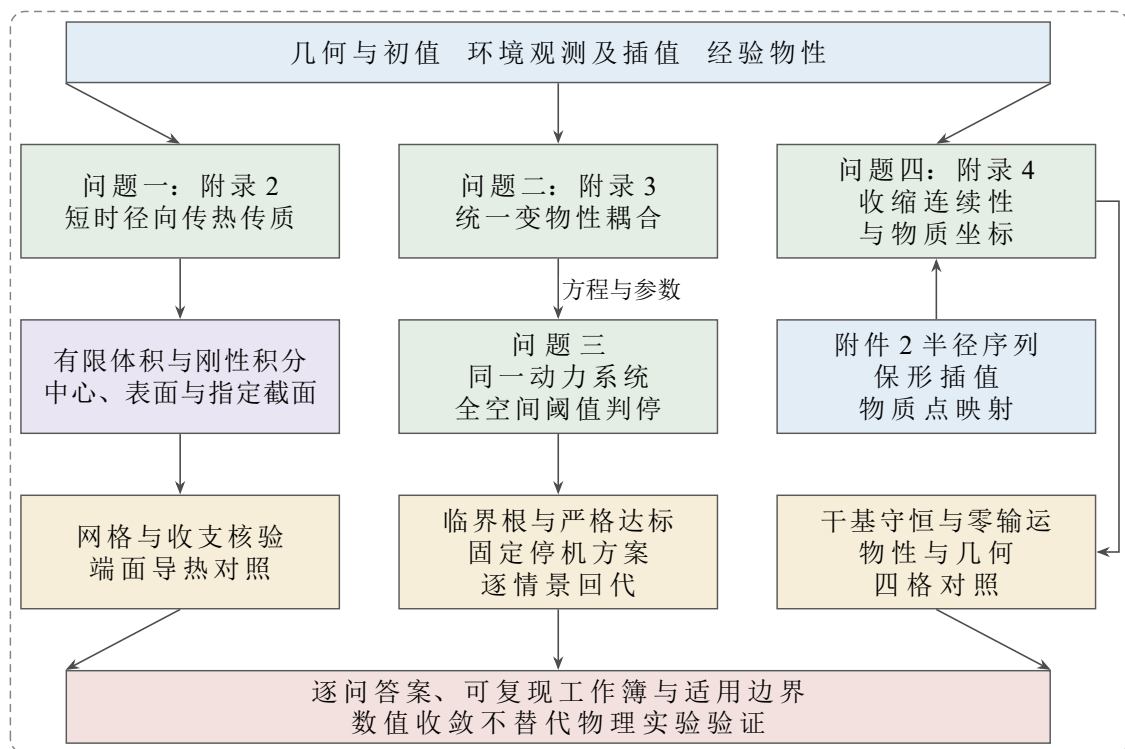


图2：总体技术路线及四问之间的真实联系

后续检验沿用同一状态定义和阈值，避免求解方程、判停对象和导出字段各自采用不同口径。

2.3 问题一：短时扩散与降维边界

空间分辨与边界。七时刻、五位置的输出要求排除仅给均温的集中参数模型。先以扩散尺度和 Biot 数判断内部梯度，再由环形控制体建立径向方程，保留轴心对称及表面 Robin 交换；端面是否可忽略另作无潜热导热诊断，不由长径比直接断言。

求解与检验。附录2热物性固定，水分扩散率仍随含水率变化，故两场分别组装、共享初态和环境输入。用指定截面、网格及收支核对输出。**潜热对照只修正热边界**；由于扩散率不含温度，所以含水率轨道应保持不变，这一不变量用于检验扩展实现。

2.4 问题二：统一变物性与非线性反馈

统一物性与联立求解。本问从原始初态起全程采用附录3，不在1800 s切换公式。温度经 Arrhenius 项影响扩散，失水又改变热容量和导热率，需要在同一状态向量内联立更新。积分通量及近状态保护处理低含水率处的扩散退化，以冻结物性和同网格对照检验其作用。

完整输出与潜热反馈。前3 h逐0.5 h截面用于展示，全过程逐秒温湿数据用于交付，第三问继续使用同一动力系统。表面蒸发冷却通过温度指数反馈扩散，故含潜热时必须重算两场，不能直接扣减温度或拼接第一问末态。

2.5 问题三：全域约束与可执行停机

判据与误差。“各处低于 0.15 kg/kg”要求检查全空间最大含水率，不用均值、表面值或预设干燥半径替代。分别计算临界相等时刻和具有执行余量的严格停机时间，再用未舍入全节点状态回代；后期斜率较小，还需检查场误差对事件时间的放大。

对照与下游任务。BDF 与 Radau 核对时间积分，网格对照检验空间分辨率；无潜热八角点测试检查固定停机方案，失败时再给统一延长时间。含潜热轨道需单独定位终点，不能按温差估算时长，也不能继承未经回代的无潜热压力情景保证。

2.6 问题四：收缩守恒与效应分离

质量测度与变域输出。从干物质、水分连续性出发，在均匀缩放假设下转入物质坐标，避免把体积变化误作额外失水。半径由附件 2 保形插值得到；固定物理位置离开药材后记为空，当前表面单独采样。

归因与潜热修正。以物性组与几何的四格对照区分条件收缩效应和交互，配合零输运特例及干基收支验证。含潜热时收缩干密度按 $\rho_d \propto R^{-2}$ 同步进入蒸发通量；经验总体密度与守恒干密度的兼容性仍需检查，不能用数值收敛替代物理闭合。

三、 模型假设

3.1 假设及其作用

1. 药材在中截面可视为均质、各向同性的径向对象；两套径向模型均令端部绝热且无水分通量。该假设把有限圆柱降为一维，端面影响另作无潜热局部导热诊断，不假定所有端部温度与中截面相同。
2. 干基含水率定义为水质量与干物质质量之比。内部水分以有效扩散表示，气固交换采用题给系数的 Robin 条件。环境的 kg/kg 与固体干基量并非天然同一热力学状态，我们将其差值解释为有效驱动力，并未建立吸附平衡。
3. 附录 3、4 未重新给出表面交换系数，问题二至四沿用附录 2 的 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。该沿用属于模型闭合，不把“附录 3 统一物性”误解为取消表面交换；其影响用独立参数扰动评估。
4. 不含潜热基线采用题给密度和热容形成的有效显热容量，不添加相变项。含潜热扩展另假定所有外流水分在侧表面蒸发，以初始经验密度按 $\rho_{d0} = \rho(C_0)/(1+C_0)$ 换算初始干物质密度，令 $j_v = \rho_d h_m (C_s - C_a)$ ，从对流供热中扣除 $L_v j_v$ ；取 $L_v = 2.45 \times 10^6 \text{ J/kg}^{[2]}$ 。这是一套明确的附加闭合，不是附件已给定的唯一相变模型；两种情况均未引入内部相变和完整水分焓输运。
5. 4 h 以后环境取附件 1 最后 1 h 观测的算术均值并保持不变。其依据是末段趋稳；它是有限观测的闭合假设，而非题目给出的长期实测。环境外推不确定性另用联合压力情景检查。
6. 问题四长度不变、径向均匀缩放，固体速度与半径成正比。干物质密度在空间上均匀并随截面积倒数变化。经验密度只进入有效热容量；若要求它同时严格代表当前总体密度，尚需额外的收缩-密度闭合数据。

上述假设给出当前数据下可计算的条件模型。四问分别并列无潜热基线与表面蒸发扩展；潜热取值来源不能证明所设气固驱动力成立，气固平衡和非均匀形变仍需补充观测。含潜热计算不修改原基线工作簿，也不继承其压力情景停机保证。未特别标明“含潜热”或“表面蒸发扩展”的数值图表，均对应无潜热基线。两组数据分别存放，受控模型对照应固定环境、物性与数值设置。第四问两套生产答案分别采用经网格复核的 2560、1280 区间，其时间差是各自细网格解的差，不能称为同网格消融；离散差异与潜热模型差异分开报告。

四、符号说明与数据准备

4.1 符号与单位

表 S1: 核心符号及物理含义

符号	含义	单位
r, z, t	径向坐标、距中截面的轴向坐标、时间	m、m、s
$R(t), L$	瞬时半径、长度	m
T, T_K	摄氏温度及绝对温度, $T_K = T + 273.15$	°C、K
C	干基含水率, m_w/m_d	kg/kg
ρ, c_p	有效密度、比热	kg/m ³ 、J/(kg · K)
k, D	导热率、扩散系数	W/(m · K)、m ² /s
h, h_m	对流换热、有效传质系数	W/(m ² · K)、m/s
T_a, C_a	烘房温度与有效环境水分	°C、kg/kg
η, ρ_d, v_s	物质坐标、干物质密度、固相速度	1、kg/m ³ 、m/s
L_v, j_v	汽化潜热、侧表面向外蒸发质量通量	J/kg、kg/(m ² · s)
ρ_{d0}, q_{in}	初始干物质密度、进入固体的净热通量	kg/m ³ 、W/m ²
N, w_i	径向区间数、归一化环形控制体权重	1、1
i, j, s	空间节点、观测时刻、情景编号；下标 s 用于表面时另行说明	1
R_0, α	初始半径、热扩散率 $k/(\rho c_p)$	m、m ² /s
$\eta_{i\pm 1/2}, \Delta\eta_i$	控制体界面、相邻节点距离	1
$J_{i+1/2}$	梯度型含水率通量，外流为其反号	m ² /s
$A(T), a, F_a$	扩散前因子、指数含水率参数、状态原函数	m ² /s、kg/kg、kg/kg
$C_*, \bar{C}, g$	达标阈值、干物质加权均值、事件函数	kg/kg
E_T, E_C, E_t	共同采样集温差、含水率差、事件时间差	°C、kg/kg、s
$\mathcal{R}_C$	初末水量与独立边界积分的残差	kg/kg
$\gamma_D, \delta_T, \delta_C$	扩散倍率、后期环境温度/水分偏移	1、°C、kg/kg
$\mathcal{S}, \tau, \mathbf{y}, \mathbf{f}$	有限情景集、候选停机时间、状态向量、半离散右端	—、s、分量单位各异
P, G, Δ_{PG}	物性组、几何条件、时间的非加性交互	—、—、h
t_*, t_{out}, t_S	临界、名义严格、有限情景停机时刻	s

表 S1 中 C 为干基含水率（以干物质质量为分母），**不是单位体积水浓度**。尤其在问题四，直接积分 C 与当前体积的乘积并不等于总水质量；这一差别决定收缩方程和守恒账本的形式。仅局部使用的求积变量 m, b, c 、相邻状态差 ΔC 、状态积分等效系数 D_{int} 、中间时间 u 及扰动参数 p 在出现处定义；上下标 c, s, a 在场变量中依次表示中心、表面、环境，情景编号 $s \in \mathcal{S}$ 只在带括号上标中使用。系数的经验式虽以 SI 数值代入，指数 a/C 和 $3850/T_K$ 必须无量纲；3850 对应温度量纲 K。热场通量与水分通量不共用物理单位。

4.2 输入预处理与外推（观测区间外的闭合）

直接读取两份官方工作簿的数值区，检查非有限值、时间严格递增、温度和半径单位；不删除观测、不以拟合曲线代替原始环境序列。环境在相邻观测之间分段线性插值。若第 $j, j+1$ 个观测时刻为 t_j, t_{j+1} ，对于 $u = T_a$ 或 C_a 有

$$u(t) = u_j + \frac{t - t_j}{t_{j+1} - t_j}(u_{j+1} - u_j), \quad t_j \leq t \leq t_{j+1}. \quad (1)$$

当 $t > 14400$ s，设 $T_a = 49.99893443$ °C、 $C_a = 0.04998754098$ kg/kg。半径转换为 m 后采用保形分段三次 Hermite 插值（PCHIP），保持给定单调数据的形状且禁止在观测时间外外推^[3,4]。局部单调三次插值依据 Fritsch 与 Butland 的方法。禁止域外外推是本文另设的数据使用限制，不是插值算法自带的物理保证。图 3 显示环境升温与收缩发生在不

同的时间范围，不应仅用末值替代全部过程。

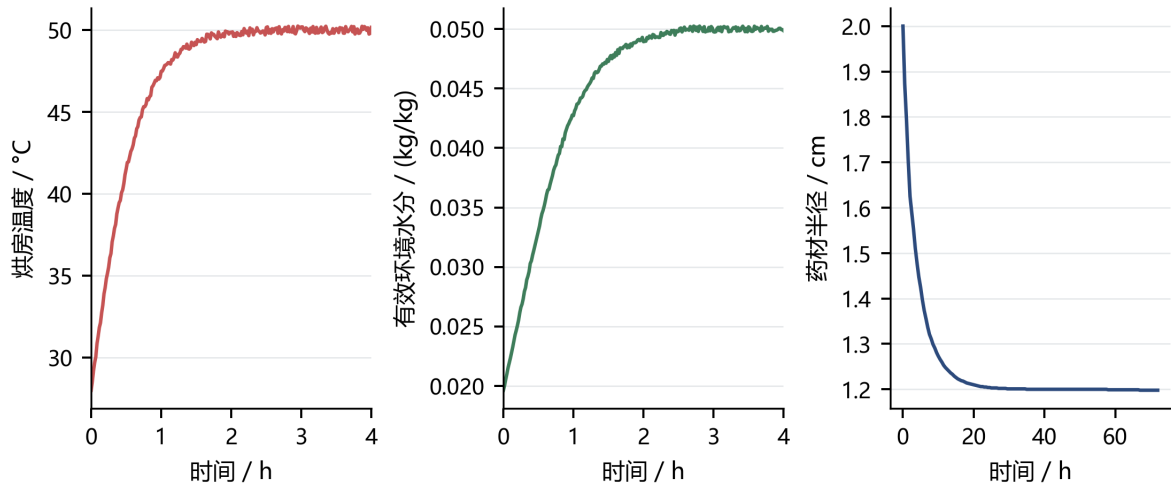


图 3: 烘房环境观测与药材半径序列

长期环境取值用于问题三、四；半径按实际停机时刻读取。

五、 问题一的模型建立与求解

5.1 模型选择与必要推导（环形控制体守恒）

控制体与储存量。先建立无潜热基线：药材不收缩，忽略轴向传递及内部相变，仅考虑径向导热与水分扩散。在 $0 < r < r + \delta r < R$ 取长度 L 的圆环，径向交换面积为 $A(r) = 2\pi rL$ ，体积为 $\Delta V = \pi L[(r + \delta r)^2 - r^2]$ 。本问 ρ, c_p, k 固定，但 D 仍随 C 变化；定热物性并不意味着整个系统线性。

含水率 C 以单位干物质质量为基准，不能直接作为体积水分浓度。设固定且空间均匀的干物质参考密度为 ρ_d ，则体积水分浓度为 $\rho_d C$ 。以任意固定温度 T_{ref} 为显热零点，圆环显热与水质质量分别为

$$E = \int_r^{r+\delta r} \rho c_p (T - T_{\text{ref}}) A(s) ds, \quad M_w = \int_r^{r+\delta r} \rho_d C A(s) ds.$$

这里 ρ 是题给的有效热容量密度参数， ρ_d 是干物质质量的换算测度，二者不混用； s 为径向积分变量。

通量定律与环形守恒。以径向向外为正，傅里叶定律给出 $q_r = -k \partial_r T$ (W/m^2)。在上述固定干物质测度下，采用 Fick 型有效水分通量 $j_r = -\rho_d D \partial_r C$ ($\text{kg}/(\text{m}^2 \cdot \text{s})$)。因此，内环面流入减外环面流出等于控制体储量增加：

$$\begin{cases} \frac{dE}{dt} = A(r)q_r(r, t) - A(r + \delta r)q_r(r + \delta r, t), \\ \frac{dM_w}{dt} = A(r)j_r(r, t) - A(r + \delta r)j_r(r + \delta r, t). \end{cases}$$

对任一径向通量 f ，将上述净流入除以 ΔV 并令 $\delta r \rightarrow 0$ ，得到

$$\lim_{\delta r \rightarrow 0} \frac{A(r)f(r) - A(r + \delta r)f(r + \delta r)}{\Delta V} = -\frac{1}{2\pi rL} \frac{\partial(2\pi rL f)}{\partial r} = -\frac{1}{r} \frac{\partial(r f)}{\partial r}.$$

$1/r$ 与通量中的 r 分别来自环形体积和交换面积, 必须成对出现。这是几何推导的结果, 不是对平板扩散方程的经验修正。代入两种通量, 并在水分方程中约去常数 ρ_d , 得到本题的核心方程组:

$$\begin{cases} \rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \\ \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right), \end{cases} \quad 0 < r < R. \quad (2)$$

第一式的两侧分别为单位体积显热增加率和导热净输入率, 单位均为 W/m^3 ; 第二式的两侧均为单位干物质含水率变化率, 单位为 $(\text{kg/kg})/\text{s}$ 。水分方程是给定通量闭合下的守恒结果, 而不是仅凭守恒就能确定扩散系数。 D 仍取题给经验式。控制体积分方法参见文献^[5, 6], 我们自行组装径向算子, 未调用 FiPy 软件。

初边值与符号约定。初态为 $T(r, 0) = 28^\circ\text{C}$ 、 $C(r, 0) = 2.55 \text{ kg/kg}$ 。轴心由对称性无径向梯度; 侧表面采用题给交换系数, $q_r(R) = h(T_s - T_a)$ 、 $j_r(R) = \rho_d h_m (C_s - C_a)$ 。代入通量并约去 ρ_d , 有

$$\begin{cases} T_r(0, t) = C_r(0, t) = 0, \\ -k T_r(R, t) = h(T_s - T_a), \\ -D C_r(R, t) = h_m (C_s - C_a). \end{cases} \quad (3)$$

升温时 $T_s < T_a$, $q_r < 0$ 表示热量向内; 失水时 $C_s > C_a$, $j_r > 0$ 表示水分向外。轴心 $r = 0$ 按对称极限处理, 离散时使用内侧面积为零的控制体, 不直接除以零。 C_a 在此作为题给有效交换关系的环境驱动量, 不额外解释为已校准的气相浓度。

向后续各问的衔接。第二、三问全程使用附录 3 变物性, 并将 $\rho(C)c_p(C)T_t$ 作为有效显热容量闭合; 不能把本问常系数储量公式直接改写为 $\partial_t[\rho(C)c_p(C)T]$ 。第四问另从干物质连续性推导收缩方程, 见式(17)–(19)。表面潜热扩展则修正式(3)的热边界, 不把内部水分迁移直接当作体积蒸发源, 具体见式(8)–(9)。

代入 $\rho = 820 \text{ kg/m}^3$ 、 $c_p = 2600 \text{ J/(kg} \cdot \text{K)}$ 、 $k = 0.36 \text{ W/(m} \cdot \text{K)}$ 、 $D = 7 \times 10^{-9} \exp(-0.89/C) \text{ m}^2/\text{s}$, 热扩散率为 $1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ 。取初始含水率计算 $D \approx 4.94 \times 10^{-9} \text{ m}^2/\text{s}$, 则 1800 s 的热、水分 Fourier 数约为 0.760、0.0222; 以半径定义的 Biot 数约为 1.389、3.24。集中参数法需要内部阻力较小, 本题不满足其典型小 Biot 数条件; 将表面直接指定为环境值也会遗漏可比的表面对流阻力。故选择式(2)–(3), 不用零维均匀模型或定表面值替代。这是适用性判别, 不是声称已经跑过所有被排除的模型。

5.2 四步计算与通量稳定化

图 4 概括本问计算责任。原始时间步与题目输出间隔分开: 积分器自适应推进, 输出在指定整数秒采样。

5.2.1 输入整理与初始化

取无量纲径向坐标 $\eta = r/R$, 用附录 2 参数设置所有节点初值。温度以 $^\circ\text{C}$ 保存, 当前扩散公式不含温度; 后问 Arrhenius 因子才使用 K。确认 $h = 25$ 、 $h_m = 8 \times 10^{-7}$ 及其单位。

5.2.2 控制体离散与基线 (界面共享通量)

记节点 η_i 及中点界面 $\eta_{i+1/2}$, 端面界面为 0 和 1, 定义

$$w_i = \frac{\eta_{i+1/2}^2 - \eta_{i-1/2}^2}{2}, \quad \dot{C}_i = \frac{J_{i+1/2} - J_{i-1/2}}{R^2 w_i}. \quad (4)$$

这里 J 定义为沿正径向的梯度型通量, 即与通常外流通量相反。中心控制体包含 $r = 0$, 内侧面积为零, 故中心 $J_{-1/2} = 0$, 无需直接计算奇异项 $1/r$; 最外侧 $J_{N+1/2} = R h_m (C_a - C_N)$ 。

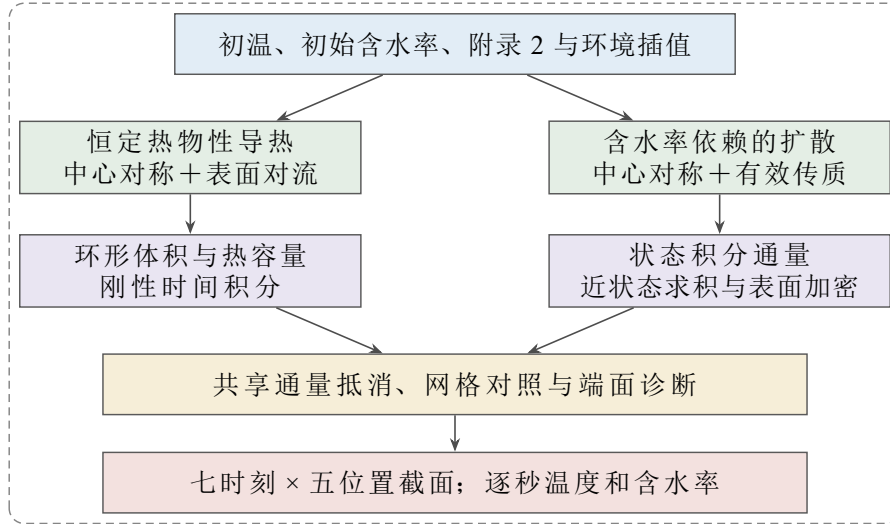


图 4: 问题一：从输入与控制体到逐秒状态和收支复核

同一个内部 J 在左右控制体中符号相反，从而离散内通量严格抵消；有限体积的这一结构可直接体现积分守恒^[5]。温度同理，容量为 $\rho_i c_{p,i}$ ，外侧热通量为 $Rh(T_a - T_N)$ 。

轴心离散与光滑径向场的极限一致，常系数下为式(25)；权重推导及控制体示意图见附录 A.1。边界账本与局部制造解联合检查，避免几何因子错误被整体收支掩盖。

基线 M0 取均匀网格和节点扩散率算术平均。M1 仍用均匀网格，但对 $D = A(T)e^{-a/C}$ 的状态依赖部分积分，其中 a 是题给含水率指数参数， $\bar{A}_{i+1/2} = [A(T_i) + A(T_{i+1})]/2$ ；本问 $A = 7 \times 10^{-9} \text{ m}^2/\text{s}$ 、 $a = 0.89$ 。令

$$F_a(C) = Ce^{-a/C} + a \text{Ei}(-a/C), \quad F'_a(C) = e^{-a/C},$$

$$J_{i+1/2} = \eta_{i+1/2} \bar{A}_{i+1/2} \frac{F_a(C_{i+1}) - F_a(C_i)}{\eta_{i+1} - \eta_i}. \quad (5)$$

指数积分的定义与实现采用标准特殊函数^[7]。式(5)来自一维局部状态积分：在冻结温度的平面连接段上积分 $D(C)dC$ ，再除以间距，避免只用两个端点表征剧变的系数。圆柱界面几何与非等温因子采用局部近似，故它不是任意非等温曲面单元的精确解析通量。

该修正保留题给扩散定律，仅改变界面离散。等效系数位于两端扩散率之间、保持正值，交换左右状态时通量反号；定义、极限及适用条件见附录 A.2。但系数为正不保证任意时间离散都保持正性，仍须数值检查。

5.2.3 稳定积分与加密求解（近状态防相消）

两相邻状态很近时，直接相减 F_a 会损失有效数字（即相近浮点数相减产生的相消误差）。取 $m = (C_L + C_R)/2$ 、 $b = (C_R - C_L)/2$ ，当 $|C_R - C_L| < 0.01m$ 时用

$$F_a(C_R) - F_a(C_L) \approx b \left[\frac{5}{9} f(m - \sqrt{3/5}b) + \frac{8}{9} f(m) + \frac{5}{9} f(m + \sqrt{3/5}b) \right],$$

$$f(C) = e^{-a/C}. \quad (6)$$

即使 $C_R < C_L$ ，带符号的 b 仍保持通量方向。M2 进一步采用

$$s_i = i/N, \quad \eta_i = 1 - (1 - s_i)^{3/2}, \quad i = 0, \dots, N, \quad (7)$$

在表面分配更密节点。指数 $3/2$ 控制加密程度，其效果由相同区间数的对照检验。本问最终取 $N = 1280$ ，用 BDF 刚性积分器推进半离散系统，相对容差为 2×10^{-9} ，温度与含水率绝对容差分别为 10^{-8} 、 10^{-10} 。稀疏 Jacobian 保留相邻节点及两场的局部联系。BDF 是变阶多步法，其时间容差仅控制积分误差的一部分^[8]。

5.2.4 复核与导出

以五阶 Radau IIA 隐式 Runge-Kutta 方法^[9] 从初态复算四问的两种热边界，仅替换时间积分器，物性、边界、空间算子、网格与容差保持一致。第一至三问为 1280 区间，第四问无潜热 2560、含潜热 1280 区间。答案表并列两算法，主答案保留 BDF；第三、四问各自求根、延时 5 s、向上舍入并回代全节点。

双算法核对的范围是指定截面和终点，检验时间积分与事件定位。共用空间算子和物理假设的偏差仍可能同时存在，因此还需网格、守恒和机制对照。第二问前 3 h 的吻合也仅适用于该窗口，全过程逐秒工作簿须另行复核。

以相同条件对照 M0、M1、M2，并以最终细网格为参考。比较指标为共同采样点集上的最大绝对温差 E_T 、含水率差 E_C ；后两问另计算事件时间差 E_t 。本问取 504 个共同时间、201 个物质位置；这是采样误差指标，不是连续全域严格上界。各方案都为 160 区间、相同物性和容差，耗时为同一环境单次运行值。表 S2 给出全部结果。M0 与

表 S2: 问题 1 同为 160 区间的数值方法比较

方法	E_T/K	$E_C/(\text{kg/kg})$	E_t/s	耗时/s
M0	3.114e-05	3.611e-03	不适用	2.16
M1	3.038e-05	3.612e-03	不适用	2.70
M2	6.161e-05	2.266e-04	不适用	2.68

M1 的短时含水率误差接近。M2 将 E_C 由约 0.003612 降至 0.0002266 kg/kg，表层加密对水分场更有效，M2 的温度误差却略增。160 区间只用于方法对照，最终答案仍由更密网格计算。

5.3 结果与端面诊断（中截面适用范围）

图 5 展示全部存储时刻的温湿场演化，指定位置的数值列于表 1、2。

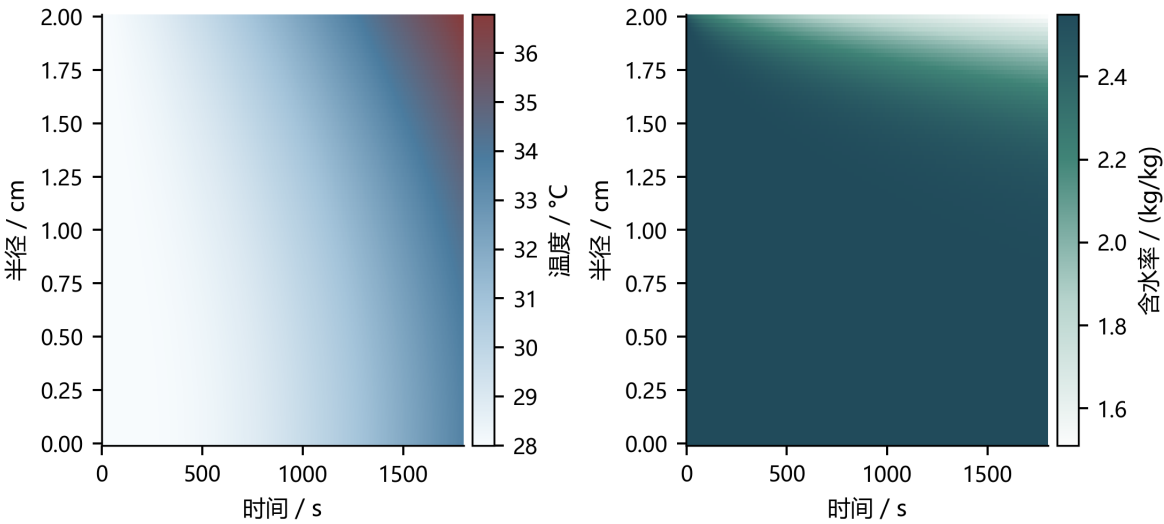


图 5: 问题一的温度与干基含水率时空演化

热量已经进入较深区域，失水主要集中在表层。中心含水率四舍五入为 2.5500 不意味着内部在数学上完全不变化；未经舍入值为 2.5499924 kg/kg。

在 1800 s，中心到表面的温差约 3.2102℃，烘房仍在升温，故“预热平衡阶段结束”不应被解释为温度处处相等。表面含水率约 1.5102 kg/kg 而非 0.15 kg/kg，此时尚未达到成品阈值。

无潜热端面导热对照在 1800 s 给出中截面最大改变量 6.04×10^{-8} ℃，端部局部最

大改变量却达 3.6970 °C。这一差别支持中截面的降维近似，但端部仍需保留轴向传热。本次检查限于导热，水分、潜热及后几问的变物性尚未覆盖；方程、网格与独立复核见附录 A.3。

5.4 物理基线比较与参数灵敏度

在同环境、同端部绝热条件下，集中参数模型式(28)仅输出均温，1800 s 为 36.3427 °C；若误作中心温度，比同网格径向中心值高约 2.7673 °C，不能回答五位置要求。曲线见图 6，推导与比较口径见附录 A.4。

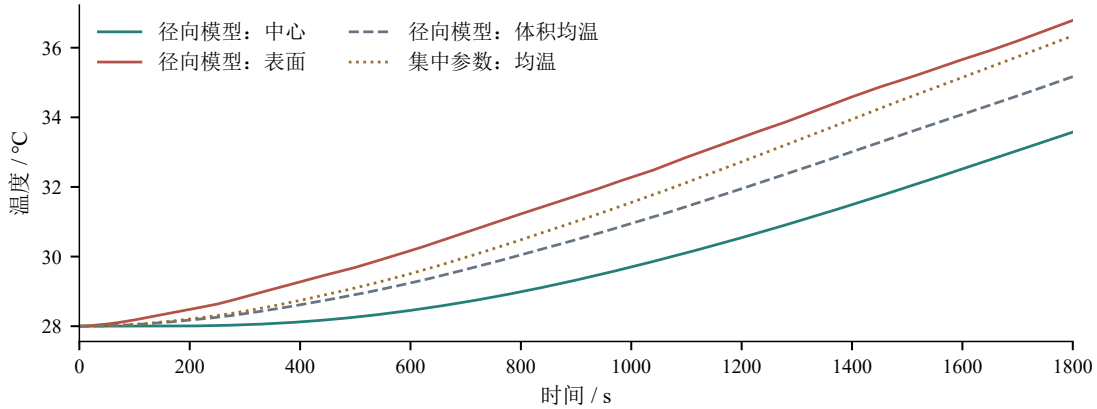


图 6: 无潜热同一环境下集中参数模型与 160 区间径向温度场的对照

该图检验均温能否代替题目要求的五位置温度。中心、表面与体积均温并不相同。我们认为，五位置输出需要保留径向分辨率。这里比较空间模型，BDF 与 Radau 的时间积分核验另见表 S4。

无潜热基线采用 160 区间，分别扰动 D, h, h_m 的 $\pm 10\%$ 。换热倍率 0.9 至 1.1 使中心温度从 33.2975 升至 33.8234 °C；传质倍率同样变化使平均含水率从 2.3117 降至 2.2767 kg/kg。短时中心含水率变化小，不能替代表层与平均失水响应。差商定义、全表和图见式(29)、表 S25。这些确定性扰动不是统计置信度；第一问 D 不含温度，故潜热不反馈水分轨道。

5.5 当考虑到蒸发潜热对本题的影响

先定义发生相变的位置和质量测度。本扩展假定内部仅迁移水分，逸出侧表面的水在该处蒸发，端面仍绝热且无传质。设干物质密度为 ρ_d ，实际水质量外向通量为 $-\rho_d DC_r$ 。将原有效水分边界乘以同一质量测度，得到

$$j_v = -\rho_d DC_r|_R = \rho_d h_m (C_s - C_a), \quad \rho_{d0} = \frac{\rho(C_0)}{1 + C_0}. \quad (8)$$

第一问取 $\rho_d = \rho_{d0} = 820/3.55 = 230.9859 \text{ kg/m}^3$ 。这里把初始经验密度解释为初始湿基总体密度以确定质量尺度，是新增假设；不能用所有时刻的 $\rho(C)/(1 + C)$ 替换而不检查干物质守恒。干基通量 $h_m(C_s - C_a)$ 的单位为 m/s，乘以 ρ_d 后才是 $\text{kg}/(\text{m}^2 \cdot \text{s})$ 。

修正的是热边界的附加项，而非重新拟合物性系数。令 $\chi = 0$ 表示无潜热基线， $\chi = 1$ 表示计入侧表面蒸发。单位面积对流供热等于进入固体的净导热与蒸发耗热之和，两套热边界统一写为

$$\begin{cases} q_{in} = h(T_a - T_s) - \chi L_v j_v, \\ -kT_r|_R = h(T_s - T_a) + \chi L_v j_v, \end{cases} \quad \chi \in \{0, 1\}. \quad (9)$$

取常数近似 $L_v = 2.45 \times 10^6 \text{ J/kg}$ ，来源为 FAO 物理参数附录所列 20 °C 附近的近似值^[2]，不是以参考温度反演的拟合参数，也不是全温域的精确常数；后文另作潜热取值情景。

此文献仅支持水的潜热量级，不验证本题的有效气固交换关系。当 $L_v j_v$ 暂时大于对流供热时，净热流可向外，故材料温度允许低于 28 °C；是否对应真实湿球冷却仍需气相状态和表面平衡数据。

内部微分方程、水分 Robin 边界及题给经验参数均保留，不重复添加体积相变热源。第二至四问的物性会随各自温湿状态更新；“参数不重拟合”不意味着两条轨道的瞬时物性数值相同。

最后组装并核对数值项。当 $\chi = 1$ 时，沿用内点离散，将表面节点的温度右端减去

$$\frac{L_v j_v}{R w_N \rho_N c_{p,N}}. \quad (10)$$

该项来自边界梯度通量 $R q_{in}$ 除以控制体容量 $R^2 w_N \rho_N c_{p,N}$ 。取 $L_v = 0$ 严格退化为基线； $C_s = C_a$ 时潜热通量为零。第一问 D 与温度无关，因此水分方程与初边值均不变，含水率解应与基线一致，而不是因“考虑了耦合”就强行改变含水率。

表 S3 在相同 1800 s 时刻比较两组温湿结果，冷却幅度直接由两次计算之差得到，不按经验倍率修正温度。

表 S3: 问题 1：不计与计入表面蒸发潜热的结果对照（1800 s）

指标	不计潜热	计入潜热	差值
中心温度/(°C)	33.5753	10.5466	-23.0287
表面温度/(°C)	36.7856	11.9026	-24.8829
中心含水率/(kg/kg)	2.5500	2.5500	0.0000
表面含水率/(kg/kg)	1.5102	1.5102	0.0000

差值为含潜热减无潜热，先用未舍入值相减再保留四位小数。两组均为 1280 区间、BDF 积分。含水率差显示为零不代表舍入前逐位相等；理论上水分方程不受本问温度影响。

常热容量下还可独立检查单位干物质的能量账本：

$$\frac{2\rho c_p}{\rho_{d0}} \sum_i w_i [T_i(t) - 28] = \int_0^t \frac{2h(T_a - T_s)}{\rho_{d0} R} du - \int_0^t \frac{2L_v j_v}{\rho_{d0} R} du. \quad (11)$$

左右量纲均为 J/kg 干物质。温度降低时左端可为负，并不违反能量守恒。640/1280 区间的终点中心、表面温度差为 3.26e-05、3.20e-05 °C。这属于数值复核，不是物理模型的实测误差。独立时间求积得到对流供热 472.443 kJ/kg 干物质、蒸发耗热 628.285 kJ/kg 干物质，显热变化 -155.842 kJ/kg 干物质，能量相对残差为 2.68e-10。因此本条件下的冷却有能量账本支持，但尚不能据此判定真实药材表面一定达到该温度。完整截面列于本问答案表 S5。图 7 概括本节引入、后续各问沿用的潜热边界：对流供热先扣除侧表面蒸发耗热，其余为进入固体的净热流；内部水分迁移不另产生体积相变热源。

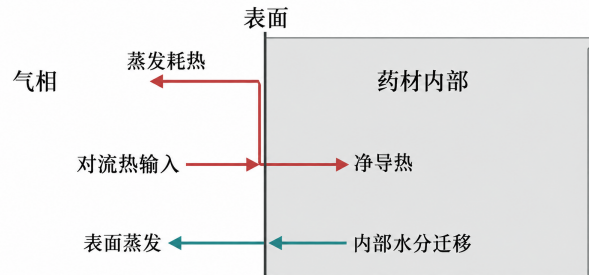


图 7: 以表面蒸发为假设的能量分配与水分迁移示意

红色为热流，青色为水分迁移；净导热以向内为正，可随收支反向。侧表面局部示意不编码数值。

5.6 双算法核验：无潜热与含潜热

表 S4 分别核对两种热边界的指定截面；同网格双算法吻合只支持时间积分一致性，不是物理验证。问题二至四的对应核验见表 S10–S21。

表 S4: 问题 1 的 BDF 与 Radau 同网格核对

模型	最大温差/℃	最大含水率差/(kg/kg)	临界时差/s
无潜热	2.91×10^{-6}	1.84×10^{-8}	—
含潜热	1.17×10^{-6}	1.07×10^{-9}	—

均报绝对差；场差限于指定时刻、径向位置及实际表面，并非连续全场上界。

5.7 本问直接答案

5.7.1 不含潜热的答案

表 1: 无潜热温度 (℃)

时间	算法	0	0.5	1	1.5	2
100	BDF	28.0001	28.0003	28.0040	28.0327	28.1801
	Radau	28.0001	28.0003	28.0040	28.0327	28.1801
300	BDF	28.0408	28.0635	28.1514	28.3681	28.8489
	Radau	28.0408	28.0635	28.1514	28.3681	28.8489
600	BDF	28.4533	28.5360	28.8039	29.3159	30.1652
	Radau	28.4533	28.5360	28.8039	29.3159	30.1652
900	BDF	29.3243	29.4583	29.8755	30.6161	31.7304
	Radau	29.3243	29.4583	29.8755	30.6161	31.7304
1200	BDF	30.5427	30.7098	31.2223	32.1126	33.4276
	Radau	30.5427	30.7098	31.2223	32.1126	33.4276
1500	BDF	31.9957	32.1867	32.7660	33.7463	35.1203
	Radau	31.9957	32.1867	32.7660	33.7463	35.1203
1800	BDF	33.5753	33.7720	34.3642	35.3621	36.7856
	Radau	33.5753	33.7720	34.3642	35.3621	36.7856

表 2: 无潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
100	BDF	2.5500	2.5500	2.5500	2.5500	2.2470
	Radau	2.5500	2.5500	2.5500	2.5500	2.2470
300	BDF	2.5500	2.5500	2.5500	2.5492	2.0508
	Radau	2.5500	2.5500	2.5500	2.5492	2.0508
600	BDF	2.5500	2.5500	2.5500	2.5353	1.8770
	Radau	2.5500	2.5500	2.5500	2.5353	1.8770
900	BDF	2.5500	2.5500	2.5497	2.5045	1.7547
	Radau	2.5500	2.5500	2.5497	2.5045	1.7547
1200	BDF	2.5500	2.5500	2.5482	2.4646	1.6586
	Radau	2.5500	2.5500	2.5482	2.4646	1.6586
1500	BDF	2.5500	2.5499	2.5445	2.4206	1.5787
	Radau	2.5500	2.5499	2.5445	2.4206	1.5787
1800	BDF	2.5500	2.5497	2.5383	2.3755	1.5102
	Radau	2.5500	2.5497	2.5383	2.3755	1.5102

左：温度；右：干基含水率。各时刻上行 BDF、下行 Radau；时间单位 s，固定径向位置单位 cm。

由此本题的直接答案是：不含潜热时，采用式(2)–(3)及附录 2 物性，七时刻五位置结果见上述两表；**1800 s 中心/表面温度为 33.5753/36.7856 ℃，含水率为 2.5500/1.5102 kg/kg**。result1.xlsx 保存 1 至 1800 s 每 1 s、0 至 2 cm 每 0.1 cm 的两场数值，四位小数显示；主答案限定为有效径向模型，中截面的端面导热诊断另如上。

5.7.2 含潜热的答案

表 S5: 含潜热温度 (℃)

时间	算法	0	0.5	1	1.5	2
100	BDF	27.9774	27.8938	27.2078	24.1967	16.7081
	Radau	27.9774	27.8938	27.2078	24.1967	16.7081
300	BDF	25.5263	24.7418	22.2550	17.9098	12.0707
	Radau	25.5263	24.7418	22.2550	17.9098	12.0707
600	BDF	18.9996	18.2853	16.2463	13.2052	9.7094
	Radau	18.9996	18.2853	16.2463	13.2052	9.7094
900	BDF	14.3008	13.8689	12.6672	10.9623	9.1520
	Radau	14.3008	13.8689	12.6672	10.9623	9.1520
1200	BDF	11.6757	11.4621	10.8900	10.1563	9.5583
	Radau	11.6757	11.4621	10.8900	10.1563	9.5583
1500	BDF	10.5817	10.5258	10.4057	10.3522	10.5371
	Radau	10.5817	10.5258	10.4057	10.3522	10.5371
1800	BDF	10.5466	10.5924	10.7614	11.1487	11.9026
	Radau	10.5466	10.5924	10.7614	11.1487	11.9026

表 S6: 含潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
100	BDF	2.5500	2.5500	2.5500	2.5500	2.2470
	Radau	2.5500	2.5500	2.5500	2.5500	2.2470
300	BDF	2.5500	2.5500	2.5500	2.5492	2.0508
	Radau	2.5500	2.5500	2.5500	2.5492	2.0508
600	BDF	2.5500	2.5500	2.5500	2.5353	1.8770
	Radau	2.5500	2.5500	2.5500	2.5353	1.8770
900	BDF	2.5500	2.5500	2.5497	2.5045	1.7547
	Radau	2.5500	2.5500	2.5497	2.5045	1.7547
1200	BDF	2.5500	2.5500	2.5482	2.4646	1.6586
	Radau	2.5500	2.5500	2.5482	2.4646	1.6586
1500	BDF	2.5500	2.5499	2.5445	2.4206	1.5787
	Radau	2.5500	2.5499	2.5445	2.4206	1.5787
1800	BDF	2.5500	2.5497	2.5383	2.3755	1.5102
	Radau	2.5500	2.5497	2.5383	2.3755	1.5102

左：温度；右：干基含水率。各时刻上行 BDF、下行 Radau；时间单位 s，固定径向位置单位 cm。

当考虑上述表面蒸发潜热时，1800 s 中心/表面温度为 10.5466/11.9026 °C，含水率为 2.5500/1.5102 kg/kg。两种答案对应不同热边界，不能混用数据文件。为检验中截面近似，可增设端面测温点，并同步记录表面温度与排湿量，核对蒸发耗热。

六、 问题二的模型建立与求解

6.1 物性耦合与模型选择（全程统一公式）

本问不是将第一问运行到 1800 s 后再换参数，而是按题意从 $t = 0$ 统一使用附录 3。无潜热基线沿用几何、初值和式(3)的 Robin 边界（保留有限的表面交换阻力）；含潜热扩展只将热边界改为式(9)。两者均采用

$$\begin{aligned} \rho(C) &= 650 + 128C, & c_p(C) &= 1450 + 2736 \frac{C}{1+C}, \\ k(C) &= 0.21 + 0.38 \frac{C}{1+C}, & D(C, T) &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/(T+273.15)}. \end{aligned} \quad (12)$$

单位按符号表采用 SI 制。升温增大扩散系数，失水又通过热容量和导热率反馈温度。这是有效物性的双向耦合，不是潜热耦合。

本问保留题给物性的状态依赖和 Robin 边界，联立求解两场。冻结初态物性用于评估反馈作用；M0/M1/M2 另在相同条件下比较离散方法。

6.2 四步求解与本问对照

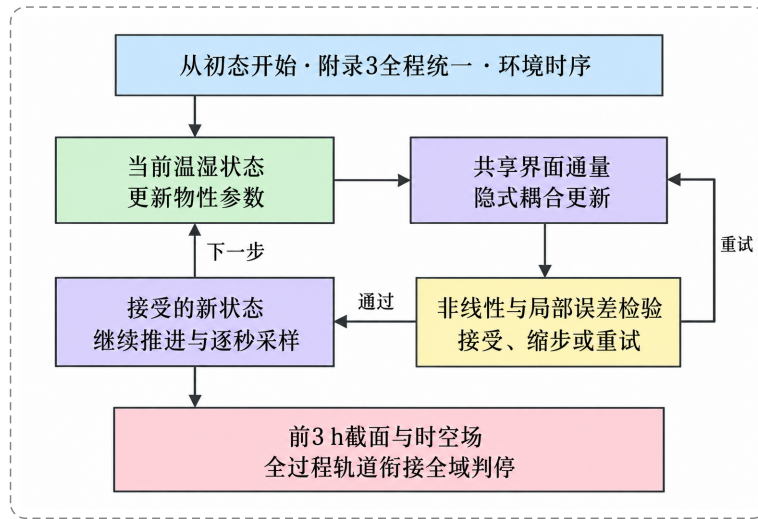


图 8: 问题二：变物性耦合与全过程输出

图 8 从原始初态求解，分别导出前 3 h 截面与全过程数据。

6.2.1 输入与状态初始化

将 $T = 28$ 、 $C = 2.55$ 重新赋给全部节点，代入式(12)。环境按式(1)插值，4 h 处单独分段推进，以免时间插值跨越外推规则变化。

6.2.2 联立可计算形式

在式(4)中按当前 T, C 计算各节点物性， $A(T) = 2.4 \times 10^{-3} e^{-3850/(T+273.15)}$ 在相邻节点算术平均， C 依赖用式(5)–(6)。热界面的 k 取相邻均值。状态向量 $\mathbf{y} = (T_0, \dots, T_N, C_0, \dots, C_N)^T$ 满足 $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$ 。非线性残差由 BDF 求解过程处理，一次显式物性更新不足以判断收敛。

生产计算采用 SciPy 变阶 BDF 及其 NDF 精度增强^[8]，利用相邻节点耦合的稀疏

Jacobian 求解非线性残差。误差按温湿分量归一化，容差沿用第一问；隐式迭代及误差范数见附录 A.6、A.5。局部误差与迭代收敛共同决定接受或缩步，隐式稳定性不替代精度检查。

6.2.3 细网格主解

仍取 $N = 1280$ 及前述容差，将最大允许步长设为前 4 h 内 30 s、之后 300 s，实际步长由局部误差控制。主表从 0.5 至 3 h 逐 0.5 h 采样；同一动力系统继续推进至问题三的严格达标终点。

6.2.4 状态与导出检查

按 505 个共同时间、201 个物质位置比较前 3 h 的三种方法，见表 S7。另外对无潜热全过程采用 640/1280 两网格回代比较，核验集含 0 s 初态、逐秒时刻及严格终点，共 206913 个时间样本；空间为 0 至 2 cm 的 21 个固定位置，其中 2 cm 即表面，不另加重复表面列。当前工作簿不输出 0 s 初态，因此每张表为 206912 条数据行。核验集最大温差 $6.25 \times 10^{-6} \text{ }^\circ\text{C}$ ，最大含水率差 $1.17 \times 10^{-5} \text{ kg/kg}$ ，后者出现在初期表面而非最后时刻。只核对 3 h 中心会漏掉这个高风险位置。M2 的采样含水率误差约为 M0 的 13.8%，本

表 S7: 问题 2 同为 160 区间的数值方法比较

方法	E_T/K	$E_C/(\text{kg/kg})$	E_t/s	耗时/s
M0	5.159e-05	9.774e-04	不适用	16.77
M1	5.174e-05	9.777e-04	不适用	21.39
M2	6.492e-05	1.345e-04	不适用	23.63

次粗网格运行时间则由 16.77 s 增至 23.63 s。M0/M1 的温度误差略小。采用 M2 着眼于含水率场的分辨率，也需接受较高计算成本。

6.3 结果及下游接口

图 9 中温度逐渐趋向环境，而含水率变化继续向内部传播；表 3、4 给出题目指定数值。

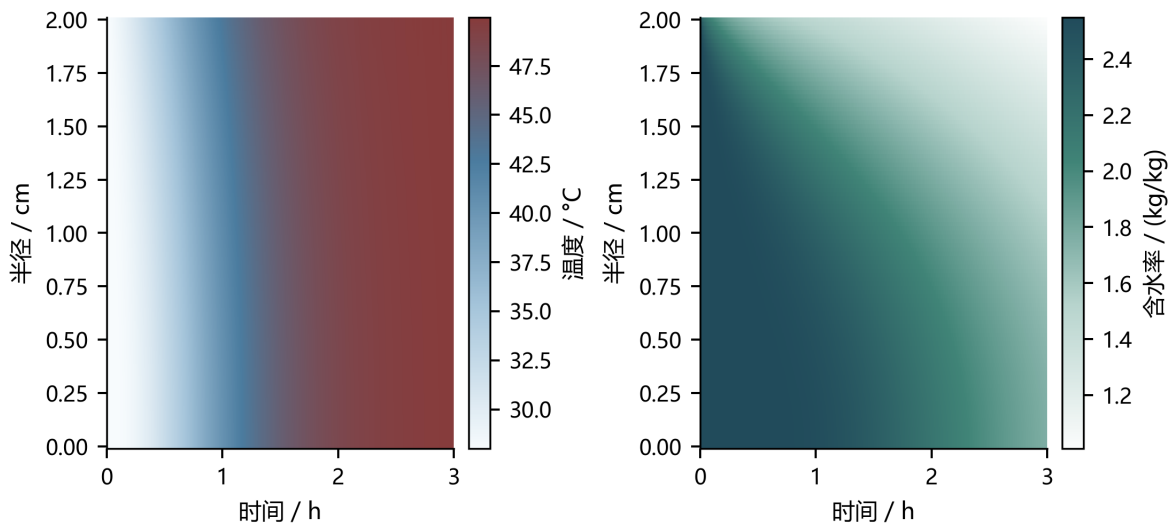


图 9: 问题二前 3 h 的变物性温湿场

3 h 时温差仅约 $0.1169 \text{ }^\circ\text{C}$ ，但中心含水率仍 1.7662 kg/kg ，远高于 0.15 kg/kg 。温度接近平衡不能作为干燥完成的替代判断。

图 10 与二维场使用同一锁定数据：后期温度曲面的径向倾斜减弱，而含水率仍保

持内部高、表面低的结构。第三轴分别为实际温度和含水率，不用透视高度比较不同物理量。

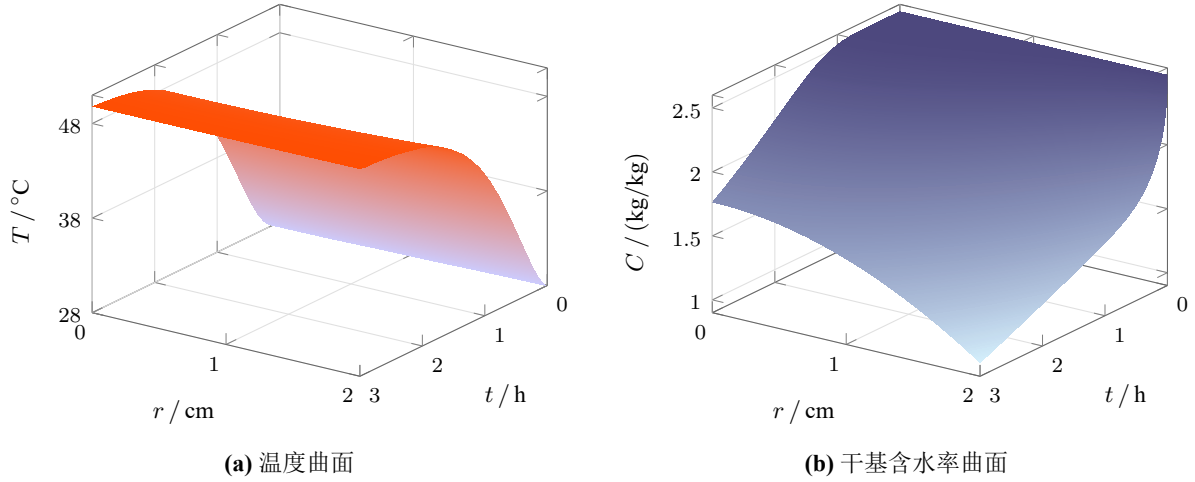


图 10: 问题二时间-半径-状态曲面

同一数值场取 44 个时刻、26 个径向节点展示，含初末状态与中心、表面；面片只连接展示节点。二维投影见图 9，精确读数以答案表为准。

本问与第一问在相同时刻的差异来自物性模型，不是连续工艺中的突变。向第三问传递的则是同一动力系统，只增加判停与输出规则；全过程工作簿延伸到 57.4754 h，不以 3 h 展示窗口或经验“2-3 天”预设终点。

6.4 冻结物性基线与耦合响应检验

将式(12)的全部物性固定在初态 $T = 28\text{ }^{\circ}\text{C}$ 、 $C = 2.55\text{ kg/kg}$ 处，构成冻结对照；保持环境、初边值、160 区间均匀网格及 BDF 容差不变，唯一删除的机制是物性随状态反馈。冻结模型 3 h 中心含水率更高，表面反而更低：内部输运较慢，水分留在内部，表

表 S8: 问题二冻结与变物性的同网格终点对照

模型	$T_c/^{\circ}\text{C}$	$T_s/^{\circ}\text{C}$	C_c	C_s
初态冻结物性	49.7351	49.8769	2.1642	0.8655
统一变物性	49.8495	49.9664	1.7662	1.0081

两者均为 160 区间均匀网格、相同初边值；终点 3 h。含水率单位 kg/kg。

层供水随之减少。我们认为，判断干燥快慢须同时检查中心、表面和全场。

无潜热三系数 $\pm 10\%$ 扰动沿用式(29)，以 3 h 为终点。扩散倍率 0.9 至 1.1 使中心含水率从 1.8255 降至 1.7151 kg/kg；换热扰动通过温度反馈水分，但本时刻影响较弱。该排序只适用于所选时刻和幅度，不能外推到后期总时长；完整表图见附录 A.7。

6.5 当考虑到蒸发潜热对本题的影响

本问从 $T_0 = 28\text{ }^{\circ}\text{C}$ 、 $C_0 = 2.55\text{ kg/kg}$ 重新开始，全程仍用附录 3，不在预热结束时切换物性。式(8)给出 $\rho_{d0} = (650 + 128 \times 2.55)/3.55 = 275.0423\text{ kg/m}^3$ ；固定几何下保持不变，热容量则继续使用当前 $\rho(C)c_p(C)$ 。沿用式(9)和(10)，热场、水分场在同一刚性系统内联立推进。

潜热反馈可直接由原扩散式说明。为避免与初温 T_0 混淆，将相同含水率下的两个比较温度记为 $T_{lo} < T_{hi}$ ，则

$$\frac{D(C, T_{lo})}{D(C, T_{hi})} = \exp \left[-3850 \left(\frac{1}{T_{lo} + 273.15} - \frac{1}{T_{hi} + 273.15} \right) \right] < 1. \quad (13)$$

因此蒸发冷却会抑制扩散。但实际两条轨道的含水率也不同，不能把此比值直接当作全程时间倍率；必须在每一步重算全部物性。温度不再接近无潜热基线的 3 h 温度，正是所设表面能量分配带来的模型差异。640/1280 区间的终点中心、表面温度差为 3.68e-07、1.05e-06 °C。这属于数值复核，不是物理模型的实测误差。完整截面列于本问答案表 S11。表 S9 比较同一 3 h 时刻的温度与含水率；两组均从原始初态求解，差异不是把第一问的降温值直接扣到第二问上。

表 S9: 问题 2: 不计与计入表面蒸发潜热的结果对照 (3 h)

指标	不计潜热	计入潜热	差值
中心温度/(°C)	49.8495	30.5356	-19.3139
表面温度/(°C)	49.9664	31.1755	-18.7909
中心含水率/(kg/kg)	1.7662	2.2830	+0.5168
表面含水率/(kg/kg)	1.0081	0.8830	-0.1251

差值为含潜热减无潜热，先用未舍入值相减再保留四位小数。两组均为 1280 区间、BDF 积分。

表面更干不等于整体干燥更快。潜热扩展在 3 h 时的中心含水率高于基线，但表面含水率反而更低。温度降低削弱内部向表面的补水能力，使表层与内部的水分差异增大；原 Robin 条件下表面含水率降低还会减小外流通量。因此，不能用表面一个点的低含水率代替全域达标判断，第三问仍需检查中心及全部内部节点。本问热容量随含水率变化， $\rho(C)c_p(C)T$ 不是已经闭合的完整内能。这里只检查有效显热率的控制体求和与表面净热输入一致，不把常容量式(11)直接套用为完整湿物料能量定律。

6.6 双算法核验：无潜热与含潜热

Radau 从 28°C、2.55 kg/kg 重新积分，不接续第一问末态。表 S10 比较前 3 h 的指定截面，物性反馈和环境插值与 BDF 一致。

表 S10: 问题 2 的 BDF 与 Radau 同网格核对

模型	最大温差/°C	最大含水率差/(kg/kg)	临界时差/s
无潜热	1.55×10^{-6}	2.65×10^{-9}	—
含潜热	1.29×10^{-6}	1.26×10^{-9}	—

均报绝对差；场差限于指定时刻、径向位置及实际表面，并非连续全场上界。

双算法比较须与分量误差控制配套。温度与含水率的量级不同，直接合并绝对误差会掩盖水分分量。附录 A.5 的控制式可写为

$$s_i = \text{atol}_i + \text{rtol}|y_i|, \quad E_{\text{loc}} = \left[\frac{1}{2(N+1)} \sum_{i=1}^{2(N+1)} \left(\frac{e_i}{s_i} \right)^2 \right]^{1/2}.$$

其中 e_i 为局部误差估计。两场绝对容差分别为 10^{-8} °C 与 10^{-10} kg/kg，相对容差为 2×10^{-9} 。局部误差控制与非线性迭代收敛共同约束推进；工作簿逐秒采样，不表示内部积分步长固定为 1 s。

无潜热、含潜热各自作 BDF/Radau 配对，形成四组独立时间积分结果。同网格吻合检验时间推进的一致性；表 S7 及 640/1280 网格复算另查空间误差。两类检验结合，才能避免把共享空间算子带来的共同偏差漏掉。

6.7 本问直接答案

6.7.1 不含潜热的答案

表 3: 无潜热温度 (°C)

时间	算法	0	0.5	1	1.5	2
0.5000	BDF	32.1892	32.3821	32.9659	33.9608	35.4130
	Radau	32.1892	32.3821	32.9659	33.9608	35.4130
1.0000	BDF	40.3816	40.5536	41.0605	41.8782	42.9976
	Radau	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	BDF	45.8468	45.9348	46.1930	46.6051	47.1401
	Radau	45.8468	45.9348	46.1930	46.6051	47.1401
2.0000	BDF	48.4502	48.4881	48.5984	48.7736	49.0033
	Radau	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	BDF	49.4670	49.4792	49.5136	49.5653	49.6609
	Radau	49.4670	49.4792	49.5136	49.5653	49.6609
3.0000	BDF	49.8495	49.8553	49.8746	49.9101	49.9664
	Radau	49.8495	49.8553	49.8746	49.9101	49.9664

表 4: 无潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
0.5000	BDF	2.5499	2.5489	2.5256	2.3257	1.6486
	Radau	2.5499	2.5489	2.5256	2.3257	1.6486
1.0000	BDF	2.5257	2.4948	2.3578	2.0230	1.4711
	Radau	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	BDF	2.3861	2.3256	2.1344	1.8020	1.3476
	Radau	2.3861	2.3256	2.1344	1.8020	1.3476
2.0000	BDF	2.1709	2.1084	1.9236	1.6259	1.2311
	Radau	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	BDF	1.9566	1.9006	1.7360	1.4720	1.1166
	Radau	1.9566	1.9006	1.7360	1.4720	1.1166
3.0000	BDF	1.7662	1.7165	1.5702	1.3333	1.0081
	Radau	1.7662	1.7165	1.5702	1.3333	1.0081

左: 温度; 右: 干基含水率。各时刻上行 BDF、下行 Radau; 时间单位 h, 固定径向位置单位 cm。

由此本题的直接答案是: 不含潜热时, 采用式(2)、(3)、(12)的变物性有效耦合模型, 前 3 h 逐 0.5 h 五位置结果见两表。3 h 中心/表面温度为 49.8495/49.9664 °C, 含水率为 1.7662/1.0081 kg/kg。result2.xlsx 是延伸至 57.4754 h 的全过程逐秒、每 0.1 cm 两场数据, 不能用早期 3 h 历史工作簿替代。

6.7.2 含潜热的答案

表 S11: 含潜热温度 (°C)

时间	算法	0	0.5	1	1.5	2
0.5000	BDF	11.7212	11.6834	11.6098	11.6173	11.8888
	Radau	11.7212	11.6834	11.6098	11.6173	11.8888
1.0000	BDF	16.1796	16.3423	16.8281	17.6323	18.7928
	Radau	16.1796	16.3423	16.8281	17.6323	18.7928
1.5000	BDF	22.2124	22.3302	22.6790	23.2476	24.0214
	Radau	22.2124	22.3302	22.6790	23.2476	24.0214
2.0000	BDF	26.1695	26.2441	26.4647	26.8231	27.3155
	Radau	26.1695	26.2441	26.4647	26.8231	27.3155
2.5000	BDF	28.6881	28.7365	28.8788	29.1086	29.4568
	Radau	28.6881	28.7365	28.8788	29.1086	29.4568
3.0000	BDF	30.5356	30.5744	30.6920	30.8902	31.1755
	Radau	30.5356	30.5744	30.6920	30.8902	31.1755

表 S12: 含潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
0.5000	BDF	2.5500	2.5500	2.5486	2.4579	1.3305
	Radau	2.5500	2.5500	2.5486	2.4579	1.3305
1.0000	BDF	2.5499	2.5486	2.5168	2.2158	1.1581
	Radau	2.5499	2.5486	2.5168	2.2158	1.1581
1.5000	BDF	2.5450	2.5285	2.4096	1.9745	1.0784
	Radau	2.5450	2.5285	2.4096	1.9745	1.0784
2.0000	BDF	2.5077	2.4593	2.2559	1.7834	1.0099
	Radau	2.5077	2.4593	2.2559	1.7834	1.0099
2.5000	BDF	2.4159	2.3428	2.0955	1.6303	0.9440
	Radau	2.4159	2.3428	2.0955	1.6303	0.9440
3.0000	BDF	2.2830	2.1997	1.9416	1.5010	0.8830
	Radau	2.2830	2.1997	1.9416	1.5010	0.8830

左: 温度; 右: 干基含水率。各时刻上行 BDF、下行 Radau; 时间单位 h, 固定径向位置单位 cm。

当考虑上述表面蒸发潜热时, 3 h 中心/表面温度为 30.5356/31.1755 °C, 含水率为 2.2830/0.8830 kg/kg。两种答案对应不同热边界, 不能混用数据文件。物性校准可优先测定不同温湿状态下的有效扩散率, 再用未参与标定的温湿曲线检验反馈关系。

七、问题三的模型建立与求解

7.1 判停对象和可计算目标 (全域最大值约束)

平均值低于阈值不排除内部湿核, 表面先达标更不能代表各处达标。故沿用第二问动力系统, 定义

$$g(t) = \max_{0 \leq \eta \leq 1} C(\eta, t) - C_*, \quad C_* = 0.15, \quad t_* = \inf\{t \geq 0 : g(t) \leq 0\}. \quad (14)$$

离散实现对全部计算节点取最大值，线性空间输出不会在节点间产生新极大值；连续PDE与离散最大值的差异仍依赖网格验证。在当前数值场中中心为最大值，但程序不预先硬编码“只检查中心”。

严格不等式构成开集，在连续单调过阈值情形下其最早时刻只有下确界。因此 t_* 表示阈值相等的临界点，不能以打印成0.1500的数值直接证明“低于”。输出采用

$$t_{\text{out}} = 0.36 \left\lceil \frac{t_* + 5}{0.36} \right\rceil \text{ s}, \quad \max_i C_i(t_{\text{out}}) < 0.15, \quad (15)$$

其中0.36 s对应0.0001 h，5 s为数值执行余量（用于越过临界相等状态），并非针对物性误差的安全保证。

7.2 四步计算及方法比较

图11把事件根、可行终点及压力场景回代分开，避免用求根精度代替干燥时间精度。

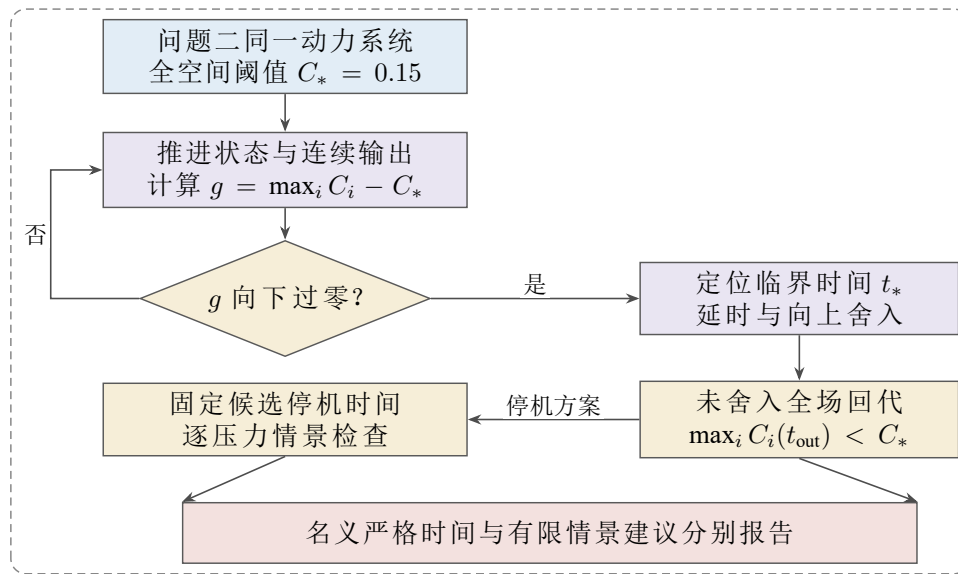


图 11: 问题三：全空间事件、严格终点和情景回代

参数改变后仍在同一个已选停机时间检查全空间最大含水率，才能判断这个停机方案是否失效。

7.2.1 继承输入并固定阈值

采用第二问初值和附录3，不从某个平均含水率重新开始。 C_* 由题目给定；记录所有场变量及完整物质网格。

7.2.2 事件定位（首次向下过阈值）

在BDF连续输出上对 $g(t)$ 作向下过零检测，首次找到零点即停止原积分段。事件器依赖步内符号变化^[8]；本模型下降型达标过程仍须结合步长和场单调检查，不能把接口本身当作不漏事件证明。

7.2.3 严格达标与后期薄层

以 $N = 1280$ 求 t_* ，从临界状态继续积分到式(15)并检查未舍入最大值。M0/M1/M2使用506个共同时间、201个位置及同一阈值比较，结果见表S13。同为160区间，M0事件误差为299.2682 s，M1降至2.5551 s，M2进一步降至0.9430 s。M1的全场含水率误差仍约0.002696 kg/kg，M2则降至 9.29×10^{-5} kg/kg。我们认为，判停时刻接近还不够，必须同时核对全场误差。积分通量与表面加密各有作用，本问采用M2也付出了更

表 S13: 问题 3 同为 160 区间的数值方法比较

方法	E_T/K	$E_C/(\text{kg/kg})$	E_t/s	耗时/s
M0	5.091e-05	4.459e-03	299.2682	19.45
M1	5.122e-05	2.696e-03	2.5551	22.41
M2	6.420e-05	9.290e-05	0.9430	43.28

长的运行时间。

7.2.4 输出及跨网格核验（区分场误差与时间误差）

最终临界时间为 57.47400138 h，严格终点 57.4754 h；未经舍入最大含水率为 0.1499985335 kg/kg。640/1280 全程复算的临界差约 0.04493 s。该差小于 5 s 执行余量，但不能由此推断未知物性、边界或潜热误差也小于 5 s。

事件的定位误差与状态误差需要分开。若最大值由同一位置实现且 t_* 附近 g 可微、 $g'(t_*) \neq 0$ ，扰动后的事件方程满足

$$0 \approx g(t_*) + g'(t_*)\delta t + \delta g, \quad \delta t \approx -\frac{\delta g}{g'(t_*)}. \quad (16)$$

这里 δg 是模型或数值场导致的含水率误差，而不是求根器的区间宽度。后期下降很缓时，即便含水率误差较小，也可形成较大的时间误差。求根定位达到微秒，不代表几十小时的物理预测也有这一精度。最大值位置切换或斜率趋零时，上述一阶估计不再适用，应直接比较扰动轨道的事件结果。

7.3 时间历程与停机扩展（有限情景回代）

图 12 用同数据的三维曲面与二维投影核对空间分布。表层先干，内部仍有湿核；后期扩散受限的局部历程见图 13，每 6 h 截面与结束行见表 5。

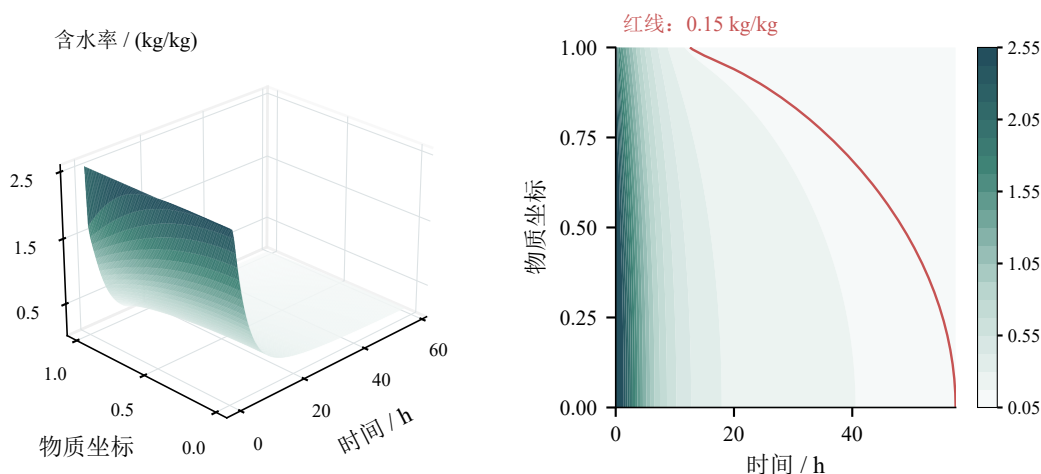


图 12: 问题三含水率三维曲面与二维投影核对

曲面与投影同取每 30 个存储时刻及末点；判停仍使用未抽稀场。

无潜热压力分析将扩散倍率 $\{0.9, 1.1\}$ 、4 h 后温度偏移 $\pm 0.5^\circ\text{C}$ 及环境含水率偏移 $\pm 0.002 \text{ kg/kg}$ 交叉为八情景。固定名义 57.4754 h 停机时四个失败；经逐情景回代，统一 64.2333 h 停机后八个均达标，最慢角点细网格最大含水率为 0.14997256 kg/kg。该方案比名义时间延长约 6.758 h，适用范围仅为所测集合。它不是概率置信区间，也不保证连续参数域达标。优化目标、网格和 60 s 余量规则见附录 A.8。

为区分不同误差来源，另对活化能、扩散前因子、初始含水率及两类表面交换系数

逐项重算总时长，见附录表 S32。活化能增加 5% 使临界时长升至 99.3886 h，而换热系数 $\pm 10\%$ 的影响不足 0.02%。这与固定时刻的状态灵敏度、八情景固定停机检验分别回答不同问题，不能互相替代；参数幅度与模型适用边界在附录说明。

7.4 当考虑到蒸发潜热对本题的影响

第三问继承第二问含潜热的同一方程与初态，从零时刻计算到全空间最大含水率首次达到 0.15 kg/kg。环境 4 h 以后仍取同一末段均值， L_v 及 ρ_d 不变，避免同时更换外推规则而混淆潜热效应。依次完成全程积分、全节点事件定位、延时 5 s 并向上对齐 0.0001 h、用未舍入状态回代四个步骤。

临界根仍不是严格不等式的可取最早时刻。扩展结果分别报告 $t_*^{(L)}$ 和 $t_{\text{out}}^{(L)}$ ，潜热造成的名义延长用 $t_*^{(L)} - t_*^{(0)}$ 衡量，不能拿临界根与另一模型严格输出混减。温度反馈较强的前期和含水率扩散很慢的后期共同决定结果，不能只对最终接近环境温度的状态进行比较。表 S14 分别比较临界时间与严格输出时间；两列使用相同阈值和执行规则，差值不混合不同判停口径。

表 S14: 问题 3：不计与计入表面蒸发潜热的结果对照（全域达标）

指标	不计潜热	计入潜热	差值
临界时间/(h)	57.4740	60.5653	+3.0913
严格输出时间/(h)	57.4754	60.5668	+3.0914

差值为含潜热减无潜热，先用未舍入值相减再保留四位小数。两组均为 1280 区间、BDF 积分。临界时间延长 5.38%；严格输出均按临界后至少 5 s、向上舍入并全节点回代确定。

640/1280 区间的终点中心、表面温度差为 $1.23\text{e-}07$ 、 $1.23\text{e-}07$ °C，临界时间差为 0.04373 s。这属于数值复核，不是物理模型的实测误差。完整截面列于本问答案表 S18。另外将 L_v 改为 2.30 和 2.50 MJ/kg 作确定性参数情景，与 2.45 MJ/kg 名义值比较。这两个端点用于检查响应，不是附件给出的误差范围或统计置信区间；结果见附录表 S30。无潜热八角点停机方案尚未在含潜热动力系统逐项回代，故不宣称原方案仍具相同适用范围。本问沿用第一问图 7 所示的潜热边界；冷却经温度相关物性反馈到扩散过程，因而需重新计算达标时间。

7.5 双算法核验：无潜热与含潜热

两算法分别定位全节点最大含水率的首次过阈值时刻。表 S15 同时给出未先舍入的临界时差和各自严格输出时刻，避免只比较四位小数而掩盖差异。

表 S15: 问题 3 的 BDF 与 Radau 同网格核对

模型	最大温差/°C	最大含水率差/(kg/kg)	临界时差/s
无潜热	9.28×10^{-9}	8.41×10^{-10}	9.32×10^{-6}
含潜热	3.35×10^{-9}	1.67×10^{-9}	7.85×10^{-6}

均报绝对差；场差限于指定时刻、径向位置及实际表面，并非连续全场上界。两算法临界/严格时刻（h，四位小数）均为：无潜热 57.4740/57.4754，含潜热 60.5653/60.5668。

有限情景共同停机的目标。沿用无潜热动力系统，八情景共享一个停机时刻。附录 A.8 的目标可简写为

$$\begin{aligned} & \min_{\tau \in [0, 72 \text{ h}]} \tau, \\ & \text{s.t.} \quad \max_i C_i^{(s)}(\tau) \leq C_*, \quad \forall s \in \mathcal{S}. \end{aligned}$$

双算法核验时间积分，情景回代检验输入改变后的可行性，二者职责不同。候选时刻由已检查下降历程中的最慢情景决定；这一有限集合结论不替换名义答案。

7.6 本问直接答案

7.6.1 不含潜热的答案

表 S16: 无潜热温度 (°C)

时间	算法	0	0.5	1	1.5	2
6.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
12.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
18.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
24.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
30.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
36.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
42.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
48.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
54.0000	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989
57.4754	BDF	49.9989	49.9989	49.9989	49.9989	49.9989
	Radau	49.9989	49.9989	49.9989	49.9989	49.9989

表 5: 无潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
6.0000	BDF	1.0170	0.9881	0.9015	0.7550	0.5328
	Radau	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	BDF	0.4561	0.4432	0.4031	0.3297	0.1642
	Radau	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	BDF	0.2988	0.2916	0.2687	0.2253	0.0845
	Radau	0.2988	0.2916	0.2687	0.2253	0.0845
24.0000	BDF	0.2378	0.2327	0.2167	0.1855	0.0665
	Radau	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	BDF	0.2058	0.2018	0.1892	0.1641	0.0598
	Radau	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	BDF	0.1859	0.1825	0.1718	0.1504	0.0566
	Radau	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	BDF	0.1721	0.1691	0.1597	0.1407	0.0548
	Radau	0.1721	0.1691	0.1597	0.1407	0.0548
48.0000	BDF	0.1618	0.1592	0.1507	0.1334	0.0537
	Radau	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	BDF	0.1539	0.1514	0.1436	0.1277	0.0530
	Radau	0.1539	0.1514	0.1436	0.1277	0.0530
57.4754	BDF	0.1500	0.1477	0.1402	0.1249	0.0526
	Radau	0.1500	0.1477	0.1402	0.1249	0.0526

左: 补充温度; 右: 干基含水率。各时刻上行 BDF、下行 Radau; 时间单位 h, 固定径向位置单位 cm。末组按 BDF 严格时刻采样; 0.1500 为舍入值, 严格性据未舍入全节点判定。

由此本题的直接答案是: 不含潜热名义条件下临界干燥时间 **57.4740 h**, 严格停机时间 **57.4754 h**; 左表 S16 补充温度, 右侧核心表 5 给出每 6 h 含水率截面。result3.xlsx 保存每 60 s、每 0.1 cm 含水率和严格结束行。另给八个指定压力角点下的统一停机建议 **64.2333 h**, 作为有边界的下游方案, 不替换名义答案。图 13 补充显示全程及阈值邻域, 解释表 5 中后期中心含水率缓慢下降的原因。

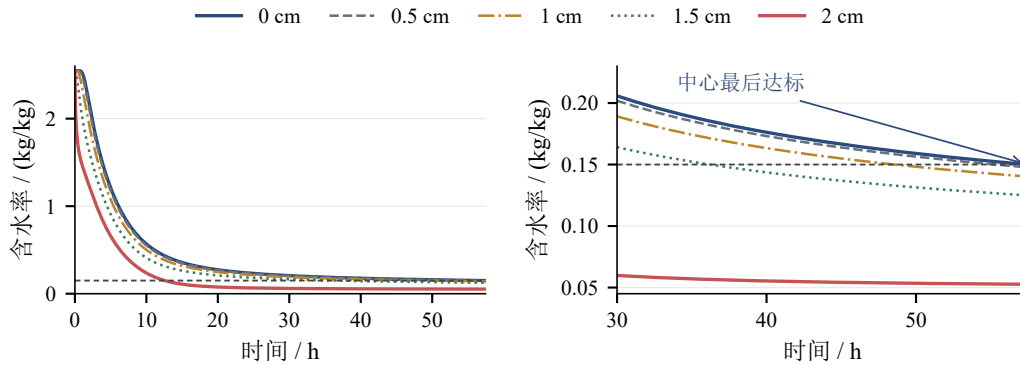


图 13: 问题三径向含水率历程与后期局部放大

统一停机时间的生成与验收。按附录 A.8, 以 320 区间遍历八情景、640 区间复算最慢角点, 将两网格较大临界值记为 $\hat{t}_*$ 。以秒为单位取

$$t_S = 60 \left\lceil \frac{\hat{t}_* + 60}{60} \right\rceil, \quad \max_i C_i^{(s)}(t_S) < 0.15 \quad (\forall s \in \mathcal{S}).$$

得到 64.2333 h 后须回代全部八情景。60 s 为共同方案的执行余量, 与名义答案的 5 s 越阈值余量分别设置, 不代表物性误差界。

7.6.2 含潜热的答案

表 S17: 含潜热温度 (°C)

时间	算法	0	0.5	1	1.5	2
6.0000	BDF	37.6948	37.7147	37.7745	37.8743	38.0171
	Radau	37.6948	37.7147	37.7745	37.8743	38.0171
12.0000	BDF	45.9370	45.9445	45.9669	46.0047	46.0595
	Radau	45.9370	45.9445	45.9669	46.0047	46.0595
18.0000	BDF	48.8081	48.8099	48.8153	48.8245	48.8379
	Radau	48.8081	48.8099	48.8153	48.8245	48.8379
24.0000	BDF	49.5047	49.5052	49.5066	49.5091	49.5126
	Radau	49.5047	49.5052	49.5066	49.5091	49.5126
30.0000	BDF	49.7272	49.7274	49.7280	49.7289	49.7303
	Radau	49.7272	49.7274	49.7280	49.7289	49.7303
36.0000	BDF	49.8246	49.8247	49.8250	49.8255	49.8261
	Radau	49.8246	49.8247	49.8250	49.8255	49.8261
42.0000	BDF	49.8760	49.8761	49.8762	49.8765	49.8769
	Radau	49.8760	49.8761	49.8762	49.8765	49.8769
48.0000	BDF	49.9067	49.9067	49.9068	49.9070	49.9072
	Radau	49.9067	49.9067	49.9068	49.9070	49.9072
54.0000	BDF	49.9265	49.9265	49.9266	49.9267	49.9269
	Radau	49.9265	49.9265	49.9266	49.9267	49.9269
60.0000	BDF	49.9402	49.9402	49.9402	49.9403	49.9404
	Radau	49.9402	49.9402	49.9402	49.9403	49.9404
60.5668	BDF	49.9412	49.9412	49.9413	49.9414	49.9415
	Radau	49.9412	49.9412	49.9413	49.9414	49.9415

表 S18: 含潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2
6.0000	BDF	1.4280	1.3730	1.2118	0.9523	0.5839
	Radau	1.4280	1.3730	1.2118	0.9523	0.5839
12.0000	BDF	0.5993	0.5803	0.5227	0.4210	0.2258
	Radau	0.5993	0.5803	0.5227	0.4210	0.2258
18.0000	BDF	0.3498	0.3405	0.3116	0.2576	0.1023
	Radau	0.3498	0.3405	0.3116	0.2576	0.1023
24.0000	BDF	0.2614	0.2555	0.2369	0.2010	0.0722
	Radau	0.2614	0.2555	0.2369	0.2010	0.0722
30.0000	BDF	0.2195	0.2150	0.2009	0.1733	0.0623
	Radau	0.2195	0.2150	0.2009	0.1733	0.0623
36.0000	BDF	0.1948	0.1912	0.1796	0.1566	0.0579
	Radau	0.1948	0.1912	0.1796	0.1566	0.0579
42.0000	BDF	0.1785	0.1754	0.1653	0.1452	0.0556
	Radau	0.1785	0.1754	0.1653	0.1452	0.0556
48.0000	BDF	0.1667	0.1639	0.1550	0.1369	0.0542
	Radau	0.1667	0.1639	0.1550	0.1369	0.0542
54.0000	BDF	0.1577	0.1552	0.1471	0.1305	0.0533
	Radau	0.1577	0.1552	0.1471	0.1305	0.0533
60.0000	BDF	0.1506	0.1483	0.1407	0.1253	0.0527
	Radau	0.1506	0.1483	0.1407	0.1253	0.0527
60.5668	BDF	0.1500	0.1477	0.1402	0.1249	0.0526
	Radau	0.1500	0.1477	0.1402	0.1249	0.0526

左：补充温度；右：干基含水率。各时刻上行 BDF、下行 Radau；时间单位 h，固定径向位置单位 cm。末组按 BDF 严格时刻采样；0.1500 为舍入值，严格性据未舍入全节点判定。

当考虑上述表面蒸发潜热时，临界时间为 **60.5653 h**，严格输出时间为 **60.5668 h**，未舍入最大含水率为 **0.1499984490 kg/kg**。该结果仅对应名义参数，不能视为原八角点方案的含潜热复验。停机方案实施前，应补采全程烘房温湿度，并检查参数区间内部的组合，重新回代最大含水率。

八、 问题四的模型建立与求解

8.1 移动坐标方程（从干基连续性推导）

Mayor 与 Sereno 的综述讨论了收缩对温湿场的影响，区分经验型与机理型模型^[10]。我们按附件半径规定几何演化并求解输运，不由应力、孔隙率和水分预测收缩。该文献不提供本题的半径数据或干燥时长。

物性与半径在本问同时改变。不能只将第三问中的 R 换成 $R(t)$ 而忽略状态定义：干基含水率附着于干物质，须先写固相连续性。令 ρ_d 为单位当前体积的干物质质量， v_s 为固体径向速度，分别对干物质和水分守恒：

$$\begin{cases} \partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}_s) = 0, \\ \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v}_s) = \nabla \cdot (\rho_d D \nabla C). \end{cases} \quad (17)$$

用第二式减去第一式乘 C ，得到 $\rho_d (C_t + v_s C_r) = \nabla \cdot (\rho_d D \nabla C)$ 。均匀径向收缩时 $v_s = r\dot{R}/R$ 、 $\rho_d(t) \propto R(t)^{-2}$ 且空间均匀。取随固体运动的 $\eta = r/R(t)$ ，链式法则给出

$$C_t|_r = C_t|_\eta - \frac{\eta \dot{R}}{R} C_\eta, \quad v_s C_r = \frac{\eta \dot{R}}{R} C_\eta. \quad (18)$$

两个项相消，因而主模型为

$$\begin{cases} C_t|_\eta = \frac{1}{R(t)^2\eta} \partial_\eta [\eta D(C, T) C_\eta], \\ \rho c_p T_t|_\eta = \frac{1}{R(t)^2\eta} \partial_\eta [\eta k(C) T_\eta]. \end{cases} \quad (19)$$

温度式仍为有效显热模型。物质坐标跟随同一固体质点，实际径向位置随收缩变化^[11]；式(17)–(19)依赖均匀缩放假设。

如附录图 23 所示，几何收缩本身不产生额外失水；零扩散、零表面交换时，随物质的含水率应保持不变，后文据此检查坐标实现。

若介质静止而边界移动，Euler 坐标变换会产生输运项；这里 η 跟随固体运动，该项已相消。是否保留输运项取决于固相运动假设，不能任意选取。

附录 4 给出的有效物性为

$$\begin{aligned} \rho(C) &= 760 + 90C, & c_p(C) &= 1850 + 2150 \frac{C}{1+C}, \\ k(C) &= 0.12 + 0.20 \frac{C}{1+C}, & D(C, T) &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/(T+273.15)}. \end{aligned} \quad (20)$$

在 $\eta = 1$ ，无潜热基线的边界导数换成 $-(k/R)T_\eta = h(T_s - T_a)$ 、 $-(D/R)C_\eta = h_m(C_s - C_a)$ ，中心仍为零梯度，初值同前。含潜热时，热边界右端另加 $L_v j_v$ ，并按式(23)的收缩干密度计算 j_v 。由于经验 $\rho(C)$ 未被强制等同于 $\rho_d(1+C)$ ，本模型不能同时声称完整孔隙结构、体积变化与总体密度关系均已闭合。

8.2 四步求解与干基收支

图 14 给出从坐标变换、变域采样到效应对照的计算顺序。

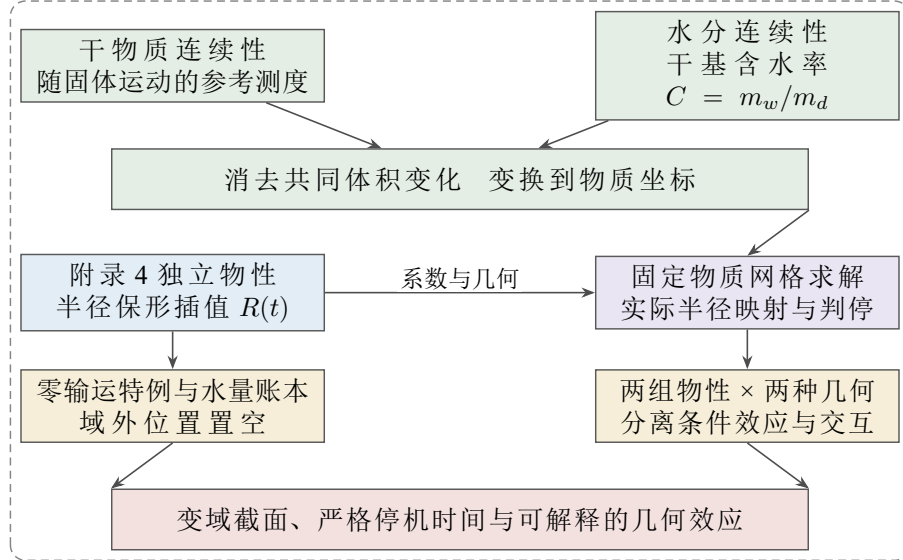


图 14: 问题四：收缩守恒、变域求解与归因核验

固定物理位置 r 与固定物质位置 η 在收缩时并不相同；导出时必须先以当前 $R(t)$ 进行映射。

8.2.1 输入、物性和收缩

读取附件 2 的 $R(t)$ ，采用保形插值且不超出数据支持时间。选择附录 4 全部公式，不能保留附录 3 的扩散前因子。固相运动只按均匀缩放假设构造，不由失水结果另拟合一条无来源收缩曲线。

8.2.2 固定物质网格离散（随固体运动）

沿用式(4)的归一化权重， $R(t)^{-2}$ 进入体积扩散，表面通量包含 $R(t)$ 。不额外加一阶迎风项。零扩散、零表面交换的非均匀初值测试中， $C(\eta)$ 应随物质保持不变；该制造解检验把坐标一致性与整体水量检查区分开。

8.2.3 加密、事件及输出位置（区分域外与表面）

无潜热生产解使用 2560 区间的表面加密网格；本问含潜热扩展使用 1280 区间，另以 640/1280 配对复核。两者均以全节点最大含水率定位事件并继续到严格结束行。对于固定位置 r_j ，仅在 $r_j \leq R(t)$ 时计算 $\eta_j = r_j/R(t)$ 并插值；超出时标为域外空值。实际表面 $C(1,t)$ 另列，不能取“最后一个仍在体内的 0.1 cm 网格点”冒充表面。

8.2.4 同口径比较和干基账本

表 S19 中，积分通量使事件误差由 62.4293 s 减至 2.2080 s，加密后为 0.9831 s。不过 M1 的全场 E_C 略高于 M0，事件时间的改善没有同步体现在全部指标上。

表 S19: 问题 4 同为 160 区间的数值方法比较

方法	E_T/K	$E_C/(kg/kg)$	E_t/s	耗时/s
M0	2.360e-04	2.774e-03	62.4293	54.24
M1	2.392e-04	2.799e-03	2.2080	58.86
M2	1.043e-04	4.703e-04	0.9831	29.25

在均匀干物质参考测度下，平均干基含水率为 $\bar{C} = 2 \sum_i w_i C_i$ 。将式(4)求和，所有内界面抵消，得到

$$\frac{d\bar{C}}{dt} = \frac{2h_m}{R(t)}[C_a(t) - C_s(t)], \quad \mathcal{R}_C = \bar{C}(t) - \bar{C}(0) - \int_0^t \frac{2h_m}{R(u)}[C_a(u) - C_s(u)]du. \quad (21)$$

以含水率变化绝对值为分母，本问边界通量独立积分的相对残差约 1.63×10^{-9} 。这个检查支持有效模型内部的数值守恒；若改成当前体积加权且没有 ρ_d ，会把几何缩小误算为额外失水。它不是潜热或完整湿空气能量守恒验证。

8.3 物性与几何的效应分离（四格对照）

临界时间为 51.08850314 h，严格终点 51.0899 h，最大含水率 0.1499974179 kg/kg，瞬时半径为 1.2000 cm。图 15 保留域外区域，并与若干实际半径截面共同解释表 6。

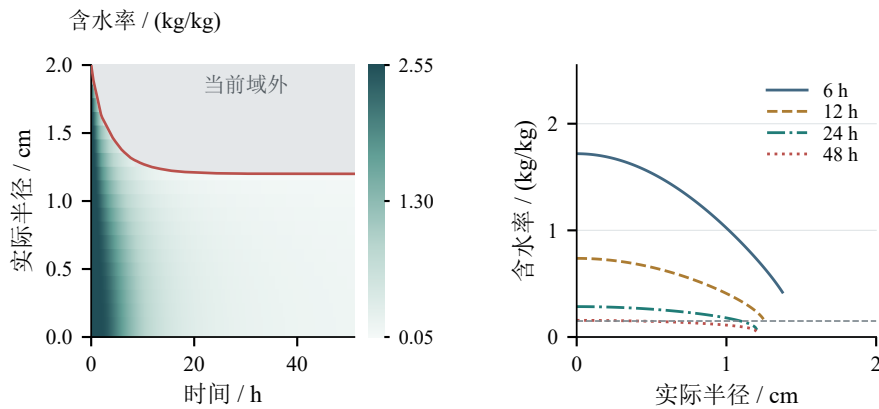


图 15: 问题四的变域水分场与实际半径截面

无潜热基线；灰色表示当前域外，实际表面按当前半径独立读取。

固定 1.5 或 2 cm 处的空值不是缺失观测，曲线端点随半径移动。

变域读数须与干基质量测度一致。在长度不变、均匀径向收缩的假设下， $r = R(t)\eta$ 及 $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$ 给出物质薄环中的干物质量及全体平均含水率：

$$dm_d = 2\pi L \rho_d(t) r dr = 2\pi L \rho_{d0} R_0^2 \eta d\eta,$$

$$\bar{C}(t) = \frac{\int C dm_d}{\int dm_d} = 2 \int_0^1 C(\eta, t) \eta d\eta.$$

同一物质薄环的干物质量不随几何缩小而改变，离散式 $\bar{C} = 2 \sum_i w_i C_i$ 因此保持固定权重。若用当前体积积分替代干基加权，却遗漏 ρ_d 的增加，就会把体积缩小误判为额外失水。零扩散、零表面交换时应有 $C_t|_\eta = 0$ ，正是式(19)与零输运测试共同要求的退化情形。

6 h 时，第四问中心含水率约 1.7188 kg/kg，高于第三问的约 1.0170 kg/kg。这说明物性与几何共同变化时，初期中心失水较慢；最终较早达标不能推出全过程都更快。两项因素的贡献须由同口径四格对照区分。

为分离竞争解释，固定数值方法为 320 区间加密积分通量，交叉使用附录 3/4 物性与固定/收缩几何，令 $t(P, G)$ 为组合下的临界时间。图 16 将同一物性下的两个结果成对连接，连接长度直接表示条件几何效应。

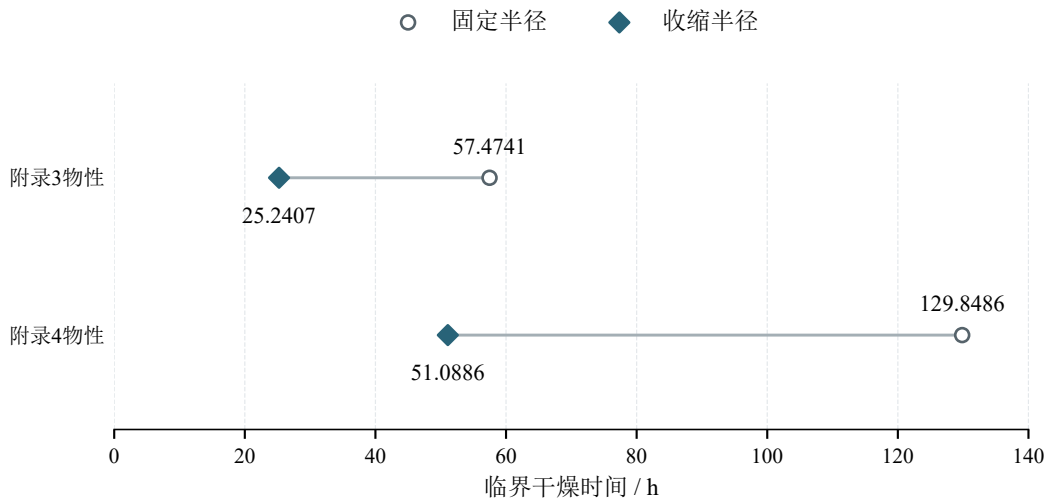


图 16: 同一数值口径下物性与几何的四格干燥时间对照

空心圆为固定半径，菱形为收缩半径；固定几何反事实沿用恒定环境，不外推收缩半径。

在附录 4 物性不变时，收缩减少 78.7600 h，即 60.6553%；在附录 3 物性下，收缩减少 32.2334 h。加性交互为

$$\Delta_{PG} = t(4, \text{缩}) - t(4, \text{固}) - t(3, \text{缩}) + t(3, \text{固}) = -46.5266 \text{ h}. \quad (22)$$

该值表示模型内两因素的非加性交互。第三、四问净临界时差约 6.3855 h，约为第三问的 11.11%；其中同时包含几何、物性及其交互作用，不能归为纯收缩贡献。我们认为，收缩贡献应在物性不变的条件下比较。四格结果限于所设模型，不具有随机试验的因果解释。

沿用第三问八角点规则，第四问临界时间范围为 46.2506 至 57.0403 h，按名义时间 51.0899 h 停机仍有四个角点未达标。统一延长至 57.0667 h 后，全部已测角点通过，最慢角点细网格最大含水率为 0.14995694 kg/kg。图 17 并列两问在名义停机与统一情景停机时的回代结果。

8.4 当考虑到蒸发潜热对本题的影响

第四问继续使用附录 4 与实测半径，不把第三问末态作为初值。由固定长度、径向均匀收缩及干物质总质量不变，有

$$\rho_d(t)\pi R^2(t)L = \rho_{d0}\pi R_0^2L, \quad \rho_d(t) = \frac{760 + 90 \times 2.55}{3.55} \left[\frac{R_0}{R(t)} \right]^2. \quad (23)$$

初始干物质密度为 278.7324 kg/m³，并非始终不变。把式(23)代入式(8)，再把蒸发耗热加入式(9)。物质坐标内的温度和干基水分守恒形式仍沿用式(19)的建模条件，不额外加入一个代表“收缩失水”的源项。收缩通过扩散距离与单位当前面积的质量换算共同影响结果。

单位干物质水量 $\bar{C} = 2 \sum_i w_i C_i$ 应满足

$$\frac{d\bar{C}}{dt} = -\frac{2j_v}{\rho_d R} = -\frac{2h_m}{R}(C_s - C_a). \quad (24)$$

这说明在质量测度前后一致时，含潜热扩展保留原干基水量账本；质量密度并非凭空改变失水规律，而是将相同水量变化换算为能量耗散。表 S20 列出两组生产答案。同为附录 4、同一半径函数和环境外推，含潜热时额外采用式(23)换算蒸发质量通量，并增加热边界耗热项；不是通过修改经验扩散系数来拟合时间。

表 S20: 问题 4: 不计与计入表面蒸发潜热的结果对照（全域达标）

指标	不计潜热	计入潜热	差值
临界时间/(h)	51.0885	56.5979	+5.5094
严格输出时间/(h)	51.0899	56.5994	+5.5095
终点半径/(cm)	1.2000	1.2000	0.0000

差值为含潜热减无潜热，先用未舍入值相减再保留四位小数。无潜热为 2560 区间，含潜热为 1280 区间；本表比较生产答案，不作为严格同网格消融。临界时间延长 10.78%；严格输出均按临界后至少 5 s、向上舍入并全节点回代确定。终点半径分别在各自严格终点读取。

640/1280 区间的终点中心、表面温度差为 2.51e-07、2.50e-07 °C，临界时间差为 0.03760 s。这属于数值复核，不是物理模型的实测误差。完整截面列于本问答案表 S24。**密度兼容性仍是本扩展的限制。**经验 $\rho(C)$ 用于有效热容量，而 $\rho_d(t)$ 来自干物质连续性；本模型不保证任意时刻二者满足 $\rho = (1 + C)\rho_d$ 。要把它升级为完整多相热力学模型，需重新闭合密度、孔隙率、气固平衡及焓输运，而不能仅凭与参考截图接近就认定物理正确。例如含潜热严格终点 $R = 1.2$ cm 时，式(23)给出 $\rho_d = 774.2567$ kg/m³；按中心未舍入含水率计算，题给经验密度为 773.4998 kg/m³，而 $(1 + C)\rho_d = 890.3931$ kg/m³。这两种湿密度解释并不相容，不能以网格收敛掩盖。因此本文仅把题给 ρ_{C_p} 保留为有效容量参数，所报含潜热时间是这套明确闭合下的结果，不是已验证的多相物理预测；若替换容量关系，须重新计算并另列模型，不能直接沿用当前答案。

8.5 双算法核验：无潜热与含潜热

保持同一物质坐标、保形半径和域外空值规则，分别替换两种热边界模型的时间积分器。表 S21 中的每一行均是同网格配对，不把无潜热与含潜热的不同网格混为算法误差。

表 S21: 问题 4 的 BDF 与 Radau 同网格核对

模型	最大温差/℃	最大含水率差/(kg/kg)	临界时差/s
无潜热	7.45×10^{-10}	1.18×10^{-8}	1.76×10^{-3}
含潜热	1.05×10^{-6}	4.85×10^{-9}	2.97×10^{-4}

均报绝对差；场差限于指定时刻、径向位置及实际表面，并非连续全场上界。两算法临界/严格时刻（h，四位小数）均为：无潜热 51.0885/51.0899，含潜热 56.5979/56.5994。

8.6 本问直接答案

8.6.1 不含潜热的答案

表 S22: 无潜热温度（℃）

时间	算法	0	0.5	1	1.5	2	表面
6.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
12.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
18.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
24.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
30.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
36.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
42.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
48.0000	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989
51.0899	BDF	49.9989	49.9989	49.9989	—	—	49.9989
	Radau	49.9989	49.9989	49.9989	—	—	49.9989

表 6: 无潜热含水率（kg/kg）

时间	算法	0	0.5	1	1.5	2	表面
6.0000	BDF	1.7188	1.5370	1.0221	—	—	0.4205
	Radau	1.7188	1.5370	1.0221	—	—	0.4205
12.0000	BDF	0.7374	0.6544	0.4082	—	—	0.1669
	Radau	0.7374	0.6544	0.4082	—	—	0.1669
18.0000	BDF	0.4086	0.3686	0.2397	—	—	0.0892
	Radau	0.4086	0.3686	0.2397	—	—	0.0892
24.0000	BDF	0.2851	0.2610	0.1790	—	—	0.0673
	Radau	0.2851	0.2610	0.1790	—	—	0.0673
30.0000	BDF	0.2263	0.2094	0.1493	—	—	0.0595
	Radau	0.2263	0.2094	0.1493	—	—	0.0595
36.0000	BDF	0.1928	0.1796	0.1317	—	—	0.0560
	Radau	0.1928	0.1796	0.1317	—	—	0.0560
42.0000	BDF	0.1712	0.1604	0.1200	—	—	0.0541
	Radau	0.1712	0.1604	0.1200	—	—	0.0541
48.0000	BDF	0.1561	0.1468	0.1116	—	—	0.0530
	Radau	0.1561	0.1468	0.1116	—	—	0.0530
51.0899	BDF	0.1500	0.1413	0.1081	—	—	0.0526
	Radau	0.1500	0.1413	0.1081	—	—	0.0526

左：补充温度；右：干基含水率。各时刻上行 BDF、下行 Radau；时间单位 h，固定径向位置单位 cm。末组按 BDF 严格时刻采样；0.1500 为舍入值，严格性据未舍入全节点判定。表面随半径移动；—表示域外。

由此本题的直接答案是：不含潜热时，采用附录 4、附件 2 半径及式(19)的收缩物质坐标模型，临界时间 **51.0885 h**，名义严格停机时间 **51.0899 h**；每 6 h 截面及实际表面见表 6。result4.xlsx 保存每 60 s、每 0.1 cm 数据，域外留空，实际表面另列。八个指定压力角点的补充统一停机建议为 **57.0667 h**。收缩贡献按同物性四格对照解释，不以跨物性净差替代。图 17 将两问的名义方案与统一情景方案分别回代到全部八角点，展示停机余量的作用。

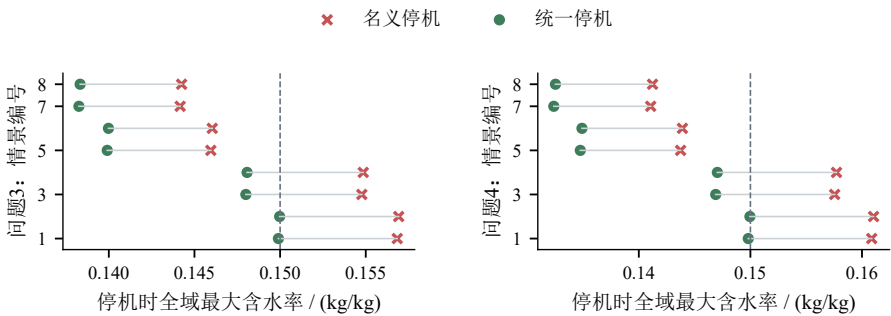


图 17: 两种停机方案在第三、四问八角点的最大含水率回代

每点对应一个指定角点，虚线右侧未达标；八角点通过不代表连续参数域保证。

8.6.2 含潜热的答案

表 S23: 含潜热温度 (°C)

时间	算法	0	0.5	1	1.5	2	表面
6.0000	BDF	34.3920	34.4090	34.4607	—	—	34.5272
	Radau	34.3920	34.4090	34.4607	—	—	34.5272
12.0000	BDF	40.2573	40.2781	40.3425	—	—	40.3952
	Radau	40.2573	40.2781	40.3425	—	—	40.3952
18.0000	BDF	45.5070	45.5193	45.5578	—	—	45.5841
	Radau	45.5070	45.5193	45.5578	—	—	45.5841
24.0000	BDF	48.0750	48.0798	48.0947	—	—	48.1042
	Radau	48.0750	48.0798	48.0947	—	—	48.1042
30.0000	BDF	49.0532	49.0552	49.0611	—	—	49.0647
	Radau	49.0532	49.0552	49.0611	—	—	49.0647
36.0000	BDF	49.4571	49.4580	49.4606	—	—	49.4623
	Radau	49.4571	49.4580	49.4606	—	—	49.4623
42.0000	BDF	49.6511	49.6515	49.6529	—	—	49.6537
	Radau	49.6511	49.6515	49.6529	—	—	49.6537
48.0000	BDF	49.7565	49.7567	49.7575	—	—	49.7580
	Radau	49.7565	49.7567	49.7575	—	—	49.7580
54.0000	BDF	49.8196	49.8198	49.8203	—	—	49.8206
	Radau	49.8196	49.8198	49.8203	—	—	49.8206
56.5994	BDF	49.8393	49.8395	49.8399	—	—	49.8401
	Radau	49.8393	49.8395	49.8399	—	—	49.8401

表 S24: 含潜热含水率 (kg/kg)

时间	算法	0	0.5	1	1.5	2	表面
6.0000	BDF	2.2805	2.0539	1.3192	—	—	0.3806
	Radau	2.2805	2.0539	1.3192	—	—	0.3806
12.0000	BDF	1.2028	1.0428	0.5967	—	—	0.2182
	Radau	1.2028	1.0428	0.5967	—	—	0.2182
18.0000	BDF	0.6168	0.5454	0.3296	—	—	0.1230
	Radau	0.6168	0.5454	0.3296	—	—	0.1230
24.0000	BDF	0.3778	0.3412	0.2220	—	—	0.0809
	Radau	0.3778	0.3412	0.2220	—	—	0.0809
30.0000	BDF	0.2743	0.2515	0.1731	—	—	0.0652
	Radau	0.2743	0.2515	0.1731	—	—	0.0652
36.0000	BDF	0.2215	0.2050	0.1466	—	—	0.0587
	Radau	0.2215	0.2050	0.1466	—	—	0.0587
42.0000	BDF	0.1902	0.1773	0.1302	—	—	0.0556
	Radau	0.1902	0.1773	0.1302	—	—	0.0556
48.0000	BDF	0.1696	0.1589	0.1191	—	—	0.0539
	Radau	0.1696	0.1589	0.1191	—	—	0.0539
54.0000	BDF	0.1551	0.1458	0.1110	—	—	0.0529
	Radau	0.1551	0.1458	0.1110	—	—	0.0529
56.5994	BDF	0.1500	0.1413	0.1081	—	—	0.0526
	Radau	0.1500	0.1413	0.1081	—	—	0.0526

左：补充温度；右：干基含水率。各时刻上行 BDF、下行 Radau；时间单位 h，固定径向位置单位 cm。末组按 BDF 严格时刻采样；0.1500 为舍入值，严格性据未舍入全节点判定。表面随半径移动；—表示域外。

当考虑上述表面蒸发潜热时，临界时间为 **56.5979 h**，严格输出时间为 **56.5994 h**，未舍入最大含水率为 **0.1499973122 kg/kg**。结束半径为 **1.2000 cm**。该结果仅对应名义参数，不能视为原八角点方案的含潜热复验。收缩模型应优先联测质量、半径与孔隙率，检验经验密度和干基守恒能否相容，再决定是否引入非均匀收缩。

九、综合检验与适用边界

9.1 数值可靠性与误差来源

数值实现由量纲、边界、收支、网格及独立求解路径共同核验，假设扰动则考察结果对输入条件的依赖。这些检查回答的是模型内部问题，物理准确性还需实验验证。

无潜热四问的相对水量残差依次约为 2.25×10^{-10} 、 3.34×10^{-10} 、 7.10×10^{-10} 、 1.63×10^{-9} ；常热容量第一问的显热收支残差约 5.47×10^{-9} 。含潜热残差见表 S29。变物性模型的 $\rho(C)c_p(C)T$ 不是完整内能，故上述检查不代表总能量闭合。

四位小数是输出格式，不代表物理准确度。观测与模型闭合、空间离散与时间积分、插值与舍入的误差来源各不相同；网格核验尤其要覆盖早期表面和干燥尾部。

附录 A.9 保留温标、判停、外推及插值对照。阶跃导热 Bessel 特例在九个指定时空点的 160、320 区间最大温差分别为 1.30×10^{-4} 、 3.48×10^{-5} °C，仅独立核对该特例的扩散尺度。

图 18 将表 S13、S19 的误差与耗时放在一起比较。M0 为端点平均，M1 改用积分通量，M2 再加密表面网格；两问均保持 160 区间，分别以各自细网格解为参考。两问的事件误差均随改进而减小，耗时却不遵循同一顺序：第三问 M2 较慢，第四问本次运行反而较快。因此，方法选择需同时考虑精度与代价；单次耗时不代表普遍速度优势。这是无潜热空间离散方案的比较，与前述 BDF/Radau 时间积分核验分开。

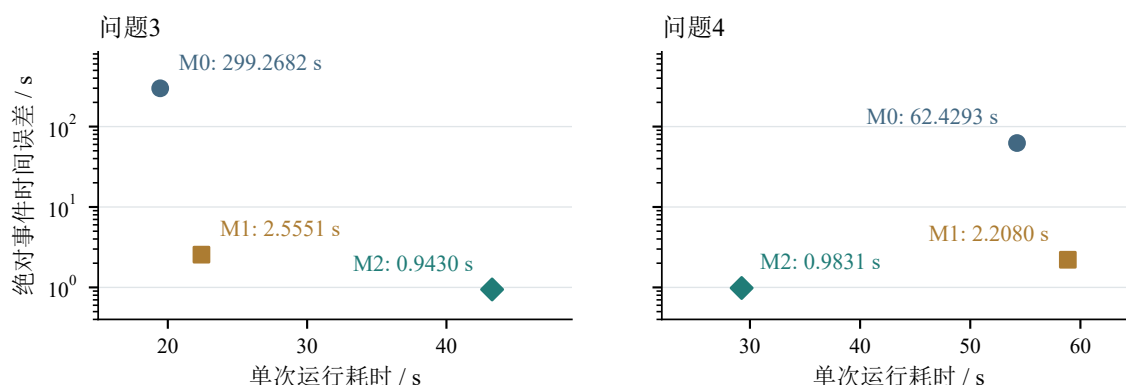


图 18: 第三、四问的事件时间误差与运行耗时对照（纵轴为对数刻度）

9.2 表面潜热扩展的验证与适用边界

四问均按式(9)扣除侧表面蒸发耗热。含潜热 640/1280 区间的第三、四问临界时差为 0.04373、0.03760 s，四问水量相对残差均不超过 4.18×10^{-10} ，见附录 A.10；生产网格双算法对照与粗网格测试分别报告。

相变位置、气固驱动力及密度闭合仍待验证。换算通量 j_v 并非实测值；扩散失水不等于当地汽化，故不另加 $-\rho L_v \partial_t C$ 体积热源。附件不足以校准表面活度、吸附等温线及湿度换算，第四问两种密度关系也尚不兼容。含潜热结果因而受这些条件限制，需联测温度、排湿、质量和尺寸加以核验。图 7 只说明能量分配。

核心推导、逐式来源与检查见第 5.1 节、附录 B.2 及表 S31。生产网格、回读记录和完整程序均保留；快速测试和冻结场导出不替代两模型的生产复算。

十、 模型评价、已完成改进及推广

10.1 有证据支持的改进

1. **稳定积分通量。**不改题给扩散定律，同为 160 区间时将第三问事件偏差由 299.27 s 降至 2.56 s；表面加密另改善场误差。
2. **守恒收缩表达。**从干基连续性推导物质坐标方程，以零输运和收支核验；四格对照分离条件几何效应，避免跨物性误归因。
3. **可计算潜热扩展。**换算质量通量并扣除表面耗热、同步收缩干密度；用零潜热退化和逐问复算检查，不重拟合经验参数。
4. **双算法答案核对。**六张核心表及扩展表并列 BDF/Radau；指定点最大温差约 $2.91 \times 10^{-6} \text{ }^\circ\text{C}$ 、最大临界时差约 0.00176 s。
5. **报告停机方案的失败情景。**两问名义停机在各自八角点中均有四例失败，统一延长后全部角点通过。结论限于所测集合。

10.2 局限与改进方向

1. **热质与收缩闭合需实验校准。**相变位置、气固驱动力及质量换算待验证，有效热容量、湿密度与守恒干密度尚未闭合。须联测温度、排湿、质量、尺寸及孔隙，数值收支不替代实测。
2. **一维模型的代表范围有限。**端部导热限制了中截面近似的适用范围；端部变物性热质及潜热作用仍需二维核验。
3. **长期预测受环境外推限制。**4 h 后采用末段均值，八角点也没有概率权重。应补采全程环境，并检查参数区间内的组合和边界极值；边界条件须依据观测设定，而非反调至经验时间，角点通过也不足以保证连续域。

推广至其他圆柱材料，应先重标物性、交换系数与收缩规律，再复核停机结果。球形还须重设面积权重并核验轴心；若出现开裂、强毛细输运或内部沸腾，则应重审扩散假设。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型优化辅助，数值计算，论文语言润色，详细使用情况见支撑材料。

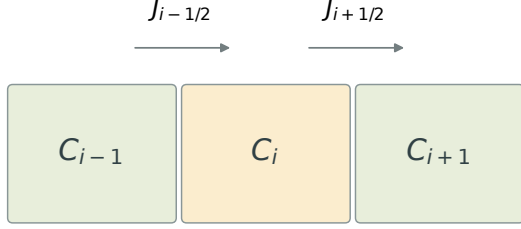
参考文献

- [1] Datta A K. Porous media approaches to studying simultaneous heat and mass transfer in food processes. I: Problem formulations[J]. Journal of Food Engineering, 2007, 80(1): 80–95. DOI: <https://doi.org/10.1016/j.jfoodeng.2006.05.013>.
- [2] Food and Agriculture Organization of the United Nations. Crop evapotranspiration: Annex 3, Background on physical parameters used in evapotranspiration computations[EB/OL]. <https://www.fao.org/4/X0490E/x0490e0k.htm>. 2026-09-13 查阅. 仅据其潜热段选取 L_v 近似值，不套用其蒸散发模型。
- [3] SciPy Developers. scipy.interpolate.PchipInterpolator: SciPy v1.13.1 Manual[EB/OL]. <https://docs.scipy.org/doc/scipy-1.13.1/reference/generated/scipy.interpolate.PchipInterpolator.html>. 2026-09-13 查阅.
- [4] Fritsch F N, Butland J. A method for constructing local monotone piecewise cubic interpolants[J]. SIAM Journal on Scientific and Statistical Computing, 1984, 5(2): 300–304. DOI: <https://doi.org/10.1137/0905021>.
- [5] National Institute of Standards and Technology. FiPy: Finite Volume Method[EB/OL]. <https://pages.nist.gov/fipy/en/stable/numerical/discret.html>. 2026-09-13 查阅.
- [6] Guyer J E, Wheeler D, Warren J A. FiPy: Partial differential equations with Python[J]. Computing in Science & Engineering, 2009, 11(3): 6–15. DOI: <https://doi.org/10.1109/MCSE.2009.52>.
- [7] SciPy Developers. scipy.special.expi: SciPy v1.13.1 Manual[EB/OL]. <https://docs.scipy.org/doc/scipy-1.13.1/reference/generated/scipy.special.expi.html>. 2026-09-13 查阅.
- [8] SciPy Developers. scipy.integrate.solve_ivp: SciPy v1.13.1 Manual[EB/OL]. https://docs.scipy.org/doc/scipy-1.13.1/reference/generated/scipy.integrate.solve_ivp.html. 2026-09-13 查阅.
- [9] SciPy Developers. scipy.integrate.Radau: SciPy v1.13.1 Manual[EB/OL]. <https://docs.scipy.org/doc/scipy-1.13.1/reference/generated/scipy.integrate.Radau.html>. 2026-09-13 查阅.
- [10] Mayor L, Sereno A M. Modelling shrinkage during convective drying of food materials: a review[J]. Journal of Food Engineering, 2004, 61(3): 373–386. DOI: [https://doi.org/10.1016/S0260-8774\(03\)00144-4](https://doi.org/10.1016/S0260-8774(03)00144-4).
- [11] COMSOL. About Frames: COMSOL Multiphysics Reference Manual, version 6.3[EB/OL]. https://doc.comsol.com/6.3/doc/com.comsol.help.comsol/comsol_ref_deformedmeshes.34.05.html. 2026-09-13 查阅.
- [12] Shampine L F, Reichelt M W. The MATLAB ODE suite[J]. SIAM Journal on Scientific Computing, 1997, 18(1): 1–22. DOI: <https://doi.org/10.1137/S1064827594276424>.
- [13] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python[J]. Nature Methods, 2020, 17(3): 261–272. DOI: <https://doi.org/10.1038/s41592-019-0686-2>.

A 补充推导与下游检验

A.1 轴心控制体与环形极限

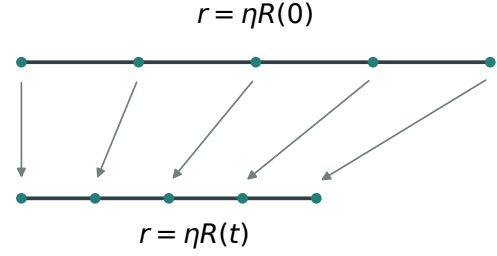
(a) 共享界面通量与环形控制体



$$w_i = (\eta_{i+1/2}^2 - \eta_{i-1/2}^2)/2$$

共享一次通量，左右相反入账
全域求和后只剩边界交换

(b) 同一物质点的几何映射



固定物质坐标 η ，不额外增加失水项

图 19: 环形控制体共享通量与收缩物质点映射

左图的三个矩形表示径向控制体的逻辑展开而非平板几何，环形几何已经包含在权重中；右图相同颜色节点具有相同物质坐标，连接线不表示新增扩散。

为核对轴心处理，把均匀节点间距记为 $\Delta\eta$ 。轴心权重为 $(\Delta\eta/2)^2/2 = \Delta\eta^2/8$ ，外侧界面为 $\Delta\eta/2$ 。常扩散系数下代入式(4)，得到

$$\dot{C}_0 = \frac{4D}{R^2\Delta\eta^2}(C_1 - C_0). \quad (25)$$

另一方面，轴对称光滑场满足 $C = C_0 + \frac{1}{2}C_{rr}(0)r^2 + O(r^4)$ ，故 $C_{rr} + C_r/r \rightarrow 2C_{rr}$ 。式(25)恰与这一极限一致；若面因子或体积因子漏去一个 2，整体收支仍可能自洽，但扩散时间尺度会错误。因此边界账本必须与局部制造解、细网格对照共同使用。

A.2 积分等效系数的性质

通量修正的对象与边界。这一步不修改题给的扩散定律，而是改进其离散表征。令 $\Delta C = C_R - C_L$ ，在冻结温度前因子 $\bar{A}$ 的连接段上，定义等效系数（表征相邻含水率之间的平均扩散作用）

$$D_{\text{int}} = \begin{cases} \frac{\bar{A}}{\Delta C} \int_{C_L}^{C_R} e^{-a/c} dc, & \Delta C \neq 0, \\ \bar{A}e^{-a/C_L}, & \Delta C = 0. \end{cases} \quad (26)$$

其中 c 为积分变量， D_{int} 的单位仍为 m^2/s 。记 $C_{\min} = \min(C_L, C_R) > 0$ 、 $C_{\max} = \max(C_L, C_R)$ 。由于 $e^{-a/c}$ 在 $c > 0$ 上单调增加，积分均值满足

$$\bar{A}e^{-a/C_{\min}} \leq D_{\text{int}} \leq \bar{A}e^{-a/C_{\max}}, \quad J = \eta_{i+1/2} D_{\text{int}} \frac{\Delta C}{\Delta\eta}.$$

故该系数为正，交换左右状态时通量反号；令 $\Delta C \rightarrow 0$ 又恢复局部扩散率。相较于直接端点平均，它保留连接段内部的状态变化；相较于擅改经验参数，它不改变连续方程。这里的正系数与共享通量结构并不能单独证明任意时间离散都保持正值，仍须结合时间积分及实际正性检查。其数值收益见正文表 S2–S19 的同网格消融，不作“首创扩散定律”的主张。

A.3 端面导热的独立诊断

针对两端截面的疑问，另在常物性导热方程中加入轴向扩散，令 $z \in [0, L/2]$ 从中截面指向端部：

$$T_t = \alpha \left[\frac{1}{r} \partial_r (r T_r) + T_{zz} \right], \quad T_z(r, 0, t) = 0, \quad -k T_z(r, L/2, t) = h_e (T - T_a). \quad (27)$$

在同一环境下比较 $h_e = 0$ 与 $h_e = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ ，用径向 128 区间、半轴向 400 区间的可分离离散谱及分段线性边界卷积复算。1800 s 时中截面最大改变量为 $6.04 \times 10^{-8} \text{ }^\circ\text{C}$ ，全域局部最大改变量为 $3.6970 \text{ }^\circ\text{C}$ ，体积均温改变量 $0.2594 \text{ }^\circ\text{C}$ ；端面轴心温度 $37.2723 \text{ }^\circ\text{C}$ ，端部边缘 $38.9197 \text{ }^\circ\text{C}$ 。64×200 到 128×400 网格的局部最大改变量之差约 $0.000346 \text{ }^\circ\text{C}$ ，独立 DOP853 时间推进在同一粗空间算子下与谱计算相差约 $2.14 \times 10^{-9} \text{ }^\circ\text{C}$ 。

这项诊断支持本问中截面近似，但否定“端面到处都无影响”的说法。它不包含水分、潜热及后几问变物性，不能作为这些耦合效应的验证。由热扩散长度 $\sqrt{\alpha t} \approx 1.74 \text{ cm}$ 远小于半轴长 12.5 cm ，可解释短时端面扰动尚未深入中截面。

A.4 第一问的集中参数与灵敏度补充

集中参数模型将内部温度替换为单一均温 $T_L(t)$ 。对整个圆柱作显热平衡，并沿用主模型的端部绝热条件，侧面积与体积之比为 $2/R$ ，因而

$$\frac{dT_L}{dt} = \frac{2h}{\rho c_p R} (T_a - T_L), \quad T_L(0) = 28. \quad (28)$$

在同一环境下积分式(28)，并用 160 区间径向模型生成对照。1800 s 时集中参数温度为 $36.3427 \text{ }^\circ\text{C}$ ；若误将它视为中心温度，将比径向中心 $33.5753 \text{ }^\circ\text{C}$ 高约 $2.7673 \text{ }^\circ\text{C}$ （差值由未舍入的同网格结果计算）。集中参数模型本来只有一个均温，因此图 6 同时给出径向体积均温，避免把两个不同统计对象混作误差。这一对照表明，低维模型虽计算简单，却不能交付题目要求的五个位置温度。

为了解参数标定的优先级，分别将 D 、 h 、 h_m 乘以 0.9 和 1.1，其余输入固定。本问终点取 1800 s，对照始终使用 160 区间表面加密网格和相同积分容差。设输出为 Y 、被扰动参数为 p ，定义设计幅度 $\varepsilon = 0.1$ 下的中心差商

$$S_{Y,p}^{(\varepsilon)} = \frac{Y((1+\varepsilon)p) - Y((1-\varepsilon)p)}{2\varepsilon}. \quad (29)$$

它表示相对参数变化对应的输出响应，保留 Y 的单位，避免以摄氏温度作相对误差分母带来的零点任意性。有限幅度差商不等于严格导数，也不是统计置信度。

表 S25: 问题 1 终点的单因素扰动结果（160 区间诊断）

参数	倍率	$T_c/^\circ\text{C}$	$T_s/^\circ\text{C}$	C_c	$\bar{C}$
扩散系数	0.9	33.5753	36.7856	2.5500	2.2977
扩散系数	1.1	33.5753	36.7856	2.5500	2.2899
换热系数	0.9	33.2975	36.4088	2.5500	2.2935
换热系数	1.1	33.8234	37.1156	2.5500	2.2935
传质系数	0.9	33.5753	36.7856	2.5500	2.3117
传质系数	1.1	33.5753	36.7856	2.5500	2.2767

含水率单位为 kg/kg；每行仅改变一项系数，其余条件不变。

换热系数由 0.9 倍到 1.1 倍使中心温度由 33.2975 升至 $33.8234 \text{ }^\circ\text{C}$ ；传质系数在相同倍率间变化使平均含水率由 2.3117 降至 2.2767 kg/kg 。扩散倍率改变对中心含水率的短时影响在四位显示下不可见，却仍改变平均含水率。因此只用中心作为灵敏度输出会掩盖表层失水信息。上述扰动表对应无潜热基线。第一问的扩散率不含温度，故改变换热

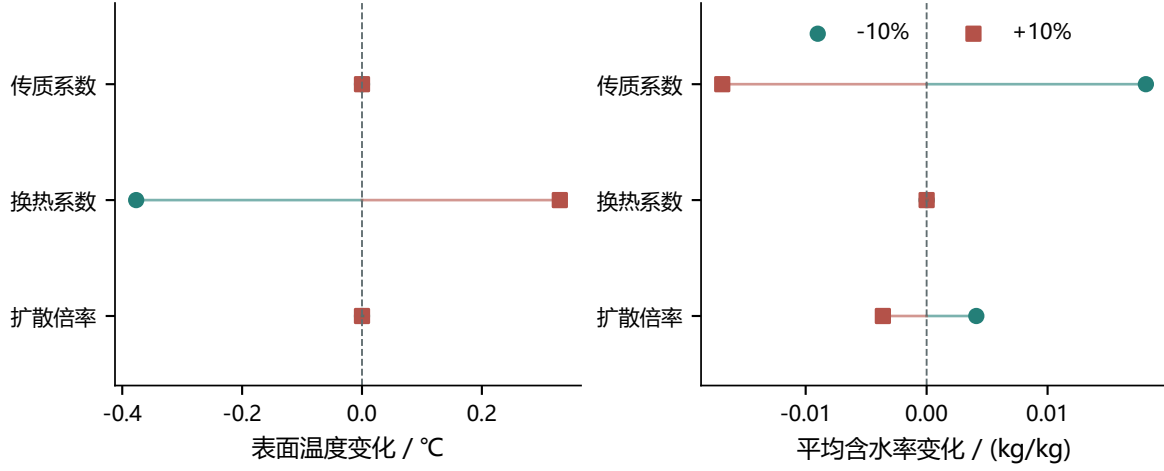


图 20: 第一问三类系数正负 10% 扰动的终点响应

横轴为各自减去未扰动基线的变化，圆点和方点分别为负、正扰动；温度与含水率采用独立单位尺度。这里不画概率误差条。

系数不改变水分方程；加入式(9)的表面潜热后，水分可通过蒸发耗热影响温度，但温度仍不反馈水分。这是单向耦合结构，不能将含水率不变误解为未加入潜热。

A.5 分量误差控制

温度与含水率的数值尺度不同，误差必须按分量归一化。设 e_i 为局部误差估计，分量数 $d = 2(N + 1)$ ，使用

$$s_i = \text{atol}_i + \text{rtol}|y_i|, \quad E_{\text{loc}} = \left[\frac{1}{d} \sum_{i=1}^d \left(\frac{e_i}{s_i} \right)^2 \right]^{1/2}. \quad (30)$$

温度、含水率分别取 $\text{atol}_T = 10^{-8} \text{ } ^\circ\text{C}$ 、 $\text{atol}_C = 10^{-10} \text{ kg/kg}$ ，公共相对容差为 2×10^{-9} 。误差检验和非线性迭代共同决定接受、缩步或重试，步长上限只限制单次推进距离。线性扩散的隐式稳定性不能替代此处的非线性收敛和精度控制，输出间隔 1 s 也不是积分器每次必取的内部步长。

A.6 隐式耦合的残差求解

以一阶后向欧拉特例说明一次隐式更新。给定 $\mathbf{y}_n$ 后，新状态应满足

$$\begin{aligned} \mathbf{R}(\mathbf{y}) &= \mathbf{y} - \mathbf{y}_n - \Delta t \mathbf{f}(t_{n+1}, \mathbf{y}) = \mathbf{0}, \\ [\mathbf{I} - \Delta t \mathbf{J}(\mathbf{y}^{(k)})] \delta \mathbf{y} &= -\mathbf{R}(\mathbf{y}^{(k)}). \end{aligned} \quad (31)$$

其中 $\mathbf{J} = \partial \mathbf{f} / \partial \mathbf{y}$ 为 Jacobian 矩阵， k 为非线性迭代编号。各行只依赖本节点及相邻节点的温湿状态，程序提供该稀疏结构，利用数值 Jacobian 和稀疏分解降低求解成本。生产计算使用 SciPy 变阶 BDF 及 NDF 增强^[8]，上述一阶特例不代表所有内部步的系数。Shampine 与 Reichelt 论文提供刚性求解器的方法背景^[12]；本文未将 MATLAB 求解器作为额外独立复算。

A.7 第二问的参数扰动与输出口径

在变物性主结构下，继续作与第一问相同的三系数正负 10% 扰动，终点改为 3 h，公式与指标沿用式(29)。这六个情景不是概率抽样，幅度也没有被解释为附件的测量误差。

扩散倍率从 0.9 增至 1.1 时，中心含水率从 1.8255 降至 1.7151 kg/kg；提高换热系

表 S26: 问题 2 终点的单因素扰动结果（160 区间诊断）

参数	倍率	$T_c/^\circ\text{C}$	$T_s/^\circ\text{C}$	C_c	$\bar{C}$
扩散系数	0.9	49.8464	49.9646	1.8255	1.4041
扩散系数	1.1	49.8522	49.9680	1.7151	1.3638
换热系数	0.9	49.7987	49.9266	1.7707	1.3847
换热系数	1.1	49.8836	49.9936	1.7626	1.3807
传质系数	0.9	49.8436	49.9615	1.8096	1.4438
传质系数	1.1	49.8548	49.9708	1.7272	1.3280

含水率单位为 kg/kg；每行仅改变一项系数，其余条件不变。

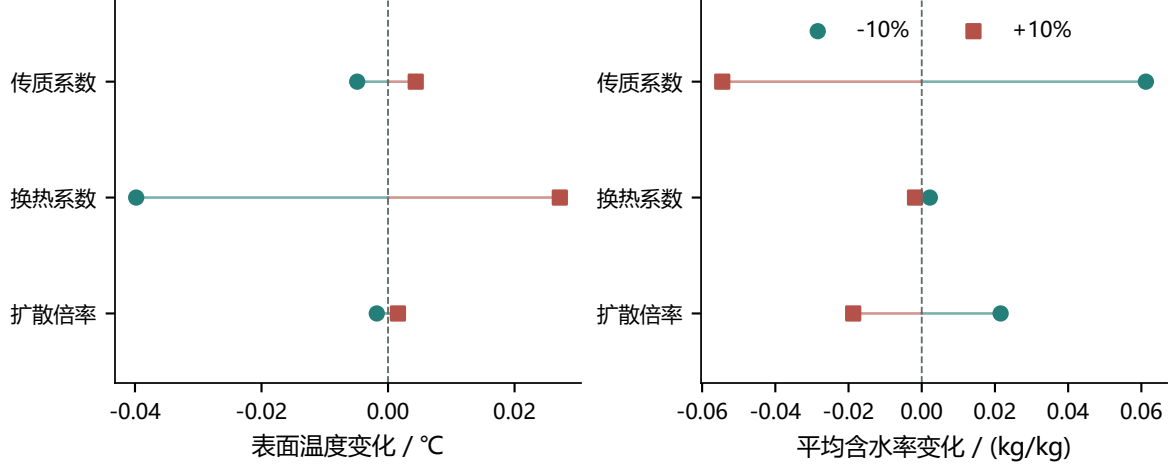


图 21: 第二问变物性耦合下的终点参数扰动响应

扩散和传质扰动通过含水率改变热物性，因而温度也出现小幅反馈；这与第一问的解耦结构形成可核验差别。

数同样略微降低中心含水率，因为升温经 Arrhenius 项促进扩散，但 3 h 时的影响弱于上述扩散扰动。传质系数也改变平均与中心状态，说明内部扩散和表面阻力均不能简单删除。此处结论只针对给定终点和扰动幅度，不能自动外推为后期全程的参数重要性排名。

对逐秒交付还需区分三种时间尺度：附件的 60 s 观测间隔、BDF 自适应内部步长、工作簿的 1 s 采样间隔。采样更密不会创造新的环境观测；积分器局部误差控制也不意味着附件精度提高。对每一输出行都用相同插值器和同一连续数值轨道采样，避免把不同工艺阶段各自重启后拼成一份看似连续的工作簿。

A.8 第三、四问的有限情景停机扩展

作为下游压力分析，令情景集合 $\mathcal{S}$ 由扩散倍率 $\gamma_D \in \{0.9, 1.1\}$ 、4 h 后环境温度偏移 $\delta_T \in \{-0.5, 0.5\}^\circ\text{C}$ 、环境水分偏移 $\delta_C \in \{-0.002, 0.002\} \text{ kg/kg}$ 的八个组合构成。幅度由本研究透明设定，不是实验拟合的误差分布。每个情景仍求同一方程，只有对应参数改变。

综上所述，有限情景的共同停机目标可集中写为

$$\begin{aligned}
 & \min_{\tau} \tau, \\
 & \dot{\mathbf{y}}^{(s)} = \mathbf{f}(t, \mathbf{y}^{(s)}; s), \quad \mathbf{y}^{(s)}(0) = \mathbf{y}_0, \quad s \in \mathcal{S}, \\
 & \max_i C_i^{(s)}(\tau) \leq C_*, \quad s \in \mathcal{S}, \quad \tau \in [0, 72 \text{ h}].
 \end{aligned} \tag{32}$$

在已检查的下降历程上，其候选最早时刻为各情景临界时间的最大值。实际建议额外保

留 60 s 并向上对齐整分钟： $t_S = 60[(\max_s t_*^{(s)} + 60)/60]$ 。采用 320 区间遍历八情景，最慢角点再用 640 区间复算，并将两个网格的较大临界值纳入。最终在统一 t_S 对所有情景回代，而非仅报告各情景重新求得的最优时间。

问题三八情景的临界时间为 52.0106 至 64.2038 h；名义 57.4754 h 停机时四个情景失败。统一停机 64.2333 h 后八个均满足要求，细网格最慢角点的最大含水率为 0.14997256 kg/kg。这是额外约 6.758 h 换取有限已测情景余量的决策扩展，并非真实概率置信区间，也未证明连续参数盒的全域鲁棒性。

第三、四问的全部角点回代见正文图 17，各点由同一固定停机方案求值，不以各情景重新优化后的结果冒充固定方案的稳健性。

A.9 原题口径、替代闭合与解析特例

A.9.1 原式、温标与判据复核

核查时区分原题要求与外部提示清单。后者可用于发现遗漏，但其中的经验答案区间、固定网格数、固定迭代次数和运行时长不能替代误差检验。尤其第二问必须输出全过程温度与含水率，不能仅保留 3 h 后再用只有水分的第三问文件补足；第四问指定的 1.5 cm 列即使全部域外也应保留并解释。原题附录 2–4 中的含水率指数分别为 $-0.89/C$ 、 $-0.45/C$ 和 $-0.30/C$ ；附录 3、4 的温度指数为 $-3850/T_K$ 。这由原页分数线确定，不能凭某个计算时间“接近两三天”反推公式。以 50 °C 为例，误将摄氏数值代入温度指数，会使扩散率约缩至正确值的 5.41×10^{-29} ，而非增大。第二问始终采用附录 3，不按预热与恒温阶段切换物性，也不把第一问末态接作第二问初态。附件 1 观测温度最大值为 50.246 °C，不能把清单中的 50.2 °C 当作精确输入上限。潜热引起冷却也不能单独作为否定扩展的理由：应检验相变位置、质量单位和热量收支。本文保留两套明确闭合的条件答案，不用某个时间区间筛选“正确”结果。

全域判据不能换成均值或表面判据；严格不等式的可行时刻集合通常没有一个取到的最早元素，因此本文先报告临界下确界 t_* ，再报告具有执行余量的可行 t_{out} 。以下回代直接取同一 640 区间轨迹的表面或均值首次达到 0.15 的时刻，并检查当时全域最大值。这类提前停止不仅改变最后一位数字，而是留下明显未达标区域。收缩半径停止变

表 S27: 错误判停准则的同轨迹回代（640 区间）

问题与错误准则	判停时间/h	最大含水率/(kg/kg)	满足全域要求
3: 表面	12.5515	0.4330	否
3: 均值	36.0254	0.1858	否
4: 表面	12.7841	0.6714	否
4: 均值	34.9696	0.1975	否

化同样不能替代水分达标检查。

A.9.2 外推与几何插值的同口径对照

附录并未规定 4 h 后的环境必须取 50 °C 和 0.05 kg/kg。本文将末段均值作为名义闭合，并保持同一物性、网格及最大值判据，分别重算圆整常值、末观测值与半径线性插值。表中均为临界时间，不能与含执行余量的严格停机时间混比较。第三问取末点而非末段均值时，临界时间缩短 0.3046 h；第四问仅改线性半径插值时，临界时间增加 12.61 s。第三问将 h_m 加倍后，临界时间为 54.6718 h，较同网格名义值缩短 2.8022 h。第四问 36 h 虽已达到半径 1.2000 cm，但最大含水率仍为 0.1928 kg/kg，尚未达标。附件 1 观测温度最大值为 50.2460 °C，不能把 50.2 °C 硬编码为风温上限。本组名义 320/640 区间临界时间差不足 0.19 s；正文最终答案仍采用既有更细网格。粗细网格的微小差异可能跨过严格停机时间的 0.0001 h 上取整边界，不能把诊断表的粗网格严格时刻直接覆盖主答案。保形插值与线性插值是不同的几何近似，不由短时单调性决定优劣；这里的比较只

表 S28: 长期环境与半径插值对临界时间的影响（640 区间）

4 h 后环境	第四问半径插值	第三问/h	第四问/h
末段均值	PCHIP	57.4740	51.0885
50 °C / 0.05	PCHIP	57.4723	51.0871
末观测值	PCHIP	57.1694	50.8207
末段均值	线性	57.4740	51.0920
50 °C / 0.05	线性	57.4723	51.0906

量化本数据上的影响。半径的 72 h 观测末值为 1.198 cm，并非恒等于整个 36–72 h 的半径。数值状态的径向、时间单调性在实际采样轨迹上核对，不将其提升为任意时变环境下的无条件规律。

A.9.3 阶跃导热解析特例

为核对扩散时间尺度，另将第一问环境改为从初态起恒定 50 °C，保持常数热物性。这是验证特例，不替代附件 1 驱动的第一问答案。令 $\theta = (50 - T)/22$ ，由分离变量得到径向方程 $(xX')' + \mu^2 xX = 0$ ，轴心正则解为 $J_0(\mu x)$ ；表面 Robin 条件给出 $\mu J_1(\mu) = \text{Bi} J_0(\mu)$ 。将初始常数 1 按权重 x 投影到这些特征函数，得到

$$\theta(r, t) = \sum_{m=1}^{\infty} b_m J_0(\mu_m r / R_0) \exp(-\mu_m^2 \alpha t / R_0^2), \tag{33}$$
$$b_m = \frac{2 J_1(\mu_m)}{\mu_m [J_0(\mu_m)^2 + J_1(\mu_m)^2]}, \quad \text{Bi} = h R_0 / k.$$

其中 J_0, J_1 为第一类 Bessel 函数， μ_m 为上述方程的正根， b_m 为初态的投影系数。取 60 项，在 100、600、1800 s 及 $r = 0, 1, 2$ cm 九个时空点比较；160、320 区间最大温差分别为 1.30×10^{-4} 、 3.48×10^{-5} °C。这为轴心、表面离散与热扩散量级提供独立谱表示核验，不意味着变物性、潜热及收缩的物理闭合均已验证。自适应内部步长与规定输出间隔也分别处理，不能由“隐式稳定”直接推出任意步长均准确。

A.10 潜热扩展的网格、收支与取值情景

表 S29: 含潜热条件扩展的网格与干基收支检查

问题	终点最大温差/°C	临界时间差/s	水量相对残差
1	3.26e-05	–	4.05e-11
2	1.05e-06	–	2.70e-10
3	1.23e-07	0.04373	8.78e-11
4	2.51e-07	0.03760	4.18e-10

温差列取中心与表面两点的较大值；网格为 640/1280 区间。完整机器报告另列共同 60 s 快照与 201 物质位置的场差，不能把终点双点误差称作全场上界。变物性情形核对有效显热率，不声称完整湿物料总能量闭合。同为 320 区间时，以 Radau 替代 BDF 复

表 S30: 潜热取值情景与临界时长（同为 320 区间）

$L_v/$ (MJ/kg)	问题三/h	问题四/h
2.30	60.3665	56.2562
2.45	60.5654	56.5980
2.50	60.6318	56.7119

算第一、三问，第一问终点双点最大温差为 $3.10\text{e-}08\text{ }^{\circ}\text{C}$ ，第三问临界时间差为 $9.151\text{e-}06\text{ s}$ 。两者共享空间离散与物理假设，属于替代时间积分核验，不是独立物理模型验证。另有四问零潜热退化、零蒸发边界及第一问水分不变性共 9 项测试。

B 支撑材料、复现与补充图

B.1 配套支撑材料文件列表

支撑材料根目录的 `result1.xlsx` 至 `result4.xlsx` 为无潜热基线，`latent/` 中的四个同名工作簿为表面潜热条件扩展。`FILE_LIST.csv` 逐文件列出路径、大小和 SHA-256，`SHA256SUMS.txt` 用于解压后的只读校验。工作簿仅压缩冗余 XML 表示，保留全部时刻、单元格数值、格式和域外空值；逐单元等价记录见 `results/workbook_equivalence.json`。

- `code/`: 生产求解器、双模型冷启动、模型比较、端面诊断、灵敏度、潜热及回归程序；`requirements-numerical.txt` 为数值环境。
- `results/`: 历史工作簿生产回执、双模型逐问未舍入结果及等价检查；`results/recheck_20260913/` 另存提交前 BDF/Radau 复算记录。
- `evidence/` 及 `supplement/`: 锁定结果、同口径对照和下游诊断 JSON/CSV；原生大容量场缓存按完整程序重新生成，不以较稀的输出替代逐秒结果表。
- `figures/` 及 `tables/`: 矢量图源、DXF、定稿图件和结果数表；生成式概念图原 PNG 随包保留，其作用范围见 AI 详情。根目录 TeX 文件保留论文排版和流程图定义。
- `sources/`: 已保存的公开方法资料；`README.md`: 输入、运行命令、输出口径和复现限制；AI 工具使用详情。`pdf`: 实际 AI 辅助记录。
- `packaging/`: 工作簿表示压缩及等价检查脚本，仅用于文件组织，不修改物理模型。

官方附件不在支撑包中重复提交，复现时须将附件 1、2 原样放为 `data/input1.xlsx` 及 `data/input2.xlsx`。大型绘图场缓存可用 `code/rebuild_plot_cache.py` 重建；该入口不代替潜热或全部诊断的独立入口。解压后运行 `pythonpackaging/verify_manifest.py` 只读检查文件身份；数值复算、图件重建和 XeLaTeX 编译的命令分别见 `README.md`。主模型计算采用 Python 3.12、NumPy 1.26.4、SciPy 1.13.1 及 `openpyxl`。源工程中 `code/` 包含完整数值核心和独立入口；`evidence/` 包含锁定数表及制图用场数据，`figures/` 包含 PNG 与可编辑 SVG，`tables/` 为程序生成的题目表格。历史冻结场重导出、双模型从零复算、环境与几何对照分别记录，不将三种运行混称为同一次计算。支撑包不重复官方附件和大型场缓存，输入恢复及缓存重建方式见配套说明。

运行以下命令完成回归及小网格检查：

```
python run_all.py --mode smoke --out quick_check
```

该入口只运行无潜热基线：改用 `--mode full --out full_check` 则按其生产网格重算四问、导出工作簿并回读核对，回读需要 `evidence/` 中的锁定记录。输出目录须为新目录。小规模测试不生成或替代主答案。增加 `--redraw` 只从已有证据重建对应本地图表，不重算所有诊断、不生成潜热结果，也不重制 AI 概念图；在线图表另有独立入口。不含历史证据的编译源包应使用下一节双模型冷启动入口进行核心复算。完整代码在下一附录列出；依赖库使用其公开安装包，不要求将第三方库全部抄入论文。

B.2 公式来源、推导与可验证链

本文把公式分为题给经验关系、由守恒和几何推导的方程、数值近似、输出规则四类。经验常数不能由本题数据重新“推导”为理论定律；需要逐项核对题给附录。数值规则则需解释近似条件并用程序验证，不能因有标准方法名称就免检。表 S31 列出每组编号公式的依据和对应验证责任。以式(5)为例，令 $x = -a/C$ ，有 $dx/dC = a/C^2$ ，由 $Ei'(x) = e^x/x$ 得到

$$\frac{d}{dC}(Ce^{-a/C}) = e^{-a/C}(1 + a/C), \quad \frac{d}{dC}\{a Ei(-a/C)\} = -(a/C)e^{-a/C}.$$

表 S31: 编号公式的来源与验证链

公式	来源或推导要点	可执行检查及边界
(1)	两相邻观测点的线性插值; 附件 1 提供数值	输入递增、观测点插值复原; 后 4 h 为外推假设
(2)、(3)	环形体积的输入减输出; 傅里叶/Fick 通量及题给表面系数	轴心正则、表面符号、零驱动力; 固定干物质测度
(4)、(25)	环形积分与轴心泰勒极限; 文献 ^[5] 提供通用方法	内界面抵消、轴心因子 4; 需额外检验时间尺度
(5)、(26)、(6)	状态积分、均值界及近状态极限; 标准指数积分 ^[7] ; 三点高斯近似	系数上下界、通量反对称、近状态相消测试; 非等温项仍近似
(7)	本文选取的表层加密映射	端点 0/1、间距为正; 同区间数对照, 不宣称最优网格
(27)、(28)	轴对称导热及整物体显热平衡	端面谱解与独立时间积分; 零维模型与径向均温对照
(29)	本文定义的有限幅度中心差商	正负扰动分别实际求解; 不是概率区间或全局导数
(12)、(20)	题给附录 3、4; 不是从附件拟合的新公式	原始参数逐项核对; K 温标、正扩散率检查
(31)、(30)	后向欧拉特例的残差和 Newton 线性化; BDF 分量误差控制 ^[8]	残差解释与实际变阶实现分开; 温湿绝对容差具有不同单位
(14)、(15)	“各处低于阈值”的最大值表述与执行舍入规则	全节点最大值、未舍入严格终点; 5 s 只为执行余量
(16)	事件方程在非零斜率处的一阶展开	解释状态差与时间差; 网格精化实际比较事件
(32)	同一停机时间满足全部有限情景的约束	八轨道逐一回代; 有限集合不代表连续盒保证
(17)–(19)	干物质与水连续性相减、链式法则相消	零输运材料制造解; 均匀缩放与空间均匀干密度假设
(21)	权重求和与边界积分; $2 \sum_i w_i = 1$	四问独立边界积分; 不能替代潜热总能量核验
(22)	四格时间的差中之差, 是本文定义的归因量	同网格四组实算; 不解释成随机试验因果效应
(33)	常物性阶跃边界下分离变量及径向正交投影	独立 Bessel 级数与有限体积比较; 仅检验导热特例
(8)–(10)	干基通量的质量换算、表面能量分配与控制体容量; L_v 据文献 ^[2]	零潜热退化、零蒸发边界、第一问水分不变性; 附加相变位置假设
(11)、(13)	常容量能量积分、同含水率下 Arrhenius 比值	第一问独立能量求积; 后问联立实算, 不用温度比推算时间
(23)、(24)	定长度收缩的干物质质量守恒及单位干物质收支	三层网格和边界水量积分; 不证明经验密度与多相焓完全兼容

两式相加即得 $F'_a = e^{-a/C}$, 因而式(5)保持原扩散定律的状态积分。程序进一步用独立数值求积检查反对称、相等状态零通量、近状态极限及正值, 以检验浮点实现。

附录式(34)是相对于同网格基准临界时间定义的有限扰动变化率, 由逐组重新求解的事件时间计算, 不是另行拟合的干燥经验式。

原始工作簿、模型程序、参数配置、机器结果与制表程序形成完整的数据链。主答案来自细网格场, 物理对照与扰动采用单独标明的诊断网格。引用资料分别支持插值、守恒离散、特殊函数、时间积分与坐标定义; 本题经验物性及边界闭合仍须通过独立实验检验。

B.3 计算复现与结果回读

复现分为三个层级：小网格检查用于识别环境或接口错误；生产网格复算用于核对主答案；物理对照、网格消融与情景试验用于解释可靠性范围。前两级由统一入口运行，第三层的完整脚本单独提供，避免把一项快速测试写成所有实验已重跑。

无潜热生产复算使用第一至三问 1280 区间、第四问 2560 区间；含潜热四问均使用 1280 区间，各自从均匀初态重新推进。每套模型的第二问指定 3 h 截面由独立短时求解核对，其全过程工作簿沿用同套模型第三问的完整轨迹。回读程序分别检查未舍入的中心、表面及临界时间，逐行检查工作簿时间严格递增、域内正值、域外空值、末时刻及行数，并把指定时刻的五位置数值与锁定表逐项比较。工作簿保留四位小数时允许的显示舍入误差为 5×10^{-5} ，该误差界只描述格式舍入，不能当作 PDE 误差。

模板复核后，导出统一保留 A1 原表头，第一、二问从 1 s 起，第三、四问从 60 s 起，并追加未落在常规采样点上的严格终点；含水率与温度单元格同时采用四位小数显示格式。第四问固定距离列为 0.0 至 1.9 cm，最后另设“药材表面”，域外为空而非零。前一轮格式修正版位于 `supplement/pitfall_audit/workbooks/`，由冻结细网格场重导出，作为历史证据保留。当前双模型从零复现入口及独立结果目录见附录，不把冻结场重导出当作再次求解 PDE。无潜热与含潜热各自核对各自锁定结果，不混用工作簿。

运行记录保存输入哈希、软件版本、入口命令、退出码和回读报告。修改物性或边界后，旧结果不能仅因文件名相同而继续进入论文；必须先重算并通过同一核对流程。守恒和解析特例属于独立的数学检查，重复运行同一程序则只证明计算可复现，两类证据分别报告。

B.4 双模型从零复现与文件清单

SciPy 的软件方法总述见文献^[13]，具体接口及运行版本仍以本文锁定的文档和环境清单为准。软件论文不能替代本题代码的复现记录。同时复现两种热边界时，在工程根目录运行：

```
python -X utf8 code/reproduce_dual.py --out dual_output --workbooks
```

入口不读取历史数值缓存：分别求解八条生产网格轨道，第一至三问均用 1280 区间，无潜热第四问用 2560 区间，含潜热第四问用 1280 区间；比较所用分辨率与本文各自网格复核一致。输出分为 `baseline/` 和 `latent/` 两个目录，各含 `result1.xlsx` 至 `result4.xlsx`。只运行核心数值复现时，以下六个模块及输入即可，不依赖已有图表或 JSON 答案：

- `code/reproduce_dual.py`：八条轨道、双套导出和回读；`code/reproduce.py`：工作簿写出。
- `code/s2_solver.py`：初边值、经验物性和事件；`code/s2_stable_graded.py`：加密网格及稳定通量。
- `code/latent_extension.py`：潜热边界；`code/experiments.py`：指定时空截面提取。
- `data/input1.xlsx`、`data/input2.xlsx`：分别由官方附件 1、2 原样提供；`requirements-numerical.txt` 列出本次数值运行环境，不含制图依赖。
- `report.json`：软件版本、源码与输入哈希、未舍入答案及逐表回读记录；`baseline/q1.json` 至 `q4.json` 和潜热同名文件保存各问截面。

第二问的 3 h 截面独立计算，全过程逐秒工作簿则由同一方程的第三问轨道输出至严格达标；第三、四问每 60 s 记录并追加真实终点。回读逐行核对时间、域内外位置及有限值，另外抽样比较未舍入场值，不能把抽样数值对照称作逐单元独立复算。`--out` 必须是不存在的新目录；若中断，另选新目录重跑，不复用未完成的结果。

冷启动记录见工程中的 `runs/cold_dual_20260912/output/report.json`。无潜热六张工作表共 423939 条数据行；含潜热六张工作表共 446713 条数据行。八份工作

簿均重新导出并回读，八条轨道均从统一初态独立推进；完整截面及关键终值另与原锁定记录比较。该复现不是对物理假设的实验验证，正式支撑包仍需检查大小与 AI 详情材料。

B.5 总烘干时长的单参数扰动

本节仅对无潜热问题三作单参数扰动，不能将结果解释为含潜热模型的灵敏度。固定环境插值、4 h 后均值延拓、半径和判停阈值，定义扩散前因子 $A_0 = 2.4 \times 10^{-3} \text{ m}^2/\text{s}$ ，活化温度 $B = E_a/\mathcal{R}_g = 3850 \text{ K}$ ，其中 E_a 为活化能、 $\mathcal{R}_g$ 为气体常数。原式可写为 $D = A_0 \exp(-0.45/C) \exp(-B/T_K)$ ，因此保持 $\mathcal{R}_g$ 不变时，扰动 E_a 等价于扰动 B ，不能将 3850 当作以 J/mol 计的活化能直接代入。这里 $A(T) = A_0 \exp(-B/T_K)$ ，与正文温度相关前因子的记号一致。

每次只改变一项： E_a 取基准的 0.95、1.05 倍； A_0, C_0, h 分别取 0.9、1.1 倍； h_m 取 0.1、10 倍。前者是人为设定的有限幅度探索，后者专门检验外部传质阻力，不是测量误差区间。改变 C_0 时，初始水量和依赖 C 的物性都随之改变，其余物性公式不另行冻结。对任一参数 p ，时间响应定义为

$$\Delta_p = 100 \left[\frac{t_*(p)}{t_*^{(0)}} - 1 \right] \%, \quad t_*(p) = \inf\{t : \max_i C_i(t; p) \leq 0.15\}. \quad (34)$$

其中 $t_*^{(0)}$ 为同网格未舍入的基准临界时间；表内严格时间另将临界点继续积分 5 s、向上对齐 0.0001 h，并回代检验。诊断积分上限扩至 240 h，以覆盖传质受阻的长历程；这不改变主解的名义环境闭合或物性。

表 S32: 问题三单参数扰动对总烘干时长的影响

扰动	临界/h	严格/h	变化/%	机理解释
基准	57.4740	57.4755	+0.000	附录 3 物性，固定半径
E_a 降低 5%	35.0099	35.0113	-39.086	温度因子增大，扩散加快
E_a 增加 5%	99.3886	99.3900	+72.928	温度因子减小，扩散变慢
A_0 降低 10%	63.1322	63.1336	+9.845	全程有效扩散系数缩小
A_0 增加 10%	52.8666	52.8680	-8.017	全程有效扩散系数增大
C_0 降低 10%	56.9717	56.9731	-0.874	初始水量及初始物性同时响应
C_0 增加 10%	57.9252	57.9266	+0.785	初始水量及初始物性同时响应
h 降低 10%	57.4828	57.4842	+0.015	主要改变早期升温过程
h 增加 10%	57.4669	57.4683	-0.012	主要改变早期升温过程
h_m 缩至 0.1 倍	144.7702	144.7716	+151.888	外部传质阻力大幅增大
h_m 放大 10 倍	53.5289	53.5304	-6.864	外部阻力降低，仍受内部扩散限制

均采用 640 区间表面加密网格；变化率相对本表未舍入的基准临界时间计算。严格时间为临界后继续 5 s 并向上对齐 0.0001 h，且回代确认全网格最大含水率低于 0.15 kg/kg。全部 11 组的 320/640 网格时间差最大 0.351 s；三组 Radau 及独立通量组装核验的最大差 0.000082 s。

参数不确定性不能靠提高求根精度消除。活化能增加使全程温度因子减小，而扩散前因子按比例改变 D ，两者均会改变后期慢尾部。降低表面传质能力会明显延长总时长；提升表面传质能力却不能解除内部扩散限制。因此，应优先核定扩散经验式及表面交换参数，再评估停机余量，而不是仅收紧事件器容差。初始含水率还会同时改变热物性和扩散率，不能以初始水量比例推算干燥时间。换热系数的结果只说明本组有效显热模型及环境条件下总时长响应较小，**不意味着真实蒸发潜热可以忽略。**

不同参数采用的扰动幅度不同，表中变化率不直接构成全局重要性排名； h_m 十倍变化也不是局部导数。全部组别采用同样的时间容差、网格族、输入和评价器，以 320/640 区间配对比较；基准、活化能增加 5% 和传质系数缩至 0.1 倍三组，另用 Radau 及独立

Gauss 通量组装核验。两条路径共享物理假设，属于数值互证，不是实验验证。表内 640 区间严格基准 57.4755 h 与正文 1280 区间 57.4754 h 相差 0.36 s，来自微小离散差异跨过输出上取整边界；**本表不替换主答案**。

在源工程根目录依次运行以下两条命令，得到逐组 JSON、汇总 CSV、验证报告和本表：

```
python code/time_sensitivity.py --out supplement/time_sensitivity --grid
↪ 320 640 --check
python code/time_sensitivity_report.py
```

程序核对输入及代码哈希，拒绝复用身份不符的缓存；本轮原始输出保留在 `supplement/time_sensitivity/`。正式复现若修改代码或数据，须以 `--out` 指定新目录，再用报告程序的 `--data` 读取该目录。

图 22 采用工程剖面表示实体几何。配套 DXF 的坐标单位为 mm，纵剖矩形为 250 mm×40 mm，端视圆半径 20 mm；论文纵剖图径向放大以便辨认，不应从显示比例反推尺寸。

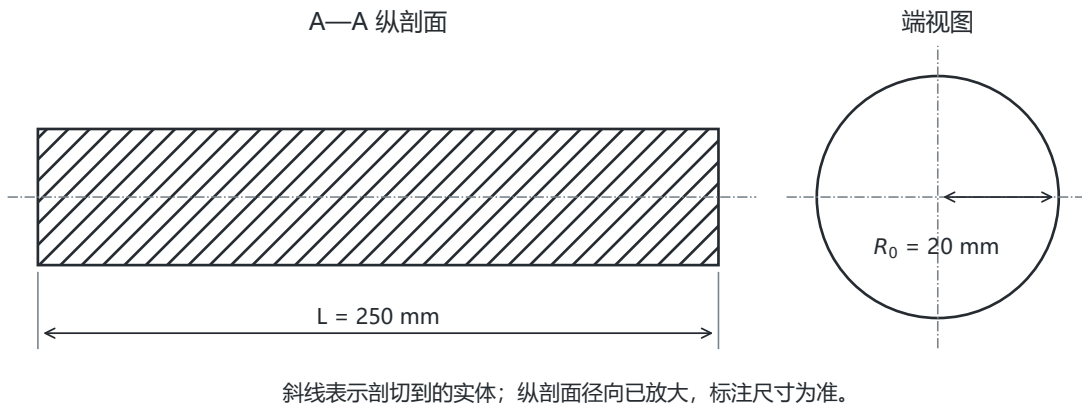


图 22: 圆柱实体的纵剖面与端视尺寸；斜线为剖面符号，不表示材料为空心。

无潜热物理对照与参数扰动由 `code/revision_experiments.py` 生成 JSON，数值图和表据此重绘。工程图由 `engineering_figures.py` 输出可编辑 SVG、PDF、450 dpi PNG 及尺寸 DXF。当前总体图与第一、三、四问流程图实际调用 `source_refinement.tex` 中的 TikZ 定义，第二问由 `ai_visuals.tex` 切换为随包提供的生成式图；二问曲面由同一 TeX 定义读取冻结场数据。`run_all.py --redraw` 只管理已有本地图表脚本，不覆盖上述 TeX 图定义或生成式图片，也不等同于从零复现所有图件。

图 23 把坐标关系与失水过程分开：相同环形标记随半径缩小而向内移动，其归一化位置不变。半径变化改变后续扩散的空间尺度，但不能凭几何变化直接扣减干基含水率。只有使用同一物质参考测度，才能把几何变化与边界失水分别记账。

正文图 7 已给出潜热扩展的能量分配；生成式示意图随源包提供，不作为计算数据。潜热结果由下列程序单独生成，不能用图片反推或填充数值。

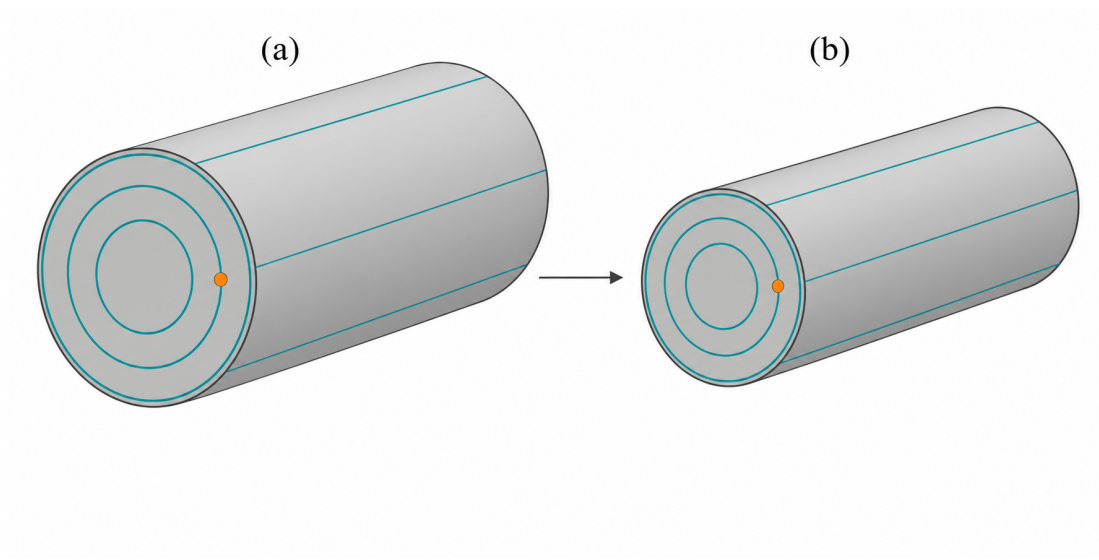


图 23: 均匀径向收缩前后的物质标记示意

(a) 初始几何；(b) 长度不变的径向收缩。青色环与橙色点仅标记对应物质位置，不表示孔隙、含水率等值线或干燥前沿。生成式概念图不按比例，不从透视长度读取收缩量；实际半径仍取附件数据。

C 完整计算程序

C.1 表面潜热扩展与独立结果目录

在论文目录依次执行以下两条命令，生成本文潜热数表与答案：

```
python -X utf8 code/latent_extension.py --tests --n 320 640 1280 --extended
python -X utf8 code/latent_report.py
```

需与下列原求解器、`experiments.py` 及两份输入工作簿一起使用；网格及潜热情景输出位于 `supplement/latent_extension`，包含 60 s 场快照和题目指定截面。该扩展试验入口会保留已有运行文件；要求不读缓存的全过程逐秒 Excel 复现时，使用 `reproduce_dual.py` 及不存在的新目录。第二问含潜热全过程沿用第三问同一动力系统，原基线工作簿不能充当含潜热输出。

```
1 """Conditional surface-evaporation extension; never fit to external answers.
2
3 Run: python -X utf8 code/latent_extension.py --n 320 640
4 Outputs are separate from the locked no-latent simulation files.
5 """
6 import argparse
7 import hashlib
8 import json
9 import platform
10 from pathlib import Path
11
12 import numpy as np
13 import scipy
14 from numpy.polynomial.legendre import leggauss
15
16 from s2_solver import Settings
17 from s2_stable_graded import StableModel
18 from experiments import requested_table
19
20 ROOT = Path(__file__).resolve().parents[1]
21 OUT = ROOT / 'supplement' / 'latent_extension'
22
23
24 class LatentModel(StableModel):
25     def __init__(self, settings, latent_heat=2.45e6):
26         super().__init__(settings)
27         if latent_heat < 0:
28             raise ValueError('Negative latent heat')
29         self.latent_heat = float(latent_heat)
30         rho = self.properties(np.array([28.]), np.array([2.55]))[0][0]
31         self.rho_d0 = float(rho / 3.55)
32
33     def dry_density(self, t):
34         return self.rho_d0 * (.02 / self.radius(t)) ** 2
35
36     def surface_terms(self, t, y):
37         Ta, Ca = self.environment(t)
38         j = self.dry_density(t) * 8e-7 * self.s.hm_scale * (y[-1] - Ca)
39         q = 25 * self.s.h_scale * (Ta - y[self.size-1])
40         return float(j), float(q)
41
42     def rhs(self, t, y):
43         dy = super().rhs(t, y)
44         rho, cp, *_ = self.properties(y[:self.size], y[self.size:])
45         j, _ = self.surface_terms(t, y)
46         dy[self.size-1] -= self.latent_heat * j / (
47             self.radius(t) * self.w[-1] * rho[-1] * cp[-1])
48         return dy
49
50
51 def ledger(model):
52     # Independent temporal quadrature of external fluxes, per kg dry solid.
53     z, weights = leggauss(4)
54     total = np.zeros(3)
55     minimum_T, minimum_j = float('inf'), float('inf')
56     rate_error = 0.
57     for piece in model.pieces:
58         lo, hi = piece.t[:-1], piece.t[1:]
59         ts = ((hi+lo)[: , None]/2 + (hi-lo)[: , None]/2*z).ravel()
```

```

60     ws = ((hi-lo)[:, None]/2*weights).ravel()
61     ys = piece.sol(ts)
62     minimum_T = min(minimum_T, float(ys[:model.size].min()))
63     for idx, t in enumerate(ts):
64         y = ys[:, idx]
65         j, q = model.surface_terms(t, y)
66         minimum_j = min(minimum_j, j)
67         R, rd = model.radius(t), model.dry_density(t)
68         total += ws[idx]*np.array([-2*j/(rd*R), 2*q/(rd*R),
69                                     2*model.latent_heat*j/(rd*R)])
70     for t in np.linspace(0, model.end, 41):
71         y = model.state([t])[0]
72         rho, cp, *_ = model.properties(y[:model.size], y[model.size:])
73         dy = model.rhs(t, y)
74         j, q = model.surface_terms(t, y)
75         lhs = 2*np.dot(model.w, rho*cp*dy[:model.size])/model.dry_density(t)
76         rhs = 2*(q-model.latent_heat*j)/(model.radius(t)*model.dry_density(t))
77         rate_error = max(rate_error, abs(lhs-rhs))
78     final = model.state([model.end])[0]
79     mass_change = float(2*np.dot(model.w, final[model.size:]-2.55))
80     result = dict(water_change_kg_per_kg=mass_change,
81                  integrated_water_flux=total[0],
82                  water_relative_residual=abs(mass_change-total[0])/abs(mass_change),
83                  convective_heat_J_per_kg_dry=total[1],
84                  latent_heat_J_per_kg_dry=total[2],
85                  local_effective_heat_rate_residual_W_per_kg_dry=rate_error,
86                  minimum_sampled_temperature_C=minimum_T,
87                  minimum_sampled_evaporation_flux=minimum_j)
88     if model.kind == 1:
89         sensible = float(2*820*2600*np.dot(model.w, final[:model.size]-28)/model.rho_d0)
90         result['sensible_energy_J_per_kg_dry'] = sensible
91         result['energy_relative_residual'] = abs(sensible-total[1]+total[2])/max(abs(total[1]),
92                                         ↪ abs(total[2]))
92     return result
93
94
95 def run_one(q, n, lv=2.45e6, method='BDF', audit=True):
96     name = f'q{q}_n{n}_L{lv:.0f}_{method}'
97     dest = OUT / name
98     if (dest/'run.json').exists():
99         print('cached', name, flush=True)
100         return
101     model = LatentModel(Settings(question=q, n=n, method=method), lv).run()
102     dest.mkdir(parents=True, exist_ok=True)
103     times = np.unique(np.r_[np.arange(0, model.end, 60.), model.end])
104     values = model.state(times)
105     np.savez_compressed(dest/'fields.npz', time_s=times, xi=model.x,
106                        T=values[:model.size].T, C=values[model.size:].T)
107     meta = dict(model.meta, latent_heat_J_kg=lv, dry_density_initial=model.rho_d0,
108                dry_density_end=model.dry_density(model.end),
109                requested_rows=requested_table(model),
110                energy_closure='surface_evaporation_with_effective_sensible_capacity',
111                external_answer_used_for_fitting=False)
112     if audit:
113         meta['ledger'] = ledger(model)
114     if q >= 3:
115         ts = [model.critical-1, model.critical, model.end]
116         meta['threshold_bracket'] = [dict(t_s=t, g=float(model.state([t])[model.size:, 0].max()-15)) for
117                                     ↪ t in ts]
117     (dest/'run.json').write_text(json.dumps(meta, indent=2), encoding='utf-8')
118     print(name, 'T=', meta['center_T'], meta['surface_T'],
119           'critical_h=', meta['critical_h'], 'strict_h=', meta['end_h'], flush=True)
120
121
122 def tests():
123     checks = []
124     for q in (1, 2, 3, 4):
125         s = Settings(question=q, n=40)
126         base, ext = StableModel(s), LatentModel(s, 0.)
127         y = np.r_[np.linspace(28, 35, 41), np.linspace(2.55, 1.8, 41)]
128         checks.append(dict(name=f'zero_latent_rhs_q{q}', passed=np.array_equal(base.rhs(600, y),
129                                     ↪ ext.rhs(600, y))))
129         ext = LatentModel(s)
130         y[-1] = ext.environment(600)[1]
131         checks.append(dict(name=f'zero_evaporation_q{q}', passed=np.array_equal(base.rhs(600, y),
132                                     ↪ ext.rhs(600, y))))
132     a = StableModel(Settings(question=1, n=80)).run()
133     b = LatentModel(Settings(question=1, n=80)).run()
134     ts = np.linspace(0, 1800, 101)

```

```

135     error = float(np.max(np.abs(a.state(ts)[a.size:]-b.state(ts)[b.size:])))
136     checks.append(dict(name='q1_moisture_invariance', error=error, passed=error < 2e-7))
137     OUT.mkdir(parents=True, exist_ok=True)
138     (OUT/'tests.json').write_text(json.dumps(checks, indent=2), encoding='utf-8')
139     if not all(c['passed'] for c in checks):
140         raise AssertionError(checks)
141     print('tests', len(checks), 'passed', flush=True)
142
143
144 def main():
145     p = argparse.ArgumentParser()
146     p.add_argument('--n', type=int, nargs='+', default=[320, 640])
147     p.add_argument('--q', type=int, nargs='+', default=[1, 2, 3, 4])
148     p.add_argument('--lv', type=float, default=2.45e6)
149     p.add_argument('--method', default='BDF')
150     p.add_argument('--tests', action='store_true')
151     p.add_argument('--extended', action='store_true')
152     args = p.parse_args()
153     OUT.mkdir(parents=True, exist_ok=True)
154     if args.tests:
155         tests()
156     for n in args.n:
157         for q in args.q:
158             run_one(q, n, args.lv, args.method)
159     if args.extended:
160         for lv in (2.30e6, 2.50e6):
161             for q in (3, 4):
162                 run_one(q, 320, lv)
163             for q in (1, 3):
164                 run_one(q, 320, method='Radau')
165     identity = {str(path.relative_to(ROOT)): hashlib.sha256(path.read_bytes()).hexdigest()
166                for path in [Path(__file__), ROOT/'code/s2_solver.py', ROOT/'code/s2_stable_graded.py',
167                             ↪ ROOT/'data/input1.xlsx', ROOT/'data/input2.xlsx']}
168     (OUT/'environment.json').write_text(json.dumps(dict(python=platform.python_version(),
169                                                         numpy=np.__version__, scipy=scipy.__version__, sha256=identity), indent=2), encoding='utf-8')
169
170
171 if __name__ == '__main__':
172     main()

```



```

1 """Backfill tables and answers from computed latent-extension records only."""
2 import json
3 from pathlib import Path
4 import numpy as np
5 from latent_effect_tables import write_effect_tables
6 from answer_table_pairs import write_pairs
7
8 ROOT = Path(__file__).resolve().parents[1]
9 OUT = ROOT/'supplement/latent_extension'
10
11
12 def read(q, n=1280, lv=2450000, method='BDF'):
13     return json.loads((OUT/f'q{q}_n{n}_L{lv}_{method}/run.json').read_text(encoding='utf-8'))
14
15
16 def write(name, value):
17     (ROOT/name).write_text(value+'\n', encoding='utf-8')
18
19
20 def table(rows, field, positions, q, caption):
21     times = sorted(set(row['time_s'] for row in rows))
22     time_label = '时间/s' if q == 1 else '时间/h'
23     lines = [r'\begin{table}[htbp]\centering\small',
24              r'\caption{' + caption + r'}\label{tab:latent-q'+str(q)+'-'+field+'}',
25              r'\begin{tabular}' + r'{' + r'*len(positions) + r'}' + '\toprule',
26              time_label + ' & '+' & '.join('实际表面' if p == 'surface' else f'{p:g} cm' for p in
27              ↪ positions) + r'\\midrule']
27     md = ['| '+time_label+' | '+' | '.join('实际表面' if p == 'surface' else f'{p:g} cm' for p in
28     ↪ positions)+' | ',
29           '|---|'+':-' + ':'*len(positions)]
29     for t in times:
30         values = []
31         for p in positions:
32             row = next(r for r in rows if r['time_s'] == t and r['r_cm'] == p)
33             value = row[field]
34             values.append('--' if value is None else f'{value:.4f}')
35         time_value = f'{t:.0f}' if q == 1 else f'{t/3600:.4f}'
36         lines.append(time_value+' & '+' & '.join(values)+r'\\')
37         md.append('| '+time_value+' | '+' | '.join(values)+' | ')

```

```

38     lines += [r'\bottomrule\end{tabular}', r'\end{table}']
39     return '\n'.join(lines), '\n'.join(md)
40
41
42 def field_difference(q):
43     a = np.load(OUT/f'q{q}_n640_L2450000_BDF/fields.npz')
44     b = np.load(OUT/f'q{q}_n1280_L2450000_BDF/fields.npz')
45     common = np.intersect1d(a['time_s'], b['time_s'])
46     ia, ib = np.searchsorted(a['time_s'], common), np.searchsorted(b['time_s'], common)
47     x = np.linspace(0, 1, 201)
48     errors = {}
49     for field in ('T', 'C'):
50         ax, bx, av, bv = a['xi'], b['xi'], a[field], b[field]
51         errors[field] = max(float(np.max(np.abs(np.interp(x, ax, av[i]) - np.interp(x, bx, bv[j])))) for i,
52                             ↪ j in zip(ia, ib))
53     a.close()
54     b.close()
55     return dict(sample_times=len(common), sample_positions=201, **errors)
56
57 def main():
58     base = {r['question']: r['meta'] for r in
59             ↪ json.loads((ROOT/'evidence/answers.json').read_text(encoding='utf-8'))}
60     runs = {q: read(q) for q in range(1, 5)}
61     md = ['# 四问答更新: 无潜热基线与表面蒸发扩展', '',
62           '本文件数值来自独立运行的程序输出。无潜热结果仍为原 RESULTS_LOCK 口径; 含潜热结果以侧表面蒸发、
63           ↪ 干物质质量换算和有效显热容量为条件, 不是已由实验确认的唯一答案。未生成或编译PDF。', '',
64           '## 假设与公式', '',
65           '逐问并列差值与网格说明见[潜热影响逐问比较](潜热影响逐问比较.md)。该补表从未舍入机器记录生成,
66           ↪ 不替换本文件的两套锁定答案。', '',
67           '- 初态仍为 28 °C、2.55 kg/kg; 第二问全程附录 3, 第四问附录 4 及原 PCHIP 半径。',
68           '- 蒸发限定在侧表面; 端面仍绝热且无传质, 不重复添加内部潜热项。',
69           '-  $\rho_d = \rho(C0)/(1+C0)$ ; 定半径时不变, 收缩时  $\rho_d = \rho_d0*(R0/R)^2$ 。',
70           '-  $j_v = \rho_d h_m (C_s - C_a)$ , 单位 kg/(m²·s)。  $-kT_r = h*(T_s - T_a) + L_v j_v$ 。',
71           '-  $L_v = 2.45e6$  J/kg, 取值依据[FAO物理参数附录](https://www.fao.org/4/X0490E/x0490e0k.htm),
72           ↪ 未根据参考答案拟合。',
73           '- 经验湿密度用于热容量, 与收缩干密度仅在初态作质量换算, 不能据此声称完整多相热力学已闭合。', '',
74           '## 直接答案', '', '| 问题与指标 | 不含潜热 | 含潜热条件扩展 |', '|---|---|---|']
75     comparisons = []
76     for q, m in runs.items():
77         b = base[q]
78         if q <= 2:
79             for key, label in [('center_T', '中心温度/°C'), ('surface_T', '表面温度/°C'), ('center_C',
80             ↪ '中心含水率/(kg/kg)'), ('surface_C', '表面含水率/(kg/kg)')]:
81                 md.append(f'| 问题 {q}, {label} | {b[key]:.4f} | **{m[key]:.4f}* |')
82             answer = f'当考虑上述表面蒸发潜热时, \textbf{{'+('1800 s' if q == 1 else '3
83             ↪ h')}}中心/表面温度为{m["center_T"]:.4f}/{m["surface_T"]:.4f} °C,
84             ↪ 含水率为{m["center_C"]:.4f}/{m["surface_C"]:.4f} kg/kg}}。两种答案对应不同热边界,
85             ↪ 不能混用数据文件。'
86         else:
87             for key, label in [('critical_h', '临界时间/h'), ('end_h', '严格输出时间/h')]:
88                 md.append(f'| 问题 {q}, {label} | {b[key]:.4f} | **{m[key]:.4f}* |')
89             answer = f'当考虑上述表面蒸发潜热时, \textbf{{临界时间为{m["critical_h"]:.4f} h,
90             ↪ 严格输出时间为{m["end_h"]:.4f} h}}, 未舍入最大含水率为{m["max_C_at_end"]:.10f} kg/kg。'
91             if q == 4:
92                 answer += f'结束半径为\textbf{{m["end_radius_cm"]:.4f} cm}}。'
93                 answer += '该结果仅对应名义参数, 不能视为原八角点方案的含潜热复验。'
94             write(f'tables/latent_q{q}_answer.tex', answer)
95             coarse = read(q, 640)
96             comparison = dict(question=q,
97                               center_T_difference=abs(m['center_T'] - coarse['center_T']),
98                               surface_T_difference=abs(m['surface_T'] - coarse['surface_T']),
99                               critical_difference_s=None if q < 3 else abs(m['critical_s'] - coarse['critical_s']),
100                               common_snapshot_difference=field_difference(q))
101             comparisons.append(comparison)
102     md += ['# 严格输出在临界根后至少延时5 s, 再向上舍入到0.0001 h, 并用未舍入状态检查所有节点。
103           ↪ 四位小数报告格式, 不是对真实过程的精度承诺。', '', '## 逐问计算结果', '',
104           rd4 = (760+90*2.55)/3.55*(2/runs[4]['end_radius_cm'])*2
105           empirical4 = 760+90*runs[4]['center_C']
106           mass4 = rd4*(1+runs[4]['center_C'])
107           md += [f'密度闭合反查: Q4含潜热严格终点的干密度为{rd4:.4f} kg/m³, 中心经验密度为{empirical4:.4f} kg/m³,
108           ↪ 而按(1+C)rho_d得到{mass4:.4f} kg/m³。两种总体密度解释不相容; 当前rho*cp仅作为题给有效容量,
109           ↪ 不能称为完整多相热力学。', '', ]
110     for q, m in runs.items():
111         b = base[q]
112         if q <= 2:
113             text = f'1280区间计算得到中心/表面温度{m["center_T"]:.4f}/{m["surface_T"]:.4f} °C;
114             ↪ 相对无潜热基线分别降低{b["center_T"] - m["center_T"]:.4f}/{b["surface_T"] - m["surface_T"]:.4f}
115             ↪ °C。中心/表面含水率为{m["center_C"]:.4f}/{m["surface_C"]:.4f} kg/kg。'

```

```

102     else:
103         delta = m['critical_h']-b['critical_h']
104         text = f'1280区间计算得到临界时间{m["critical_h"]:.8f} h, 严格输出时间{m["end_h"]:.4f} h,
        ↳ 最大含水率为{m["max_C_at_end"]:.10f} kg/kg。相对无潜热临界时间延长{delta:.4f} h
        ↳ ( $\{100*\delta/b["critical_h"]:.2f\}\%$ )。'
105     check = comparisons[q-1]
106     text += f'640/1280区间的终点中心、表面温度差为{check["center_T_difference"]:.2e}、
        ↳ {check["surface_T_difference"]:.2e} °C'
107     if q >= 3:
108         text += f', 临界时间差为 {check["critical_difference_s"]:.5f} s'
109     text += '。这属于数值复核, 不是物理模型的实测误差。'
110     if q == 1:
111         l = m['ledger']
112         text += f'独立时间求积得到对流供热{1["convective_heat_J_per_kg_dry"]/1000:.3f} kJ/kg干物质、
        ↳ 蒸发耗热{1["latent_heat_J_per_kg_dry"]/1000:.3f} kJ/kg干物质,
        ↳ 显热变化{1["sensible_energy_J_per_kg_dry"]/1000:.3f} kJ/kg干物质,
        ↳ 能量相对残差为{1["energy_relative_residual"]:.2e}。因此本条件下的冷却有能量账本支持,
        ↳ 但尚不能据此判定真实药材表面一定达到该温度。'
113     md += [f'### 问题 {q}', '', text.replace(r'\%', '%'), '']
114     summary_text = text[text.index('640/1280 区间'):]
115     write(f'tables/latent_q{q}_summary.tex', summary_text +
        ↳ f'完整截面列于本问答案表\ref{{tab:latent-q{q}}-} + ('T_C' if q <= 2 else 'C_kg_kg') + '}。')
116     tex = []
117     fields = ['T_C'] if q == 1 else ['T_C', 'C_kg_kg'] if q == 2 else ['C_kg_kg']
118     positions = [0., .5, 1., 1.5, 2.] if q != 4 else [0., .5, 1., 1.5, 2., 'surface']
119     for field in fields:
120         caption = f'问题{q}表面蒸发扩展的'+('温度 (°C)' if field == 'T_C' else '干基含水率 (kg/kg)')
121         t, markdown = table(m['requested_rows'], field, positions, q, caption)
122         tex.append(t)
123         md += [caption, '', markdown, '']
124     write(f'tables/latent_q{q}_results.tex', '\n\n'.join(tex))
125     validation = [r'\begin{table}[htbp]\centering\small',
126                 r'\caption{含潜热条件扩展的网格与干基收支检查}\label{tab:latent-validation}',
127                 r'\begin{tabular}{crrr}\toprule',
128                 r' 问题 & 终点最大温差/°C & 临界时间差/s & 水量相对残差\\midrule']
129     for c in comparisons:
130         q = c['question']
131         event = '--' if q < 3 else f'{c["critical_difference_s"]:.5f}'
132         validation.append(f'{q} & {max(c["center_T_difference"], c["surface_T_difference"]):.2e} &
        ↳ {event} & {runs[q]["ledger"]["water_relative_residual"]:.2e}+r'\')
133     validation += [r'\bottomrule\end{tabular}\end{table}',
134                 '温差列取中心与表面两点的较大值; 网格为640/1280区间。完整机器报告另列共同60 s快照与201物质位置的场差,
        ↳ 不能把终点双点误差称作全场上界。变物性情形核对有效显热率, 不声称完整湿物料总能量闭合。']
135     sensitivity = []
136     for lv in (2300000, 2450000, 2500000):
137         sensitivity.append(dict(latent_heat=lv, q3_h=read(3, 320, lv)['critical_h'], q4_h=read(4, 320,
        ↳ lv)['critical_h']))
138     validation += [r'\begin{table}[htbp]\centering\small',
139                 r'\caption{潜热取值情景与临界时长 (同为 320 区间)}\label{tab:latent-sensitivity}',
140                 r'\begin{tabular}{crrr}\toprule', r'$Lv$/(MJ/kg) & 问题三/h & 问题四/h\\midrule']
141     md += ['## 潜热取值情景', '', '| Lv/(MJ/kg) | 第三问临界/h | 第四问临界/h |', '|---|---|---|']
142     for s in sensitivity:
143         validation.append(f'{s["latent_heat"]/1e6:.2f} & {s["q3_h"]:.4f} & {s["q4_h"]:.4f}+r'\')
144         md.append(f'| {s["latent_heat"]/1e6:.2f} | {s["q3_h"]:.4f} | {s["q4_h"]:.4f} |')
145     validation += [r'\bottomrule\end{tabular}\end{table}']
146     independent = []
147     for q in (1, 3):
148         a, b = read(q, 320), read(q, 320, method='Radau')
149         independent.append(dict(question=q,
        ↳ endpoint_temperature_difference=max(abs(a['center_T']-b['center_T']),
        ↳ abs(a['surface_T']-b['surface_T'])),
        ↳ event_difference_s=None if q == 1 else abs(a['critical_s']-b['critical_s'])))
150     validation.append(f'同为320区间时, 以Radau替代BDF复算第一、三问,
        ↳ 第一问终点双点最大温差为{independent[0]["endpoint_temperature_difference"]:.2e} °C.
        ↳ 第三问临界时间差为{independent[1]["event_difference_s"]:.3e} s。两者共享空间离散与物理假设,
        ↳ 属于替代时间积分核验, 不是独立物理模型验证。另有四问零潜热退化、零蒸发边界及第一问水分不变性共9项测试。')
152     write('tables/latent_validation.tex', '\n\n'.join(validation))
153     external = [dict(q=1, metric='center_T', reference=11.1867), dict(q=1, metric='surface_T',
        ↳ reference=12.5943),
        ↳ dict(q=2, metric='center_T', reference=30.9795), dict(q=2, metric='surface_T',
        ↳ reference=31.6111),
        ↳ dict(q=3, metric='critical_h', reference=60.46750514), dict(q=4, metric='end_h',
        ↳ reference=56.4362)]
154     md += ['', '## 与用户提供截图的比较', '', '截图只有数字, 没有方程、潜热取值、质量换算或源码; 来源未核实,
        ↳ 不能称为官方标准答案。下面严格区分第三问临界根与第四问严格时间。', '']
155     ' | 指标 | 本次计算 | 截图参考 | 本次减参考 |', '|---|---|---|---|']
156     for e in external:
157         value = runs[e['q']][e['metric']]
158         e.update(computed=value, difference=value-e['reference'])
159

```

```

161         md.append(f'| Q{e["q"]} {e["metric"]} | {value:.8f} | {e["reference"]:.8f} |  

    ↳ {value-e["reference"]:.8f} |')
162 md += [' ', '差异明显大于本轮网格误差，不能归为显示舍入。潜热数值、干物质密度、潜热位置、  

    ↳ 长期环境外推和收缩质量换算都可能造成模型差异；没有对方公式时不能确定是哪一项，也未反向调参逼近截图。', ' ',  

163     '## 验证与复现', ' ', f'- 9 项结构测试，详见 `supplement/latent_extension/tests.json`。',  

164     '- 主解320/640/1280三层网格，另有四个潜热情景和两次Radau复算，共18次主/对照求解；测试中另运行第一问两次。  

    ↳ ',  

165     '- 保存JSON截面、每60 s原生网格NPZ快照、环境版本与源码/输入哈希；双模型工作簿冷启动状态与入口见下节。',  

166     '- 第三问与第二问采用同一动力系统，第三问 NPZ 轨道同时给出第二问含潜热长期演化。',  

167     '- 不重新渲染 PDF，故本轮没有排版/页数/逐页视觉验收。', ' ', '``powershell`,  

168     'python -X utf8 code/latent_extension.py --tests --n 320 640 1280 --extended',  

169     'python -X utf8 code/latent_report.py', '``', '`,  

170     '机器检查只支持实现、收支和离散一致性；气固平衡、真实相变位置、经验密度兼容性 & 端面潜热效应仍未经实验验证。']  

171 cold_path = ROOT/'runs/cold_dual_20260912/output/report.json'  

172 md += [' ', '## 双模型从零复现', ' ',  

173     'python -X utf8 code/reproduce_dual.py --out dual_output --workbooks', ' ',  

174     '输出目录必须不存在。两套结果分入baseline/latent，各生成result1--4.xlsx；Q2为全过程逐秒温湿数据，  

    ↳ Q3/4为60 s水分数据并追加严格终点。源码不读取锁定答案来计算。']  

175 if cold_path.exists():  

176     cold = json.loads(cold_path.read_text(encoding='utf-8'))  

177     if cold['ready']:  

178         md += [' ', '本轮已经在仅有代码和两份输入的新目录完成八条生产网格轨道复算、八份工作簿导出和回读。  

    ↳ 实际文件: `runs/cold_dual_20260912/output/`。', ' ',  

179         '| 模型 | Q2 结束/h | Q2 每张表行数 | 六张工作表数据行合计 |', ' |---|---|---|---|']  

180         receipt = []  

181         for name, family in cold['cases'].items():  

182             rows = sum(c['workbook']['rows']*len(c['workbook']['verification']['sheets']) for c in  

    ↳ family.values())  

183             md.append(f'| {name} | {family["3"]["meta"]["end_h"]:.4f} |  

    ↳ {family["2"]["workbook"]["rows"]} | {rows} |')  

184             receipt.append(('无潜热' if name == 'baseline' else '含潜热')+f'六张工作表共{rows}条数据行')  

185             text = '冷启动记录见工程中的\\texttt{runs/cold\\_dual\\_20260912/output/report.json}。' + '  

    ↳ '.join(receipt)+'。八份工作簿均重新导出并回读，八条轨道均从统一初态独立推进；  

    ↳ 完整截面及关键终值另与原锁定记录比较。该复现不是对物理假设的实验验证，  

    ↳ 正式支撑包仍需检查大小与AI详情材料。'  

186             write('tables/cold_dual_receipt.tex', text)  

187     else:  

188         md += [' ', '本轮冷启动尚未形成完成报告，不能据此声称双套工作簿已通过。']  

189 write(' 答案更新 _ 潜热对照.md', '\n'.join(md))  

190 matrix = [' # 本轮逐问答案与证据矩阵', ' ',  

191     '无潜热为原有效模型主口径；含潜热为侧表面蒸发条件扩展，不能混用工作簿。', ' ',  

192     '| 问题 | 模型与任务 | 无潜热直接答案 | 含潜热直接答案 | 输出及验证 |', ' |---|---|---|---|']  

193 tasks = {1: '附录 2，恒热物性、非线性水分扩散；七时刻五位置',  

194         2: '初态起统一附录 3；六个半小时时刻、全过程逐秒输出',  

195         3: '附录 3 同一动力系统、全域最大值临界与严格判停',  

196         4: '附录 4、PCHIP 半径、干基物质坐标与当前表面'}  

197 evidence = {1: 'result1 两表：网格、Bessel 导热特例、端面导热、能量账本和水分不变量',  

198             2: 'result2 两表至各自严格终点；联立反馈、冻结物性、网格与字段回读',  

199             3: 'result3 每 60 s 与严格末行；全节点回代、网格、单因素、潜热情景',  

200             4: 'result4 固定列加实际表面、域外留空；质量测度、零输运、四格、潜热网格'}  

201 for q in range(1,5):  

202     b,m = base[q],runs[q]  

203     if q <= 2:  

204         values = [f'Tc/Ts={r["center_T"]:.4f}/{r["surface_T"]:.4f} °C;  

    ↳ Cc/Cs={r["center_C"]:.4f}/{r["surface_C"]:.4f} kg/kg' for r in (b,m)]  

205     else:  

206         values = [f' 临界 {r["critical_h"]:.4f} h; 严格 {r["end_h"]:.4f} h' for r in (b,m)]  

207     matrix.append(f'| {q} | {tasks[q]} | {values[0]} | {values[1]} | {evidence[q]} |')  

208 matrix += [' ', '证据源: evidence/answers.json、supplement/latent_extension/comparison.json;  

    ↳ 最新冷启动身份与未舍入对照见audit/SYNC_CHECK.json。',  

209     '不含潜热八角点方案不适用于未经回代的含潜热情景；数值复现不等于物理实验、完整密度闭合或PDF验收。']  

210 write('audit/ANSWER_MATRIX.md', '\n'.join(matrix))  

211 result = dict(runs=runs, grid_comparison=comparisons, sensitivity=sensitivity,  

212             alternative_integrator=independent, external_unverified_comparison=external,  

213             baseline_source='evidence/answers.json',  

    ↳ status='conditional_extension_not_experimentally_validated')  

214 (OUT/'comparison.json').write_text(json.dumps(result, ensure_ascii=False, indent=2),  

    ↳ encoding='utf-8')  

215 write_effect_tables(ROOT)  

216 write_pairs(ROOT)  

217 print('Generated four question tables, answers, verification tables and Markdown; no PDF.')  

218  

219  

220 if __name__ == '__main__':  

221     main()

```

逐问潜热并列表由下列程序读取未舍入记录生成，并核对独立冷启动结果。仅重建这四张对照表可运行 `code/latent_effect_tables.py`；该操作不重新求解微分方程，所

需记录与来源哈希见 `supplement/latent_effect_tables/report.json`。

```
1 """Build paired latent/no-latent tables from frozen, unrounded records."""
2 import argparse
3 import hashlib
4 import json
5 import math
6 from pathlib import Path
7
8 ROOT = Path(__file__).resolve().parents[1]
9 FIELDS = [('center_T', '中心温度', '°C'), ('surface_T', '表面温度', '°C'),
10          ('center_C', '中心含水率', 'kg/kg'), ('surface_C', '表面含水率', 'kg/kg')]
11
12
13 def digest(path):
14     return hashlib.sha256(path.read_bytes()).hexdigest()
15
16
17 def read_records(root):
18     sources = [root / 'evidence/answers.json']
19     baseline = {a['question']: a['meta'] for a in json.loads(sources[0].read_text(encoding='utf-8'))}
20     pairs = {}
21     for q in range(1, 5):
22         path = root / f'supplement/latent_extension/q{q}_n1280_L2450000_BDF/run.json'
23         sources.append(path)
24         latent = json.loads(path.read_text(encoding='utf-8'))
25         b = baseline[q]
26         if latent['latent_heat_J_kg'] != 2450000 or latent['external_answer_used_for_fitting']:
27             raise ValueError(f'Q{q}: unexpected latent configuration')
28         if {k: v for k, v in b['settings'].items() if k != 'n'} != {
29             k: v for k, v in latent['settings'].items() if k != 'n':
30             raise ValueError(f'Q{q}: settings differ beyond the declared mesh')
31         if (b['settings']['n'], latent['settings']['n']) != ((2560, 1280) if q == 4 else (1280, 1280)):
32             raise ValueError(f'Q{q}: unexpected production meshes')
33         for name, record in [('baseline', b), ('latent', latent)]:
34             cold_path = root / f'runs/cold_dual_20260912/output/{name}/q{q}.json'
35             sources.append(cold_path)
36             cold = json.loads(cold_path.read_text(encoding='utf-8'))['meta']
37             for key in ['center_T', 'surface_T', 'center_C', 'surface_C', 'end_h', 'end_radius_cm'] + (
38                 ['critical_h'] if q >= 3 else []):
39                 if not math.isclose(record[key], cold[key], rel_tol=0, abs_tol=1e-7):
40                     raise ValueError(f'Q{q}/{name}/{key}: cold-run mismatch')
41                 if q >= 3 and not record['max_C_at_end'] < .15:
42                     raise ValueError(f'Q{q}/{name}: strict endpoint invalid')
43                 if q <= 2 and b['end_s'] != latent['end_s']:
44                     raise ValueError(f'Q{q}: unequal comparison times')
45             pairs[q] = (b, latent)
46     return pairs, {str(p.relative_to(root)): digest(p) for p in sources}
47
48
49 def signed(value):
50     return '0.0000' if abs(value) < .00005 else f'{{value:+.4f}}'
51
52
53 def write_effect_tables(root=ROOT, output_root=None):
54     root = Path(root)
55     out = Path(output_root) if output_root else root
56     pairs, sources = read_records(root)
57     (out / 'tables').mkdir(parents=True, exist_ok=True)
58     (out / 'supplement/latent_effect_tables').mkdir(parents=True, exist_ok=True)
59     report = dict(source_sha256=sources, latent_heat_J_kg=2450000,
60                  difference='latent minus baseline, before rounding', questions=[])
61     markdown = ['# 蒸发潜热: 四问并列结果', '',
62                '本表整理已锁定的两套模型输出, 并以独立冷启动记录核对; 本轮未重新求解生产 PDE.', '',
63                '修正方式: `q_in = h*(T_a-T_s) - chi*L_v*j_v`, 其中`chi=0/1` 分别对应不计/计入潜热;',
64                ' `j_v=rho_d*h_m*(C_s-C_a)`, `L_v=2.45e6 J/kg`. 内部不重复添加相变热源, 不重新拟合题给物性参数。',
65                'Q1-3 干物质参考密度固定; Q4 按`rho_d=rho_d0*(R0/R)^2` 更新.', '',
66                '这些是侧表面相变及有效热容量假设下的条件结果, 不是实验验证的唯一答案.', '']
67     for q, (b, latent) in pairs.items():
68         fields = FIELDS if q <= 2 else [('critical_h', '临界时间', 'h'), ('end_h', '严格输出时间', 'h')]
69         if q == 4:
70             fields = fields + [('end_radius_cm', '终点半径', 'cm')]
71         rows = [dict(key=key, label=label, unit=unit, baseline=b[key], latent=latent[key],
72                     difference=latent[key]-b[key]) for key, label, unit in fields]
73         time = '1800 s' if q == 1 else '3 h' if q == 2 else '全域达标'
74         caption = f'问题 {q}: 不计与计入表面蒸发潜热的结果对照 ({time})'
75         note = f'差值为含潜热减无潜热, 先用未舍入值相减再保留四位小数。'
76         if q <= 3:
```

```

77     note += ' 两组均为 1280 区间、BDF 积分。'
78 else:
79     note += ' 无潜热为 2560 区间，含潜热为 1280 区间；本表比较生产答案，不作为严格同网格消融。'
80 if q == 1:
81     note += ' 含水率差显示为零不代表舍入前逐位相等；理论上水分方程不受本问温度影响。'
82 if q >= 3:
83     percent = 100 * (latent['critical_h'] - b['critical_h']) / b['critical_h']
84     note += f' 临界时间延长 {percent:.2f}%；严格输出均按临界后至少 5 s、向上舍入并全节点回代确定。'
85     if q == 4:
86         note += ' 终点半径分别在各自严格终点读取。'
87 tex = [r'\begin{table}[htbp]\centering\small',
88       r'\caption{' + caption + r'}\label{tab:latent-effect-q'+str(q)+'}',
89       r'\begin{tabular}{lrrr}\toprule',
90       r' 指标 & 不计潜热 & 计入潜热 & 差值\\midrule']
91 markdown += [f'## 问题 {q} ({time})', '',
92             '| 指标 | 不计潜热 | 计入潜热 | 差值 |', '|---|---|---|---|']
93 for row in rows:
94     label = f'{row["label"]} / ({row["unit"]})'
95     values = [f'{row["baseline"]:.4f}', f'{row["latent"]:.4f}', signed(row["difference"])]
96     tex.append(label + ' & '+' & '.join(values) + r'\\')
97     markdown.append('| '+label+' | '+' | '.join(values)+' |')
98 tex += [r'\bottomrule\end{tabular}',
99       r'\par\smallskip\begin{minipage}{.95\linewidth}\footnotesize',
100       note.replace('%', r'\%'), r'\end{minipage}', r'\end{table}']
101 (out / f'tables/latent_q{q}_comparison.tex').write_text('\n'.join(tex) + '\n', encoding='utf-8')
102 markdown += ['', note, '']
103 report['questions'].append(dict(question=q, baseline_n=b['settings']['n'],
104                                latent_n=latent['settings']['n'], rows=rows))
105 markdown += ['## 使用边界', '',
106             '第一、二问比较同一物理时刻的状态；第三、四问比较同一个全域阈值的临界时间及同一执行规则的严格输出。',
107             '↪ 第四问两套模型的几何、物性经验式、环境外推均一致，但生产网格不同，',
108             '↪ 不能将全部未舍入差异归因于单一效应。',
109             '表面冷却不等于模型已经完成气固平衡、内部相变与密度/焓的实验闭合。',
110             '参考依据：[FAO潜热参数说明](https://www.fao.org/4/X0490E/x0490e0k.htm)只支持所取潜热量级。',
111             '↪ 复现表格：`python -X utf8 code/latent_effect_tables.py`。',
112             '↪ 来源哈希与未舍入差值见 `supplement/latent_effect_tables/report.json`。']
111 (out / '潜热影响逐问比较.md').write_text('\n'.join(markdown) + '\n', encoding='utf-8')
112 (out / 'supplement/latent_effect_tables/report.json').write_text(
113     json.dumps(report, ensure_ascii=False, indent=2) + '\n', encoding='utf-8')
114 return report
115
116
117 if __name__ == '__main__':
118     parser = argparse.ArgumentParser(description=__doc__)
119     parser.add_argument('--out', type=Path, default=ROOT)
120     args = parser.parse_args()
121     result = write_effect_tables(output_root=args.out)
122     print(f'Generated {len(result["questions"])} paired tables; no PDE solve or PDF compilation.')

```

完整答案的并列表由以下程序直接读取指定时空截面的未舍入记录生成；运行 `code/answer_table_pairs.py` 可重建八组并排表（四问各含两种热边界），不重新求解 PDE。第三、四问的温度为补充输出。

```

1 """Generate adjacent answer tables from the locked baseline and latent records."""
2 import argparse
3 import json
4 from pathlib import Path
5
6 from latent_effect_tables import ROOT, digest, read_records
7
8
9 def panel(rows, caption, label, core=False, methods=False):
10     count = len(rows[0])
11     columns = ([ '算法' ] if methods else []) + [ '0', '0.5', '1', '1.5', '2' ] + ([ '表面' ] if count -
12     ↪ int(methods) == 7 else [])
13     lines = [r'\begin{minipage}[t]{.485\linewidth}\vspace{0pt}\centering',
14             r'\captionsetup{font=footnotesize,skip=4pt}']
15     if core:
16         lines.append(r'\coreanswercaption{' + caption + '}' + label + '}')
17     else:
18         lines.append(r'\caption{' + caption + r'}\label{' + label + '}')
19     lines += [r'\linespread{1}\fontsize{8.5pt}\selectfont\setlength{\tabcolsep}{1pt}',
20             r'\begin{tabular*}{\linewidth}{@{\extracolsep{\fill}}+' + r'*count+r*@{}}\toprule',
21             r' 时间 & '+' & '.join(columns) + r'\\midrule']
22     for row in rows:

```

```

22     lines.append(' & '.join(row)+r'\')
23     lines += [r'\bottomrule\end{tabular*}', r'\end{minipage}']
24     return '\n'.join(lines)
25
26
27 def write_pairs(root=ROOT, output_root=None, baseline_only=False):
28     root = Path(root)
29     out = Path(output_root) if output_root else root
30     if (root / 'supplement/algorithm_crosscheck/report.json').exists():
31         from algorithm_report import load_comparison
32         load_comparison(root)
33     (out / 'tables').mkdir(parents=True, exist_ok=True)
34     pairs, hashes = ({}, {'evidence/answers.json': digest(root / 'evidence/answers.json')}) if
↪ baseline_only else read_records(root)
35     answers = {a['question']: a for a in json.loads((root /
↪ 'evidence/answers.json').read_text(encoding='utf-8'))}
36     manifest = dict(source_sha256=hashes, panels=[])
37     modes = [('baseline', (1, 2))] if baseline_only else [('baseline', (1, 2)), ('latent', (1, 2, 3, 4))]
38     for mode, questions in modes:
39         for q in questions:
40             panels = []
41             for kind, field, symbol in [('temperature_C', 'T_C', '温度 (°C)'), ('moisture', 'C_kg_kg',
↪ '含水率 (kg/kg)')]:
42                 rows = []
43                 if mode == 'baseline':
44                     for record in answers[q]['table_rows']:
45                         time = record['time_s'] / (1 if q == 1 else 3600)
46                         rows.append([f'{time:.0f}' if q == 1 else f'{time:.4f}' +
47                                     [f'{v:.4f}' for v in record[kind][:5]]])
48                         label = f'tab:q{q}-{kind}'
49                 else:
50                     records = pairs[q][1]['requested_rows']
51                     positions = [0., .5, 1., 1.5, 2.] + (['surface'] if q == 4 else [])
52                     for time_s in sorted({r['time_s'] for r in records}):
53                         time = time_s / (1 if q == 1 else 3600)
54                         row = [f'{time:.0f}' if q == 1 else f'{time:.4f}']
55                         for position in positions:
56                             matches = [r for r in records if r['time_s'] == time_s and r['r_cm'] ==
↪ position]
57                             if len(matches) != 1:
58                                 raise ValueError(f'Nonunique Q{q} time/position record')
59                             value = matches[0][field]
60                             row.append('--' if value is None else f'{value:.4f}')
61                             rows.append(row)
62                         label = f'tab:latent-q{q}-{field}'
63                     prefix = '无潜热' if mode == 'baseline' else '含潜热'
64                     panels.append(panel(rows, prefix+symbol, label, core=mode == 'baseline'))
65                     manifest['panels'].append(dict(mode=mode, question=q, field=field, label=label,
↪ rows=rows))
66     timeunit = 's' if q == 1 else 'h'
67     note = f'左表为温度, 右表为干基含水率; 时间单位为 {timeunit}, 固定径向位置单位为 cm.'
68     if q >= 3:
69         note += '含水率表回答题目要求, 温度表为补充结果; 末行为严格达标时刻.'
70     if q == 4:
71         note += '"表面"为瞬时移动表面; --表示该固定位置已在药材外, 不以零代替.'
72     tex = '\n'.join([r'\begin{table}[htbp]\centering', panels[0]+r'\hfill%', panels[1],
73                     r'\par\smallskip\begin{minipage}{\linewidth}\footnotesize',
74                     note, r'\end{minipage}', r'\end{table}', ''])
75     name = f'q{q}_baseline_pair.tex' if mode == 'baseline' else f'latent_q{q}_results.tex'
76     (out / 'tables' / name).write_text(tex, encoding='utf-8')
77     if (root / 'supplement/algorithm_crosscheck/report.json').exists():
78         from algorithm_report import write_comparison
79         manifest['algorithm_comparison'] = write_comparison(root, out, baseline_only)
80     target = out / 'supplement/answer_table_pairs'
81     target.mkdir(parents=True, exist_ok=True)
82     (target / ('report_baseline.json' if baseline_only else 'report.json')).write_text(
83         json.dumps(manifest, ensure_ascii=False, indent=2), encoding='utf-8')
84     return manifest
85
86
87 if __name__ == '__main__':
88     parser = argparse.ArgumentParser(description=__doc__)
89     parser.add_argument('--out', type=Path, default=ROOT)
90     args = parser.parse_args()
91     result = write_pairs(output_root=args.out)
92     actual = result.get('algorithm_comparison', result)['panels']
93     print(f'Generated {len(actual)} answer panels in adjacent temperature/moisture pairs; no PDF.')

```

C.2 第二算法的生产网格复算与答案回填

将 `code/` 和 `data/` 复制到新的空工作目录，先运行 `python-Xutf8code/reproduce_dual.py--outruns/cold_dual_20260912/output` 生成 BDF 生产记录，再运行 `python-Xutf8code/algorithm_crosscheck.py--outsupplement/algorithm_crosscheck`。两处输出目录必须不存在，不能直接在带有旧结果的副本中使用同一路径。程序核对 BDF 生产核心及原始输入哈希，从初态重算八条 Radau 轨道，保存指定截面、各自终点和未舍入差值。重算须使用相同 SciPy 环境；更改核心或输入后，应重建基线及其身份记录。

数值复算与排版表重建是两项操作。在保留完整证据目录的源工程中，运行 `python-Xutf8code/answer_table_pairs.py--outtable_rebuild` 可将双算法答案表写入新目录。该步骤还读取 `evidence/answers.json` 及 `supplement/latent_extension/` 中的冻结记录，并与冷启动结果核对；不能宣称只运行上述两条求解命令就已重建全部图表。

```
1 """Fresh Radau solves against hash-verified production BDF records."""
2 import argparse
3 import gc
4 import hashlib
5 import json
6 import platform
7 import time
8 from pathlib import Path
9
10 import numpy as np
11 import scipy
12
13 from s2_solver import ROOT, Settings
14 from s2_stable_graded import StableModel
15 from latent_extension import LatentModel
16 from experiments import requested_table
17
18 CORE = ('s2_solver.py', 's2_stable_graded.py', 'latent_extension.py', 'experiments.py')
19
20
21 def digest(path):
22     return hashlib.sha256(path.read_bytes()).hexdigest()
23
24
25 def write(path, value):
26     path.write_text(json.dumps(value, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
27
28
29 def verify_bdf_identity():
30     path = ROOT / 'runs/cold_dual_20260912/output/report.json'
31     cold = json.loads(path.read_text(encoding='utf-8'))
32     if not cold['ready'] or cold['exit_code'] != 0 or cold['cache_read']:
33         raise ValueError('Historic BDF production did not pass its execution gate')
34     for name in CORE:
35         if digest(ROOT / 'code' / name) != cold['code'][name]:
36             raise ValueError(f'Production core changed: {name}')
37     for name, expected in cold['inputs'].items():
38         if digest(ROOT / 'data' / name) != expected:
39             raise ValueError(f'Production input changed: {name}')
40     return cold, dict(report_sha256=digest(path), code={n: cold['code'][n] for n in CORE},
41 ↪ inputs=cold['inputs'])
42
43
44 def sample_like(model, rows):
45     times = sorted({r['time_s'] for r in rows})
46     states = model.state(times)
47     index = {t: i for i, t in enumerate(times)}
48     out = []
49     for row in rows:
50         t, p = row['time_s'], row['r_cm']
51         radius = model.radius(t)
52         inside = p == 'surface' or p/100 <= radius + 1e-12
53         x = 1.0 if p == 'surface' else p/100/radius
54         state = states[:, index[t]]
55         out.append(dict(time_s=t, time_h=t/3600, r_cm=p, radius_cm=radius*100,
56 ↪ inside=bool(inside),
57 ↪ T_C=float(np.interp(x, model.x, state[:model.size])) if inside else None,
```

```

57         C_kg_kg=float(np.interp(x, model.x, state[model.size:])) if inside else None))
58     return out
59
60
61 def compare(base, other):
62     errors, mismatches, count = {'T_C': 0., 'C_kg_kg': 0.}, {'T_C': 0, 'C_kg_kg': 0}, 0
63     for b, r in zip(base, other, strict=True):
64         if (b['time_s'], b['r_cm'], b['inside']) != (r['time_s'], r['r_cm'], r['inside']):
65             raise ValueError('Different sampling domain')
66         for field in errors:
67             if b[field] is None:
68                 if r[field] is not None:
69                     raise ValueError('Undefined value replaced by a number')
70             else:
71                 errors[field] = max(errors[field], abs(b[field] - r[field]))
72                 mismatches[field] += f'{b[field]:.4f}' != f'{r[field]:.4f}'
73                 count += 1
74     return dict(max_absolute_difference=errors, four_decimal_mismatches=mismatches,
75               finite_values=count, scope='Requested times/positions, not a continuous-field bound')
76
77
78 def run(out, smoke_n=None):
79     out.mkdir(parents=True, exist_ok=False)
80     cold, identity = verify_bdf_identity()
81     report = dict(ready=False, identity=identity, python=platform.python_version(),
82                 ↪ scipy=scipy.__version__,
83                 numpy=np.__version__, radau_cache_read=False, bdf_reference='fresh_smoke' if smoke_n
84                 ↪ else 'verified_cold_production',
85                 production=smoke_n is None, cases={}, runner_sha256=digest(Path(__file__)))
86     write(out / 'progress.json', report)
87     started = time.perf_counter()
88     for family, cls in [('baseline', StableModel), ('latent', LatentModel)]:
89         for q in range(1, 5):
90             name = f'{family}_q{q}'
91             if smoke_n:
92                 bdf = cls(Settings(question=q, n=smoke_n)).run()
93                 baseline = dict(meta=bdf.meta, requested_rows=requested_table(bdf))
94                 del bdf
95             else:
96                 path = ROOT / f'runs/cold_dual_20260912/output/{family}/q{q}.json'
97                 baseline = json.loads(path.read_text(encoding='utf-8'))
98                 if baseline['meta'] != cold['cases'][family][str(q)]['meta']:
99                     raise ValueError('Per-case BDF record differs from run report')
100                 settings = dict(baseline['meta']['settings'], method='Radau')
101                 print(f'SOLVE {name} Radau N={settings["n"]}', flush=True)
102                 begin = time.perf_counter()
103                 model = cls(Settings(**settings)).run()
104                 own_meta = dict(model.meta)
105                 own_rows = requested_table(model)
106                 original_end = model.end
107                 common_end = max(r['time_s'] for r in baseline['requested_rows'])
108                 # Extend by an actual solve if the BDF display endpoint is later.
109                 # Never extrapolate Radau dense output past its solved interval.
110                 if common_end > model.end + 1e-8:
111                     initial = model.state([model.end])[0]
112                     model.pieces.append(model._solve(model.end, common_end, initial))
113                     model.end = common_end
114                 matched = sample_like(model, baseline['requested_rows'])
115                 metrics = compare(baseline['requested_rows'], matched)
116                 metrics['critical_difference_s'] = None if q <= 2 else
117                 ↪ own_meta['critical_s']-baseline['meta']['critical_s']
118                 metrics['strict_difference_s'] = own_meta['end_s']-baseline['meta']['end_s']
119                 metrics['max_C_at_bdf_end'] = float(model.state([common_end])[model.size:, 0].max())
120                 metrics['adequate_for_table_comparison'] = (
121                     metrics['max_absolute_difference']['T_C'] < 1e-3
122                     and metrics['max_absolute_difference']['C_kg_kg'] < 1e-4
123                     and (q <= 2 or abs(metrics['critical_difference_s']) < 1.0))
124                 case = dict(bdf=baseline, radau_meta=own_meta, radau_own_requested_rows=own_rows,
125                           radau_at_bdf_rows=matched, metrics=metrics,
126                           continuation_s=max(0., common_end-original_end),
127                           ↪ elapsed_s=time.perf_counter()-begin)
128                 write(out / f'{name}.json', case)
129                 report['cases'][name] = dict(file=f'{name}.json', sha256=digest(out / f'{name}.json'),
130                                             n=settings['n'], metrics=metrics, elapsed_s=case['elapsed_s'])
131                 write(out / 'progress.json', report)
132                 print(f'DONE {name}: {metrics}; elapsed={case["elapsed_s"]:.1f}s', flush=True)
133                 del model
134                 gc.collect()
135     report.update(ready=True, exit_code=0, elapsed_s=time.perf_counter()-started)

```

```

132     report['comparison_gate'] = all(c['metrics']['adequate_for_table_comparison'] for c in
↪     report['cases'].values())
133     write(out / 'report.json', report)
134     print(f'FINISHED: {len(report["cases"])} Radau runs; comparison_gate={report["comparison_gate"]}',
↪     flush=True)
135     return report
136
137
138 if __name__ == '__main__':
139     parser = argparse.ArgumentParser(description=__doc__)
140     parser.add_argument('--out', required=True, type=Path)
141     parser.add_argument('--smoke-n', type=int)
142     args = parser.parse_args()
143     run(args.out, args.smoke_n)

1 """Back-fill verified Radau results without changing locked BDF answers."""
2 import argparse
3 import json
4 from pathlib import Path
5
6 from latent_effect_tables import ROOT, digest
7
8
9 def load_comparison(root):
10     folder = root / 'supplement/algorithm_crosscheck'
11     report = json.loads((folder / 'report.json').read_text(encoding='utf-8'))
12     if not report['ready'] or not report['production'] or report['exit_code'] != 0:
13         raise ValueError('Not a completed production comparison')
14     for group, parent in [(('code', 'code'), ('inputs', 'data'))]:
15         for name, expected in report['identity'][group].items():
16             if digest(root / parent / name) != expected:
17                 raise ValueError(f'Comparison source changed: {name}')
18     cases = {}
19     for family in ['baseline', 'latent']:
20         for q in range(1, 5):
21             name = f'{family}_q{q}'
22             item = report['cases'][name]
23             path = folder / item['file']
24             if digest(path) != item['sha256']:
25                 raise ValueError(f'Comparison case changed: {name}')
26             cases[name] = json.loads(path.read_text(encoding='utf-8'))
27     return report, cases
28
29
30 def rows_for(case, q, field):
31     positions = [0., .5, 1., 1.5, 2.] + (['surface'] if q == 4 else [])
32     bdf = case['bdf']['requested_rows']
33     radau = case['radau_at_bdf_rows']
34     indexes = {method: {(r['time_s'], r['r_cm']): r for r in records}
35                 for method, records in [(('BDF', bdf), ('Radau', radau))]}
36     rows = []
37     for t in sorted({r['time_s'] for r in bdf}):
38         for method in ['BDF', 'Radau']:
39             time = f'{t:.0f}' if q == 1 else f'{t/3600:.4f}'
40             row = [time if method == 'BDF' else '', method]
41             for position in positions:
42                 value = indexes[method][t, position][field]
43                 row.append('--' if value is None else f'{value:.4f}')
44             rows.append(row)
45     return rows
46
47
48 def sci(value):
49     if value == 0:
50         return '$0$'
51     mantissa, exponent = f'{abs(value):.2e}'.split('e')
52     return '$'+mantissa+r'\times10^{'+str(int(exponent))+'}$'
53
54
55 def write_comparison(root=ROOT, out=None, baseline_only=False):
56     from answer_table_pairs import panel
57     root, out = Path(root), Path(out) if out else Path(root)
58     report, cases = load_comparison(root)
59     target = out / 'tables'
60     target.mkdir(parents=True, exist_ok=True)
61     manifest = dict(report_sha256=digest(root / 'supplement/algorithm_crosscheck/report.json'),
62 ↪     panels=[])
63     modes = ['baseline'] if baseline_only else ['baseline', 'latent']
64     for mode in modes:

```

```

64 for q in range(1, 5):
65     case = cases[f'{mode}_q{q}']
66     fields = [('T_C', 'temperature_C', '温度 (°C)'), ('C_kg_kg', 'moisture', '含水率 (kg/kg)')]
67     panels = []
68     for field, kind, symbol in fields:
69         rows = rows_for(case, q, field)
70         label = f'tab:q{q}-{kind}' if mode == 'baseline' else f'tab:latent-q{q}-{field}'
71         caption = ('无潜热' if mode == 'baseline' else '含潜热')+symbol
72         if len(fields) == 2:
73             core = mode == 'baseline' and (q <= 2 or field == 'C_kg_kg')
74             panels.append(panel(rows, caption, label, core, methods=True))
75         else:
76             count = len(rows[0])
77             cols = ['时间/h', '算法', '0 cm', '0.5 cm', '1 cm', '1.5 cm', '2 cm'] + ([ '实际表面' ]
78                                     ↪ if q == 4 else [])
79             panels.append('\n'.join([r'\caption{'+caption+r'}\label{'+label+'}',
80                                     r'\small\begin{tabular}{'+r'*count+r'}\toprule',
81                                     '& '.join(cols)+r'\\midrule'+
82                                     ['& '.join(row)+r'\\' for row in rows]+r'\bottomrule\end{tabular}'))
83             manifest['panels'].append(dict(mode=mode, question=q, field=field, label=label,
84                                     ↪ rows=rows))
85     unit = 's' if q == 1 else 'h'
86     note = f' 每个时刻上行为 BDF、下行为 Radau: 时间单位为 {unit}, 固定径向位置单位为 cm.'
87     if len(fields) == 2:
88         note += ' 左表温度、右表干基含水率。'+note
89     if q >= 3:
90         note += ' 末组共用BDF严格达标时刻: 两算法各自终点见本问交叉验证表。0.1500是舍入显示,
91         ↪ 严格性由未舍入全节点值判定。'
92         note += ' 温度为补充输出。'
93     if q == 4:
94         note += ' 表面为瞬时移动表面; --表示该固定位置已在药材外。'
95     env = 'coretable' if len(fields) == 1 else 'table'
96     tex = '\n'.join([r'\begin{'+env+r'}[htbp]\centering',
97                     ('\hfill\n'.join(panels)),
98                     r'\par\smallskip\begin{minipage}{\linewidth}\footnotesize', note,
99                     r'\end{minipage}', r'\end{'+env+r'}', ''])
100     name = f'q{q}_baseline_pair.tex' if mode == 'baseline' and q <= 2 else (
101         f'q{q}_moisture.tex' if mode == 'baseline' else f'latent_q{q}_results.tex')
102     (target / name).write_text(tex, encoding='utf-8')
103 if not baseline_only:
104     md = ['# BDF 与 Radau 逐问答案核对', '',
105           'BDF使用经源码和输入哈希核对的既有冷启动生产结果; 已保存的交叉核从初态实际重算了8条Radau轨道。
106           ↪ 运行本表格脚本只重建表格, 不重新积分生产轨道。',
107           '两算法共用空间离散、物性、边界、网格、容差和采样位置; 比较检验时间积分实现, 不构成独立物理模型验证。
108           ↪ ',
109           '| 问题 | 模型 | N | 最大温差/°C | 最大含水率差/(kg/kg) | 临界时差/s | 严格时间 BDF/Radau/h |',
110           '| ---|---|---|---:|---:|---:|---|']
111     for q in range(1, 5):
112         lines = [r'\begin{table}[htbp]\centering\small',
113                 r'\caption{问题'+str(q)+'的BDF与Radau同网络核对}\label{tab:algorithm-q'+str(q)+'}',
114                 r'\begin{tabular}{lrrr}\toprule',
115                 r'模型 & 最大温差/°C & 最大含水率差/(kg/kg) & 临界时差/s\\midrule']
116         for mode, title in [('baseline', '无潜热'), ('latent', '含潜热')]:
117             c = cases[f'{mode}_q{q}']
118             m, b, r = c['metrics'], c['bdf'], c['radau_meta']
119             dt = m['critical_difference_s']
120             errors = m['max_absolute_difference']
121             lines.append(title+' & '+sci(errors['T_C'])+' & '+sci(errors['C_kg_kg'])+' & '+'--' if
122                 ↪ dt is None else sci(dt))+r'\\')
123             end = '--' if q <= 2 else f'{b['end_s']/3600:.4f}/{r['end_s']/3600:.4f}'
124             md.append(f'|{q}|{title}|{b['settings']['n']}|{errors['T_C']:.8e}|{errors['C_kg_kg']:.8e}|
125                 ↪ |{dt if dt is not None else '--}|{end}|')
126             lines += [r'\bottomrule\end{tabular}',
127                 ↪ r'\par\smallskip\begin{minipage}{\linewidth}\footnotesize',
128                 ' 差值取绝对值; 场差仅覆盖题目指定时刻、径向位置及实际表面, 不是连续全场上界。']
129         if q >= 3:
130             for mode, title in [('baseline', '无潜热'), ('latent', '含潜热')]:
131                 c = cases[f'{mode}_q{q}']
132                 b, r = c['bdf'], c['radau_meta']
133                 lines.append(f'{title}的临界时间 (BDF/Radau)
134                 ↪ 为 {b['critical_s']/3600:.8f}/{r['critical_s']/3600:.8f} h,
135                 ↪ 严格时间为 {b['end_s']/3600:.4f}/{r['end_s']/3600:.4f} h。")
136             lines += [r'\end{minipage}', r'\end{table}', '']
137         (target / f'algorithm_q{q}.tex').write_text('\n'.join(lines), encoding='utf-8')
138     md += ['', '主答案保留BDF口径。答案表按同一时刻逐行并列, 不取两算法平均; 机器报告保存未舍入值、
139     ↪ 四位小数不一致个数和各自终点。',
140           '复算入口: `python -X utf8 code/algorithm_crosscheck.py --out
141           ↪ supplement/algorithm_crosscheck` (输出目录须不存在)。',

```

```

130         '表格入口: `python -X utf8 code/answer_table_pairs.py`。报告缺失时保留单算法版;
        ↳ 已有报告但哈希失配时拒绝回填。',
131         '当前仅核对指定截面及判停, 不额外导出Radau全过程逐秒Excel, 也不替代原工作簿、
        ↳ 空间网格收敛或气固闭合审计。', '')
132     (out / '答案更新_双算法核对.md').write_text('\n'.join(md), encoding='utf-8')
133     max_t = max(c['metrics']['max_absolute_difference']['T_C'] for c in cases.values())
134     max_c = max(c['metrics']['max_absolute_difference']['C_kg_kg'] for c in cases.values())
135     max_time = max(abs(c['metrics']['critical_difference_s'] or 0.) for c in cases.values())
136     mismatches = sum(sum(c['metrics']['four_decimal_mismatches'].values()) for c in cases.values())
137     note = ('\n\n## 双算法复核补充\n\n'
138            f' 已保存的生产核验证重算四问两种热边界下的 8 条 Radau 轨道, 并与同网格 BDF 冷启动记录核对。'
139            f' 指定截面最大温差 {max_t:.8e} °C、最大含水率差 {max_c:.8e} kg/kg、'
140            f' 最大临界时差 {max_time:.8e} s; 四位小数不一致计数为 {mismatches}。主答案保留 BDF。'
141            '六张核心表、两张无潜热温度补充表和八张潜热表并列两算法。详见工程根目录的答案更新_双算法核对.md及'
142            'supplement/algorithm_crosscheck/report.json。该比较是时间积分交叉核验, '
143            '不是独立物理模型验证。本脚本只回填冻结记录; PDF 编译与交付验收另有记录。')
144     for name in ['答案更新_潜热对照.md', 'audit/ANSWER_MATRIX.md']:
145         path = out / name
146         if path.exists():
147             existing = path.read_text(encoding='utf-8').split('\n## 双算法复核补充')[0]
148             existing = existing.split('\n## 2026-09-12 双算法补充')[0]
149             path.write_text(existing.rstrip()+note, encoding='utf-8')
150     return manifest
151
152
153 if __name__ == '__main__':
154     parser = argparse.ArgumentParser(description=__doc__)
155     parser.add_argument('--out', type=Path, default=ROOT)
156     args = parser.parse_args()
157     result = write_comparison(out=args.out)
158     print(f'Generated {len(result["panels"])} dual-algorithm answer panels; no PDF.')

```

以下代码为可移植副本, 数学算子与已锁定生产核心保持一致, 仅输入和输出路径改为相对路径。生成的图表代码作为完整支撑源提供, 不以截图代替。

C.3 模型、输入与积分

```

1 """Approved effective radial model; outputs are conditional, pending team review."""
2 from dataclasses import dataclass, asdict
3 from pathlib import Path
4 from functools import lru_cache
5 import hashlib
6 import json
7 import time
8
9 import numpy as np
10 import openpyxl
11 from scipy.integrate import solve_ivp
12 from scipy.interpolate import PchipInterpolator
13 from scipy.sparse import diags, bmat
14 from scipy.special import expi
15
16 ROOT = Path(__file__).resolve().parents[1]
17 SOURCE = ROOT/'data'
18
19
20 @lru_cache(None)
21 def inputs():
22     arrays = []
23     for name in ('input1.xlsx', 'input2.xlsx'):
24         book = openpyxl.load_workbook(SOURCE/name, read_only=True, data_only=True)
25         arr = np.asarray(list(book.active.values)[1:], dtype=float)
26         book.close()
27         if not np.isfinite(arr).all() or not np.all(np.diff(arr[:, 0]) > 0):
28             raise ValueError('Nonfinite input or invalid time ordering')
29         arrays.append(arr)
30     return arrays
31
32
33 @dataclass
34 class Settings:
35     question: int = 3
36     n: int = 160
37     property_set: int = 0
38     moving: bool | None = None
39     frozen: bool = False
40     flux: str = 'kirchhoff'

```

```

41 method: str = 'BDF'
42 boundary: str = 'mean'
43 tail_start_h: float = 3.0
44 radius_kind: str = 'pchip'
45 D_scale: float = 1.0
46 h_scale: float = 1.0
47 hm_scale: float = 1.0
48 tail_T_shift: float = 0.0
49 tail_C_shift: float = 0.0
50 rtol: float = 2e-9
51 atol_T: float = 1e-8
52 atol_C: float = 1e-10
53 max_hours: float = 72.0
54 safe_margin_s: float = 5.0
55
56
57 class Model:
58     def __init__(self, settings):
59         self.s = settings
60         if settings.n < 4 or settings.question not in (1, 2, 3, 4):
61             raise ValueError('Invalid grid or question')
62         if min(settings.D_scale, settings.h_scale, settings.hm_scale) < 0:
63             raise ValueError('Negative transport coefficient')
64         self.env, self.rad = inputs()
65         self.kind = settings.property_set or (1 if settings.question == 1 else 4 if settings.question ==
        ↪ 4 else 3)
66         self.moving = settings.question == 4 if settings.moving is None else settings.moving
67         self.x = np.linspace(0, 1, settings.n+1)
68         self.faces = np.r_[0, (self.x[1:]+self.x[:-1])/2, 1]
69         self.w = np.diff(self.faces**2)/2
70         self.dx = np.diff(self.x)
71         self.size = len(self.x)
72         self.tail = self.env[self.env[:, 0] >= settings.tail_start_h*3600, 1:].mean(axis=0)
73         self.Rfit = PchipInterpolator(self.rad[:, 0], self.rad[:, 1]/100, extrapolate=False)
74         band = diags([np.ones(self.size-1), np.ones(self.size), np.ones(self.size-1)], [-1, 0, 1])
75         self.sparse = bmat([[band, band], [band, band]], format='csc')
76         self.atol = np.r_[np.full(self.size, settings.atol_T), np.full(self.size, settings.atol_C)]
77         self.min_trial_C = 2.55
78
79     def environment(self, t):
80         if t <= 14400:
81             return np.interp(t, self.env[:, 0], self.env[:, 1]), np.interp(t, self.env[:, 0], self.env[:,
            ↪ 2])
82         val = self.env[-1, 1:] if self.s.boundary == 'last' else self.tail
83         return val[0]+self.s.tail_T_shift, val[1]+self.s.tail_C_shift
84
85     def radius(self, t):
86         if not self.moving:
87             return 0.02
88         if t < 0 or t > self.rad[-1, 0]+1e-8:
89             raise ValueError('Outside measured radius time domain')
90         return float(np.interp(t, self.rad[:, 0], self.rad[:, 1])/100 if self.s.radius_kind == 'linear'
        ↪ else self.Rfit(t))
91
92     def properties(self, T, C):
93         if np.min(C) <= 0 or np.min(T+273.15) <= 0:
94             raise ValueError('Nonpositive C or absolute temperature; no clipping permitted')
95         c = np.full_like(C, 2.55) if self.s.frozen else C
96         temp = np.full_like(T, 28.0) if self.s.frozen else T
97         if self.kind == 1:
98             return np.full_like(c, 820), np.full_like(c, 2600), np.full_like(c, .36), np.full_like(c,
            ↪ 7e-9), .89
99         if self.kind == 3:
100             return 650+128*c, 1450+2736*c/(1+c), .21+.38*c/(1+c), 2.4e-3*np.exp(-3850/(temp+273.15)), .45
101             return 760+90*c, 1850+2150*c/(1+c), .12+.20*c/(1+c), 4.2e-4*np.exp(-3850/(temp+273.15)), .30
102
103     @staticmethod
104     def primitive(c, a):
105         if np.any(c <= 0):
106             raise ValueError('Primitive requires positive dry-basis moisture')
107         return c*np.exp(-a/c)+a*expi(-a/c)
108
109     def rhs(self, t, y):
110         T, C = y[:self.size], y[self.size:]
111         self.min_trial_C = min(self.min_trial_C, float(C.min()))
112         rho, cp, k, A, a = self.properties(T, C)
113         R = self.radius(t)
114         Ta, Ca = self.environment(t)
115         heat, water = np.zeros(self.size+1), np.zeros(self.size+1)

```

```

116     heat[1:-1] = self.faces[1:-1]*(k[1:]+k[:-1])/2*np.diff(T)/self.dx
117     heat[-1] = R*25*self.s.h_scale*(Ta-T[-1])
118     if self.s.frozen:
119         water[1:-1] = self.faces[1:-1]*(A[1:]+A[:-1])/2*np.exp(-a/2.55)*np.diff(C)/self.dx
120     elif self.s.flux == 'kirchhoff':
121         water[1:-1] = self.faces[1:-1]*(A[1:]+A[:-1])/2*np.diff(self.primitive(C, a))/self.dx
122     else:
123         D = A*np.exp(-a/C)
124         water[1:-1] = self.faces[1:-1]*(D[1:]+D[:-1])/2*np.diff(C)/self.dx
125     water[1:-1] *= self.s.D_scale
126     water[-1] = R*8e-7*self.s.hm_scale*(Ca-C[-1])
127     return np.r_[np.diff(heat)/(R*R*self.w*rho*cp), np.diff(water)/(R*R*self.w)]
128
129 def _solve(self, left, right, y, event=None):
130     out = solve_ivp(self.rhs, (left, right), y, method=self.s.method,
131                     rtol=self.s.rtol, atol=self.atol, jac_sparsity=self.sparse,
132                     max_step=30 if left < 14400 else 300, events=event, dense_output=True)
133     if not out.success or not np.isfinite(out.y).all() or out.y[self.size:].min() <= 0:
134         raise RuntimeError('Solver or physical state guard failed: '+out.message)
135     return out
136
137 def run(self):
138     start = time.perf_counter()
139     y = np.r_[np.full(self.size, 28.), np.full(self.size, 2.55)]
140     stop = 1800 if self.s.question == 1 else 10800 if self.s.question == 2 else self.s.max_hours*3600
141     def event(t, y):
142         return np.max(y[self.size:])-15
143     event.terminal, event.direction = True, -1
144     self.pieces, self.critical = [], None
145     for left, right in zip([0, min(stop, 14400)], [min(stop, 14400), stop]):
146         if right <= left:
147             continue
148         sol = self._solve(left, right, y, event if self.s.question >= 3 else None)
149         self.pieces.append(sol)
150         y = sol.y[:, -1]
151         if self.s.question >= 3 and len(sol.t_events[0]):
152             self.critical = float(sol.t_events[0][0])
153             safe = float(np.ceil((self.critical+self.s.safe_margin_s)/.36)*.36)
154             if safe > stop:
155                 raise RuntimeError('No radius-supported strict feasible endpoint')
156             sol = self._solve(self.critical, safe, y)
157             self.pieces.append(sol)
158             y = sol.y[:, -1]
159             if np.max(y[self.size:]) >= .15:
160                 raise RuntimeError('Strict threshold guard failed')
161             break
162     self.end = float(self.pieces[-1].t[-1])
163     if self.s.question >= 3 and self.critical is None:
164         raise RuntimeError('Threshold not reached within authorized time domain')
165     self.meta = dict(settings=asdict(self.s), critical_s=self.critical,
166                     critical_h=None if self.critical is None else self.critical/3600,
167                     end_s=self.end, end_h=self.end/3600, elapsed_s=time.perf_counter()-start,
168                     min_trial_C=self.min_trial_C, min_accepted_C=min(float(p.y[self.size:].min())
169                             for p in self.pieces),
170                     max_C_at_end=float(y[self.size:].max()),
171                     end_radius_cm=100*self.radius(self.end),
172                     center_T=float(y[0]), surface_T=float(y[self.size-1]),
173                     center_C=float(y[self.size]), surface_C=float(y[-1]),
174                     nfev=sum(p.nfev for p in self.pieces), nlu=sum(p.nlu for p in self.pieces),
175                     status='computed_conditional_pending_team_review', clipping_used=False)
176     return self
177
178 def state(self, times):
179     times = np.asarray(times)
180     if np.any(times < 0) or np.any(times > self.end+1e-7):
181         raise ValueError('Sampling outside solved interval')
182     states = np.empty((2*self.size, len(times)))
183     for p in self.pieces:
184         mask = (times >= p.t[0]-1e-8) & (times <= p.t[-1]+1e-8)
185         if mask.any():
186             states[:, mask] = p.sol(times[mask])
187     if not np.isfinite(states).all() or np.min(states[self.size:]) <= 0:
188         raise RuntimeError('Dense-output physical state guard failed')
189     return states
190
191 def export(self, name):
192     dest = ROOT/'results/stage2'/name
193     dest.mkdir(parents=True, exist_ok=False)
194     step = 1 if self.s.question <= 2 else 60

```

```

193     times = np.unique(np.r_[np.arange(0, self.end, step), self.end])
194     r_fixed = np.linspace(0, .02, 21)
195     xi = np.linspace(0, 1, 101)
196     T, C, U, Tc, Rs = [], [], [], [], []
197     for first in range(0, len(times), 256):
198         batch = times[first:first+256]
199         states = self.state(batch)
200         for i, t in enumerate(batch):
201             R = self.radius(t)
202             coords = r_fixed/R
203             tv = np.interp(coords, self.x, states[:self.size, i], right=np.nan)
204             cv = np.interp(coords, self.x, states[self.size:, i], right=np.nan)
205             T.append(np.r_[tv, states[self.size-1, i]])
206             C.append(np.r_[cv, states[-1, i]])
207             U.append(np.interp(xi, self.x, states[self.size:, i]))
208             Tc.append(np.interp(xi, self.x, states[:self.size, i]))
209             Rs.append(R)
210     np.savez_compressed(dest/'fields.npz', times_s=times, radius_m=Rs, r_fixed_m=r_fixed,
211                        T=T, C=C, xi=xi, C_xi=U, T_xi=Tc)
212     if self.critical is not None:
213         near = np.array([self.critical-1, self.critical, self.end])
214         near_state = self.state(near)[self.size:]
215         self.meta['threshold_bracket'] = [dict(t_s=float(t), g=float(near_state[:, i].max()-0.15)) for
216             ↪ i, t in enumerate(near)]
217     (dest/'run.json').write_text(json.dumps(self.meta, indent=2), encoding='utf-8')
218     return self.meta

```

C.4 稳定状态积分与表层网格

```

1 """Stable close-state Kirchhoff differences, avoiding cancellation on graded grids."""
2 import json
3 import numpy as np
4 from scipy.integrate import quad
5 from s2_solver import Model, Settings, ROOT
6
7
8 def integral_difference(left, right, a):
9     delta = right-left
10    midpoint = (left+right)/2
11    close = abs(delta) < .01*midpoint
12    out = np.empty_like(left)
13    far = ~close
14    if far.any():
15        out[far] = Model.primitive(right[far], a)-Model.primitive(left[far], a)
16    # Three-point Gaussian quadrature has no subtraction of nearly equal primitives.
17    mid, half = midpoint[close], delta[close]/2
18    offset = np.sqrt(3/5)*half
19    out[close] = half*(5/9*np.exp(-a/(mid-offset))+8/9*np.exp(-a/mid)+5/9*np.exp(-a/(mid+offset)))
20    return out
21
22
23 class StableModel(Model):
24     def __init__(self, settings, graded=True):
25         super().__init__(settings)
26         self.graded = graded
27         if graded:
28             s = np.linspace(0, 1, self.s.n+1)
29             self.x = 1-(1-s)**1.5
30             self.faces = np.r_[0, (self.x[1:]+self.x[:-1])/2, 1]
31             self.w = np.diff(self.faces**2)/2
32             self.dx = np.diff(self.x)
33
34     def rhs(self, t, y):
35         T, C = y[:self.size], y[self.size:]
36         self.min_trial_C = min(self.min_trial_C, float(C.min()))
37         rho, cp, k, A, a = self.properties(T, C)
38         R = self.radius(t)
39         Ta, Ca = self.environment(t)
40         heat, water = np.zeros(self.size+1), np.zeros(self.size+1)
41         heat[1:-1] = self.faces[1:-1]*(k[1:]+k[:-1])/2*np.diff(T)/self.dx
42         heat[-1] = R*25*self.s.h_scale*(Ta-T[-1])
43         water[1:-1] = self.faces[1:-1]*(A[1:]+A[:-1])/2*integral_difference(C[:-1], C[1:],
44             ↪ a)/self.dx*self.s.D_scale
45         water[-1] = R*8e-7*self.s.hm_scale*(Ca-C[-1])
46         return np.r_[np.diff(heat)/(R*R*self.w*rho*cp), np.diff(water)/(R*R*self.w)]
47
48     def run(self):
49         super().run()
50         self.meta['mesh'] = dict(kind='surface_graded' if self.graded else 'uniform',

```

```

50             mapping='xi=1-(1-s)^1.5' if self.graded else 'xi=s',
51             stable_close_state_flux=True,
52             min_dxi=float(self.dx.min()), max_dxi=float(self.dx.max()))
53     return self
54
55
56 def main():
57     tests = []
58     for a in (.89, .45, .30):
59         for c in (.05, .15, 1., 2.55):
60             for fraction in (1e-8, .001, .009, .02):
61                 right = c*(1+fraction)
62                 value = float(integral_difference(np.array([c]), np.array([right]), a)[0])
63                 exact, _ = quad(lambda x: np.exp(-a/x), c, right, epsabs=1e-20)
64                 error = abs(value-exact)/max(abs(exact), 1e-30)
65                 tests.append(dict(a=a, c=c, fraction=fraction, relative_error=error, passed=error <
66                               ↪ 1e-8))
67
68     testpath = ROOT/'audit/stage2/STABLE_FLUX_TESTS.json'
69     testpath.write_text(json.dumps(tests, indent=2), encoding='utf-8')
70     if not all(t['passed'] for t in tests):
71         raise AssertionError('Stable flux quadrature check failed')
72     for q in (1, 2, 3, 4):
73         for n in (320, 640, 1280):
74             name = f'q{q}_stable_graded_n{n}'
75             if (ROOT/'results/stage2'/name/'run.json').exists():
76                 continue
77             meta = StableModel(Settings(question=q, n=n)).run().export(name)
78             with (ROOT/'logs/stage2/stable_graded.jsonl').open('a', encoding='utf-8') as f:
79                 f.write(json.dumps(dict(name=name, exit_code=0, **meta))+'\n')
80             print(name, meta['critical_h'], round(meta['elapsed_s'], 2), flush=True)
81
82 if __name__ == '__main__':
83     main()

```

C.5 四问运行、工作簿与基本验证入口

```

1  """Portable numerical entry. No TeX compiler and no manuscript overwrites."""
2  from pathlib import Path
3  import argparse
4  import json
5  import platform
6  import time
7  import numpy as np
8  import scipy
9  from scipy.integrate import quad
10 from openpyxl import Workbook, load_workbook
11 from openpyxl.cell import WriteOnlyCell
12 from s2_solver import Settings, ROOT
13 from s2_stable_graded import StableModel, integral_difference
14
15 def workbook(model, path, step, temperature=False):
16     book=Workbook(write_only=True)
17     names=[' 温度', ' 水分浓度'] if temperature else ['Sheet1']
18     sheets=[book.create_sheet(name) for name in names]
19     distances=np.arange(20 if model.moving else 21)/10
20     for sheet in sheets:
21         sheet.append([' 时间\\到药材中心的距离']+distances.tolist()+(['药材表面'] if model.moving else []))
22     times=np.unique(np.r_[np.arange(step,model.end,step),model.end])
23     for start in range(0,len(times),256):
24         ts=times[start:start+256]
25         ys=model.state(ts)
26         for j,t in enumerate(ts):
27             xx=distances/100/model.radius(t)
28             for sheet in sheets:
29                 values=ys[:model.size,j] if sheet.title==' 温度' else ys[model.size:,j]
30                 row=np.interp(xx,model.x,values,right=np.nan)
31                 if model.moving:
32                     row=np.r_[row,values[-1]]
33                 cells=[]
34                 for v in row:
35                     cell=WriteOnlyCell(sheet, value=None if not np.isfinite(v) else round(float(v),4))
36                     cell.number_format='0.0000'
37                     cells.append(cell)
38                 sheet.append([float(t)]+cells)
39     book.save(path)
40     check=load_workbook(path,read_only=True,data_only=True)
41     for ws in check:
42         assert sum(1 for _ in ws.values)==len(times)+1

```

```

43     check.close()
44     return dict(rows=len(times),end_s=model.end,step_s=step)
45
46 def smoke(out):
47     tests=[]
48     for a in (.30,.45,.89):
49         for c in (.05,.15,1.,2.55):
50             right=c*(1+1e-8)
51             v=integral_difference(np.array([c]),np.array([right]),a)[0]
52             exact=quad(lambda x:np.exp(-a/x),c,right,epsabs=1e-24)[0]
53             err=abs(v-exact)/abs(exact)
54             assert err<1e-8
55             tests.append(err)
56     m=StableModel(Settings(question=1,n=40)).run()
57     assert abs(m.meta['center_T']-33.57534471765)<.01
58     assert abs(m.meta['surface_C']-1.5102350883)<.01
59     n=StableModel(Settings(question=4,n=40,D_scale=0,hm_scale=0))
60     y=np.r_[np.full(n.size,28.),.8-.3*n.x**2]
61     assert np.max(abs(n.rhs(100,y)[n.size:] ))==0
62     ledger=workbook(m,out/'smoke.xlsx',60,True)
63     return dict(positive_C=bool(m.meta['min_accepted_C']>0),
64                 flux_max_relative_error=max(tests),material_zero_flux_pass=True,
65                 q1_small_grid=m.meta,workbook=ledger,
66                 scope='Small-grid smoke only; not production rerun or physical validation')
67
68 def full(out):
69     models={q:StableModel(Settings(question=q,n=2560 if q==4 else 1280)).run() for q in (1,2,3,4)}
70     report={str(q):m.meta for q,m in models.items()}
71     for q in (1,2,3,4):
72         # Q2 and Q3 have identical governing systems; Q3 supplies the full trajectory.
73         m=models[3] if q==2 else models[q]
74         report[str(q)]['workbook']=workbook(m,out/f'result{q}.xlsx',1 if q<=2 else 60,q<=2)
75     return report
76
77 if __name__=='__main__':
78     ap=argparse.ArgumentParser()
79     ap.add_argument('--mode',choices=['smoke','full'],default='smoke')
80     ap.add_argument('--out',type=Path,default=ROOT/'reproduced')
81     args=ap.parse_args();args.out.mkdir(parents=True,exist_ok=True)
82     if (args.out/f'{args.mode}_report.json').exists():
83         raise FileExistsError('Choose a new --out directory to preserve previous reports')
84     start=time.perf_counter()
85     report=(smoke if args.mode=='smoke' else full)(args.out)
86     payload=dict(mode=args.mode,python=platform.python_version(),numpy=np.__version__,scipy=scipy.__version__,
87                 ion__,
88                 elapsed_s=time.perf_counter()-start,result=report,exit_code=0)
89     (args.out/f'{args.mode}_report.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf8')
90     print(json.dumps(payload,ensure_ascii=False))

```

C.6 方法对照、收支、情景与四格试验

```

1 """S2 model ablations, material moisture ledger and finite-scenario stopping audit."""
2 import csv
3 import hashlib
4 import itertools
5 import json
6 import platform
7 import sys
8 import time
9 import traceback
10 from pathlib import Path
11
12 import numpy as np
13 import scipy
14 from numpy.polynomial.legendre import leggauss
15 from scipy.integrate import quad
16
17 from s2_solver import Model, Settings, ROOT, SOURCE
18 from s2_stable_graded import StableModel, integral_difference
19
20 OUT = ROOT/'reproduced/experiments'
21 AUDIT = ROOT/'reproduced/experiment_audit'
22 LOG = AUDIT/'runs.jsonl'
23
24
25 def save(path, value):
26     path.parent.mkdir(parents=True, exist_ok=True)

```

```

27     path.write_text(json.dumps(value, indent=2, ensure_ascii=False, allow_nan=False), encoding='utf-8')
28
29
30 def digest(path):
31     return hashlib.sha256(path.read_bytes()).hexdigest()
32
33
34 def calculate(label, settings, family='graded'):
35     model = (Model(settings) if family == 'arithmetic' else
36             StableModel(settings, graded=family != 'uniform'))
37     model.run()
38     record = dict(label=label, family=family, **model.meta)
39     save(OUT/'runs'/f'{label}.json', record)
40     with LOG.open('a', encoding='utf-8') as stream:
41         stream.write(json.dumps(dict(label=label, exit_code=0, elapsed_s=model.meta['elapsed_s']))+'\n')
42     print(label, 'hours=', model.meta['critical_h'], 'seconds=', round(model.meta['elapsed_s'], 2),
43           ↪ flush=True)
44     return model
45
46 def requested_table(model):
47     q = model.s.question
48     ts = ([100, 300, 600, 900, 1200, 1500, 1800] if q == 1 else
49          list(np.arange(1800, 10801, 1800)) if q == 2 else
50          list(np.arange(21600, model.end, 21600))+[model.end])
51     states = model.state(ts)
52     rows = []
53     for j, t in enumerate(ts):
54         for r in (0., .005, .01, .015, .02, None):
55             radius = model.radius(t)
56             inside = r is None or r <= radius+1e-12
57             x = 1. if r is None else r/radius
58             rows.append(dict(time_s=float(t), time_h=float(t/3600),
59                             r_cm='surface' if r is None else r*100,
60                             radius_cm=radius*100, inside=bool(inside),
61                             ↪ T_C=float(np.interp(x, model.x, states[:model.size, j])) if inside else
62                             ↪ None,
63                             C_kg_kg=float(np.interp(x, model.x, states[model.size:, j])) if inside else
64                             ↪ None))
65     return rows
66
67 def profile_errors(model, reference):
68     end = min(model.end, reference.end)
69     times = np.unique(np.r_[np.linspace(0, end, 501), np.minimum([100, 300, 600, 1800, 10800], end)])
70     xi = np.linspace(0, 1, 201)
71     errors = np.zeros(2)
72     for start in range(0, len(times), 64):
73         batch = times[start:start+64]
74         left, right = model.state(batch), reference.state(batch)
75         for j in range(len(batch)):
76             for k in (0, 1):
77                 a = np.interp(xi, model.x, left[k*model.size:(k+1)*model.size, j])
78                 b = np.interp(xi, reference.x, right[k*reference.size:(k+1)*reference.size, j])
79                 errors[k] = max(errors[k], np.max(abs(a-b)))
80     return dict(sample_time_count=len(times), material_locations=len(xi),
81               ↪ max_T_error_C=float(errors[0]), max_C_error=float(errors[1]),
82               ↪ event_error_s=None if model.critical is None else abs(model.critical-reference.critical),
83               ↪ elapsed_s=model.meta['elapsed_s'])
84
85 def integrate_boundary(model, order):
86     nodes, weights = leggauss(order)
87     water, heat = 0., 0.
88     for piece in model.pieces:
89         left, right = piece.t[:-1], piece.t[1:]
90         centers, half = (left+right)/2, (right-left)/2
91         times = (centers[:, None]+half[:, None]*nodes).ravel()
92         ws = (half[:, None]*weights).ravel()
93         for first in range(0, len(times), 128):
94             t = times[first:first+128]
95             y = piece.sol(t)
96             env = np.array([model.environment(v) for v in t])
97             radii = np.array([model.radius(v) for v in t])
98             water += np.dot(ws[first:first+128], 2*8e-7*model.s.hm_scale/radii*(env[:, 1]-y[-1]))
99             heat += np.dot(ws[first:first+128], 2*25*model.s.h_scale/radii*(env[:, 0]-y[model.size-1]))
100     return float(water), float(heat)
101
102

```

```

103 def conservation(model):
104     final = model.state([model.end])[:, 0]
105     change = float(2*np.dot(model.w, final[model.size:])-2.55)
106     water7, heat7 = integrate_boundary(model, 7)
107     water3, heat3 = integrate_boundary(model, 3)
108     relative = abs(change-water7)/max(abs(change), 1e-15)
109     local = []
110     for t in np.linspace(0, model.end, 31):
111         y = model.state([t])[:, 0]
112         rate = model.rhs(t, y)[model.size:]
113         expected = 2*8e-7*model.s.hm_scale/model.radius(t)*(model.environment(t)[1]-y[-1])
114         local.append(abs(2*np.dot(model.w, rate)-expected))
115     record = dict(mean_C_change=change, integrated_surface_exchange=water7,
116                 relative_residual=relative, quadrature_3_vs_7_abs=abs(water3-water7),
117                 max_instantaneous_balance_residual=max(local), passed=relative < 1e-5,
118                 measure='normalized dry-solid mass, 2*sum(w*C); not unweighted current-volume water
119                 ↪ mass')
119     if model.s.question == 1:
120         energy = float(820*2600*(2*np.dot(model.w, final[:model.size])-28))
121         heat_error = abs(energy-heat7)/abs(energy)
122         record['constant_capacity_heat'] = dict(change_J_m3=energy, exchange_J_m3=heat7,
123                                             relative_residual=heat_error, passed=heat_error < 1e-5)
124     return record
125
126
127 def extend(model, until):
128     if until <= model.end:
129         return
130     y = model.state([model.end])[:, 0]
131     piece = model._solve(model.end, until, y)
132     model.pieces.append(piece)
133     model.end = float(until)
134
135
136 def unit_tests():
137     tests = []
138     for a, lo, hi in itertools.product((.89, .45, .3), (.05, .15, 2.55), (1e-8, .02)):
139         right = lo*(1+hi)
140         val = float(integral_difference(np.array([lo]), np.array([right]), a)[0])
141         ref = quad(lambda c: np.exp(-a/c), lo, right, epsabs=1e-22)[0]
142         tests.append(dict(name=f'flux_{a}_{lo}_{hi}', error=abs(val-ref)/abs(ref),
143                         ↪ passed=abs(val-ref)/abs(ref) < 1e-8))
143     for q in (1, 2, 3, 4):
144         m = StableModel(Settings(question=q, n=16, hm_scale=0, h_scale=0))
145         y = np.r_[np.full(m.size, 28.), np.full(m.size, 2.55)]
146         error = max(float(np.max(abs(m.rhs(t, y)))) for t in (0., 1800., 10000.))
147         tests.append(dict(name=f'no_exchange_uniform_material_q{q}', error=error, passed=error < 1e-12))
148     for label, settings in [('negative_D', Settings(D_scale=-1)), ('small_grid', Settings(n=3))]:
149         rejected = False
150         try:
151             StableModel(settings)
152         except ValueError:
153             rejected = True
154         tests.append(dict(name=label, passed=rejected))
155     return tests
156
157
158 def scenario_audit(question, nominal_end):
159     models, records = [], []
160     for ds, dt, dc in itertools.product((.9, 1.1), (-.5, .5), (-.002, .002)):
161         label = f'q{question}_scenario_{len(models)}'
162         model = calculate(label, Settings(question=question, n=320, D_scale=ds, tail_T_shift=dt,
163                                         ↪ tail_C_shift=dc))
164         models.append(model)
165         records.append(dict(label=label, D_scale=ds, tail_T_shift=dt, tail_C_shift=dc,
166                             critical_h=model.critical/3600))
167     worst = int(np.argmax([m.critical for m in models]))
168     source = models[worst].s
169     fine = calculate(f'q{question}_worst_n640', Settings(question=question, n=640,
170                                         ↪ D_scale=source.D_scale,
171                                         ↪ tail_T_shift=source.tail_T_shift,
172                                         ↪ tail_C_shift=source.tail_C_shift))
173     robust_end = float(np.ceil((max(fine.critical, max(m.critical for m in models))+60)/60)*60)
174     for model, row in zip(models, records):
175         extend(model, max(robust_end, nominal_end))
176         row['max_C_at_nominal_stop'] = float(model.state([nominal_end])[model.size:].max())
177         row['max_C_at_scenario_stop'] = float(model.state([robust_end])[model.size:].max())
178         row['nominal_stop_passed'] = row['max_C_at_nominal_stop'] < .15
179         row['scenario_stop_passed'] = row['max_C_at_scenario_stop'] < .15

```

```

177     sample = np.linspace(0, model.end, 601)
178     y = model.state(sample)
179     np.savez_compressed(OUT/'runs'/f'{row['label']}_trajectory.npz", times_s=sample,
180                        xi=model.x, T=y[:model.size], C=y[model.size:])
181     extend(fine, robust_end)
182     grid_error = abs(fine.critical-models[worst].critical)
183     output = dict(question=question, grid_n=320, scenarios=records, worst_index=worst,
184                  fine_worst_critical_h=fine.critical/3600, worst_grid_event_difference_s=grid_error,
185                  worst_grid_passed=grid_error < 1,
186                  nominal_stop_h=nominal_end/3600, scenario_stop_h=robust_end/3600,
187                  fine_worst_max_C_at_stop=float(fine.state([robust_end])[fine.size:].max()),
188                  nominal_failures=sum(not r['nominal_stop_passed'] for r in records),
189                  all_tested_scenarios_passed=all(r['scenario_stop_passed'] for r in records),
190                  interpretation='8 finite stress scenarios only; not a confidence interval or
191                               ↳ continuous-box guarantee')
192     save(OUT/f'q{question}_scenarios.json', output)
193     return output
194
195 def main():
196     started = time.perf_counter()
197     OUT.mkdir(parents=True, exist_ok=True)
198     AUDIT.mkdir(parents=True, exist_ok=True)
199     if (OUT/'SUMMARY.json').exists() or LOG.exists():
200         raise RuntimeError('Existing experiment evidence must not be overwritten')
201     protected = [ROOT/'evidence/RESULTS_LOCK.md', ROOT/'evidence/answers.json', ROOT/'code/s2_solver.py',
202                 ↳ ROOT/'code/s2_stable_graded.py']
203     save(AUDIT/'BASELINE_IDENTITY.json', {str(p): digest(p) for p in protected})
204     tests = unit_tests()
205     save(AUDIT/'UNIT_TESTS.json', tests)
206     assert all(t['passed'] for t in tests)
207     summary = dict(stage='S2_PENDING_USER_G2', nominal={}, comparisons={}, ledgers={}, scenarios={})
208     for q in (1, 2, 3, 4):
209         n = 2560 if q == 4 else 1280
210         fine = calculate(f'q{q}_nominal_n{n}', Settings(question=q, n=n))
211         summary['nominal'][str(q)] = fine.meta
212         rows = requested_table(fine)
213         save(OUT/f'q{q}_requested_table.json', rows)
214         with (OUT/f'q{q}_requested_table.csv').open('w', newline='', encoding='utf-8-sig') as f:
215             writer = csv.DictWriter(f, fieldnames=rows[0].keys())
216             writer.writeheader()
217             writer.writerows(rows)
218         historical =
219         ↳ json.loads((ROOT/'evidence/improvement.json').read_text(encoding='utf-8'))['nominal'][str(q)]
220         regression = {key: abs(fine.meta[key]-historical[key]) for key in ('center_T', 'surface_T',
221                                ↳ 'center_C', 'surface_C')}
222         regression['event_s'] = 0. if fine.critical is None else
223         ↳ abs(fine.critical-historical['critical_s'])
224         regression['passed'] = (max(regression['center_T'], regression['surface_T']) < 2e-5 and
225                                max(regression['center_C'], regression['surface_C']) < 2e-7 and
226                                ↳ regression['event_s'] < .05)
227         summary['nominal'][str(q)]['historical_regression'] = regression
228         summary['ledgers'][str(q)] = conservation(fine)
229         comparisons = []
230         for family in ('arithmetic', 'uniform', 'graded'):
231             s = Settings(question=q, n=160, flux='arithmetic' if family == 'arithmetic' else 'kirchhoff')
232             model = calculate(f'q{q}_{family}_n160', s, family)
233             comparisons.append(dict(family=family, n=160, **profile_errors(model, fine)))
234         summary['comparisons'][str(q)] = comparisons
235         save(OUT/'PARTIAL.json', summary)
236         del fine, model
237     for q in (3, 4):
238         summary['scenarios'][str(q)] = scenario_audit(q, summary['nominal'][str(q)]['end_s'])
239     factorial = {}
240     for props, moving in itertools.product((3, 4), (False, True)):
241         model = calculate(f'factor_p{props}_moving{int(moving)}_n320',
242                          Settings(question=3, n=320, property_set=props, moving=moving, max_hours=72 if
243                                  ↳ moving else 168))
244         factorial[f'p{props}_moving{int(moving)}'] = model.critical/3600
245     a, b, c, d = [factorial[key] for key in ('p3_moving0', 'p4_moving0', 'p3_moving1', 'p4_moving1')]
246     summary['factorial'] = dict(times_h=factorial, property_effect_at_fixed_h=b-a,
247                                geometry_effect_at_p3_h=c-a, interaction_h=d-b-c+a,
248                                geometry_effect_at_p4_h=d-b, net_q4_minus_q3_h=d-a,
249                                geometry_reduction_p4_percent=100*(b-d)/b)
250     summary['elapsed_s'] = time.perf_counter()-started
251     summary['protected_unchanged'] = all(digest(Path(p)) == h for p, h in
252     ↳ json.loads((AUDIT/'BASELINE_IDENTITY.json').read_text(encoding='utf-8')).items())
253     save(OUT/'SUMMARY.json', summary)
254     passed = all(v['historical_regression']['passed'] for v in summary['nominal'].values())

```

```

248     passed &= all(v['passed'] for v in summary['ledgers'].values())
249     passed &= summary['ledgers']['1']['constant_capacity_heat']['passed']
250     passed &= all(v['worst_grid_passed'] and v['all_tested_scenarios_passed'] and
    ↪ v['fine_worst_max_C_at_stop'] < .15 for v in summary['scenarios'].values())
251     passed &= summary['protected_unchanged']
252     save(AUDIT/'CHECKS.json', dict(computational_checks_passed=bool(passed), G2_approved=False,
    ↪ real_world_validation=False, test_count=len(tests)))
253
254     if not passed:
255         raise AssertionError('One or more frozen experiment gates failed; inspect saved evidence')
256
257
258 if __name__ == '__main__':
259     code = 0
260     try:
261         main()
262     except Exception:
263         code = 1
264         traceback.print_exc()
265     save(AUDIT/'EXECUTION.json', dict(command='python code/experiments.py', exit_code=code,
    ↪ python=sys.version, executable=sys.executable,
    ↪ platform=platform.platform(),
    ↪ numpy=np.__version__, scipy=scipy.__version__, stochastic=False))
266
267     sys.exit(code)
268

```

C.7 第一问端面导热与独立时间积分

```

1 """Isolated Q1 end-face audit; never changes production model or result lock."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import platform
6 import shutil
7 import time
8
9 import numpy as np
10 import scipy
11 from scipy.linalg import eigh_tridiagonal
12 from scipy.integrate import solve_ivp
13 from PIL import Image
14 from s2_solver import inputs
15
16 ROOT = Path(__file__).resolve().parents[1]
17 OUT = ROOT / 'reproduced/endface'
18 ALPHA = .36 / (820 * 2600)
19
20
21 def sha(p):
22     return hashlib.sha256(p.read_bytes()).hexdigest()
23
24
25 def spectrum(n, length, radial, h):
26     x = np.linspace(0, length, n + 1)
27     faces = np.r_[0, (x[:-1] + x[1:]) / 2, length]
28     w = np.diff(faces**2) / 2 if radial else np.diff(faces)
29     conductance = ALPHA / np.diff(x)
30     if radial:
31         conductance *= faces[1:-1]
32     diag = -np.r_[conductance, 0] - np.r_[0, conductance]
33     diag[-1] -= h / (820 * 2600) * (length if radial else 1)
34     d = diag / w
35     off = conductance / np.sqrt(w[:-1] * w[1:])
36     lam, q = eigh_tridiagonal(d, off)
37     assert lam.max() < 1e-10
38     beta = q.T @ np.sqrt(w)
39     return x, w, lam, q, beta, d, off
40
41
42 def ramp_weights(lam, env, stop):
43     total = np.zeros_like(lam)
44     for a, b in zip(env[:-1], env[1:]):
45         left, right = a[0], min(b[0], stop)
46         if left >= stop:
47             break
48         slope = (b[1] - a[1]) / (b[0] - a[0])
49         dt = right - left
50         # Stable integral exp(lambda * age) over this linear forcing segment.
51         term = np.full_like(lam, dt)
52         mask = np.abs(lam) > 1e-14
53         term[mask] = np.expm1(lam[mask] * dt) / lam[mask]

```

```

54         total += slope * np.exp(lam * (stop - right)) * term
55     return total
56
57
58 def calculation(nr, nz, h_end):
59     env, _ = inputs()
60     assert env[0, 0] == 0 and env[0, 1] == 28
61     r, wr, lr, qr, br, _ = spectrum(nr, .02, True, 25)
62     z, wz, lz, qz, bz, _ = spectrum(nz, .125, False, h_end)
63     ta = float(np.interp(1800, env[:, 0], env[:, 1]))
64     deficit_r = qr @ (br * ramp_weights(lr, env, 1800)) / np.sqrt(wr)
65     deficit = (qr @ (br[:, None] * bz[None, :] *
66                    ramp_weights(lr[:, None] + lz[None, :], env, 1800)) @ qz.T)
67     deficit /= np.sqrt(wr[:, None] * wz[None, :])
68     T = ta - deficit
69     radial = ta - deficit_r
70     delta = T - radial[:, None]
71     assert T.min() >= 28 - 1e-8 and T.max() <= ta + 1e-8
72     assert delta.min() > -1e-7
73     return dict(nr=nr, nz=nz, h_end=h_end, ambient_C=ta,
74               radial_center_C=float(radial[0]), radial_surface_C=float(radial[-1]),
75               cylinder_center_C=float(T[0, 0]), end_axis_C=float(T[0, -1]),
76               end_rim_C=float(T[-1, -1]),
77               midplane_max_delta_C=float(delta[:, 0].max()),
78               full_domain_max_delta_C=float(delta.max()),
79               mean_delta_C=float(wr @ delta @ wz / (wr.sum() * wz.sum())),
80               radial_mean_C=float(wr @ radial / wr.sum()),
81               temperature_min_C=float(T.min()), temperature_max_C=float(T.max()))
82
83
84 def independent_time_test():
85     env, _ = inputs()
86     x, w, lam, q, beta, d, off = spectrum(20, .02, True, 25)
87     sqrtw = np.sqrt(w)
88     def rhs(t, v):
89         ta = np.interp(t, env[:, 0], env[:, 1])
90         u = sqrtw * (v - ta)
91         out = d * u
92         out[1:] += off * u[:-1]
93         out[:-1] += off * u[1:]
94         return out / sqrtw
95     sol = solve_ivp(rhs, (0, 1800), np.full(len(x), 28.), method='DOP853',
96               rtol=1e-10, atol=1e-11, max_step=1)
97     assert sol.success
98     exact = np.interp(1800, env[:, 0], env[:, 1]) - q @ (
99         beta * ramp_weights(lam, env, 1800)) / sqrtw
100    err = float(np.max(np.abs(exact - sol.y[:, -1])))
101    assert err < 1e-6
102    return dict(method='DOP853 independent time integration, same coarse spatial operator',
103              ↪ max_error_C=err)
104
105 def main():
106     start = time.perf_counter()
107     OUT.mkdir(parents=True, exist_ok=True)
108     protected = [ROOT/'evidence/RESULTS_LOCK.md', ROOT/'evidence/answers.json', ROOT/'code/s2_solver.py',
109                 ROOT/'code/s2_stable_graded.py']
110     before = {str(p): sha(p) for p in protected}
111     runs = [calculation(64, 200, 25), calculation(128, 400, 25), calculation(64, 200, 0)]
112     assert runs[-1]['full_domain_max_delta_C'] < 1e-7
113     tests = independent_time_test()
114     ratios = []
115     for qnum, a, prefactor in [(1, .89, 7e-9), (3, .45, 2.4e-3*np.exp(-3850/323.15)),
116                               (4, .30, 4.2e-4*np.exp(-3850/323.15))]:
117         for c in [2.55, .15]:
118             wrong = prefactor*np.exp(-c if qnum == 1 else -a*c)
119             correct = prefactor*np.exp(-a/c)
120             ratios.append(dict(question=qnum, C=c, correct_D=correct, candidate_D=wrong,
121                               wrong_over_correct=wrong/correct,
122                               fixed_temperature_C=None if qnum == 1 else 50))
123     # Scenario only: effective dry-basis exchange is NOT an observed evaporation flux.
124     rho_d = 820/(1+2.55)
125     ca = float(np.interp(1800, inputs()[0][:, 0], inputs()[0][:, 2]))
126     j = rho_d*8e-7*(1.51023509-ca)
127     sensible_flux = 25*(runs[1]['ambient_C']-36.78556033)
128     result = dict(scope='Q1 constant properties, 1800 s, no moisture or latent coupling; diagnostic
129     ↪ only',
130                  geometry=dict(L_m=.25, R_m=.02, end_over_side=.02/.25,
131                               end_fraction_total=.02/((.25+.02), alpha_m2_s=ALPHA,
```

```

131         diffusion_length_1800_m=float(np.sqrt(ALPHA*1800))),
132         endface_runs=runs, time_integration_check=tests, diffusivity_comparison=ratios,
133         latent_scenario=dict(status='hypothetical conversion, not physical validation or
↳ production prediction',
134         assumed_lv_j_kg=2.4e6, assumed_dry_density=rho_d, implied_evap_flux_kg_m2_s=j,
135         implied_latent_W_m2=2.4e6*j, baseline_convective_W_m2=sensible_flux,
136         ratio=2.4e6*j/sensible_flux),
137         production_sha256=before, production_unchanged=before=={str(p):sha(p) for p in
↳ protected},
138         runtime=dict(python=platform.python_version(),
↳ numpy=np.__version__, scipy=scipy.__version__,
139         elapsed_s=time.perf_counter()-start),
140         source_inventory='official inputs only; no candidate image dependency')
141     assert result['production_unchanged']
142     (OUT/'DIAGNOSTICS.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
143     print(json.dumps({k:v for k,v in result.items() if
↳ k!='source_inventory'},ensure_ascii=False,indent=2))
144
145
146 if __name__ == '__main__':
147     main()

```

C.8 数表与图的完整生成程序

```

1  """Assemble locked numerical evidence; never compile TeX or modify S2."""
2  from pathlib import Path
3  import hashlib
4  import json
5  import shutil
6  import sys
7  import numpy as np
8  import matplotlib
9  matplotlib.use('Agg')
10 import matplotlib.pyplot as plt
11 from matplotlib.colors import LinearSegmentedColormap
12 from matplotlib.patches import FancyArrowPatch, Rectangle
13
14 HERE = Path(__file__).resolve().parent
15 PROJECT = HERE
16 TEX = HERE
17 DATA = TEX/'evidence'
18 FIG = TEX/'figures'
19 AUDIT = HERE/'redraw_audit'
20 for p in (TEX, DATA, FIG, TEX/'tables', TEX/'code', TEX/'data', AUDIT):
21     p.mkdir(parents=True, exist_ok=True)
22
23 plt.rcParams.update({'font.family':'sans-serif', 'font.sans-serif':['Microsoft YaHei','Arial','DejaVu
↳ Sans'],
24     'font.size':8.5, 'axes.labelsize':8.5, 'xtick.labelsize':8, 'ytick.labelsize':8,
25     'axes.spines.top':False, 'axes.spines.right':False, 'axes.linewidth':.7,
26     'legend.frameon':False, 'legend.fontsize':8, 'svg.fonttype':'none',
27     'axes.unicode_minus':False, 'savefig.dpi':360})
28 BLUE, GREEN, RED, GOLD, GREY = '#2C4A7C', '#3E7D5A', '#C65353', '#B9852D', '#66717E'
29 CMAP = LinearSegmentedColormap.from_list('moisture', ['#fbfcfc', '#b7d4cd', '#408477', '#214b5b'])
30 HEAT = LinearSegmentedColormap.from_list('heat', ['#f7fbfd', '#a4c4da', '#4b7c9f', '#873b3b'])
31 identities = {}
32 figures = []
33 def sha(p):
34     return hashlib.sha256(p.read_bytes()).hexdigest()
35 def copy(source, target):
36     shutil.copy2(source,target)
37     identities[str(source.relative_to(PROJECT)) if source.is_relative_to(PROJECT) else source.name] =
↳ sha(source)
38 def write_json(p, obj):
39     p.write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding='utf8')
40 def save(fig, name, source, role='main'):
41     fig.savefig(FIG/f'{name}.svg')
42     fig.savefig(FIG/f'{name}.png', dpi=360)
43     fig.canvas.draw()
44     figures.append(dict(id=name,source=source,role=role,width_mm=float(fig.get_figwidth()*25.4),
45     height_mm=float(fig.get_figheight()*25.4),dpi=360,png_sha256=sha(FIG/f'{name}.png')))
46     plt.close(fig)
47 def grid(ax):
48     ax.grid(axis='y', color='#e4e7e9', lw=.5)
49     ax.set_axisbelow(True)
50 def box(ax, x,y,w,h, label, color=BLUE):
51     ax.add_patch(Rectangle((x,y),w,h,facecolor='#f5f8f9',edgecolor=color,lw=1))
52     ax.text(x+w/2,y+h/2,label,ha='center',va='center',fontsize=8.5,linespacing=1.55)

```

```

53 def arrow(ax, a,b, label=None):
54     ax.add_patch(FancyArrowPatch(a,b,arrowstyle='->',mutation_scale=11, lw=1,color=GREY))
55     if label:
56         ax.text((a[0]+b[0])/2,(a[1]+b[1])/2+.03,label,ha='center',fontsize=8,color=GREY)
57
58 answers = json.loads((DATA/'answers.json').read_text(encoding='utf8'))
59 summary = json.loads((DATA/'improvement.json').read_text(encoding='utf8'))
60 with np.load(DATA/'inputs.npz') as source:
61     env, radius = source['environment'], source['radius']
62 write_json(DATA/'input_statistics.json',dict(environment_rows=len(env),radius_rows=len(radius),
63     tail_mean=env[env[:,0]>=10800,1:].mean(axis=0).tolist(),missing=0,removed=0))
64
65 fields = {}
66 for a in answers:
67     q=a['question']
68     fields[q] = dict(np.load(DATA/f'q{q}_fields.npz'))
69     # Q4 physical positions outside the current body remain empty, never zero.
70     for kind in (['temperature_C','moisture'] if q<2 else ['moisture']):
71         cap = (' 温度 ($^\circ\text{C}$)' if kind=='temperature_C' else ' 干基含水率 (kg/kg) ')
72         rows=[r'\begin{coretable}[htbp]\centering\small',
73             f'\caption{{问题 {q} 指定时刻与径向位置的 {cap}: 无潜热基线}}\label{{tab:q{q}-{kind}}}',
74             r'\begin{tabular}{r'+r*(6 if q==4 else 5)+r'}\toprule',
75             ('时间/s' if q==1 else '时间/h')+r' & 0 cm & 0.5 cm & 1 cm & 1.5 cm & 2 cm'+(r' & 实际表面' if
76             ↪ q==4 else '')+r'\midrule']
77         for r in a['table_rows']:
78             vals=r[kind][:6 if q==4 else 5]
79             t=r['time_s']/(1 if q==1 else 3600)
80             rows.append(f'{t:.0f}' if q==1 else f'{t:.4f}')
81             rows[-1] += ' & '+' & '.join(r'\textendash' if x is None else f'{x:.4f}' for x in vals)+r'\\"
82             rows.extend([r'\bottomrule\end{tabular}',r'\end{coretable}'])
83         (TEX/'tables'/f'q{q}_{kind}.tex').write_text('\n'.join(rows),encoding='utf8')
84     rows=[r'\begin{table}[htbp]\centering\small',
85         f'\caption{{问题 {q} 同为 160 区间的数值方法比较}}\label{{tab:ablation{q}}}',
86         r'\begin{tabular}{lrrrr}\toprule 方法 & $E_T$/K & $E_C$/ (kg/kg) & $E_t$/s & 耗时/s\\ \midrule']
87     for i,v in enumerate(summary['comparisons'][str(q)]):
88         event=' 不适用' if v['event_error_s'] is None else f'{v["event_error_s"]:.4f}'
89         rows.append(f"M{i} & {v['max_T_error_C']:.3e} & {v['max_C_error']:.3e} & {event} &
90         ↪ {v['elapsed_s']:.2f}"+r'\\"')
91     rows += [r'\bottomrule\end{tabular}',r'\end{table}']
92     (TEX/'tables'/f'q{q}_ablation.tex').write_text('\n'.join(rows),encoding='utf8')
93
94 sys.path.insert(0, str(HERE/'code'))
95 from answer_table_pairs import write_pairs
96 write_pairs(HERE, baseline_only=True)
97
98 fig,axs=plt.subplots(1,3,figsize=(155/25.4,65/25.4),layout='constrained')
99 for ax,y,label,col in [(axs[0],1,' 烘房温度 / °C',RED),(axs[1],2,' 有效环境水分 / (kg/kg)',GREEN)]:
100     ax.plot(env[:,0]/3600,env[:,y],color=col,lw=1.3)
101     ax.set(xlabel=' 时间 / h',ylabel=label,xlim=(0,4)); grid(ax)
102     axs[2].plot(radius[:,0]/3600,radius[:,1],color=BLUE,lw=1.3)
103     axs[2].set(xlabel=' 时间 / h',ylabel=' 药材半径 / cm'); grid(axs[2])
104 save(fig,'environment','inputs.npz; all original observations')
105
106 fig,ax=plt.subplots(figsize=(155/25.4,88/25.4))
107 ax.set(xlim=(0,1),ylim=(0,1));ax.axis('off')
108 box(ax,.02,.77,.96,.18,'附件 1: 温湿环境 • 附件 2: 收缩半径 • 附录 2/3/4: 分别给定物性')
109 for x,lab in [(0.02,'问题一\附录2, 固定半径\0-1800 s'),(.36,'问题二 / 三\附录3, 同一动力系统\全过程 +
110     ↪ 最大值判停'),(.70,'问题四\附录4, 物质坐标\半径变化 + 最大值判停')]:
111     box(ax,x,.42,.23,lab)
112     arrow(ax,(x+.14,.77),(x+.14,.65))
113 box(ax,.02,.06,.96,.22,'统一计算: 守恒通量 + 稳定状态积分 + 表面加密 + BDF\逐问检验: 基线对照 / 全场网格 /
114     ↪ 干基账本 / 严格阈值\下游: 有限情景停机回代; 物性 × 几何归因; 端面导热诊断',GREEN)
115 for x in (.16,.50,.84):arrow(ax,(x,.42),(x,.28))
116 fig.subplots_adjust(left=.01,right=.99,top=.98,bottom=.02)
117 save(fig,'overview','STAGE_AUTHORIZATION.md; frozen dependency graph')
118
119 flows={
120 1:['Step 1\附录2初始化\两场均匀初值','Step 2\柱坐标控制体\Robin边界','Step 3\薄层加密\刚性积分','Step
121     ↪ 4\逐秒导出\显热/水量核对'],
122 2:['Step 1\重新从t=0开始\附录3变物性','Step 2\温湿耦合\状态积分通量','Step 3\全程推进\前3h截面','Step
123     ↪ 4\全程逐秒回读\双网格比较'],
124 3:['Step 1\继承Q2动力系统\各处C<0.15','Step 2\全空间最大值\向下过零事件','Step
125     ↪ 3\加5s后向上舍入\名义严格停机','Step 4\8个联合角点\统一停机回代'],
126 4:['Step 1\附录4 + R(t)\干基量定义','Step 2\固相连续性\物质坐标抵消','Step
127     ↪ 3\变域高精度求解\实际表面采样','Step 4\干基账本\物性×几何消融']]
128 for q,labels in flows.items():
129     fig,ax=plt.subplots(figsize=(155/25.4,38/25.4)); ax.axis('off');ax.set(xlim=(0,1),ylim=(0,1))
130     for i,lab in enumerate(labels):
131         x=.015+i*.252;box(ax,x,.18,.215,.64,lab,GREEN if i==3 else BLUE)

```

```

124         if i<3:arrow(ax,(x+.215,.5),(x+.252,.5))
125     fig.subplots_adjust(left=0,right=1,top=1,bottom=0)
126     save(fig,f'q{q}_flow','frozen model and actual S2 steps')
127
128 for q in (1,2):
129     d=fields[q];t=d['times_s']/(1 if q==1 else 3600)
130     fig,axs=plt.subplots(1,2,figsize=(155/25.4,70/25.4),layout='constrained')
131     for ax,key,cmap,lab in [(axs[0],'T_xi',HEAT,'温度 / °C'),(axs[1],'C_xi',CMAP,'含水率 / (kg/kg)')]:
132         im=ax.pcolormesh(t,2*d['xi'],d[key].T,shading='auto',cmap=cmap,rasterized=True)
133         cb=fig.colorbar(im,ax=ax,pad=.025);cb.set_label(lab)
134         ax.set(xlabel='时间 / '+'(s' if q==1 else 'h'),ylabel='半径 / cm')
135     save(fig,f'q{q}_fields',f'q{q}_fields.npz; all stored time and xi nodes')
136
137 d=fields[3];t=d['times_s']/3600
138 fig,axs=plt.subplots(1,2,figsize=(155/25.4,72/25.4),layout='constrained')
139 for i,(col,ls) in enumerate(zip([BLUE,GREY,GOLD,GREEN,RED],['-','--','-.-',':','-'])):
140     j=i*25
141     for ax in axs:ax.plot(t,d['C_xi'][:,j],color=col,ls=ls,lw=1.35 if i in (0,4) else
142         ↪ .95,label=f'{i*.5:g} cm')
143     ax.axhline(.15,color='#242a30',lw=.8,ls='--');ax.set(xlabel='时间 / h',ylabel='含水率 /
144         ↪ (kg/kg)');grid(ax)
145     axs[0].set(xlim=(0,t[-1]),ylim=(0,2.6));axs[0].legend(ncol=2,loc='upper right')
146     axs[1].set(xlim=(30,t[-1]),ylim=(.045,.23))
147     axs[1].annotate('中心最后达标',xy=(t[-1],d['C_xi'][-1,0]),xytext=(39,.21),arrowprops=dict(arrowstyle='->'
148         ↪ ,color=BLUE),color=BLUE)
149     save(fig,'q3_drying','q3_fields.npz; all time nodes; exact 0/.5/1/1.5/2 cm')
150
151 d=fields[4];t=d['times_s']/3600
152 fig,axs=plt.subplots(1,2,figsize=(155/25.4,75/25.4),layout='constrained')
153 axs[0].set_facecolor('#e4e7e9')
154 im=axs[0].pcolormesh(t,d['r_fixed_m']*100,np.ma.masked_invalid(d['C'][:,21].T),shading='auto',cmap=CMAP)
155     ↪ ,vmin=.05,vmax=2.55,rasterized=True)
156     axs[0].plot(t,d['radius_m']*100,color=RED,lw=1.2)
157     axs[0].fill_between(t,d['radius_m']*100,2,color='#e4e7e9',zorder=2)
158     axs[0].plot(t,d['radius_m']*100,color=RED,lw=1.2,zorder=3)
159     axs[0].text(25,1.80,'灰色: 当前域外',ha='center',fontsize=8)
160     axs[0].set(xlabel='时间 / h',ylabel='绝对半径 / cm',ylim=(0,2))
161     fig.colorbar(im,ax=axs[0],pad=.02,label='含水率 / (kg/kg)')
162     for target,col,ls in zip([6,12,24,48],[BLUE,GOLD,GREEN,RED],['-','--','-.-',':']):
163         idx=np.argmin(abs(t-target));axs[1].plot(d['xi']*d['radius_m'][idx]*100,d['C_xi'][idx],color=col,ls=
164             ↪ ls,label=f'{target} h')
165     axs[1].axhline(.15,color=GREY,lw=.7,ls='--');axs[1].set(xlabel='实际半径 / cm',ylabel='含水率 /
166         ↪ (kg/kg)',xlim=(0,2),ylim=(0,2.55));axs[1].legend();grid(axs[1])
167     save(fig,'q4_domain','q4_fields.npz; NaN mask retained, selected exact times in right panel')
168
169 f=summary['factorial']['times_h']
170 fig,ax=plt.subplots(figsize=(155/25.4,60/25.4),layout='constrained')
171 for y,p,col in [(1,3,BLUE),(0,4,GREEN)]:
172     fixed,moving=f[f'p{p}_moving0'],f[f'p{p}_moving1']
173     ax.plot([moving,fixed],[y,y],color=col,lw=2)
174     ax.scatter([fixed],[y],marker='s',s=45,color=GREY,label='固定半径' if y==1 else None)
175     ax.scatter([moving],[y],marker='o',s=45,color=col,label='收缩半径' if y==1 else None)
176     for val in (fixed,moving):ax.annotate(f'{val:.4f}',(val,y),xytext=(0,9),textcoords='offset
177         ↪ points',ha='center')
178     ax.set(yticks=[0,1],yticklabels=['附录4物性','附录3物性'],ylim=(-.55,1.65),xlim=(0,145),xlabel='临界干燥时间
179         ↪ / h')
180     ax.legend(loc='upper right',ncol=2);grid(ax)
181     save(fig,'factorial','improvement.json factorial; N=320 all four cases')
182
183 fig,axs=plt.subplots(1,2,figsize=(155/25.4,75/25.4),layout='constrained')
184 for ax,q in zip(axs,(3,4)):
185     sc=summary['scenarios'][str(q)];rows=sc['scenarios']
186     for i,r in enumerate(rows):
187         ax.plot([r['max_C_at_scenario_stop'],r['max_C_at_nominal_stop']],i,i,color='#ccd2d8',lw=1)
188     ax.scatter([r['max_C_at_nominal_stop'] for r in
189         ↪ rows],range(8),marker='x',color=RED,s=28,label='名义停机回代')
190     ax.scatter([r['max_C_at_scenario_stop'] for r in
191         ↪ rows],range(8),marker='o',color=GREEN,s=21,label='统一情景停机回代')
192     ax.axvline(.15,color=GREY,lw=.8,ls='--');ax.set(xlabel='停机时全空间最大含水率 /
193         ↪ (kg/kg)',ylabel=f'问题{q}: 情景编号',yticks=range(8),yticklabels=range(1,9));ax.legend(loc='lower
194         ↪ left',fontsize=7.8)
195     save(fig,'scenarios','improvement.json scenarios; all 8 corners for each q')
196
197 # Identical display grid in 3D and 2D. Full-data 2D curve is retained in q3_drying.
198 d=fields[3];ids=np.unique(np.r_[np.arange(0,len(d['times_s']),30),len(d['times_s'])-1])
199 t=d['times_s'][ids]/3600;x=d['xi'];z=d['C_xi'][ids].T
200 tt,xx=np.meshgrid(t,x)

```

```

190 fig=plt.figure(figsize=(155/25.4,80/25.4),layout='constrained')
191 ax=fig.add_subplot(121,projection='3d');ax.plot_surface(tt,xx,z,cmap=CMAP,rstride=1,cstride=1,linewidth=]
    ↪ 0,rasterized=True)
192 ax.set(xlabel=' 时间 / h',ylabel=' 物质坐标');ax.view_init(28,225);ax.tick_params(labelsize=7)
193 ax.text2D(.02,.96,' 纵轴: 含水率 / (kg/kg)',transform=ax.transAxes,fontsize=8)
194 ax2=fig.add_subplot(122);im=ax2.contourf(tt,xx,z,levels=np.linspace(.05,2.55,21),cmap=CMAP)
195 ax2.contour(tt,xx,z,levels=[.15],colors=[RED],linewidths=1)
196 ax2.set(xlabel=' 时间 / h',ylabel=' 物质坐标');fig.colorbar(im,ax=ax2,label=' 含水率 / (kg/kg)')
197 save(fig,'q3_surface_candidate',f'q3_fields.npz; displayed {len(ids)} of {len(d["times_s"])} times, every
    ↪ 30 stored rows plus endpoint; both panels same grid','candidate')
198
199 write_json(AUDIT/'DRAWING_SCOPE.json',dict(
200     image_generation_used=False, external_bitmap_required=False,
201     mechanism_source='engineering_figures.py',
202     flowcharts='Run engineering_figures.py after this data-plot baseline.',
203     website_chart='website_figures.cjs; frozen export is included.'))
204 write_json(AUDIT/'FIGURES.json',figures)
205 write_json(AUDIT/'SOURCE_HASHES.json',identities)
206 print(json.dumps(dict(figures=len(figures),tables=len(list((TEX/'tables').glob('*.tex'))),tail=env[env[:]
    ↪ ,0]>=10800,1:].mean(axis=0).tolist()),ensure_ascii=False))

```

C.9 本轮物理模型对照与参数扰动

```

1 """Additional diagnostics, not replacement primary answers. Run from any cwd."""
2 from pathlib import Path
3 import json
4 import numpy as np
5 from scipy.integrate import solve_ivp
6 from s2_solver import Model, Settings
7 from s2_stable_graded import StableModel
8
9 BASE = Path(__file__).resolve().parents[1]
10 OUT = BASE / 'supplement'
11 OUT.mkdir(exist_ok=True)
12
13
14 def simulate(q, name, **kwargs):
15     path = OUT / f'q{q}_{name}.json'
16     if path.exists():
17         return json.loads(path.read_text(encoding='utf8'))
18     s = Settings(question=q, n=160, **kwargs)
19     model = Model(s) if s.frozen else StableModel(s, graded=True)
20     model.run()
21     ts = np.linspace(0, 1800 if q == 1 else 10800, 181)
22     field = model.state(ts)
23     C, T = field[model.size:], field[:model.size]
24     result = {'meta': model.meta, 'time_s': ts.tolist(),
25             'Tc': T[0].tolist(), 'Ts': T[-1].tolist(),
26             'Cc': C[0].tolist(), 'Cs': C[-1].tolist(),
27             'Cmean': (2*model.w @ C).tolist(),
28             'Tmean': (2*model.w @ T).tolist()}
29     path.write_text(json.dumps(result, indent=2), encoding='utf8')
30     print(f'completed q{q} {name}', flush=True)
31     return result
32
33
34 def main():
35     runs = {}
36     for q in (1, 2):
37         runs[f'q{q}_base'] = simulate(q, 'base')
38         for par in ('D_scale', 'h_scale', 'hm_scale'):
39             for scale, label in ((.9, 'minus'), (1.1, 'plus')):
40                 key = f'q{q}_{par}_{label}'
41                 runs[key] = simulate(q, f'{par}_{label}', **{par: scale})
42     frozen = simulate(2, 'frozen', frozen=True)
43     envmodel = Model(Settings(question=1, n=160))
44     ts = np.asarray(runs['q1_base']['time_s'])
45     sol = solve_ivp(lambda t, y: 2*25/(820*2600*.02)*(envmodel.environment(t)[0]-y),
46                     (0, 1800), [28.], t_eval=ts, rtol=1e-10, atol=1e-11, max_step=2.)
47     assert sol.success
48     lumped = {'time_s': ts.tolist(), 'T': sol.y[0].tolist()}
49     (OUT/'q1_lumped.json').write_text(json.dumps(lumped, indent=2), encoding='utf8')
50     sensitivities = []
51     for q in (1, 2):
52         base = runs[f'q{q}_base']
53         for par in ('D_scale', 'h_scale', 'hm_scale'):
54             low, high = (runs[f'q{q}_{par}_{v}'] for v in ('minus', 'plus'))
55             row = {'question': q, 'parameter': par}

```

```

56         for metric in ('Tc', 'Ts', 'Cc', 'Cs', 'Cmean'):
57             b, lo, hi = (v[metric][-1] for v in (base, low, high))
58             row[metric] = {'base': b, 'minus': lo, 'plus': hi,
59                             'central_difference_per_scale': (hi-lo)/.2}
60         sensitivities.append(row)
61     # A uniform frozen model is compared to a uniform variable model as well.
62     uniform = Model(Settings(question=2, n=160))
63     uniform.run()
64     y = uniform.state(np.array([10800.]))[:, 0]
65     comparison = {'q1_lumped_T1800': float(sol.y[0, -1]),
66                  'q1_radial_Tc': runs['q1_base']['Tc'][-1],
67                  'q1_radial_Ts': runs['q1_base']['Ts'][-1],
68                  'q2_frozen': {k: frozen[k][-1] for k in ('Tc', 'Ts', 'Cc', 'Cs')},
69                  'q2_variable_uniform': dict(zip(('Tc', 'Ts', 'Cc', 'Cs'),
70                                                  map(float, (y[0], y[uniform.size-1], y[uniform.size], y[-1]))))}
71     summary = {'scope': '160 intervals, diagnostics only, +/-10% is a designed perturbation, not a
72     ↪ confidence interval',
73               'comparison': comparison, 'sensitivity': sensitivities}
74     (OUT/'SUMMARY.json').write_text(json.dumps(summary, indent=2), encoding='utf8')
75     print(json.dumps(comparison, indent=2), flush=True)
76
77 if __name__ == '__main__':
78     main()

```

C.10 总时长扰动、独立数值核验与表格生成

```

1  """Q3 OAT time diagnostics. No changes to the production solver or answers."""
2  from pathlib import Path
3  from dataclasses import dataclass
4  import argparse
5  import hashlib
6  import json
7  import platform
8  import sys
9  import numpy as np
10 import scipy
11 from scipy.integrate import solve_ivp
12 from s2_solver import Settings
13 from s2_stable_graded import StableModel
14
15 ROOT = Path(__file__).resolve().parents[1]
16
17
18 @dataclass
19 class Perturbation:
20     name: str = 'base'
21     ea_scale: float = 1.0
22     initial_scale: float = 1.0
23     D_scale: float = 1.0
24     h_scale: float = 1.0
25     hm_scale: float = 1.0
26
27
28 CASES = [Perturbation()]
29 for key, values in [('ea_scale', (.95, 1.05)), ('D_scale', (.9, 1.1)),
30                   ('initial_scale', (.9, 1.1)), ('h_scale', (.9, 1.1)),
31                   ('hm_scale', (.1, 10.))]:
32     for value in values:
33         CASES.append(Perturbation(name=f'{key}_{value:g}', **{key: value}))
34
35
36 class PerturbedModel(StableModel):
37     def __init__(self, case, n, method='BDF'):
38         if min(case.ea_scale, case.initial_scale, case.D_scale,
39               case.h_scale, case.hm_scale) <= 0:
40             raise ValueError('All perturbation multipliers must be positive')
41         self.case = case
42         super().__init__(Settings(question=3, n=n, max_hours=240,
43                               D_scale=case.D_scale, h_scale=case.h_scale,
44                               hm_scale=case.hm_scale, method=method))
45
46     def properties(self, T, C):
47         rho, cp, k, A, beta = super().properties(T, C)
48         return rho, cp, k, A*np.exp(-3850*(self.case.ea_scale-1)/(T+273.15)), beta
49
50     def _solve(self, left, right, y, event=None):
51         # Intercept only the original initial condition, not later restarts.
52         if left == 0:

```

```

53         y = y.copy()
54         y[self.size:] = 2.55*self.case.initial_scale
55         return super()._solve(left, right, y, event)
56
57
58 class QuadratureCheck(PerturbedModel):
59     """Separate flux assembly and Radau integration, shared physical assumptions."""
60     def rhs(self, t, y):
61         T, C = y[:self.size], y[self.size:]
62         if np.min(C) <= 0:
63             raise ValueError('Nonpositive moisture')
64         rho, cp, k = 650+128*C, 1450+2736*C/(1+C), .21+.38*C/(1+C)
65         A = 2.4e-3*np.exp(-3850*self.case.ea_scale/(T+273.15))
66         z, w = np.polynomial.legendre.leggauss(12)
67         mid, half = (C[:-1]+C[1:])/2, np.diff(C)/2
68         integral = half*np.sum(w[:, None]*np.exp(-.45/(mid+z[:, None]*half)), axis=0)
69         radius = .02
70         Ta, Ca = self.environment(t)
71         heat, water = np.zeros(self.size+1), np.zeros(self.size+1)
72         heat[1:-1] = self.faces[1:-1]*(k[:-1]+k[1:])/2*np.diff(T)/self.dx
73         water[1:-1] = self.faces[1:-1]*(A[:-1]+A[1:])/2*integral/self.dx*self.case.D_scale
74         heat[-1] = radius*25*self.case.h_scale*(Ta-T[-1])
75         water[-1] = radius*8e-7*self.case.hm_scale*(Ca-C[-1])
76         return np.r_[np.diff(heat)/(radius**2*self.w*rho*cp),
77                     np.diff(water)/(radius**2*self.w)]
78
79
80 def sha(path):
81     return hashlib.sha256(path.read_bytes()).hexdigest()
82
83
84 def formula_tests():
85     errors = []
86     for case in CASES:
87         model = PerturbedModel(case, 8)
88         T = np.array([28., 40., 50.])
89         C = np.array([.05, .15, 2.55])
90         _, A, beta = model.properties(T, C)
91         actual = A*np.exp(-beta/C)*case.D_scale
92         expected = 2.4e-3*case.D_scale*np.exp(-3850*case.ea_scale/(T+273.15)-.45/C)
93         errors.append(float(np.max(np.abs(actual/expected-1))))
94     model = PerturbedModel(Perturbation(initial_scale=1.1), 8)
95     sol = model._solve(0, .01, np.r_[np.full(9, 28.), np.full(9, 2.55)])
96     assert np.all(sol.y[9:, 0] == 2.55*1.1)
97     rejected = False
98     try:
99         PerturbedModel(Perturbation(initial_scale=0), 8)
100     except ValueError:
101         rejected = True
102     assert rejected and max(errors) < 1e-13
103     return dict(cases=len(errors), max_relative_error=max(errors),
104               actual_initial_state_checked=True, nonpositive_rejected=rejected)
105
106
107 def main():
108     parser = argparse.ArgumentParser()
109     parser.add_argument('--out', required=True)
110     parser.add_argument('--grid', nargs='+', type=int, default=[320, 640])
111     parser.add_argument('--check', action='store_true')
112     args = parser.parse_args()
113     out = Path(args.out).resolve()
114     out.mkdir(parents=True, exist_ok=True)
115     identity = {str(p.relative_to(ROOT)): sha(p) for p in
116                [Path(__file__), ROOT/'code/s2_solver.py', ROOT/'code/s2_stable_graded.py',
117                 ROOT/'data/input1.xlsx', ROOT/'data/input2.xlsx']}
118     contract = dict(identity=identity, python=sys.version, scipy=scipy.__version__,
119                    numpy=np.__version__, platform=platform.platform(),
120                    command=sys.argv, test=formula_tests())
121     (out/'environment.json').write_text(json.dumps(contract, indent=2), encoding='utf8')
122     runs = [(case, n, False) for n in args.grid for case in CASES]
123     if args.check:
124         runs += [(case, 320, True) for case in CASES
125                  if case.name in ('base', 'ea_scale_1.05', 'hm_scale_0.1')]
126     for case, n, independent in runs:
127         name = f'{case.name}_n{n}' + ('_radau' if independent else '')
128         path = out/f'{name}.json'
129         if path.exists():
130             saved = json.loads(path.read_text(encoding='utf8'))
131             if saved['identity'] != identity:

```

```

132         raise RuntimeError('Refusing stale cache: '+name)
133     continue
134     print('START '+name, flush=True)
135     model = (QuadratureCheck(case, n, 'Radau') if independent else PerturbedModel(case, n)).run()
136     times = np.array([0., model.critical-1, model.critical, model.end])
137     states = model.state(times)
138     g = states[model.size:].max(axis=0)-.15
139     assert g[1] > 0 and abs(g[2]) < 1e-10 and g[3] < 0
140     accepted = np.concatenate([p.y[model.size:] for p in model.pieces], axis=1)
141     payload = dict(identity=identity, case=vars(case), meta=model.meta,
142                    verification_method='Radau + 12-point Gauss flux' if independent else 'BDF +
143                    ↪ stable integral flux',
144                    initial_C=float(states[model.size, 0]), threshold_times_s=times.tolist(),
145                    threshold_g=g.tolist(), minimum_accepted_C=float(accepted.min()),
146                    maximum_accepted_C=float(accepted.max()),
147                    max_outward_increase=float(np.diff(accepted, axis=0).max()), exit_code=0)
148     path.write_text(json.dumps(payload, indent=2), encoding='utf8')
149     print(f'DONE {name}: {model.critical/3600:.8f} h; {model.meta["elapsed_s"]:.1f} s', flush=True)
150
151 if __name__ == '__main__':
152     main()

1 """Validate saved OAT runs, then generate the paper table from JSON (no PDF)."""
2 import csv
3 import argparse
4 import json
5 from pathlib import Path
6 from time_sensitivity import CASES, ROOT, sha
7
8 DATA = ROOT/'supplement/time_sensitivity'
9 LABELS = {
10     'base': ('基准', '附录 3 物性, 固定半径'),
11     'ea_scale_0.95': (r'$E_a$ 降低 5%', '温度因子增大, 扩散加快'),
12     'ea_scale_1.05': (r'$E_a$ 增加 5%', '温度因子减小, 扩散变慢'),
13     'D_scale_0.9': (r'$A_0$ 降低 10%', '全程有效扩散系数缩小'),
14     'D_scale_1.1': (r'$A_0$ 增加 10%', '全程有效扩散系数增大'),
15     'initial_scale_0.9': (r'$C_0$ 降低 10%', '初始水量及初始物性同时响应'),
16     'initial_scale_1.1': (r'$C_0$ 增加 10%', '初始水量及初始物性同时响应'),
17     'h_scale_0.9': (r'$h$ 降低 10%', '主要改变早期升温过程'),
18     'h_scale_1.1': (r'$h$ 增加 10%', '主要改变早期升温过程'),
19     'hm_scale_0.1': (r'$h_m$ 缩至 0.1 倍', '外部传质阻力大幅增大'),
20     'hm_scale_10': (r'$h_m$ 放大 10 倍', '外部阻力降低, 仍受内部扩散限制'),
21 }
22
23
24 def main():
25     global DATA
26     parser = argparse.ArgumentParser()
27     parser.add_argument('--data', type=Path, default=DATA)
28     args = parser.parse_args()
29     DATA = args.data.resolve()
30     checks, rows = [], []
31     def check(name, condition, **detail):
32         checks.append(dict(name=name, passed=bool(condition), **detail))
33
34     def read(name, n, suffix=''):
35         path = DATA/f'{name}_n{n}{suffix}.json'
36         item = json.loads(path.read_text(encoding='utf8'))
37         check(path.name+'identity', all(sha(ROOT/k) == v for k, v in item['identity'].items()))
38         check(path.name+'_threshold', item['threshold_g'][1] > 0
39              and abs(item['threshold_g'][2]) < 1e-10 and item['threshold_g'][3] < 0)
40         check(path.name+'_bounds', item['minimum_accepted_C'] > 0
41              and item['maximum_accepted_C'] <= item['initial_C']+1e-8)
42         check(path.name+'_initial', abs(item['initial_C']-2.55*item['case']['initial_scale']) < 1e-12)
43         return item
44
45     base = read('base', 640)['meta']['critical_h']
46     for case in CASES:
47         low, high = read(case.name, 320), read(case.name, 640)
48         diff = abs(high['meta']['critical_s']-low['meta']['critical_s'])
49         check(case.name+'_mesh', diff < 1, difference_s=diff)
50         varied = sum(v != 1 for k, v in high['case'].items() if k != 'name')
51         check(case.name+'_OAT', varied == (0 if case.name == 'base' else 1))
52         rows.append(dict(case=case.name, critical_h=high['meta']['critical_h'],
53                         strict_h=high['meta']['end_h'],
54                         relative_percent=100*(high['meta']['critical_h']/base-1),
55                         grid_difference_s=diff,
56                         max_C_at_strict=high['meta']['max_C_at_end']))

```

```

57 cross = []
58 for name in ('base', 'ea_scale_1.05', 'hm_scale_0.1'):
59     bdf, radau = read(name, 320), read(name, 320, '_radau')
60     diff = abs(bdf['meta']['critical_s']-radau['meta']['critical_s'])
61     check(name+'independent_numerics', diff < 1, difference_s=diff)
62     cross.append(dict(case=name, difference_s=diff))
63 check('baseline_locked_regression', abs(base-57.47400138312419)*3600 < 1)
64 report = dict(ready=all(c['passed'] for c in checks), checks=checks,
65             baseline_critical_h=base, rows=rows, crosschecks=cross,
66             maximum_grid_difference_s=max(r['grid_difference_s'] for r in rows),
67             maximum_crosscheck_difference_s=max(r['difference_s'] for r in cross),
68             scope='Designed OAT, not confidence intervals or global sensitivity; shared physical
        ↪ closures')
69 (DATA/'VALIDATION.json').write_text(json.dumps(report, indent=2, ensure_ascii=False),
    ↪ encoding='utf8')
70 if not report['ready']:
71     raise RuntimeError('Validation failed; no paper table generated')
72 with (DATA/'time_sensitivity.csv').open('w', encoding='utf-8-sig', newline='') as f:
73     writer = csv.DictWriter(f, fieldnames=list(rows[0]))
74     writer.writeheader()
75     writer.writerows(rows)
76 lines = [r'\begin{table}[htbp]\centering\small',
77         r'\caption{问题三单参数扰动对总烘干时长的影响}\label{tab:q3-time-sensitivity}',
78         r'\begin{group}\setlength{\tabcolsep}{3pt}\renewcommand{\arraystretch}{1.12}',
79         r'\begin{tabularx}{\linewidth}{@{}lrrr>{\raggedright\arraybackslash}X@{}}\toprule',
80         r' 扰动 & 临界/h & 严格/h & 变化/\% & 机理解释\\midrule']
81 for r in rows:
82     label, note = LABELS[r['case']]
83     lines.append(f'{label} & {r["critical_h"]:.4f} & {r["strict_h"]:.4f} & '
84                f'{r["relative_percent"]:+.3f} & {note}' + r'\\')
85 lines += [r'\bottomrule\end{tabularx}\end{group}',
86          r'\par\smallskip\begin{minipage}{\linewidth}\footnotesize',
87          '均采用640区间表面加密网格；变化率相对本表未舍入的基准临界时间计算。严格时间为临界后继续5
        ↪ s并向上对齐0.0001 h，且回代确认全网格最大含水率低于0.15 kg/kg。',
88          f'全部11组的320/640网格时间差最大{report["maximum_grid_difference_s"]:.3f} s；
        ↪ 三组Radau及独立通量组装核验的最大差{report["maximum_crosscheck_difference_s"]:.6f} s。',
89          r'\end{minipage}\end{table}']
90 (ROOT/'tables/q3_time_sensitivity.tex').write_text('\n'.join(lines)+'\n', encoding='utf8')
91 print(json.dumps({k: report[k] for k in ('ready', 'baseline_critical_h',
92     'maximum_grid_difference_s', 'maximum_crosscheck_difference_s', 'rows')},
93     ensure_ascii=False, indent=2))
94
95
96 if __name__ == '__main__':
97     main()

```

C.11 分层框架与新增数值图表

```

1 """Deterministic diagrams and actual supplementary experiment plots."""
2 from pathlib import Path
3 import json
4 import numpy as np
5 import matplotlib
6 matplotlib.use('Agg')
7 import matplotlib.pyplot as plt
8 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch, Rectangle
9
10 ROOT = Path(__file__).resolve().parent
11 FIG = ROOT/'figures'
12 plt.rcParams.update({'font.family': 'sans-serif', 'font.sans-serif': ['Microsoft YaHei'],
13     'font.size': 10, 'axes.unicode_minus': False, 'svg.fonttype': 'none',
14     'pdf.fonttype': 42, 'axes.spines.top': False, 'axes.spines.right': False,
15     'axes.linewidth': .65, 'lines.linewidth': 1.4, 'legend.frameon': False})
16 COL = {'input': '#E8EEDB', 'model': '#DBE7F0', 'solve': '#FBEDCE',
17     'check': '#E4E0EF', 'output': '#F4DFDA', 'ink': '#304044', 'green': '#247F78', 'red': '#B45249'}
18
19
20 def save(fig, name):
21     for ext in ('svg', 'pdf', 'png'):
22         fig.savefig(FIG/f'{name}.{ext}', dpi=450, facecolor='white')
23     plt.close(fig)
24
25
26 def canvas(height):
27     fig = plt.figure(figsize=(155/25.4, height/25.4))
28     ax = fig.add_axes([0, 0, 1, 1], xlim=(0, 155), ylim=(0, height))
29     ax.axis('off')
30     return fig, ax
31

```

```

32
33 def box(ax,x,y,w,h,label,kind='model',fs=9):
34     ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='round,pad=0.15,rounding_size=.7',
35                               lw=.55,ec='#899697',fc=COL[kind]))
36     ax.text(x+w/2,y+h/2,label,ha='center',va='center',fontsize=fs,color=COL['ink'],linespacing=1.35)
37
38
39 def arrow(ax,start,end):
40     ax.add_patch(FancyArrowPatch(start,end,arrowstyle='->',mutation_scale=7,lw=.65,color='#737E80'))
41
42
43 QUESTIONS = [
44     ('问题一', '附录 2 物性 · 环境观测 · 均匀初态',
45      ['径向热传导', '含水率相关扩散', '中心与表面边界', '传递时间尺度'],
46      ['显热收支', '非线性通量', 'Robin 交换', 'Biot / Fourier 数'],
47      '环形控制体守恒离散 + 表层加密 + BDF 刚性积分',
48      ['零维模型对照', '系数扰动 / 端面诊断', '网格误差 / 水热收支'],
49      '七时刻 × 五位置表: 逐秒温度与含水率'),
50     ('问题二', '附录 3 统一公式 · 从初态重新开始',
51      ['温度影响扩散', '含水率改变物性', '强退化状态积分', '环境时序插值'],
52      ['Arrhenius 项', '热容量与导热率', '近状态高斯积分', '观测区间线性插值'],
53      '两场联立有效模型 + 稀疏 Jacobian + 自适应积分',
54      ['冻结物性实算对照', '三系数扰动检验', '全程逐秒网格复核'],
55      '前 3 小时截面表: 全过程逐秒数据与第三问同源'),
56     ('问题三', '完整继承问题二动力系统 · 全域阈值 0.15',
57      ['全空间最大值', '首次向下过阈值', '后期薄层误差', '外推环境不确定性'],
58      ['不以均值判停', '连续输出事件', '三种数值方案', '八个压力角点'],
59      '临界时间定位 → 继续积分 → 未舍入严格可行检查',
60      ['网格 / 事件误差', '同一停机时间回代', '情景范围与代价'],
61      '名义严格停机: 每 6 小时截面: 有限情景停机'),
62     ('问题四', '附录 4 独立物性 · 实测半径 · 干基状态',
63      ['干物质连续性', '水分连续性', '随固体坐标', '半径保形插值'],
64      ['固体运动测度', '相对扩散通量', '坐标项相消', '禁止无依据外推'],
65      '固定物质网格求解 + 实际表面读取 + 域外掩膜',
66      ['材料制造解 / 收支', '物性 × 几何四格对照', '八角点统一停机'],
67      '收缩模型停机: 变域截面: 有条件的效应归因')]
68
69
70 def workflows():
71     for q, (_, inp, top, bottom, solve, checks, out) in enumerate(QUESTIONS, 1):
72         fig, ax = canvas(91)
73         box(ax, 24, 79, 107, 10, inp, 'input', 10)
74         for i in range(4):
75             x = 3 + i * 38
76             box(ax, x, 60, 35, 10, top[i], 'model', 9.3)
77             box(ax, x, 45, 35, 10, bottom[i], 'solve', 8.9)
78             arrow(ax, (77.5, 79), (x + 17.5, 70))
79             arrow(ax, (x + 17.5, 60), (x + 17.5, 55))
80             arrow(ax, (x + 17.5, 45), (77.5, 39))
81             box(ax, 10, 28, 135, 11, solve, 'model', 9.2)
82             for i in range(3):
83                 x = 3 + i * 51
84                 box(ax, x, 12, 47, 10, checks[i], 'check', 8.8)
85                 arrow(ax, (77.5, 28), (x + 23.5, 22))
86                 arrow(ax, (x + 23.5, 12), (77.5, 8))
87             box(ax, 10, 5, 135, 7.5, out, 'output', 9)
88             save(fig, f'q{q}_flow')
89         fig, ax = canvas(173)
90         for q, (title, inp, top, bottom, solve, checks, out) in enumerate(QUESTIONS):
91             y = 132 - q * 43
92             ax.add_patch(Rectangle((1, y), 153, 37, fill=False, ec='#ACBABB', lw=.65, linestyle=(0, (3, 2))))
93             box(ax, 3, y + 2, 14, 33, title.replace('问题', '问题\n'), 'input', 10)
94             box(ax, 21, y + 27, 129, 8, inp, 'input', 9)
95             for j, label in enumerate(top):
96                 x = 21 + j * 33
97                 box(ax, x, y + 14, 30, 8, label, 'model', 8.5)
98                 arrow(ax, (85.5, y + 27), (x + 15, y + 22))
99             box(ax, 21, y + 2, 81, 8, ['环形控制体守恒离散', '两场联立有效模型', '全空间事件 +
100 ↪ 严格可行回代', '固定物质网格求解'][q], 'solve', 8.8)
101             box(ax, 108, y + 2, 42, 8, ['两场输出与端面诊断', '全场输出与扰动检验', '严格判停与情景回代', '变域输出与四格归
102 ↪ 因'][q], 'output', 8.5)
103             for j in range(4): arrow(ax, (36 + j * 33, y + 14), (61.5, y + 10))
104             arrow(ax, (102, y + 6), (108, y + 6))
105             if q < 3:
106                 ax.text(85, y - 1, ['不同物性, 独立重新初始化', '相同动力系统, 增加终止判据', '独立物性与收缩闭合,
107 ↪ 不直接继承数值'][q],
108                        ha='center', va='top', fontsize=7.8, color='#596568')
109             save(fig, 'overview')
110
111

```

```

109 def control_volume():
110     fig,ax=canvas(64)
111     ax.text(3,60,'(a) 共享界面通量与环形控制体',fontsize=10,weight='bold')
112     for i in range(3):
113         box(ax,5+i*23,27,22,18,[r'$C_{i-1}$',r'$C_i$',r'$C_{i+1}$'][i],'input' if i!=1 else 'solve',11)
114     for x in (28,51):
115         arrow(ax,(x-7,50),(x+7,50))
116     ax.text(28,54,r'$J_{i-1/2}$',ha='center',fontsize=9)
117     ax.text(51,54,r'$J_{i+1/2}$',ha='center',fontsize=9)
118     ax.text(39,21,r'$w_i=(\eta_{i+1/2}^2-\eta_{i-1/2}^2)/2$',ha='center',fontsize=9)
119     ax.text(39,9,'共享一次通量,左右相反入账\n 全域求和后只剩边界交换',ha='center',va='center',fontsize=9)
120     ax.plot([79,79],[4,60],color='#C5CECE',lw=.6)
121     ax.text(83,60,'(b) 同一物质点的几何映射',fontsize=10,weight='bold')
122     ax.plot([88,150],[44,44],color=COL['ink'])
123     ax.plot([88,127],[26,26],color=COL['ink'])
124     for eta in (0,.25,.5,.75,1):
125         a,b=88+62*eta,88+39*eta
126         ax.plot(a,44,'o',ms=3,color=COL['green'])
127         ax.plot(b,26,'o',ms=3,color=COL['green'])
128         arrow(ax,(a,42),(b,28))
129     ax.text(119,49,r'$r=\eta R(0)$',ha='center',fontsize=10)
130     ax.text(111,20,r'$r=\eta R(t)$',ha='center',fontsize=10)
131     ax.text(117,7,'固定物质坐标  $\eta$ , 不额外增加失水项',ha='center',fontsize=8.8)
132     save(fig,'contro1_volume')
133
134
135 def diagnostic_figures():
136     p=ROOT/'supplement/SUMMARY.json'
137     if not p.exists(): return
138     data=json.loads(p.read_text(encoding='utf8'))
139     for q in (1,2):
140         fig,axes=plt.subplots(1,2,figsize=(155/25.4,63/25.4),layout='constrained')
141         rows=[r for r in data['sensitivity'] if r['question']==q]
142         for ax,metric,label in zip(axes,['Ts','Cmean'],['表面温度变化 / °C','平均含水率变化 / (kg/kg)']):
143             for y,row in enumerate(rows):
144                 d=row[metric]
145                 for key,marker,color in [('-', 'o', COL['green']), ('+', 's', COL['red'])]:
146                     val=d[key]-d['base']
147                     ax.plot([0,val],[y,y],color=color,alpha=.6,lw=1)
148                     ax.scatter(val,y,s=26,marker=marker,color=color,label=(-'10%' if key=='minus' else
149                                     ↪ '+10%') if y==0 else None)
150                     ax.axvline(0,lw=.7,color='#606D70',ls='--')
151                     ax.set_yticks(range(3),['扩散倍率','换热系数','传质系数'],fontsize=9)
152                     ax.set_xlabel(label,fontsize=9)
153                     ax.tick_params(labelsize=8)
154                     ax.set_ylim(-.6,2.7)
155                     axes[1].legend(fontsize=8,ncol=2,loc='upper center')
156                     save(fig,f'q{q}_sensitivity')
157     base=json.loads((ROOT/'supplement/q1_base.json').read_text())
158     lump=json.loads((ROOT/'supplement/q1_lumped.json').read_text())
159     fig,ax=plt.subplots(figsize=(155/25.4,59/25.4),layout='constrained')
160     for key,label,color,ls in [ ('Tc','径向模型: 中心','247F78','-'),('Ts','径向模型: 表面','#B45249','-'),
161                               ('Tmean','径向模型: 体积均温','#677582','--')]:
162         ax.plot(base['time_s'],base[key],label=label,color=color,ls=ls)
163         ax.plot(lump['time_s'],lump[key],label='集中参数: 均温',color='#996F32',ls='-')
164         ax.set_xlabel='时间 / s',ylabel='温度 / °C',xlim=(0,1800))
165         ax.legend(fontsize=8.5,ncol=2)
166         save(fig,'q1_physical_compare')
167
168 def tables():
169     p=ROOT/'supplement/SUMMARY.json'
170     if not p.exists(): return
171     s=json.loads(p.read_text())
172     for q in (1,2):
173         rows=[r for r in s['sensitivity'] if r['question']==q]
174         text=[r'\begin{table}[htbp]\centering\small',
175              rf'\caption{{问题 {q} 终点的单因素扰动结果 (160 区间诊断)}}\label{{tab:sensitivity{q}}}',
176              r'\begin{tabular}{llrrrr}\toprule',
177              r' 参数 & 倍率 & $T_c$/°C & $T_s$/°C & $C_c$ & $\bar{C}_s$\\midrule']
178         for row,name in zip(rows,['扩散系数','换热系数','传质系数']):
179             for k,mult in [('-',.9),('+',1.1)]:
180                 vals=' & '.join(f'{row[z][k]:.4f}' for z in ('Tc','Ts','Cc','Cmean'))
181                 text.append(f'{name} & {mult:.1f} & {vals}'+r'\\')
182         text += [r'\bottomrule\end{tabular}',r'\par\smallskip\footnotesize 含水率单位为kg/kg;
183                 ↪ 每行仅改变一项系数,其余条件不变.',r'\end{table}']
184         (ROOT/f'tables/q{q}_sensitivity.tex').write_text('\n'.join(text),encoding='utf8')
185     c=s['comparison']
186     text=[r'\begin{table}[htbp]\centering\small',
187          r'\caption{{问题二冻结与变物性的同网格终点对照}}\label{{tab:frozen}}',
188          r'\begin{tabular}{llrrrr}\toprule 模型 & $T_c$/°C & $T_s$/°C & $C_c$ & $C_s$\\midrule']

```

```

188 for key,label in [('q2_frozen',' 初态冻结物性'),('q2_variable_uniform',' 统一变物性')]:
189     text.append(label+' & '+' & '.join(f'{c[key][k]:.4f}' for k in ('Tc','Ts','Cc','Cs'))+r'\\\')
190 text += [r'\bottomrule\end{tabular}',r'\par\smallskip\footnotesize 两者均为160区间均匀网格、相同初边值;
↪ 终点3 h。含水率单位kg/kg。',r'\end{table}']
191 (ROOT/'tables/q2_frozen.tex').write_text('\n'.join(text),encoding='utf8')
192
193
194 if __name__=='__main__':
195     workflows()
196     control_volume()
197     diagnostic_figures()
198     tables()

```

C.12 工程示意与分问逻辑图

```

1 """Editable engineering diagrams; no textures, generated images or measured pores."""
2 from pathlib import Path
3 import json
4 import numpy as np
5 import matplotlib
6 matplotlib.use('Agg')
7 import matplotlib.pyplot as plt
8 from matplotlib.patches import Rectangle, Circle, Polygon, FancyArrowPatch
9
10 ROOT = Path(__file__).resolve().parent
11 FIG = ROOT / 'figures'
12 plt.rcParams.update({'font.family':'sans-serif', 'font.sans-serif':['Microsoft YaHei'],
13 'font.size':9, 'svg.fonttype':'none', 'pdf.fonttype':42,
14 'axes.unicode_minus':False, 'lines.linewidth':.8})
15 INK='#252B30'; BLUE='#286378'; GRAY='#79848B'
16
17
18 def canvas(h):
19     fig=plt.figure(figsize=(155/25.4,h/25.4))
20     ax=fig.add_axes([0,0,1,1],xlim=(0,155),ylim=(0,h))
21     ax.axis('off')
22     return fig,ax
23
24
25 def save(fig,name):
26     fig.savefig(FIG/f'{name}.svg',facecolor='white')
27     fig.savefig(FIG/f'{name}.pdf',facecolor='white')
28     fig.savefig(FIG/f'{name}.png',dpi=450,facecolor='white')
29     plt.close(fig)
30
31
32 def label(ax,x,y,s,size=9,align='center',color=INK):
33     ax.text(x,y,s,fontsize=size,ha=align,va='center',color=color,linespacing=1.45)
34
35
36 def node(ax,x,y,w,h,s,accent=False):
37     ax.add_patch(Rectangle((x,y),w,h,ec=BLUE if accent else INK,
38 fc='#EEF4F6' if accent else 'white',lw=.8))
39     label(ax,x+w/2,y+h/2,s)
40
41
42 def edge(ax,points,s=None,at=None):
43     xx,yy=zip(*points)
44     ax.plot(xx,yy,color=GRAY,lw=.85)
45     ax.add_patch(FancyArrowPatch(points[-2],points[-1],arrowstyle='->',
46 mutation_scale=8,lw=.85,color=GRAY,shrinkA=0,shrinkB=0))
47     if s: label(ax,*at,s,8,color=INK)
48
49
50 def dim(ax,a,b,s,offset=(0,2)):
51     ax.annotate('',xy=b,xytext=a,arrowprops={'arrowstyle':'<->','lw':.65,'color':INK})
52     label(ax,(a[0]+b[0])/2+offset[0],(a[1]+b[1])/2+offset[1],s,8)
53
54
55 def mechanisms():
56     fig,ax=canvas(72)
57     for x in (52,104): ax.plot([x,x],[9,67],color='#CED3D5',lw=.5)
58     label(ax,26,66,'(a) 纵剖面: 表面交换')
59     ax.add_patch(Rectangle((8,27),36,17,fc='white',ec=INK,hatch='///',lw=1))
60     ax.plot([4,48],[35.5,35.5],color=GRAY,ls='-.',lw=.7)
61     for x in (13,26,39):
62         edge(ax,[(x,56),(x,45)])
63         edge(ax,[(x,26),(x,17)])
64     label(ax,26,60,' 热量进入: h(Ta - Ts)',8)

```

```

65 label(ax,26,12,'水分外迁: hm(Cs - Ca)',8)
66 label(ax,78,66,'(b) 横截面: 径向扩散')
67 ax.add_patch(Circle((78,36),18,fill=False,ec=INK,lw=1))
68 ax.add_patch(Circle((78,36),10,fill=False,ec=GRAY,lw=.6,ls='--'))
69 ax.plot([55,101],[36,36],color=GRAY,ls='-.',lw=.6)
70 ax.plot([78,78],[13,59],color=GRAY,ls='-.',lw=.6)
71 for angle in (45,135,225,315):
72     a=np.deg2rad(angle); d=np.array([np.cos(a),np.sin(a)])
73     edge(ax,[np.array([78,36])+11*d,np.array([78,36])+22*d])
74 dim(ax,(78,36),(96,36),'R',(0,3))
75 label(ax,78,9,'虚线为示意等半径线',8)
76 label(ax,130,66,'(c) 均匀收缩: 物质映射')
77 ax.add_patch(Circle((130,36),19,fill=False,ec=GRAY,ls='--',lw=.8))
78 ax.add_patch(Circle((130,36),11.4,fill=False,ec=INK,lw=1))
79 ax.plot([107,151],[36,36],color=GRAY,ls='-.',lw=.6)
80 for a in (35,145,215,325):
81     d=np.array([np.cos(np.deg2rad(a)),np.sin(np.deg2rad(a))])
82     edge(ax,[np.array([130,36])+18*d,np.array([130,36])+12*d])
83 label(ax,130,60,'虚线 R(0): 实线 R(t)',8)
84 label(ax,130,9,'同一物质点: r = ηR(t)',8)
85 save(fig,'mechanism_engineering')
86
87 fig,ax=canvas(59)
88 label(ax,52,53,'A—A 纵剖面')
89 ax.add_patch(Rectangle((7,21),90,18,fc='white',ec=INK,hatch='///',lw=1))
90 ax.plot([3,102],[30,30],color=GRAY,ls='-.',lw=.6)
91 ax.plot([7,7],[10,20],color=GRAY,lw=.6)
92 ax.plot([97,97],[10,20],color=GRAY,lw=.6)
93 dim(ax,(7,12),(97,12),'L = 250 mm')
94 label(ax,126,53,'端视图')
95 ax.add_patch(Circle((126,30),16,fill=False,ec=INK,lw=1))
96 ax.plot([106,146],[30,30],color=GRAY,ls='-.',lw=.6)
97 ax.plot([126,126],[10,50],color=GRAY,ls='-.',lw=.6)
98 dim(ax,(126,30),(142,30),r'$R_0$ = 20 mm',(-8,-7))
99 label(ax,77.5,4,'斜线表示剖切到的实体: 纵剖面径向已放大, 标注尺寸为准。',8)
100 save(fig,'cylinder_engineering')
101 # CAD source uses true millimetres; PDF presentation exaggerates radial scale.
102 # Minimal ASCII DXF: simple editable LINE/CIRCLE/TEXT entities, in mm.
103 pairs=[(0,'SECTION'),(2,'HEADER'),(9,'$ACADVER'),(1,'AC1009'),
104         (0,'ENDSEC'),(0,'SECTION'),(2,'ENTITIES')]
105 def line(p,q,layer='OUTLINE'):
106     pairs.extend([(0,'LINE'),(8,layer),(10,p[0]),(20,p[1]),(30,0),
107                  (11,q[0]),(21,q[1]),(31,0)])
108 pts=[(0,0),(250,0),(250,40),(0,40),(0,0)]
109 for p,q in zip(pts[:-1],pts[1:]): line(p,q)
110 pairs.extend([(0,'CIRCLE'),(8,'OUTLINE'),(10,310),(20,20),(30,0),(40,20)])
111 for p,q in [((0,-10,20),(260,20)),((280,20),(340,20)),((310,-10),(310,50))]:
112     line(p,q,'CENTER')
113 for x in range(-40,251,8):
114     a=max(0,-x);b=min(40,250-x)
115     if b>a: line((x+a,a),(x+b,b),'SECTION')
116 for p,q in [((0,-15),(250,-15)),((0,-18),(0,-2)),((250,-18),(250,-2))]:
117     line(p,q,'DIMENSION')
118 for x,y,s in [(85,-12,'L = 250 mm'),(285,48,'R0 = 20 mm'),(0,52,'LONGITUDINAL SECTION - SOLID
119 ↪ CYLINDER')]:
120     pairs.extend([(0,'TEXT'),(8,'DIMENSION'),(10,x),(20,y),(30,0),(40,4),(1,s)])
121 pairs.extend([(0,'ENDSEC'),(0,'EOF')])
122 (FIG/'cylinder_engineering.dxf').write_text('').join(f'{k}\n{v}\n' for k,v in pairs),encoding='ascii')
123
124 def overview():
125     fig,ax=canvas(135)
126     node(ax,12,119,131,12,'环境观测、均匀初态与圆柱几何: 共同输入',True)
127     edge(ax,[(38,119),(38,109)])
128     edge(ax,[(111,119),(111,109)])
129     node(ax,6,91,64,18,'问题一 / 附录 2\n 定热物性, 0—1800 s')
130     node(ax,85,91,64,18,'问题二 / 附录 3\n 统一变物性, 从 t = 0 重算')
131     edge(ax,[(38,91),(38,81)])
132     node(ax,6,66,64,15,'七时刻截面、逐秒输出\n 端面与集中参数对照')
133     edge(ax,[(117,91),(117,81)],,'同一动力系统',97,86))
134     node(ax,85,63,64,18,'问题三 / 全域阈值\n 事件定位与严格停机')
135     edge(ax,[(117,63),(117,51)])
136     node(ax,85,33,64,18,'名义时间与八情景回代\n 输出全程状态')
137     node(ax,6,33,64,22,'问题四 / 附录 4 + 半径观测\n 干基守恒 → 物质坐标\n 独立求解收缩过程',True)
138     edge(ax,[(12,124),(2,124),(2,44),(6,44)])
139     edge(ax,[(38,33),(38,22)])
140     edge(ax,[(117,33),(117,22)])
141     node(ax,6,4,143,18,'同口径核: 网格与收支 / 物性×几何对照 / 有限情景约束\n 输出逐问答案、
    ↪ 工作簿和可复现程序',True)

```

```

142     label(ax,37,59,' 问题一末态不传入问题二',8)
143     save(fig,'overview')
144
145
146 def q1():
147     fig,ax=canvas(80)
148     node(ax,3,58,32,18,' 几何与初态\n 附录 2、环境')
149     node(ax,48,59,46,17,' 热量支路\n 显热容量 + 导热通量')
150     node(ax,48,32,46,17,' 水分支路\n 干基容量 + 状态积分')
151     edge(ax,[(35,67),(48,67)])
152     edge(ax,[(35,67),(41,67),(41,40),(48,40)])
153     node(ax,109,32,43,44,' 共同环形网格\n 中心正则\n 表面交换\nBDF 联立积分',True)
154     edge(ax,[(94,67),(109,67)])
155     edge(ax,[(94,40),(109,40)])
156     node(ax,3,4,68,18,' 物理检验: 端面 / 集中参数\n 数值检验: 收支 / 网格 / 扰动')
157     node(ax,87,4,65,18,'T(r,t)、C(r,t)\n 七个截面时刻 + 逐秒工作簿',True)
158     edge(ax,[(130,32),(130,22)])
159     edge(ax,[(87,13),(71,13)])
160     label(ax,19,42,' 输入分流, \n 状态联立求解',8)
161     save(fig,'q1_flow')
162
163
164 def q2():
165     fig,ax=canvas(88)
166     node(ax,32,73,91,11,' 附录 3 统一适用: 从 28 °C、2.55 kg/kg 初始化',True)
167     node(ax,6,45,45,17,' 含水率 C\n 更新  $\rho(C)$ 、 $cp(C)$ 、 $k(C)$ ')
168     node(ax,103,45,45,17,' 温度 T\n 更新  $D(C,T)$ ')
169     edge(ax,[(32,78),(28,78),(28,62)])
170     edge(ax,[(123,78),(125,78),(125,62)])
171     edge(ax,[(51,57),(103,57)], ' 热方程的系数', (77,61))
172     edge(ax,[(103,49),(51,49)], ' 水分方程的系数', (77,45))
173     node(ax,31,21,94,14,' 稀疏半离散系统: 联立更新 [T, C]\n 状态积分通量 + 自适应刚性积分',True)
174     edge(ax,[(28,45),(28,28),(31,28)])
175     edge(ax,[(125,45),(135,45),(135,28),(125,28)])
176     node(ax,6,1,65,11,'0.5—3 h 截面: 题问表格')
177     node(ax,84,1,64,11,' 全程轨迹: 第三问继续判停')
178     edge(ax,[(53,21),(53,12)])
179     edge(ax,[(108,21),(108,12)])
180     save(fig,'q2_flow')
181
182
183 def q3():
184     fig,ax=canvas(112)
185     node(ax,34,97,87,11,' 第二问动力系统 + 0.15 kg/kg 阈值',True)
186     node(ax,34,76,87,11,' 推进状态, 计算全节点最大含水率')
187     edge(ax,[(77,97),(77,87)])
188     ax.add_patch(Polygon([(77,69),(105,58),(77,47),(49,58)],fc='white',ec=INK,lw=.8))
189     label(ax,77,58,' 首次向下穿越阈值? ',9)
190     edge(ax,[(77,76),(77,69)])
191     edge(ax,[(49,58),(11,58),(11,81),(34,81)], ' 否: 继续', (25,63))
192     edge(ax,[(77,47),(77,38)], ' 是', (85,42))
193     node(ax,34,23,87,15,' 连续输出定位临界根\n 续算 5 s, 时间向上舍入')
194     edge(ax,[(77,23),(77,13)])
195     node(ax,34,1,87,12,' 未舍入全域严格检查 → 名义停机',True)
196     edge(ax,[(121,7),(139,7),(139,23)],None)
197     label(ax,139,32,' 检查失败\n 报错退出',8)
198     label(ax,18,26,' 独立扩展\n 八角点在\n 统一时刻\n 逐一回代',8)
199     save(fig,'q3_flow')
200
201
202 def q4():
203     fig,ax=canvas(105)
204     node(ax,3,85,59,15,' 干物质连续性\n 水分相对扩散连续性')
205     node(ax,82,85,70,15,' 附件 2: R(t) 保形插值\n 附录 4: 独立经验物性')
206     edge(ax,[(62,92),(72,92),(72,76)])
207     edge(ax,[(117,85),(117,76)])
208     node(ax,19,61,133,15,' 随固体坐标  $\eta = r/R(t)$ \n 相消坐标输运, 保留扩散与边界收支',True)
209     edge(ax,[(85,61),(85,51)])
210     node(ax,19,39,133,12,' 固定  $\eta$  网格求解 → 全域判停 → 域外掩膜与实际表面')
211     edge(ax,[(85,39),(85,32)])
212     label(ax,79,28,' 控制变量对照: 物性 × 几何',9)
213     for x in (42,97,152): ax.plot([x,x],[1,23],color=GRAY,lw=.6)
214     for y in (1,12,23): ax.plot([42,152],[y,y],color=GRAY,lw=.6)
215     label(ax,69,18,' 附录 3 / 固定半径',8)
216     label(ax,125,18,' 附录 3 / 收缩半径',8)
217     label(ax,69,6,' 附录 4 / 固定半径',8)
218     label(ax,125,6,' 附录 4 / 收缩半径',8)
219     label(ax,20,12,' 同物性比较\n 分离几何效应',8)
220     save(fig,'q4_flow')

```

```

221
222
223 if __name__ == '__main__':
224     FIG.mkdir(exist_ok=True)
225     mechanisms(); overview(); q1(); q2(); q3(); q4()
226     print(json.dumps({'figures':7,'cad_units':'mm','raster_dpi':450,
227         'mechanism':'orthographic structural schematic, not measured pores'}))

```

C.13 回归测试与统一复现入口

`code/pitfall_audit.py` 用于环境外推、半径插值和错误判停回代；`code/test_pitfalls.py` 检查公式、工作簿及阶跃导热特例。`code/reexport_locked.py` 只从冻结细网格场重导出格式，不声称再次求解 PDE。

```

1 """Focused regression tests for the revised manuscript diagnostics."""
2 import json
3 from pathlib import Path
4 import unittest
5 import numpy as np
6 from scipy.integrate import quad
7 from s2_solver import Model, Settings
8 from s2_stable_graded import StableModel, integral_difference
9
10 ROOT = Path(__file__).resolve().parents[1]
11
12
13 class RevisionTests(unittest.TestCase):
14     def test_integral_against_independent_quadrature(self):
15         for a in (.30,.45,.89):
16             for c in (.05,.15,1.,2.55):
17                 for d in (0.,1e-8,.009,.02):
18                     right=c*(1+d)
19                     got=integral_difference(np.array([c]),np.array([right]),a)[0]
20                     ref=quad(lambda x:np.exp(-a/x),c,right,epsabs=1e-24)[0]
21                     self.assertAlmostEqual(got/max(abs(ref),1e-30),ref/max(abs(ref),1e-30),places=8)
22                     rev=integral_difference(np.array([right]),np.array([c]),a)[0]
23                     self.assertAlmostEqual(got+rev,0.,places=17)
24
25     def test_annular_weights(self):
26         for n in (20,160):
27             m=StableModel(Settings(question=1,n=n))
28             self.assertAlmostEqual(float(2*m.w.sum()),1.)
29             self.assertTrue(np.all(m.dx>0))
30             self.assertTrue(np.all(m.w>0))
31
32     def test_dry_mass_material_manufactured_state(self):
33         m=StableModel(Settings(question=4,n=20,D_scale=0,hm_scale=0))
34         y=np.r_[np.full(m.size,28.),2.55-.1*m.x**2]
35         for t in (0,3600,10800):
36             np.testing.assert_array_equal(m.rhs(t,y)[m.size:],np.zeros(m.size))
37
38     def test_boundary_ledger_rhs(self):
39         for q in (1,2,3,4):
40             m=StableModel(Settings(question=q,n=20))
41             y=np.r_[28+3*m.x**2,2.55-.2*m.x**2]
42             t=1800.
43             got=float(2*m.w @ m.rhs(t,y)[m.size:])
44             ref=2*8e-7/m.radius(t)*(m.environment(t)[1]-y[-1])
45             self.assertAlmostEqual(got,ref,places=14)
46
47     def test_supplementary_outputs_and_decoupling(self):
48         s=json.loads((ROOT/'supplement/SUMMARY.json').read_text())
49         self.assertEqual(len(s['sensitivity']),6)
50         for row in s['sensitivity']:
51             for field in ('Tc','Ts','Cc','Cs','Cmean'):
52                 self.assertTrue(np.isfinite(list(row[field].values()))).all()
53                 if row['question']==1 and row['parameter']=='h_scale':
54                     self.assertLess(abs(row['Cmean']['plus']-row['Cmean']['minus']),1e-8)
55         c=s['comparison']
56         self.assertGreater(c['q2_frozen']['Cc'],c['q2_variable_uniform']['Cc'])
57         self.assertLess(c['q2_frozen']['Cs'],c['q2_variable_uniform']['Cs'])
58
59     def test_frozen_property_identity(self):
60         m=Model(Settings(question=2,n=20,frozen=True))
61         a=m.properties(np.full(m.size,28.),np.full(m.size,2.55))
62         b=m.properties(np.full(m.size,50.),np.full(m.size,.15))
63         for x,y in zip(a,b): np.testing.assert_array_equal(x,y)

```

```

64
65
66 if __name__ == '__main__':
67     unittest.main(verbosity=2)

1 """Reproducible primary answers, tests and deterministic manuscript figures."""
2 from pathlib import Path
3 import argparse
4 import datetime
5 import hashlib
6 import json
7 import platform
8 import subprocess
9 import sys
10 import time
11
12 ROOT=Path(__file__).resolve().parent
13
14
15 def main():
16     p=argparse.ArgumentParser()
17     p.add_argument('--mode',choices=['smoke','full'],default='smoke')
18     p.add_argument('--out',type=Path)
19     p.add_argument('--redraw',action='store_true')
20     args=p.parse_args()
21     out=(args.out or ROOT/'runs'/datetime.datetime.now().strftime('%Y%m%d_%H%M%S')).resolve()
22     out.mkdir(parents=True,exist_ok=False)
23     log=[]
24     def run(parts,name):
25         command=[sys.executable,'-X','utf8',*parts]; start=time.perf_counter()
26         with (out/f'{name}.log').open('w',encoding='utf8') as stream:
27             result=subprocess.run(command,cwd=ROOT,stdout=stream,stderr=subprocess.STDOUT)
28             log.append(dict(command=command,exit_code=result.returncode,seconds=time.perf_counter()-start))
29             (out/'execution.json').write_text(json.dumps(log,indent=2),encoding='utf8')
30             if result.returncode: raise RuntimeError(f'{name} failed; inspect log; no success report issued')
31             print(name,'PASS',flush=True)
32             inputs={str(x.relative_to(ROOT)):hashlib.sha256(x.read_bytes()).hexdigest()
33                     for x in (ROOT/'data').glob('*.xlsx')}
34             (out/'environment.json').write_text(json.dumps(dict(python=platform.python_version(),
35                     platform=platform.platform(),inputs=inputs,seed='not applicable: deterministic
36                     ↪ PDE'),indent=2),encoding='utf8')
37         run(['-m','unittest','discover','-s','code','-p','test_revision.py','-v'],'tests')
38         run(['code/reproduce.py','--mode',args.mode,'--out',str(out)],'calculation')
39         if args.mode=='full': run(['code/verify_reproduction.py','--out',str(out)],'verification')
40         if args.redraw:
41             for script in ('redraw.py','revision_figures.py','engineering_figures.py'):
42                 run([script],Path(script).stem)
43         print('Output:',out)
44
45 if __name__ == '__main__':
46     main()

1 """Check a fresh production run against frozen answers and all workbook rows."""
2 from pathlib import Path
3 import argparse
4 import hashlib
5 import json
6 import math
7 from openpyxl import load_workbook
8 from s2_solver import ROOT, Model, Settings
9
10
11 def verify(out):
12     old=json.loads((ROOT/'evidence/answers.json').read_text(encoding='utf8'))
13     fresh=json.loads((out/'full_report.json').read_text(encoding='utf8'))
14     checks=[]; books=[]
15     for answer in old:
16         q=answer['question']; ref=answer['meta']; got=fresh['result'][str(q)]
17         for key,tol in [('center_T',5e-6),('surface_T',5e-6),('center_C',5e-8),
18             ('surface_C',5e-8),('critical_s',.02),('end_s',1e-7)]:
19             if ref.get(key) is None: continue
20             delta=abs(got[key]-ref[key])
21             checks.append(dict(question=q,metric=key,delta=delta,tolerance=tol,pass_=delta<=tol))
22             checks.append(dict(question=q,metric='unclipped_positive_state',
23                 pass_=got['min_accepted_C']>0 and not got['clipping_used']))
24             if q>=3:
25                 ↪ checks.append(dict(question=q,metric='strict_unrounded_stop',pass_=got['max_C_at_end']<.15))
26     path=out/f'result{q}.xlsx'; book=load_workbook(path,read_only=True,data_only=True)

```

```

26     assert book.sheetnames==([' 温度', ' 水分浓度'] if q<=2 else ['Sheet1'])
27     targets={round(row['time_s'],7):row for row in answer['table_rows']}
28     model=Model(Settings(question=q,n=4)); sheet_reports=[]
29     for sheet in book:
30         thermal=sheet.title==' 温度'; field='temperature_C' if thermal else 'moisture'
31         rows=iter(sheet.rows); header=[c.value for c in next(rows)]; prev=0.; count=0; matched=0;
32         ↪ max_error=0.
33         assert header==['时间\\到药材中心的距离']+[i/10 for i in range(20 if q==4 else
34         ↪ 21)]+(['药材表面'] if q==4 else [])
35         for cells in rows:
36             row=[c.value for c in cells]
37             count+=1; t=float(row[0]); assert t>prev; prev=t
38             if count==1: assert t==(1 if q<=2 else 60)
39             if t<got['workbook']['end_s']-1e-7: assert t==count*(1 if q<=2 else 60)
40             radius=100*model.radius(t)
41             for i in range(20 if q==4 else 21):
42                 v=row[i+1]
43                 if q==4 and i/10>radius+1e-10: assert v is None
44                 else:
45                     assert v is not None and math.isfinite(v) and v>0
46                     assert cells[i+1].number_format=='0.0000'
47             target=targets.get(round(t,7))
48             if target:
49                 matched+=1
50                 for j,column in enumerate((1,6,11,16,21)):
51                     expected=target[field][j]
52                     if q==4 and j==4:
53                         assert expected is None
54                         continue
55                     if expected is None: assert row[column] is None
56                     else:
57                         err=abs(row[column]-expected);max_error=max(max_error,err)
58                         assert err<=.0000501,(q,t,field,j,err)
59             if q==4:
60                 assert row[21] is not None and row[21]>0
61                 if target: assert abs(row[21]-target[field][-1])<=.0000501
62             assert count==got['workbook']['rows']
63             assert abs(prev-got['workbook']['end_s'])<1e-7
64             assert matched==len(targets)
65             sheet_reports.append(dict(sheet=sheet.title,rows=count,table_times=matched,max_rounding_erro
66             ↪ r=max_error))
67         book.close()
68         books.append(dict(question=q,sha256=hashlib.sha256(path.read_bytes()).hexdigest(),sheets=sheet_r
69         ↪ eports))
70     report=dict(ready=all(x['pass_'] for x in checks),checks=checks,workbooks=books,
71     scope=('Frozen-field re-export integrity; not a PDE rerun.' if
72     ↪ fresh.get('mode')== 'frozen_field_reexport'
73     else 'Fresh rerun and rounded export integrity; not independent physical validation.'))
74     (out/'verification.json').write_text(json.dumps(report,indent=2),encoding='utf8')
75     if not report['ready']: raise AssertionError('Production regression mismatch')
76     return report
77
78 if __name__=='__main__':
79     parser=argparse.ArgumentParser();parser.add_argument('--out',type=Path,required=True)
80     args=parser.parse_args();print(json.dumps(verify(args.out),ensure_ascii=False))

```

C.14 避坑核验、解析特例与冻结结果重导出

下列前三个脚本分别承担受控对照、结构与解析测试、以及从机器记录生成表格。冻结结果重导出不是从零求解；运行最后一个脚本须显式提供--q2-fields 全过程 NPZ 和新的--out 目录，并保留对应 evidence/场文件。没有冻结场时，应使用上述双模型冷启动入口，不能使用 3 h 短轨道替代全过程。

```

1 """Targeted checklist audit; no manuscript or PDF is generated here."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import platform
6 import time
7
8 import numpy as np
9 import scipy
10 from scipy.optimize import brentq
11 from s2_solver import ROOT, Settings, inputs

```

```

12 from s2_stable_graded import StableModel
13 from experiments import conservation, requested_table
14
15
16 def first_crossing(model, mode):
17     def residual(t):
18         c = model.state([t])[model.size:, 0]
19         value = c[-1] if mode == 'surface' else 2 * model.w @ c
20         return float(value - .15)
21     ts = np.linspace(0, model.end, 1501)
22     for left, right in zip(ts[:-1], ts[1:]):
23         if residual(left) > 0 >= residual(right):
24             root = brentq(residual, left, right, xtol=1e-7)
25             c = model.state([root])[model.size:, 0]
26             return dict(time_h=root / 3600, actual_max_C=float(c.max()),
27                         actually_feasible=bool(c.max() < .15))
28     raise AssertionError('No crossing found')
29
30
31 def main():
32     out = ROOT / 'supplement/pitfall_audit'
33     out.mkdir(parents=True, exist_ok=True)
34     if (out / 'report.json').exists():
35         raise FileExistsError('Preserve the completed report')
36     started = time.perf_counter()
37     env, rad = inputs()
38     tail = env[env[:, 0] >= 10800, 1:].mean(axis=0)
39     cases = [('q1_nominal', 1, {}), ('q2_nominal', 2, {})]
40     for q in (3, 4):
41         cases += [(f'q{q}_nominal', q, {}),
42                  (f'q{q}_nominal_n320', q, dict(n=320)),
43                  (f'q{q}_constant50', q, dict(tail_T_shift=50-tail[0], tail_C_shift=.05-tail[1])),
44                  (f'q{q}_last', q, dict(boundary='last'))]
45     cases += [(f'q4_linear', 4, dict(radius_kind='linear')),
46              ('q4_linear_constant50', 4, dict(radius_kind='linear', tail_T_shift=50-tail[0],
47                                               ↪ tail_C_shift=.05-tail[1])),
48              ('q3_hm2', 3, dict(hm_scale=2))]
49     records = []
50     for name, q, changes in cases:
51         settings = dict(question=q, n=640)
52         settings.update(changes)
53         model = StableModel(Settings(**settings)).run()
54         record = dict(name=name, meta=model.meta, table=requested_table(model))
55         if name in ('q3_nominal', 'q4_nominal'):
56             record['conservation'] = conservation(model)
57             record['wrong_stop'] = {mode: first_crossing(model, mode) for mode in ('surface', 'mean')}
58             ts = np.unique(np.r_[np.arange(0, model.end, 60), model.end])
59             states = model.state(ts)
60             cs = states[model.size:]
61             record['sampled_state_checks'] = dict(
62                 step_s=60, max_radial_C_increase=float(np.diff(cs, axis=0).max()),
63                 max_temporal_C_increase=float(np.diff(cs, axis=1).max()),
64                 max_temperature=float(states[:model.size].max()),
65                 note='Sampled diagnostics, not a continuous-domain monotonicity proof')
66         if q == 4:
67             c36 = model.state([36*3600])[model.size:, 0]
68             record['at_36h'] = dict(mean_C=float(2*model.w@c36), max_C=float(c36.max()))
69             record['radii_cm'] = {str(h): 100*model.radius(h*3600) for h in (0, 1, 6, 36, 72)}
70             (out / (name+'.json')).write_text(json.dumps(record, ensure_ascii=False, indent=2),
71                                               ↪ encoding='utf8')
72             records.append(record)
73             print(name, json.dumps(model.meta, ensure_ascii=False), flush=True)
74     report = dict(scope='Conditional recomputation; alternatives are not replacements for the locked
75     ↪ fine-grid answers',
76                  python=platform.python_version(), numpy=np.__version__, scipy=scipy.__version__,
77                  inputs=dict(environment_rows=len(env), radius_rows=len(rad),
78                               observed_max_T=float(env[:,1].max()), last_environment=env[-1,1:].tolist(),
79                               mean_tail=tail.tolist(), observed_last_radius_cm=float(rad[-1,1])),
80                  celsius_error_ratio=float(np.exp(-3850/50 + 3850/(50+273.15))),
81                  cases=records, elapsed_s=time.perf_counter()-started, exit_code=0,
82                  hashes={str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
83                           for p in [Path(__file__), ROOT/'code/s2_solver.py', ROOT/'code/s2_stable_graded.py',
84                                    ROOT/'data/input1.xlsx', ROOT/'data/input2.xlsx']})
85             (out/'report.json').write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding='utf8')
86             print('COMPLETE', report['elapsed_s'], flush=True)
87
88
89 if __name__ == '__main__':
90     main()

```

```

1 """Formula, export and independent cylindrical step-heating checks."""
2 import json
3 from types import SimpleNamespace
4 import unittest
5 import numpy as np
6 from openpyxl import load_workbook
7 from scipy.integrate import solve_ivp
8 from scipy.optimize import brentq
9 from scipy.special import j0, j1
10 from scipy.sparse import diags
11 from reproduce import workbook
12 from s2_solver import ROOT, Settings
13 from s2_stable_graded import StableModel
14
15
16 class PitfallTests(unittest.TestCase):
17     def test_template_export_all_questions(self):
18         out=ROOT/'supplement/pitfall_audit/schema_tests'
19         out.mkdir(parents=True,exist_ok=True)
20         for q in (1,2,3,4):
21             moving=q==4
22             x=np.linspace(0,1,21)
23             model=SimpleNamespace(moving=moving, end=120.36, size=21, x=x,
24                                   radius=lambda t: .012 if moving else .02,
25                                   state=lambda ts:np.repeat(np.r_[28+x,2.55-x][:,None],len(ts),axis=1))
26             step=1 if q<=2 else 60
27             path=out/f'result{q}.xlsx'
28             ledger=workbook(model,path,step,q<=2)
29             book=load_workbook(path,read_only=True)
30             self.assertEqual(book.sheetnames,[' 温度', ' 水分浓度'] if q<=2 else ['Sheet1'])
31             for sheet in book:
32                 rows=list(sheet.rows)
33                 self.assertEqual(rows[0][0].value, ' 时间\\到药材中心的距离')
34                 self.assertEqual(rows[1][0].value, step)
35                 self.assertEqual(rows[-1][0].value, 120.36)
36                 self.assertEqual(len(rows)-1, ledger['rows'])
37                 self.assertEqual(len(rows[0]), 22)
38                 for row in rows[1:]:
39                     for cell in row[1:]:
40                         if cell.value is not None: self.assertEqual(cell.number_format, '0.0000')
41             if moving:
42                 self.assertEqual(rows[0][-1].value, ' 药材表面')
43                 self.assertTrue(all(c.value is None for c in rows[-1][14:21]))
44                 self.assertEqual(rows[-1][13].value, rows[-1][-1].value)
45             book.close()
46
47     def test_formula_units_and_direction(self):
48         for q,A,a in ((1,7e-9,.89),(2,2.4e-3,.45),(4,4.2e-4,.30)):
49             m=StableModel(Settings(question=q,n=4))
50             T=np.full(m.size,50.); C=np.full(m.size,.15)
51             rho,cp,k,coef,beta=m.properties(T,C)
52             expected=A if q==1 else A*np.exp(-3850/323.15)
53             np.testing.assert_allclose(coef,expected,rtol=1e-14)
54             self.assertEqual(beta,a)
55             self.assertLess(np.exp(-a/.15),np.exp(-a/2.55))
56             self.assertLess(np.exp(-3850/50),np.exp(-3850/323.15))
57
58     def test_constant_step_heat_bessel(self):
59         R=.02; alpha=.36/(820*2600); bi=25*R/.36
60         f=lambda z: z*j1(z)-bi*j0(z)
61         samples=np.linspace(1e-8,65*np.pi,10000)
62         roots=np.array([brentq(f,a,b) for a,b in zip(samples[:-1],samples[1:]) if f(a)*f(b)<0])[ :60]
63         coeff=2*j1(roots)/(roots*(j0(roots)**2+j1(roots)**2))
64         times=np.array([100.,600.,1800.]); positions=np.array([0.,.5,1.])
65         exact=50-22*np.array([np.sum(coeff[:,None]*j0(roots[:,None]*positions)*
66                                   np.exp(-roots[:,None]**2*alpha*t/R**2),axis=0) for t in times])
67         results=[]
68         for n in (160,320):
69             m=StableModel(Settings(question=1,n=n))
70             m.environment=lambda t:(50.,.05)
71             def rhs(t,T):
72                 return m.rhs(t,np.r_[T,np.full(m.size,2.55)])[:m.size]
73             band=diags([np.ones(n),np.ones(n+1),np.ones(n)],[-1,0,1],format='csc')
74             sol=solve_ivp(rhs,(0,1800),np.full(m.size,28.),method='BDF',t_eval=times,
75                           rtol=1e-11,atol=1e-10,max_step=5,jac_sparsity=band)
76             self.assertTrue(sol.success)
77             got=np.array([np.interp(positions,m.x,sol.y[:,i]) for i in range(3)])
78             error=float(np.max(abs(got-exact)))
79             self.assertLess(error,.01)

```

```

80         results.append(dict(n=n,computed=got.tolist(),max_error_C=error))
81     self.assertLess(results[1]['max_error_C'],results[0]['max_error_C'])
82     report=dict(times_s=times.tolist(),radius_fractions=positions.tolist(),
83                 analytic_T=exact.tolist(),roots=roots.tolist(),results=results,
84                 scope='Constant-property step-heating test only, not the observed-boundary
            ↳ answer',pass_=True)
85     out=ROOT/'supplement/pitfall_audit'
86     out.mkdir(parents=True,exist_ok=True)
87     (out/'bessel_check.json').write_text(json.dumps(report,indent=2),encoding='utf8')
88
89
90 if __name__=='__main__':
91     unittest.main(verbosity=2)

1 """Generate TeX diagnostic tables and the answer update from machine evidence."""
2 import json
3 from s2_solver import ROOT
4
5
6 def main():
7     report=json.loads((ROOT/'supplement/pitfall_audit/report.json').read_text(encoding='utf8'))
8     cases={c['name']:c for c in report['cases']}
9     locked=json.loads((ROOT/'evidence/answers.json').read_text(encoding='utf8'))
10    lines=[r'\begin{table}[htbp]\centering\small',
11           r'\caption{错误判停准则的同轨迹回代 (640 区间)}\label{tab:pitfall_stop}',
12           r'\begin{tabular}{llll}\toprule',
13           r' 问题与错误准则 & 判停时间/h & 最大含水率/(kg/kg) & 满足全域要求\\\midrule']
14    for q in (3,4):
15        for mode,title in [('surface','表面'),('mean','均值')]:
16            d=cases[f'q{q}_nominal'][f'wrong_stop'][mode]
17            lines.append(f'{q}: {title} & {d["time_h"]:.4f} & {d["actual_max_C"]:.4f} & 否'+r'\')
18    lines += [r'\bottomrule\end{tabular}',r'\end{table}']
19    (ROOT/'tables/pitfall_wrong_stop.tex').write_text('\n'.join(lines)+'\n',encoding='utf8')
20    combos=[(' 末段均值','PCHIP','q3_nominal','q4_nominal'),
21            ('50 °C / 0.05','PCHIP','q3_constant50','q4_constant50'),
22            (' 末观测值','PCHIP','q3_last','q4_last'),
23            (' 末段均值','线性','q3_nominal','q4_linear'),
24            ('50 °C / 0.05','线性','q3_constant50','q4_linear_constant50')]
25    lines=[r'\begin{table}[htbp]\centering\small',
26           r'\caption{长期环境与半径插值对临界时间的影响 (640 区间)}\label{tab:pitfall_closure}',
27           r'\begin{tabular}{llll}\toprule',
28           r'4 h 后环境 & 第四问半径插值 & 第三问/h & 第四问/h\\\midrule']
29    for environment,kind,c3,c4 in combos:
30        lines.append(f'{environment} & {kind} & {cases[c3]["meta"]["critical_h"]:.4f} &
            ↳ {cases[c4]["meta"]["critical_h"]:.4f}'+r'\')
31    lines += [r'\bottomrule\end{tabular}',r'\end{table}']
32    (ROOT/'tables/pitfall_closures.tex').write_text('\n'.join(lines)+'\n',encoding='utf8')
33    q3=cases['q3_nominal'];q4=cases['q4_nominal']
34    linear=(cases['q4_linear']['meta']['critical_h']-q4['meta']['critical_h'])*3600
35    hm=cases['q3_hm2']['meta']['critical_h']
36    summary=(f'第三问取末点而非末段均值时,
            ↳ 临界时间缩短{q3["meta"]["critical_h"]-cases["q3_last"]["meta"]["critical_h"]:.4f} h; '
37            f' 第四问仅改线性半径插值时, 临界时间增加 {linear:.2f} s。'
38            f' 第三问将$h_m$加倍后, 临界时间为{hm:.4f} h, 较同网格名义值缩短{q3["meta"]["critical_h"]-hm:.4f}
            ↳ h。'
39            f' 第四问36 h虽已达到半径1.2000 cm, 但最大含水率仍为{q4["at_36h"]["max_C"]:.4f} kg/kg, 尚未达标。'
40            f' 附件1观测温度最大值为{report["inputs"]["observed_max_T"]:.4f} °C, 不能把50.2
            ↳ °C硬编码为风温上限。'
41            '\n 本组名义 320/640 区间临界时间差不足 0.19 s; 正文最终答案仍采用既有更细网格。'
42            '\n 粗细网格的微小差异可能跨过严格停机时间的0.0001 h上取整边界,
            ↳ 不能把诊断表的粗网格严格时刻直接覆盖主答案。'\n')
43    (ROOT/'tables/pitfall_summary.tex').write_text(summary,encoding='utf8')
44    md=[ '# 四问答案更新与避坑复核', '',
45         ' 更新时间: 2026-09-12. 本轮不生成或编译 PDF。', '',
46         ' ## 结论', '',
47         ' ***未发现需要推翻当前主模型及细网格主答案的数值证据。保留主答案, 补充闭合假设、13条新复算轨迹、
            ↳ 错误判停反证和独立解析特例; 修正工作簿模板接口。***',
48         ' 这不是“所有坑都不存在”; 端面全程二维效应、潜热、气固平衡及密度闭合仍是已披露的物理限制。', '',
49         '| 问题 | 模型与口径 | 当前主答案 | 本轮处理 |', '|---|---|---|---|',
50         '| 一 | 端面绝热的一维径向; 附录2, D仍随C变化 | 1800 s中心/表面T: 33.5753/36.7856 °C; C: 2.5500/1.5102
            ↳ | 640区间重新核对, 保留1280区间锁定表 |',
51         '| 二 | 全程统一附录3; 从原初态联立求解 | 3 h中心/表面T: 49.8495/49.9664 °C; C: 1.7662/1.0081 |
            ↳ | 640区间重新核对; 保留全过程逐秒两场输出 |',
52         '| 三 | 同第二问; 全域最大含水率事件 | 临界57.4740 h; 严格停机57.4754 h | 名义320/640复算;
            ↳ | 增加外推/判据对照 |',
53         '| 四 | 附录4; 实测半径PCHIP; 均匀收缩物质坐标 | 临界51.0885 h; 严格停机51.0899 h; 半径1.2000 cm |
            ↳ | 名义320/640复算; 增加线性插值/外推对照 |', '',
54         ' 严格终点原始含水率分别为0.1499985335和0.1499974179 kg/kg; 显示0.1500是四舍五入, 不是将等号当成严格小于。
            ↳ 临界时刻是可行集合的下确界, 执行时刻另舍5 s余量和向上舍入。', '',

```

```

55     '## 新增同口径对照', '',
56     '下表为本轮 640 区间实际计算的 ** 临界时间 **，并非主细网格答案或实验真值。', '',
57     '| 4 h 后环境 | 第四问半径插值 | 第三问/h | 第四问/h |', '|---|---|---|---|']
58 for env, kind, c3, c4 in combos:
59     md.append(f'| {env} | {kind} | {cases[c3]["meta"]["critical_h"]:.4f} |
60     ↪ {cases[c4]["meta"]["critical_h"]:.4f} |')
61 md += [' ', f'- 第三问 h_m 加倍: {hm:.4f} h, 缩短 {q3["meta"]["critical_h"]-hm:.4f} h。',
62     f'- 第四问仅由 PCHIP 改线性插值，临界时间增加 {linear:.2f} s，不能将插值选择写成唯一标准。',
63     f'- “50/0.05” 和 “末点保持” 都只是 4 h 之后的闭合，不是题目指定的长期观测。', '',
64     '| 错误停止标准 | 第三问时刻/h | 当时全域 max C | 第四问时刻/h | 当时全域 max C |',
65     '|---|---|---|---|']
66 for mode, title in [('surface', '只看表面'), ('mean', '只看干物质加权均值')]:
67     a = q3['wrong_stop'][mode]; b = q4['wrong_stop'][mode]
68     md.append(f'| {title} | {a["time_h"]:.4f} | {a["actual_max_C"]:.4f} | {b["time_h"]:.4f} |
69     ↪ {b["actual_max_C"]:.4f} |')
70 md += [' ', '** 上表四种提前停止均不满足全域要求。**', '',
71     '## 各问完整指定截面', '', '数表直接读取既有细网格机器答案；单位为℃或kg/kg，
72     ↪ 不由本轮640区间诊断值替换。域外位置用“域外”表示。']
73 for answer in locked:
74     q = answer['question']
75     fields = [('temperature_C', '温度'), ('moisture', '含水率')] if q <= 2 else [('moisture', '含水率')]
76     for field, title in fields:
77         md += [' ', f'### 问题 {q}: {title}', '',
78             '| 时间/' + ('s' if q == 1 else 'h') + ' | 0 cm | 0.5 cm | 1 cm | 1.5 cm |' + ('表面 | 半径/cm
79             ↪ |' if q == 4 else '2 cm |'),
80             '|---|---|---|---|---|---|' + ('---|' if q == 4 else '')]
81         for row in answer['table_rows']:
82             vals = row[field][:5] if q != 4 else row[field][:4] + [row[field][-1]]
83             t = row['time_s'] if q == 1 else row['time_s']/3600
84             md.append('| ' + f'{t:.4f}' + ' | ' + ' | '.join('域外' if v is None else f'{v:.4f}' for v in
85             ↪ vals) + ' | ' + (f'{row["radius_cm"]:.4f}' if q == 4 else ''))
86 md += [' ', '## 清单需要纠正的说法', '',
87     f'1. 用℃误代Arrhenius指数会使D大幅变小。在50℃处，
88     ↪ 错误/正确比值为{report["celsius_error_ratio"]:.5e}，不是增大。',
89     f'2. 附件 1 实际最大风温为 {report["inputs"]["observed_max_T"]:.4f}℃，不是 50.2℃。',
90     f'3. 不能凭“约 17 h”排他地诊断一种代码错误；必须查原式、边界、坐标及判据。',
91     f'4. result2 仍保留完整过程的温度和水分逐秒数据，不能用只有含水率的 result3 替代。',
92     f'5. “未给潜热”不能证明潜热可忽略，未经闭合的 655/118 W/m² 比较也不能证明与题意矛盾。',
93     f'6. 隐式稳定、每秒输出、严格阈值与四位小数精度是不同概念；不能互相替代。', '',
94     '## 验证与文件', '',
95     f'- 本轮 13 条轨迹总脚本耗时 {report["elapsed_s"]:.2f} s；全部返回成功。',
96     f'- 新Q3/Q4名义干基收支相对残差: {q3["conservation"]["relative_residual"]:.3e}、
97     ↪ {q4["conservation"]["relative_residual"]:.3e}。',
98     f'- Bessel阶跃导热九点对照: 160/320区间最大误差0.0001296364/0.0000348449℃。这是常物性特例，
99     ↪ 不是实验验证。',
100    f'- 主稿: `main.tex`，新增论证: `pitfall_validation.tex`，新数表: `tables/pitfall_*.tex`。
101    ↪ 编译主稿时需保留整个源工程。',
102    f'- 修正版工作簿: `supplement/pitfall_audit/workbooks/result1.xlsx` 至 `result4.xlsx`。
103    ↪ 从已锁定细网格场直接重导出，未改PDE值；格式及所有指定截面阅读见同目录`verification.json`。',
104    f'- 逐项审计: `audit/PITFALL_REVIEW.md`；新运行记录: `supplement/pitfall_audit/report.json`；
105    ↪ 完整锁定依据仍为项目`RESULTS_LOCK.md`。',
106    f'- 旧PDF、历史压缩包和submission没有更新。本轮未进行页面视觉验收；历史PDF报告不代表新TeX通过版面审查。
107    ↪ ', '']
108 (ROOT/'答案更新 _ 避坑复核.md').write_text('\n'.join(md), encoding='utf8')
109 print('Generated three TeX tables and the answer Markdown from machine records')
110
111 if __name__ == '__main__':
112     main()
113
114 """Re-export frozen fine-grid fields without solving or altering their numbers."""
115 from pathlib import Path
116 import copy
117 import argparse
118 import hashlib
119 import json
120 import numpy as np
121 from openpyxl import Workbook
122 from openpyxl.cell import WriteOnlyCell
123 from s2_solver import ROOT
124 from verify_reproduction import verify
125
126 def main():
127     parser = argparse.ArgumentParser(description=__doc__)
128     parser.add_argument('--q2-fields', type=Path, required=True,
129         help='Frozen full-process Q2 NPZ, not the short 3 h field')
130     parser.add_argument('--out', type=Path, required=True)
131     args = parser.parse_args()

```

```

20 if not args.q2_fields.is_file():
21     raise FileNotFoundError(args.q2_fields)
22 out=args.out
23 out.mkdir(parents=True, exist_ok=False)
24 answers=json.loads((ROOT/'evidence/answers.json').read_text(encoding='utf8'))
25 result={}
26 sources=[]
27 for answer in answers:
28     q=answer['question']
29     path=args.q2_fields if q==2 else ROOT/f'evidence/q{q}_fields.npz'
30     with np.load(path) as data:
31         times=data['times_s']
32         step=1 if q<=2 else 60
33         selected=np.flatnonzero(times>=step)
34         names=[' 温度', ' 水分浓度'] if q<=2 else ['Sheet1']
35         book=Workbook(write_only=True)
36         for name in names:
37             sheet=book.create_sheet(name)
38             sheet.append(['时间\\到药材中心的距离']+[i/10 for i in range(20 if q==4 else
↪ 21)]+(['药材表面'] if q==4 else []))
39             field=data['T' if name==' 温度' else 'C']
40             for idx in selected:
41                 vals=field[idx,:21] if q!=4 else np.r_[field[idx,:20],field[idx,-1]]
42                 cells=[]
43                 for value in vals:
44                     cell=WriteOnlyCell(sheet, value=None if not np.isfinite(value) else
↪ round(float(value),4))
45                     cell.number_format='0.0000'
46                     cells.append(cell)
47                 sheet.append([float(times[idx]))+cells)
48             book.save(out/f'result{q}.xlsx')
49             meta=copy.deepcopy(answer['meta'])
50             meta['workbook']=dict(rows=len(selected), end_s=float(times[-1]), step_s=step)
51             result[str(q)]=meta
52             sources.append(dict(question=q, path=str(path),
↪ sha256=hashlib.sha256(path.read_bytes()).hexdigest()))
53             print('EXPORTED',q,len(selected),flush=True)
54 report=dict(mode='frozen_field_reexport', result=result, sources=sources,
55            scope='No PDE rerun in this exporter; frozen fine-grid values are reused without
↪ interpolation')
56 (out/'full_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf8')
57 check=verify(out)
58 assert check['ready']
59 print('VERIFIED',len(check['checks']),flush=True)
60
61
62 if __name__=='__main__':
63     main()

```

C.15 双模型冷启动完整程序

```

1 """Rebuild baseline plotting arrays via the existing production export API."""
2 import argparse
3 import json
4 from pathlib import Path
5 import shutil
6 import uuid
7 import numpy as np
8 from s2_solver import ROOT, Settings, inputs
9 from s2_stable_graded import StableModel
10
11
12 def main():
13     parser = argparse.ArgumentParser()
14     parser.add_argument('--out', required=True, type=Path)
15     parser.add_argument('--smoke', action='store_true')
16     args = parser.parse_args()
17     args.out.mkdir(parents=True, exist_ok=False)
18     env, radius = inputs()
19     np.savez_compressed(args.out/'inputs.npz', environment=env, radius=radius)
20     records = {}
21     for q in ((1,) if args.smoke else (1, 2, 3, 4)):
22         n = 32 if args.smoke else (2560 if q == 4 else 1280)
23         model = StableModel(Settings(question=q, n=n)).run()
24         label = f'plot_cache_q{q}_{uuid.uuid4().hex[:12]}'
25         model.export(label)
26         folder = ROOT/'results/stage2'/label
27         shutil.copy2(folder/'fields.npz', args.out/f'q{q}_fields.npz')

```

```

28     records[str(q)] = model.meta
29     print(f'EXPORTED Q{q} N{n}', flush=True)
30     (args.out/'cache_rebuild.json').write_text(json.dumps(dict(
31         ready=True, smoke=args.smoke, cases=records,
32         scope='Existing solver/export; baseline plot arrays only, not all diagnostics'),
33         ensure_ascii=False, indent=2), encoding='utf-8')
34
35
36 if __name__ == '__main__':
37     main()

1 """Cold production reproduction of both closures, without result-cache reads."""
2 import argparse
3 import gc
4 import hashlib
5 import json
6 import platform
7 import time
8 from pathlib import Path
9
10 import numpy as np
11 import scipy
12 import openpyxl
13
14 from s2_solver import Settings, ROOT
15 from s2_stable_graded import StableModel
16 from latent_extension import LatentModel
17 from experiments import requested_table
18 from reproduce import workbook
19
20
21 def digest(path):
22     return hashlib.sha256(path.read_bytes()).hexdigest()
23
24
25 def verify_book(model, path, step, temperature):
26     """Read every output row; independently compare sampled values to the solution."""
27     book = openpyxl.load_workbook(path, read_only=True, data_only=True)
28     expected_names = ['温度', '水分浓度'] if temperature else ['Sheet1']
29     assert book.sheetnames == expected_names
30     times = np.unique(np.r_[np.arange(step, model.end, step), model.end])
31     distances = np.arange(20 if model.moving else 21)/10
32     header = ['时间\\到药材中心的距离'] + distances.tolist()
33     if model.moving:
34         header += ['药材表面']
35     sampled = 0
36     for sheet in book:
37         rows = iter(sheet.values)
38         assert list(next(rows)) == header
39         count = 0
40         for count, row in enumerate(rows, 1):
41             assert count <= len(times)
42             t = times[count-1]
43             assert abs(row[0]-t) < 1e-7
44             assert len(row) == len(header)
45             inside = distances/100 <= model.radius(t)
46             if model.moving:
47                 inside = np.r_[inside, True]
48             values = row[1:]
49             assert all((v is not None) == bool(ok) for v, ok in zip(values, inside))
50             assert all(np.isfinite(v) for v in values if v is not None)
51             if sheet.title != '温度':
52                 assert all(v >= 0 for v in values if v is not None)
53             if count == 1 or count == len(times) or count % 1800 == 0:
54                 y = model.state([t])[0]
55                 field = y[model.size] if sheet.title == '温度' else y[model.size:]
56                 expected = np.interp(distances/100/model.radius(t), model.x, field, right=np.nan)
57                 if model.moving:
58                     expected = np.r_[expected, field[-1]]
59                 for value, target in zip(values, expected):
60                     if np.isfinite(target):
61                         assert abs(value-target) <= 5.00001e-5
62                         sampled += 1
63             assert count == len(times)
64     book.close()
65     return dict(ready=True, rows_per_sheet=len(times), sheets=expected_names,
66                 sampled_cells=sampled, sha256=digest(path),
67                 scope='Every row: time/domain/finite; sampled cells: unrounded trajectory')
68

```

```

69
70 def main():
71     parser = argparse.ArgumentParser()
72     parser.add_argument('--out', type=Path, required=True)
73     parser.add_argument('--workbooks', action='store_true')
74     args = parser.parse_args()
75     args.out.mkdir(parents=True, exist_ok=False)
76     start = time.perf_counter()
77     report = dict(ready=False, cache_read=False, workbooks_requested=args.workbooks,
78                  python=platform.python_version(), numpy=np.__version__,
79                  scipy=scipy.__version__, openpyxl=openpyxl.__version__,
80                  inputs={p.name:digest(p) for p in sorted((ROOT/'data').glob('input*.xlsx'))},
81                  code={p.name:digest(p) for p in sorted((ROOT/'code').glob('*.py'))}, cases={})
82     for family, cls in [('baseline', StableModel), ('latent', LatentModel)]:
83         folder = args.out/family
84         folder.mkdir()
85         cases = report['cases'][family] = {}
86         for q in range(1, 5):
87             n = 2560 if family == 'baseline' and q == 4 else 1280
88             print(f'SOLVE {family} Q{q} N{n}', flush=True)
89             model = cls(Settings(question=q, n=n)).run()
90             row = dict(meta=model.meta, requested_rows=requested_table(model),
91                       latent_heat=0 if family == 'baseline' else model.latent_heat)
92             if q >= 3:
93                 row['max_C_at_end'] = float(model.state([model.end])[model.size:, 0].max())
94                 assert row['max_C_at_end'] < .15
95             cases[str(q)] = row
96             (folder/f'q{q}.json').write_text(json.dumps(row, ensure_ascii=False, indent=2),
97                                               ↪ encoding='utf-8')
98             print(f'SOLVED {family} Q{q}: {model.meta}', flush=True)
99             if args.workbooks and q != 2:
100                 # Identical Q2/Q3 dynamics: Q3 supplies all Q2 times through strict completion.
101                 for output_q in ([2, 3] if q == 3 else [q]):
102                     path = folder/f'result{output_q}.xlsx'
103                     step = 1 if output_q <= 2 else 60
104                     thermal = output_q <= 2
105                     print(f'EXPORT {family} result{output_q}', flush=True)
106                     info = workbook(model, path, step, thermal)
107                     info['verification'] = verify_book(model, path, step, thermal)
108                     cases[str(output_q)]['workbook'] = info
109                     print(f'VERIFIED {family} result{output_q}: {info}', flush=True)
110             del model
111             gc.collect()
112             (args.out/'progress.json').write_text(json.dumps(report, ensure_ascii=False, indent=2),
113                                                   ↪ encoding='utf-8')
114             report.update(ready=True, exit_code=0, elapsed_s=time.perf_counter()-start)
115             (args.out/'report.json').write_text(json.dumps(report, ensure_ascii=False, indent=2),
116                                                 ↪ encoding='utf-8')
117             print('PASS: eight fresh production solves; see report.json', flush=True)
118
119 if __name__ == '__main__':
120     main()

```

```

1 # Verified with Python 3.12; numerical reproduction only.
2 numpy==1.26.4
3 scipy==1.13.1
4 openpyxl==3.1.5

```

C.16 正文定稿图的重绘程序

以下三个脚本分别生成物理基线与四格对照、第三问场图与径向历程、第四问变域图与停机对照。它们读取随源工程提供的冻结结果及来源记录，统一图形样式，不重新求解微分方程；不能用重绘成功替代数值复算。生成式示意图另随源包提供，不由这些脚本生成。

```

1 """Typography and compact layout for existing controlled comparisons."""
2 from pathlib import Path
3 import hashlib
4 import json
5 import matplotlib
6 matplotlib.use("Agg")
7 import matplotlib.pyplot as plt
8
9 ROOT = Path(__file__).resolve().parents[1]

```

```

10 OUT = ROOT / "figures"
11 plt.rcParams.update({
12     "font.family": ["Times New Roman", "SimSun"], "font.size": 9,
13     "mathtext.fontset": "stix", "axes.unicode_minus": False,
14     "axes.spines.top": False, "axes.spines.right": False,
15     "axes.linewidth": .6, "legend.frameon": False,
16     "pdf.fonttype": 42, "svg.fonttype": "none",
17     "xtick.labelsize": 8, "ytick.labelsize": 8,
18 })
19
20 def save(fig, name):
21     for ext in ("pdf", "svg", "png"):
22         fig.savefig(OUT / f"{name}.{ext}", dpi=360, facecolor="white")
23     plt.close(fig)
24
25 paths = [ROOT / "supplement/q1_base.json",
26          ROOT / "supplement/q1_lumped.json", ROOT / "evidence/improvement.json"]
27 base, lump, summary = [json.loads(p.read_text(encoding="utf8")) for p in paths]
28 fig, ax = plt.subplots(figsize=(155/25.4, 59/25.4), layout="constrained")
29 for key, label, color, ls in [
30     ("Tc", " 径向模型: 中心", "#247F78", "-"),
31     ("Ts", " 径向模型: 表面", "#B45249", "-"),
32     ("Tmean", " 径向模型: 体积均温", "#677582", "--")]:
33     ax.plot(base["time_s"], base[key], label=label, color=color, ls=ls, lw=1.1)
34 ax.plot(lump["time_s"], lump["T"], label=" 集中参数: 均温", color="#996F32", ls=":", lw=1.1)
35 ax.set(xlabel=" 时间 / s", ylabel=" 温度 / °C", xlim=(0, 1800))
36 ax.legend(fontsize=8.5, ncol=2)
37 save(fig, "q1_physical_compare_refined")
38
39 times = summary["factorial"]["times_h"]
40 fig, ax = plt.subplots(figsize=(155/25.4, 78.91/25.4))
41 fig.subplots_adjust(left=.17, right=.95, bottom=.20, top=.78)
42 for y, prop in [(0, 4), (1, 3)]:
43     fixed, moving = times[f"p{prop}_moving0"], times[f"p{prop}_moving1"]
44     ax.plot([moving, fixed], [y, y], color="#a4afb5", lw=1.1)
45     ax.scatter(fixed, y, s=27, facecolor="white", edgecolor="#56636b",
46               label=" 固定半径" if y == 0 else None, zorder=3)
47     ax.scatter(moving, y, marker="D", s=27, color="#286378",
48               label=" 收缩半径" if y == 0 else None, zorder=3)
49     for val, offset in [(fixed, 9), (moving, -17)]:
50         ax.annotate(f"{val:.4f}", (val, y), xytext=(0, offset),
51                    textcoords="offset points", ha="center", fontsize=8.5)
52 ax.set(xlim=(0, 140), ylim=(-.6, 1.6), yticks=[0, 1],
53        yticklabels=[" 附录 4 物性", " 附录 3 物性"], xlabel=" 临界干燥时间 / h")
54 ax.grid(axis="x", color="#e4e8eb", linestyle="--", lw=.45)
55 ax.spines["left"].set_visible(False)
56 ax.tick_params(axis="y", length=0)
57 ax.legend(loc="upper center", bbox_to_anchor=(.5, 1.27), ncol=2, fontsize=9)
58 save(fig, "factorial_refined")
59
60 fig, axes = plt.subplots(1, 2, figsize=(140/25.4, 51/25.4))
61 fig.subplots_adjust(left=.085, right=.99, bottom=.25, top=.78, wspace=.35)
62 for ax, q in zip(axes, [3, 4]):
63     rows = summary["scenarios"][str(q)]["scenarios"]
64     assert len(rows) == 8
65     for i, row in enumerate(rows):
66         ax.plot([row["max_C_at_scenario_stop"], row["max_C_at_nominal_stop"]],
67                [i+1, i+1], color="#cbd2d7", lw=.7)
68     ax.scatter([r["max_C_at_nominal_stop"] for r in rows], range(1, 9),
69               marker="x", color="#C65353", s=17, label=" 名义停机")
70     ax.scatter([r["max_C_at_scenario_stop"] for r in rows], range(1, 9),
71               marker="o", color="#3E7D5A", s=15, label=" 统一停机")
72     ax.axvline(.15, color="#66717e", ls="--", lw=.7)
73     ax.set(ylabel=f" 问题 {q}: 情景编号", yticks=[1, 3, 5, 7, 8],
74            xlabel=" 停机时全域最大含水率 / (kg/kg)", ylim=(.5, 8.5))
75 fig.legend(*axes[0].get_legend_handles_labels(), ncol=2, loc="upper center",
76            bbox_to_anchor=(.52, 1.02), fontsize=8.5)
77 save(fig, "scenarios_refined")
78 (OUT / "comparison_refined_identity.json").write_text(json.dumps(
79     {"inputs": {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
80                for p in paths}, "model_recomputed": False,
81     "factorial_times": times, "scenario_counts": [8, 8]},
82     ensure_ascii=False, indent=2), encoding="utf8")
83 print("Three comparison figures exported from unchanged numerical records.")

```

```

1 """Restyle two question-three plots from locked fields without recomputation."""
2 from pathlib import Path
3 import hashlib
4 import json

```

```

5
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 from matplotlib.colors import LinearSegmentedColormap
10 import numpy as np
11
12 ROOT = Path(__file__).resolve().parents[1]
13 SOURCE = ROOT / "evidence/q3_fields.npz"
14 OUT = ROOT / "figures"
15 plt.rcParams.update({
16     "font.family": ["Times New Roman", "SimSun"],
17     "font.size": 9, "axes.labelsize": 9, "xtick.labelsize": 8,
18     "ytick.labelsize": 8, "legend.fontsize": 8,
19     "mathtext.fontset": "stix", "axes.unicode_minus": False,
20     "axes.spines.top": False, "axes.spines.right": False,
21     "axes.linewidth": .6, "legend.frameon": False,
22     "pdf.fonttype": 42, "svg.fonttype": "none",
23     "savefig.facecolor": "white",
24 })
25 CMAP = LinearSegmentedColormap.from_list(
26     "moisture", ["#fbfcfc", "#b7d4cd", "#408477", "#214b5b"])
27 COLORS = ["#2C4A7C", "#66717E", "#B9852D", "#3E7D5A", "#C65353"]
28 LINES = ["-", "--", "-.", ":", "-"]
29 with np.load(SOURCE) as source:
30     d = {k: source[k] for k in source.files}
31 t = d["times_s"] / 3600
32 xi, c = d["xi"], d["C_xi"]
33 indices = [int(np.argmax(abs(xi - i / 4))) for i in range(5)]
34 assert np.allclose(xi[indices], np.arange(5) / 4)
35 assert indices == [0, 25, 50, 75, 100]
36
37 def save(fig, name):
38     for ext in ("pdf", "svg", "png"):
39         fig.savefig(OUT / f"{name}.{ext}", dpi=360)
40     plt.close(fig)
41
42 fig, axes = plt.subplots(1, 2, figsize=(140/25.4, 51/25.4))
43 fig.subplots_adjust(left=.09, right=.99, bottom=.22, top=.81, wspace=.32)
44 for i, j in enumerate(indices):
45     for ax in axes:
46         ax.plot(t, c[:, j], color=COLORS[i], ls=LINES[i],
47                lw=1.25 if i in (0, 4) else .95, label=f"{i*.5:g} cm")
48     for ax in axes:
49         ax.axhline(.15, color="#454545", lw=.7, ls=(0, (3, 2)))
50         ax.set(xlabel=" 时间 / h", ylabel=" 含水率 / (kg/kg)")
51         ax.grid(axis="y", color="#e7ebed", lw=.45)
52         ax.set_axisbelow(True)
53     axes[0].set(xlim=(0, t[-1]), ylim=(0, 2.6), xticks=[0, 10, 20, 30, 40, 50])
54     axes[1].set(xlim=(30, t[-1]), ylim=(.045, .23), xticks=[30, 40, 50])
55     axes[1].annotate(" 中心最后达标", xy=(t[-1], c[-1, 0]),
56                    xytext=(34, .211), fontsize=8, color=COLORS[0],
57                    arrowprops={"arrowstyle": "->", "color": COLORS[0], "lw": .65})
58     fig.legend(*axes[0].get_legend_handles_labels(), ncol=5, loc="upper center",
59              bbox_to_anchor=(.52, 1.01), handlelength=2.1, columnspacing=1.35)
60     save(fig, "q3_drying_refined")
61
62 ids = np.unique(np.r_[np.arange(0, len(t), 30), len(t)-1])
63 tt, xx = np.meshgrid(t[ids], xi)
64 z = c[ids].T
65 fig = plt.figure(figsize=(145/25.4, 69/25.4))
66 ax = fig.add_axes([.005, .12, .47, .77], projection="3d")
67 ax.plot_surface(tt, xx, z, cmap=CMAP, rstride=1, cstride=1,
68               linewidth=0, antialiased=True, rasterized=True,
69               vmin=.05, vmax=2.55)
70 ax.set(xlabel=" 时间 / h", ylabel=" 物质坐标", xticks=[0, 20, 40, 60],
71       yticks=[0, .5, 1], zticks=[.5, 1.5, 2.5])
72 ax.view_init(26, 225)
73 ax.set_box_aspect((1.25, 1, .82))
74 ax.tick_params(labelsize=7, pad=0)
75 ax.xaxis.labelpad = 0
76 ax.yaxis.labelpad = 0
77 ax.text2D(.03, 1.0, " 含水率 / (kg/kg)", transform=ax.transAxes, fontsize=8)
78 for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
79     axis.pane.fill = False
80     axis._axinfo["grid"].update(color="#dce2e4", linewidth=.4)
81 ax2 = fig.add_axes([.58, .20, .31, .69])
82 im = ax2.contourf(tt, xx, z, levels=np.linspace(.05, 2.55, 21), cmap=CMAP)
83 ax2.contour(tt, xx, z, levels=[.15], colors=[COLORS[-1]], linewidths=.9)

```

```

84 ax2.set(xlabel=" 时间 / h", ylabel=" 物质坐标", xticks=[0, 20, 40],
85         yticks=[0, .25, .5, .75, 1])
86 ax2.text(.02, 1.045, " 红线: 0.15 kg/kg", transform=ax2.transAxes,
87         fontsize=8, color=COLORS[-1])
88 cax = fig.add_axes([.91, .20, .018, .69])
89 cb = fig.colorbar(im, cax=cax, ticks=[.05, .55, 1.05, 1.55, 2.05, 2.55])
90 cb.ax.tick_params(labelsize=7, length=2, pad=2)
91 cb.outline.set_linewidth(.4)
92 save(fig, "q3_surface_refined")
93
94 report = {
95     "source_sha256": hashlib.sha256(SOURCE.read_bytes()).hexdigest(),
96     "full_curve_shape": list(c[:, indices].shape),
97     "radial_indices": indices,
98     "curve_sha256": hashlib.sha256(c[:, indices].tobytes()).hexdigest(),
99     "surface_display_rows": ids.tolist(),
100    "surface_sha256": hashlib.sha256(z.tobytes()).hexdigest(),
101    "surface_and_projection_same_array": True,
102    "model_recomputed": False,
103 }
104 (OUT / "q3_refined_identity.json").write_text(
105     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf8")
106 print(json.dumps({k: v for k, v in report.items() if k != "surface_display_rows"}))

1 """Redraw locked evidence and add a numerical error/runtime comparison."""
2 from pathlib import Path
3 import csv
4 import hashlib
5 import json
6
7 import matplotlib
8 matplotlib.use("Agg")
9 import matplotlib.pyplot as plt
10 from matplotlib.colors import LinearSegmentedColormap
11 import numpy as np
12
13 ROOT = Path(__file__).resolve().parents[1]
14 OUT = ROOT / "figures"
15 plt.rcParams.update({
16     "font.family": ["Times New Roman", "SimSun"],
17     "font.size": 9, "mathtext.fontset": "stix",
18     "axes.unicode_minus": False, "axes.linewidth": .65,
19     "axes.spines.top": False, "axes.spines.right": False,
20     "xtick.labelsize": 8, "ytick.labelsize": 8,
21     "legend.frameon": False, "pdf.fonttype": 42, "svg.fonttype": "none",
22 })
23 BLUE, TEAL, GOLD, RED = "#426882", "#207C77", "#AC7C32", "#BA514C"
24
25
26 def digest(path):
27     return hashlib.sha256(path.read_bytes()).hexdigest()
28
29
30 def save(fig, name):
31     fig.savefig(OUT / (name + ".pdf"), facecolor="white", dpi=600)
32     fig.savefig(OUT / (name + ".svg"), facecolor="white", dpi=600)
33     fig.savefig(OUT / (name + ".png"), facecolor="white", dpi=600)
34     plt.close(fig)
35
36
37 summary_path = ROOT / "evidence/improvement.json"
38 summary = json.loads(summary_path.read_text(encoding="utf8"))
39 fig, axes = plt.subplots(1, 2, figsize=(155 / 25.4, 58 / 25.4), sharey=True)
40 fig.subplots_adjust(left=.10, right=.985, bottom=.24, top=.85, wspace=.19)
41 points = []
42 for ax, q in zip(axes, [3, 4]):
43     rows = summary["comparisons"][str(q)]
44     assert [r["family"] for r in rows] == ["arithmetic", "uniform", "graded"]
45     assert all(r["n"] == 160 for r in rows)
46     for m, row, color, marker in zip(range(3), rows, [BLUE, GOLD, TEAL], ["o", "s", "D"]):
47         x, y = row["elapsed_s"], row["event_error_s"]
48         ax.scatter(x, y, color=color, marker=marker, s=32, zorder=3)
49         dx = -7 if (q == 3 and m == 2) or (q == 4 and m in [0, 1]) else 7
50         ax.annotate(f"M{m}: {y:.4f} s", (x, y), xytext=(dx, 7),
51                   textcoords="offset points", ha="right" if dx < 0 else "left",
52                   color=color, fontsize=8.5)
53         points.append({"question": q, "method": f"M{m}",
54                       "elapsed_s": x, "event_error_s": y})
55     ax.set(yscale="log", ylim=(.4, 850), xlabel=" 单次运行耗时 / s")

```

```

56 ax.set_title(f" 问题 {q}", loc="left", fontsize=9.5, pad=6)
57 ax.grid(axis="y", which="major", color="#E0E5E8", lw=.5)
58 axes[0].set(xlim=(15, 49), xticks=[20, 30, 40], ylabel=" 绝对事件时间误差 / s")
59 axes[1].set(xlim=(22, 65), xticks=[30, 40, 50, 60])
60 save(fig, "event_runtime_comparison")
61 with (OUT / "event_runtime_comparison.csv").open("w", newline="", encoding="utf8") as f:
62     writer = csv.DictWriter(f, fieldnames=list(points[0]))
63     writer.writeheader()
64     writer.writerows(points)
65
66 field_path = ROOT / "evidence/q4_fields.npz"
67 d = np.load(field_path)
68 t = d["times_s"] / 3600
69 r = d["radius_m"] * 100
70 fixed = d["r_fixed_m"] * 100
71 C = d["C"][:, :21]
72 assert len(t) == 3067 and C.shape == (3067, 21)
73 assert np.all(np.diff(t) > 0) and np.all(np.diff(r) <= 1e-12)
74 cmap = LinearSegmentedColormap.from_list("water", ["#F5F9F8", "#B9D6CF", "#629F96", "#204E57"])
75 cmap.set_bad("#E4E7E9")
76 fig = plt.figure(figsize=(120 / 25.4, 58 / 25.4))
77 ax = fig.add_axes([.12, .22, .29, .62])
78 cbax = fig.add_axes([.425, .22, .018, .62])
79 right = fig.add_axes([.655, .22, .33, .62])
80 im = ax.pcolormesh(t, fixed, np.ma.masked_invalid(C.T), shading="auto",
81                  cmap=cmap, vmin=.05, vmax=2.55, rasterized=True)
82 ax.fill_between(t, r, 2, color="#E4E7E9", zorder=2)
83 ax.plot(t, r, color=RED, lw=1.0, zorder=3)
84 ax.text(30, 1.8, " 当前域外", ha="center", fontsize=8, color="#50585E")
85 ax.set(xlim=(0, t[-1]), ylim=(0, 2), xlabel=" 时间 / h", ylabel=" 实际半径 / cm",
86        xticks=[0, 20, 40], yticks=[0, .5, 1, 1.5, 2])
87 cb = fig.colorbar(im, cax=cbax, ticks=[.05, 1.3, 2.55])
88 cb.ax.tick_params(labelsize=7.5, length=2)
89 cb.outline.set_visible(False)
90 fig.text(.095, .925, " 含水率 / (kg/kg)", fontsize=8.5)
91 selected = []
92 for target, color, style in zip([6, 12, 24, 48],
93                                [BLUE, GOLD, TEAL, RED], ["-", "--", "-.", ":"]):
94     idx = int(np.argmax(abs(t - target)))
95     assert abs(t[idx] - target) < 1e-10
96     x, y = d["xi"] * r[idx], d["C_xi"][idx]
97     right.plot(x, y, color=color, ls=style, lw=1.05, label=f"{target} h")
98     selected.append({"time_h": float(t[idx]), "points": len(y),
99                    "x_y_sha256": hashlib.sha256(np.c_[x, y].tobytes()).hexdigest()})
100 right.axhline(.15, color="#808A90", lw=.65, ls="--")
101 right.set(xlim=(0, 2), ylim=(0, 2.55), xlabel=" 实际半径 / cm",
102          ylabel=" 含水率 / (kg/kg)", xticks=[0, 1, 2], yticks=[0, 1, 2])
103 right.grid(axis="y", color="#E2E7E9", lw=.45)
104 right.legend(fontsize=7.5, loc="upper right", handlelength=2.0, labelspacing=.15)
105 save(fig, "q4_domain_refined")
106 identity = {
107     "inputs": {str(p.relative_to(ROOT)): digest(p) for p in [summary_path, field_path]},
108     "model_recomputed": False, "comparison_points": points,
109     "q4_times": len(t), "q4_fixed_locations": len(fixed), "q4_sections": selected,
110     "q4_nan_mask_sha256": hashlib.sha256(np.isnan(C).tobytes()).hexdigest(),
111     "q4_field_sha256": hashlib.sha256(C.tobytes()).hexdigest(),
112     "outputs": {str(p.name): digest(p) for name in ["q4_domain_refined", "event_runtime_comparison"]
113               for p in [OUT / (name + "." + ext) for ext in ["pdf", "svg", "png"]]},
114 }
115 (OUT / "final_figures_identity.json").write_text(
116     json.dumps(identity, ensure_ascii=False, indent=2), encoding="utf8")
117 print("Exported two figures; source data and equations unchanged.")

```

C.17 在线图表的完整导出代码

```

1 /* Online ECharts editor export. Source values remain in the local browser. */
2 const fs=require('fs'),path=require('path'),crypto=require('crypto');
3 let pw;try {pw=require('playwright');} catch(e){pw=require(path.join(process.env.USERPROFILE,'.cache/cod
    ↪ ex-runtimes/codex-primary-runtime/dependencies/node/node_modules/playwright'));}
4 const ROOT=__dirname;
5 const hash=p=>crypto.createHash('sha256').update(fs.readFileSync(p)).digest('hex');
6 (async()=>{
7     const source=path.join(ROOT,'evidence/improvement.json');
8     const t=JSON.parse(fs.readFileSync(source,'utf8')).factorial.times_h;
9     const fixed=[[t.p4_moving0,0],[t.p3_moving0,1]];
10    const moving=[[t.p4_moving1,0],[t.p3_moving1,1]];
11    const option={animation:false,backgroundColor:'#ffffff',
12        textStyle:{fontFamily:'Microsoft YaHei',fontSize:22,color:'#252B30'},

```

```

13   legend:{data:[' 固定半径',' 收缩半径'],top:13,left:'center',itemWidth:27,itemHeight:17,
14     itemGap:36,textStyle:{fontSize:22,fontFamily:'Microsoft YaHei'}},
15   grid:{left:166,right:90,top:100,bottom:90},
16   xAxis:{type:'value',min:0,max:140,interval:20,name:'临界干燥时间 /
    ↪ h',nameLocation:'middle',nameGap:48,
17     nameTextStyle:{fontSize:23,axisLabel:{fontSize:21,color:'#333'},axisLine:{show:true,lineStyle:{co
    ↪ lor:'#657078'}}},
18     splitLine:{lineStyle:{color:'#E4E8EB',type:'dashed'}}},
19   yAxis:{type:'category',data:['附录4物性','附录3物性'],axisTick:{show:false},axisLine:{show:false},
20     axisLabel:{fontSize:23,margin:24,color:'#252B30'}},
21   series:[0,1].map(i=>({type:'line',data:[moving[i],fixed[i]],symbol:'none',silent:true,
22     lineStyle:{color:'#A4AFB5',width:3,z:1}).concat([
23     {name:' 固定半径',type:'scatter',data:fixed,symbol:'circle',symbolSize:18,
24       itemStyle:{color:'#fff',borderColor:'#56636B',borderWidth:3},
25       label:{show:true,position:'top',distance:15,fontSize:23,color:'#414A50',formatter:p=>p.value[0].
    ↪ toFixed(4)}}),
26     {name:' 收缩半径',type:'scatter',data:moving,symbol:'diamond',symbolSize:21,
27       itemStyle:{color:'#286378'},label:{show:true,position:'bottom',distance:15,fontSize:23,
28         color:'#286378',formatter:p=>p.value[0].toFixed(4)}})}));
29   const code='option = '+JSON.stringify(option,(k,v)=>typeof v==='function'? '__FORMATTER__':v,2)
30     .replaceAll('__FORMATTER__','function(p){return p.value[0].toFixed(4)}')+'';
31   fs.mkdirSync(path.join(ROOT,'audit',{recursive:true}));
32   fs.writeFileSync(path.join(ROOT,'audit/factorial_online_option.js'),code);
33   const browser=await pw.chromium.launch({channel:'msedge',headless:true});
34   try{
35     const page=await browser.newPage({viewport:{width:1400,height:950},deviceScaleFactor:2});
36     const url='https://echarts.apache.org/examples/en/editor.html?c=line-simple';
37     await page.goto(url,{waitUntil:'domcontentloaded',timeout:45000});
38     await
    ↪ page.waitForFunction(()=>window.ace&&document.getElementById('code-panel'),null,{timeout:30000});
39     await page.waitForTimeout(5000);
40     await page.evaluate(code=>ace.edit('code-panel').setValue(code,-1),code);
41     await page.getByText('Run',{exact:true}).click();
42     await page.waitForTimeout(6000);
43     const frame=page.frames().find(f=>f!==page.mainFrame());
44     const actual=await frame.evaluate(()=>{
45       const el=document.getElementById('chart-container');
46       const chart=echarts.getInstanceByDom(el);
47       return chart.getOption().series.filter(x=>x.type==='scatter').map(x=>({name:x.name,data:x.data}));
48     });
49     if(JSON.stringify(actual.map(x=>x.data))!==JSON.stringify([fixed,moving])) throw Error('Website
    ↪ source-data mismatch');
50     await page.screenshot({path:path.join(ROOT,'audit/factorial_website.png')});
51     const exported=await frame.evaluate(()=>{
52       const el=document.getElementById('chart-container'),old=echarts.getInstanceByDom(el),opt=old.getOp
    ↪ tion();
53       old.dispose();el.style.width='1100px';el.style.height='560px';
54       const chart=echarts.init(el,null,{renderer:'svg',width:1100,height:560});chart.setOption(opt);
55       return {svg:chart.renderToSVGString(),version:echarts.version};
56     });
57     fs.writeFileSync(path.join(ROOT,'figures/factorial_online.svg'),exported.svg);
58     const preview=await browser.newPage({viewport:{width:1100,height:560},deviceScaleFactor:3});
59     await preview.setContent('<html><head><style>@page{margin:0}html,body{margin:0;width:100%;height:100
    ↪ %}svg{width:100%;height:100%;display:block}</style></head><body>'+exported.svg+'</body></html>');
60     await preview.screenshot({path:path.join(ROOT,'figures/factorial_online.png')});
61     await preview.pdf({path:path.join(ROOT,'figures/factorial_online.pdf'),width:'155mm',height:'78.91mm'
    ↪ ',printBackground:true,margin:{top:0,right:0,bottom:0,left:0}});
62     fs.writeFileSync(path.join(ROOT,'audit/WEBSITE_EXPORT.json'),JSON.stringify({url,date:new
    ↪ Date().toISOString(),
63       tool:'Apache ECharts online editor, Edge headless',version:exported.version,
64       input_sha256:hash(source),data:actual,stages:{access:true,editor_config:true,run:true,visible:true}
    ↪ ,export:true,numeric_check:true},
65     export:['svg','pdf','png'],effective_png_dpi:3300/155*25.4,
66     note:'Editor code executed in official website iframe; no Share/publish or raw attachment
    ↪ upload.',null,2));
67     console.log('Online ECharts data and SVG/PDF/PNG exports verified.');
```